Disassembly of the running p-system on the TI 99/4A.
Anders Persson, Sweden, 2026

The syntax of this file is NOT for assembly. It's for information and reference.

```
;=================================================
;
; This is kind of a table of contents, to make some things easier to find.
;
; 8 K RAM expansion memory
; 2000 80-column virtual screen
; 2780 Interpreters memory in VDP RAM
; 2784 Sound interrupt handling
; 2824 BIOS workspace
; 2896 Subsidiary workspace
; 28B6 Simulated interrupt workspace
; 2964 IO workspace
; 2984 Table with CRU addresses to cards with an interrupt DSR
; 29AB IO control blocks
; 29DC Peripheral Control Block pointers
; 2AB2 Current date
; 2B6E SYSCOM, starts with IO-result code
; 2BD0 Main task's Task Information Block (TIB)
; 2BFC IO device's pointer table
; 2C68 BIOS pointers
; 2CFA GROM tables, execution maps
; 2DB2 Op-code table for p-code interpretation
; 2FB2 Kernel procedure pointers
; 2FFE Kernel procedures
; 3026 Tables for IDSEARCH
; 3250 PME core routines
; 32DC P-system interrupt service
; 337E DSR search
; 33F6 BIOS call
; 3580 Kernel global data
; 3696 Device name table
; 3822 System file names
; 3902 Unused

; P-code card starts here
; 4000 Card valider
; 4038 Power up starting point
; 4142 PME support routines
; 41B6 Keyboard scan
; 4292 Screen handling
; 43FA Sprite movement list processing
; 4556 Sound list player
; 4774 Support routines for video
; 4886 PME p-code interpretation
; 580E Kernel procedures, code segment loader
; 5E28 PME p-code interpretation, continued

; P-code card page 2
; 5000 BIOS. Implements unitread, unitwrite, unitclear and unitstatus for all devices
;
;=================================================
```

When attempting to understand how the p-system works under the hood, you have to figure out what is accessed when handling the system's structures on assembly level. There is information about these data structures in the Internal Architecture Guide, but for convenience some of them are listed here.

```
* Code pool description *
Pooldes = Record
              PoolBase : fulladdress;
              PoolSize : integer;
              MinOffset : memptr;
              Maxoffset : memptr;
              Resolution : integer; (in bytes)
              PoolHead : SIB_P;
              Perm SIB : SIB_P;
              Extended : boolean;
End;

* Segment information block *
SIB = RECORD
//      The first item is not in the TI 99/4A, for some reason
              Seg_Pool: Pool_Ptr; {points to the description of the code pool 0
                                   where the segment resides}
              Seg_Base: Mem_Ptr; {byte offset within code pool of segment's 2
                                   memory location}
//            Seg_Refs: integer; {# of active calls to the seg} 0    // Or is it this one that's missing?
              Time_Stamp: integer; {memory swap activity} 4
              Link_Count: integer; {number of links to the SIB} 6
```

```
              Residency: -1..maxint;{-1 = pos lock, 0 = swap, n = mem lock} 8
              Seg_Name: PACKED ARRAY [0-7] OF CHAR; 10
              Seg_Leng: integer; {# of words in segment} 18
              Seg_Addr: integer; {disk address of segment} 20
              Vol_Info: VIP; {pointer to disk drive info} 22
              Data_Size: integer; {number of words in data segment} 24
              Res_SIBs: RECORD {code pool management record}
                           Next_SIB, {next SIB in list} 28
                           Prev_SIB: SIB_P; {previous SIB in list} 26
                           CASE boolean OF {scratch area}
                             TRUE: (Next_Sort: SIB_P); {next SIB in sort list} 30
                             FALSE: (New_Loc: Mem_Ptr); {temporary address} 30
              END {of Res_SIBs};
              M_Type: integer; 32
END {of SIB};

* Task information block *
TIB = RECORD {Task Information Block}
              Regs: PACKED RECORD
                           Wait_Q: TIB_Ptr; 0
                           Prior: byte; 2
                           Flags: byte; 3
                           SP_Low: Mem_Ptr; 4
                           SP_Upr: Mem_Ptr; 6
                           SP: Mem_Ptr; 8
                           MP: MSCW_Ptr; 10
                           Reserved: Integer 12
                           IPC: integer; 14
                           Env: ERec_Ptr; 16
                           Proc_Num: byte; 18
                           TIB_IOResult: byte; 19
                           Hang_Ptr: Sem_Ptr; 20
                           M_Depend: integer; 22
              END {of Regs}
              Main_Task: Boolean; 24
              Start_MSCW: MSCW_Ptr; 26
END {of TIB}

* Environment records *
E_Vect_P = ^E_Vect;
E_Rec_P = ^E_Rec;

E_Vect = RECORD
              Vec_Length: integer; {number of local segments} 0
              Map: ARRAY [1..Vec_Length] OF E_Rec_P; 2
                                        {local environment mapping}
              END {of E_Vect};

E_Rec = RECORD
              Env_Data: Mem_Ptr; {pointer to global data} 0
              Env_Vect: E_Vect_P; {pointer to environment} 2
              Env_SIB: SIB_P; {pointer to SIB for seg number} 4
              CASE boolean OF
                TRUE : (Link-Count: integer; {number of links to E_Rec} 6
                           Next_Rec: E_Rec_P); {next environment record} 8
END {of E_Rec};

* File information block *
FIB = RECORD
              FWindow: Window_P;
              FEOF, FEOLN: Boolean;
              FState : (FJandW, FNeedChar, FGotChar);
              FRecsize: integer;
              FLock: semaphore;
              CASE FIsOpen: Boolean OF
                true: (FIsBlkd: Boolean;
                           FUNIT:UNITNUM;
                           FVID:VID:
                           FReptCnt,
                           FNxtBlk,
                           FMaxBlk: integer;
                           FModified: Boolean;
                           FHeader: DirEntry;
                           CASE FSoftBuf: Boolean OF
                             true: (FNxtByte, FMaxByte: integer;
                                        FBUfChngd: Boolean;
                                        FBuffer: PACKED ARRAY [0..FBlkSize]
                                                     OF CHAR))
END {of FIB}

* ------------------------------------------------------------------------------ *

Disassembly starts here. It's taken from a running p-system on a real TI 99/4A and includes the following:
* 2000-3FFF  8 K RAM, which is used for tables and code.
```

```
* 4000-5FFF   8 K ROM residing on the p-code card. Visible when CRU bit 1F00 is ON.
* 5000-5FFF   4 K ROM also on the p-code card. Visible when CRU bits 1F00 and 1F80 are ON.
* 8300-83FF   256 bytes RAM pad, used for code, workspaces and some data.

CONSCAN       .EQU 000EH    ; Console keyscan routine
PCODE         .EQU 1F00H    ; P-code card base CRU address
PCODE2        .EQU 1F80H    ; P-code card second page CRU address
GROMCOD .EQU E804H          ; Code repository in GROM. Goes to CPU RAM 2000-351EH.
GROMCOD2 .EQU ED22H         ; Secondary code repository. Goes to CPU RAM 2AA6-357EH.
SNDGEN        .EQU 8400H    ; Sound generator chip
VDPWA   .EQU 8C02H          ; VDP write address
WAOFFS  .EQU 0402H          ; Write address offset from base address
VSTATUS .EQU 8802H          ; VDP Status
VSCREEN .EQU 800H           ; Start of screen in VDP RAM
SPRATT        .EQU 0C00H    ; Sprite attribute table
SPRMOT        .EQU SPRATT+80H             ; Sprite motion data
SP            .EQU 10                     ; R10 is replaced with SP in this file only when PASCALWS is in use.

; Mark stack control word offsets.
MSSTAT        .EQU 0                      ; Data in MSCW
MSDYN         .EQU 2
MSIPC         .EQU 4
MSENV         .EQU 6
MSPROC        .EQU 8

; Tape recorder control bits (TMS9901)
CONTROL       .EQU 0                      ; PSI control bit (interrupt/timer)
EXTMASK       .EQU 1                      ; PSI external interrupt mask
VDPMASK       .EQU 2                      ; PSI VDP interrupt mask
VDPIRQ        .EQU 2                      ; PSI VDP interrupt request
TMRMASK       .EQU 3                      ; Timer interrupt mask
INTREQ        .EQU 15                     ; PSI interrupt request pin
CS1MOT        .EQU 22                     ; Tape 1 motor control
CS2MOT        .EQU 23                     ; Tape 2 motor control
AUDGATE       .EQU 24                     ; Audio gate control
MAGOUT        .EQU 25                     ; Magnetic tape output
MAGINP        .EQU 27                     ; Magnetic tape input


;=============================================================================================================
==
; Code transferred from GROM to CPU RAM at power up reset sequence.
; The data between 2000H and 251CH is loaded from GROM by code @405EH on the p-code card. That's what's
listed below.
; The same memory is then used by the 80 column virtual screen, so this code lives only during startup.

; Startup sequence prepares the system for operation
;
; * Find out how much VDP RAM is available
; * Scans the keyboard
; * Collect CRU addresses of all PEB cards with interrupt routines
; * Resets CONSOLE:
; * Turns off physical screen
; * Clears physical screen
; * Sets up keyboard layout area
; * Resets disk drive #4:
; * Calculates VDP RAM available after this operation
; * Maps disk drives to BIOS driver
; * Sets up PRINTER: and REMOTE: devices
; * Checks if there is any GROM cartride present
; * Reads code file dictionary from OS:SYSTEM.PASCAL, the operating system
; * Builds Segment information blocks (SIB) for the operating system on the stack
; * Loads the proper PME core to 8300H
; * Sets up dictionary, SIB, segment disk address and constant pool pointers for operating system
; * Loads address of first p-code instruction to execute
; * Clears 80 column virtual screen
; * Starts the PME, which begins by calling a procedure in the Kernel segment
;

              2000 2000    .WORD 2000H
              2002 051C    .WORD 051CH
INITCOD       2004 C80B    mov  R11,@PCONT             ; Set up return address
              2006 23F8
              2008 06A0    BL   @>GROMLOD; Load yet another set of data from GROM, this time between 2AA6H
(SYSDATA) and 3580H (KRNGLOB)
              200A 4070
              200C C2A0    MOV  @INTERP,SP             ; Set up p-system stack pointer
              200E 4016
              2010 04CF    CLR  R15           ; Code flag CPU memory
              2012 0201    LI   R1,>INTMEM
              2014 2780
CLRSYS        2016 04F1    CLR  *R1+
              2018 0281    CI   R1,>SYSDATA
              201A 2AA6
```

```
              201C 16FC    JNE  CLRSYS
              201E 0201    LI   R1,0DF8H ; Initial value of INTMEM
              2020 0DF8
              2022 C801    MOV  R1,@INTMEM
              2024 2780
              2026 0201    LI   R1,3EDFH ; Initial value of TOPMEM
              2028 3EDF
              202A C801    MOV  R1,@TOPMEM
              202C 2782
              202E 0201    LI   R1,25D   ; Initial value of EXECERR
              2030 0019
              2032 C801    MOV  R1,@EXECERR
              2034 2A46
              2036 0200    LI   R0,INTWS+6             ; INTWS R3
              2038 83C6
              203A 0201    LI   R1,KSCSAVE
              203C 2AA0
              203E CC70    MOV  *R0+,*R1+; Load keyscan data to INTWS R3-R5
              2040 CC70    MOV  *R0+,*R1+
              2042 CC70    MOV  *R0+,*R1+
              2044 0200    LI   R0,>7c00 ; 7C is initial value of GPS substack pointer
              2046 7C00
              2048 D800    MOVB R0,@GPLSTK             ; GPL substack pointer
              204A 8373
              204C 0200    LI   R0,04FFH ; Pascal keyboard code
              204E 04FF
              2050 C800    MOV  R0,@KBD
              2052 8374
              2054 02E0    LWPI SUB2WS   ; Subsidiary workspace
              2056 2896
              2058 06A0    BL   @KSCAN2
              205A 42CE
              205C 02E0    LWPI PASCALWS
              205E 8380

; Scan for interrupt links on PEB cards
              2060 04C1    CLR  R1
              2062 020C    LI   R12,1F00H; P-code card CRU base address
              2064 1F00
LOOP200       2066 1E00    SBZ  0                      ; Disable p-code card
              2068 022C    AI   R12,-256 ; Reverse one card
              206A FF00
              206C 028C    CI   R12,0F00 ; Before first PEB card?
              2070 130C    JEQ  ENDSRCH
              2072 1D00    SBO  0
              2074 9820    CB   @VALDSR,@CARDVAL       ; Check if a valid card
              2076 2CAC
              2078 4000
              207A 16F5    JNE  LOOP200
              207C C020    MOV  @INTLINK,R0    ; Any interrupt link?
              207E 400C
              2080 13F2    JEQ  LOOP200
              2082 C84C    MOV  R12,@INTLIST(R1)       ; Store address in interrupt list
              2084 2986
              2086 05C1    INCT R1                     ; Advance list pointer
              2088 10EE    JMP  LOOP200

ENDSRCH       208A 020C    LI   R12,1F00H; P-code card CRU address
              208C 1F00
              208E 1D00    SBO  0                      ; Enable p-code card
              2090 0201    LI   R1,VDPWA
              2092 8C02
              2094 C801    MOV  R1,@SUBWIN+20          ; To SUBWIN R10
              2096 83B4
              2098 064A    DECT SP
              209A C6A0    MOV  @SBITMSK,*SP           ; Value is 1
              209C 2D92
              209E 06A0    BL   @UNITCL  ; Call UNITCLEAR from assembly level. Console: init.
              20A0 34DC
              20A2 C820    MOV  @>23fa,@VDPCOP2        ; Data is 81F0 for VDP R1
              20A4 23FA
              20A6 2CB0
              20A8 02E0    LWPI SUBWIN
              2044 83A0
              20AC 04C9    CLR  R9
              20AE 06A0    BL   @VDPWAW  ; Load R9 to *R10 (VDPWA)
              20B0 47C4
              20B2 0202    LI   R2,0020H ; chr(0)+chr(32)
              20B4 0020
              20B6 0201    LI   R1,2048D
              20B8 0800
LOOP201       20BA DA82    MOVB R2,@-2(R10)            ; Clear the screen image table
              20BC FFFE
              20BE 0601    DEC  R1
```

```
            20C0 15FC    JGT  LOOP201
            20C2 06C2    SWPB R2                    ; Time for the space
            20C4 0201    LI   R1,960D  ; Size of physical screen in text mode
            20C6 03C0
LOOP202 20C8 DA82    MOVB R2,@-2(R10)    ; Fill the screen with spaces
            20CA FFFE
            20CC 0601    DEC  R1
            20CE 15FC    JGT  LOOP202
            20D0 02E0    LWPI PASCALWS
            20D2 8380
            20D4 0420    BLWP @VMBW
            20D6 478E
            20D8 24A2    .WORD LAYOUT  ; Source pointer
            20DA 251A    .WORD LAYADDR ; Pointer to VDP address, 0D80, keyboard layout area
            20DC 0078    .WORD 120D         ; Byte count
            20DE 0420    BLWP @WMBW
            20E0 478E
            20E2 2476    .WORD DISKPCB
            20E4 29E4    .WORD DRV4PCB ; Points to PCB pointer table for drive #4
            20E6 0013    .WORD 19D
            20E8 064A    DECT SP
            20EA C6A0    MOV  @SBITMSK+4,*SP        ; Data is 4
            20EE 06A0    BL   @UNITCL  ; Call UNITCLEAR from assembly level
            20F0 34DC
            20F2 0420    BLWP @VMBR    ; Returns 1100H to R0 during startup
            20F4 4774
            20F6 23FC    .WORD 23FC                 ; Points to pointer to address
            20F8 8380    .WORD PASCALWS
            20FA 0002    .WORD 2
            20FC C060    MOV  @VDPTOP,R1            ; GPL VDP top returns 37D7 during startup
            20FE 8370
            2100 C000    MOV  R0,R0                 ; Drive CRU base
            2102 1611    JNE  ISDRIVE
            2104 C801    MOV  R1,@>2400
            2106 2400
            2108 0221    AI   R1,-255
            210A FF01
            210C C801    MOV  R1,@DSKBUF            ; Location of disk buffer in VDP RAM
            210E 2CEA
            2110 0221    AI   R1,-4
            2112 FFFC
            2114 0420    BLWP @VMBW
            2116 478E
            2118 23FF    .WORD 23FF
            211A 8382    .WORD PASCALWS+2
            211C 0004    .WORD 4
ISDRIVE     2126 0281    CI   R1,3FFF  ; Last address in VDP RAM
            2128 3FFF
            212A 134A    JEQ  NOTVAL
            212C 0581    INC  R1
            212E 04C4    CLR  R4
            2130 0420    BLWP @VMBR
            2132 4774
            2134 8382    .WORD PASCALWS+2           ; PASCALWS R1
            2136 8385    .WORD PASCALWS+5           ; PASCALWS R2 Lsby
            2138 0004    .WORD 4
            213A 06C2    SWPB R2
            213C 9802    CB   R2,@VALDSR
            213E 2CAC
            2140 163F    JNE  NOTVAL
            2142 8100    C    R0,R4
            2144 1302    JEQ  214AH
            2146 C043    MOV  R3,R1
            2148 10EE    JMP  ISDRIVE
            214A C143    MOV  R3,R5                 ; Copy last address in VDP RAM
            214C 0225    AI   R5,-283  ; Subtract required space
            214E FEE5
            2150 C081    MOV  R1,R2                 ; Computed from VDPTOP. Calculations about top of VDP
RAM
            2152 0222    AI   R2,3
            2154 0003
            2156 C105    MOV  R5,R4
            2158 6102    S    R2,R4
            215A C1E0    MOV  @VDPTOP,R7
            215C 8370
LOOP206     215E 0587    INC  R7
            2160 8047    C    R7,R1                 ; Does it match?
            2162 130F    JEQ  VMEMOK
            2164 0420    BLWP @VMBR
            2166 4774
            2168 838E    .WORD PASCALWS+14
            216A 8391    .WORD PASCALWS+17
            216C 0003    .WORD 3
            216E C289    MOV  R9,SP
```

```
             2170 A244    A    R4,R9
             2172 0609    DEC  R9
             2174 0420    BLWP @VMBW
             2176 478E
             2178 8391    .WORD PASCALWS+17D
             217A 838E    .WORD PASCALWS+14D
             217C 0003    .WORD 3
             217E C1CA    MOV  SP,R7
             2180 10EE    JMP  LOOP206
VMEMOK       2182 C182    MOV  R2,R6
             2184 61A0    S    @VDPTOP,R6
             2186 8370
             2188 0586    INC  R6
             218A C1E0    MOV  @VDPTOP,R7
             218C 8370
             218E C047    MOV  R7,R1
             2190 A1C4    A    R4,R7
             2192 C087    MOV  R7,R2
             2194 A046    A    R6,R1
             2196 A086    A    R6,R2
             2198 0602    DEC  R2
             219A 020A    LI   R10,VDPWA
             219C 8C02
LOOP203      219E C241    MOV  R1,R9                ; Copy data (PCB?) from original place to right before
buffer area in VDP RAM
             21A0 0601    DEC  R1
             21A2 06A0    BL   @VDPWAR
             21A4 47C8
             21A6 D12A    MOVB @-1026D(SP),R4       ; VDPRD
             21A8 FBFE
             21AA C242    MOV  R2,R9
             21AC 0602    DEC  R2
             21AE 06A0    BL   @VDPWAW
             21B0 47C4
             21B2 DA84    MOVB R4,@-2(SP)           ; VPDWD
             21B4 FFFE
             21B6 0606    DEC  R6
             21B8 16F2    JNE  LOOP203
             21BA 0607    DEC  R7
             21BC C807    MOV  R7,@VDPTOP
             21BE 8370
NOTVAL       21C0 C1E0    MOV  @VDPTOP,R7
             21C2 8370
             21C4 0227    AI   R7,-18D  ; Space for a drive PCB
             21C6 FFEE
             21C8 0201    LI   R1,DRV4PCB
             21CA 29E4
             21CC CC47    MOV  R7,*R1+  ; Store PCB address for #4
             21CE C447    MOV  R7,*R1   ; For #5
             21D0 0221    AI   R1,8
             21D2 0008
LOOP204      21D4 CC47    MOV  R7,*R1+  ; For #9 to #30
             21D6 0281    CI   R1,TAPEPCB
             21D8 2A1A
             21DA 16FC    JNE  LOOP204
             21DC 0420    BLWP @VMBW
             21DE 478E
             21E0 2476    .WORD DISKPCB
             21E2 29E4    .WORD DRV4PCB
             21E4 0013    .WORD 19D
             21E6 0201    LI   R1,PRTPCB
             21E8 29E8
             21EA 0227    AI   R7,-61D  ; Difference between PCB locations for REMOTE and PRINTER
             21EC FFC3
             21EE CC47    MOV  R7,*R1+
             21F0 0420    BLWP @VMBW                ; Store default PRINTER PCB
             21F2 478E
             21F4 2432    .WORD PRTDEF
             21F6 29E8    .WORD PRTPAB
             21F8 003D    .WORD 61D                 ; Max PCB size
             21FA 0227    AI   R7,-61
             21FC FFC3
             21FE CC47    MOV  R7,*R1+  ; To REMIN and REMOUT
             2200 CC47    MOV  R7,*R1+
             2202 0420    BLWP @VMBW                ; Store default REMOTE PCB
             2204 478E
             2206 2404    .WORD REMDEF
             2208 29EC    .WORD REMPCB
             220A 003D    .WORD 61D
             220C 0227    AI   R7,-22
             220E FFEA
             2210 C807    MOV  R7,@TPPAB; To thermal printer
             2212 2A1C
             2214 0420    BLWP @VMBW                ; Store default thermal printer (TP) PCB
```

```
             2216 478E
             2218 2460    .WORD TPDEF
             221A 2A1C    .WORD TPPCB
             221C 0016    .WORD 22D
             221E 0607    DEC  R7
             2220 C807    MOV  R7,@TOPMEM            ; Establish top of VDP RAM
             2222 2782
             2224 020A    LI   R10,IOPAR1            ; Check if there is a GROM module available
             2226 248A
             2228 06A0    BL   @UNITRD  ; Read from device
             222A 34AC
             222C C020    MOV  @SYSCOM,R0            ; IO result code
             222E 2B6E
             2230 1616    JNE  NOCART   ; IO error?
             2232 06A0    BL   @UNITRD
             2234 34AC
             2236 C020    MOV  @SYSCOM,R0            ; IO result code
             2238 2B6E
             223A 1611    JNE  NOCART   ; IO error, so no cartridge
             223C 0201    LI   R1,TAPEIO4            ; Set up to compare two buffers, 64 bytes
             223E 2846
             2240 0202    LI   R2,THRMBUF
             2242 28D8
             2244 0203    LI   R3,64D
             2246 0040  @
LOOP207      2248 8CB1    C    *R1+,*R2+
             224A 1609    JNE  NOCART
             224C 0643 C  DECT R3
             224E 16FC    JNE  LOOP207
             2250 0201    LI   R1,CODREAD+8
             2252 2D02
             2254 0202    LI   R2,15D
             2256 000F
LOOP208      2258 04F1    CLR  *R1+     ; Clear GROM library addresses
             225A 0602    DEC  R2
             225C 16FD -  JNE  LOOP208

; No GROM cartridge found
NOCART       225E C2A0    MOV  @INTERP,SP            ; Restore stack base
             2260 4016
             2262 064A    DECT SP
             2264 C6A0    MOV  @FAULCOD+2,*SP
             2266 2B72
             2268 064A    DECT SP
             226A C6A0    MOV  @BUFADDR,*SP
             226C 251C
             226E 064A    DECT SP
             2270 C6A0    MOV  @BUFIX,*SP
             2272 40BC
             2274 064A    DECT SP
             2276 C6A0    MOV  @CON200H,*SP          ; 512, one block
             2278 40B2
             227A 064A    DECT SP
             227C C6A0    MOV  @INTERP+2,*SP         ; Block address on disk
             227E 4018
             2280 064A    DECT SP
             2282 C6A0    MOV  @BUFIX,*SP
             2284 40BC
             2286 06A0    BL   @UNITRD  ; Try to read from GROM unit on p-code card (OS:)
             2288 34AC
             228A C160    MOV  @BUFADDR,R5
             228C 251C
             228E 020E    LI   R14,KRNGLOB
             2290 3580
             2292 0204    LI   R4,2
             2294 0002
             2296 C060    MOV  @EXECERR,R1
             2298 2A46

; Segment information blocks for the operating system are built on the stack
BULDSIB      229A 8825    c    @0(R5),@BUFIX         ; BUFIX is zero
             229C 0000
             229E 40BC
             22A0 133E    JEQ  >231E
             22A2 022A    AI   SP,-34   ; Push a SIB ON stack
             22A4 FFDE
             22A6 04EA    CLR  @0(SP)
             22A8 0000
             22AA 04EA    CLR  @2(SP)
             22AC 0002
             22AE 04EA    CLR  @4(SP)
             22B0 0004
             22B2 CAA0    MOV  @SBITMSK,@6(SP)       ; Data is 1, number of links
             22B4 2D92
```

```
22B6 0006
22B8 04EA    CLR  @8(SP)
22BA 0008
22BC 04EA    CLR  @10D(SP)
22BE 000A
22C0 04EA    CLR  @12D(SP)
22C2 000C
22C4 04EA    CLR  @14D(SP)
22C6 000E
22C8 04EA    CLR  @16D(SP)
22CA 0010
22CC CAA5    MOV  @2(R5),@18D(SP)        ; Segment length
22CE 0002
22D0 0012
22D2 CAA5    MOV  @0(R5),@20D(SP)        ; Segment address
22D4 0000
22D6 0014
22D8 AAA0    A    @INTERP+2,@20D(SP)     ; Address offset to start
22DA 4018
22DC 0014
22DE CAA0    MOV  @FAULCOD+2,@22D(SP)    ; Disk drive info poiner
22E0 2B72
22E2 0016
22E4 04EA    CLR  @24D(SP) ; Data size
22E6 0018
22E8 04EA    CLR  @28D(SP)
22EA 001C
22EC 04EA    CLR  @26D(SP)
22EE 001A
22F0 04EA    CLR  @30D(SP)
22F2 001E
22F4 022A    AI   SP,-10   ; Push on stack MSCW?
22F6 FFF6
22F8 CA8E    MOV  R14,@0(SP)             ; MSSTAT
22FA 0000
22FC 0202    LI   R2,EXECERR
22FE 2A46
2300 CA82    MOV  R2,@2(SP); MSDYN
2302 0002
2304 CA8A    MOV  SP,@4(SP); MSIPC
2306 0004
2308 0203    LI   R3,10D
230A 000A
230C AA83    A    R3,@4(SP); MSIPC
230E 0004
2310 CAA0    MOV  @SBITMSK,@6(SP)        ; Value is 1
2312 2D92
2314 0006
2316 04EA    CLR  @8(SP)   ; MSPROC
2318 0008
231A C90A    MOV  SP,@EXECERR(R4)        ; Environment record pointer
231C 2A46
231E 05C5    INCT R5       ; Advance in buffer
2320 05C5    INCT R5
2322 05C4    INCT R4
2324 0601    DEC  R1       ; Number of segments
2326 15B9    JGT  BULDSIB
2328 0201    LI   R1,EXECERR
232A 2A46
232C C061    MOV  @2(R1),R1; Environment record pointer
232E 0002
2330 C061    MOV  @4(R1),R1; SIB pointer
2332 0004
2334 C860    MOV  @MINUS1,@8(R1)         ; Position locked segment
2336 40AE
2338 0008
233A 022A    AI   SP,-10   ; Push another MSCW?
233C FFF6
233E C24A    MOV  SP,R9
2340 CA89    MOV  R9,@0(SP); Stack pointer value to top of stack
2342 0000
2344 CA89    MOV  R9,@2(SP)
2346 0002
2348 04EA    CLR  @>4(SP)
234A 0004
234C 04EA    CLR  @>6(SP)
234E 0006
2350 04EA    CLR  @>8(SP)
2352 0008
2354 C120    MOV  @SBIMSK+4,R4           ; Value is 4
2356 2D96
2358 C804    MOV  R4,@FAULDCOD+2
235A 2B72
235C C364    MOV  @CODREAD(R4),R13
```

```
            235E 2CFA
            2360 C3E4    MOV  @CODETYP(R4),R15
            2362 2D20
            2364 C064    MOV  @PMECORE(R4),R1
            2366 2D46
            2368 0202    LI   R2,FETCH
            236A 8300
            236C 0200    LI   R0,37D
            236E 0025
LOOP205     2370 CCB1    MOV  *R1+,*R2+; Move correct PME core in position
            2372 0600    DEC  R0
            2374 15FD    JGT  LOOP205
            2376 0201    LI   R1,EXECERR
            2378 2A46
            237A C061    MOV  @32D(R1),R1
            237C 0020
            237E C801    MOV  R1,@ERECP
            2380 2AB4
            2382 C0E1    MOV  @4(R1),R3; SIB pointer
2384 0004
            2386 C803    MOV  R3,@CURSIB
            2388 2AB2
            238A C820    MOV  @SBITMSK,@PMEFLIP      ; Value is 1
            238C 2D92
            238E 2AE8
            2390 C063    MOV  @20D(R3),R1            ; Segment disk address
            2392 0014
            2394 05C1    INCT R1
            2396 0A11    SLA  R1,1
            2398 A064    A    @ADDROFS(R4),R1
            239A 2D6C
            239C 06A0    BL   @GETDAT2
            239E 4178
            23A0 C802    MOV  R2,@SEGBASE
            23A2 2AE2
            23A4 C102    MOV  R2,R4
            23A6 06A0    BL   @SETPNT  ; Constant pool and dictionary pointers
            23A8 4FCA
            23AA C044    MOV  R4,R1
            23AC 0641    DECT R1
            23AE 06A0    BL   @GETDAT2 ; Where is first instruction?
            23B0 4178
            23B2 0A12    SLA  R2,1                   ; Word pointer made byte
            23B4 A0A0    A    @SEGBASE,R2            ; Segment relative pointer made absolute
            23B6 2AE2
            23B8 05C2    INCT R2
            23BA C202    MOV  R2,R8                  ; Load IPC with first instruction's address
            23BC 06A0    BL   @LOADAD1 ; Load p-code address, then return here
            23BE 4144
            23C0 C060    MOV  @BUFADDR,R1
            23C2 251C
            23C4 C061    MOV  @0120H(R1),R1          ; Comes to 263E, get 01BE
            23C6 0120
            23C8 0A11    SLA  R1,1                   ; 037C
            23CA C0CE    MOV  R14,R3   ; Has 3580, kernel global data
            23CC C8C3    MOV  R3,@0(R3); Store address to itself in kernel global data area
            23CE 0000
            23D0 0223    AI   R3,10D   ; Advance to SYSCOM pointer
            23D2 000A
            23D4 0204    LI   R4,SYSCOM
            23D6 2B6E
            23D8 C4C4    MOV  R4,*R3
            23DA 0203    LI   R3,A000
            23DC A000
            23DE 0201    LI   R1,MAINTIB
            23E0 2BD0
            23E2 C843    MOV  R3,@4(R1); SP_Low stack pointer, beginning of 24 K RAM
            23E4 0004
            23E6 04E0    CLR  @2A7A                  ; ?
            23E8 2A7A
            23EA 04E0    CLR  @2A7C                  ; ?
            23EC 2A7C
            23EE C820    MOV  @SBITMSK,@CURPRNO-1    ; Current procedure number set to 1
            23F4 0460    B    @CLRSCR  ; Clear the screen and start running the PME
            23F6 3568

PCONT       23F8 4066    .WORD PMERUN
            23FA 81F0    .WORD 81F0
            23FC 0004    .WORD 4
            23FE 00AA    .WORD AAH
            2400 0000
            2402 0000
REMDEF      2404 0000     ; Default REMOTE: PCB
            2406 0700
```

```
             2408 0000
             240A 0008
             240C 0000
             240E 3F00 ?
             2410 0100
             2412 0000
             2414 001C
             2416 5253 RS
             2418 3233 23
             241A 322F 2/
             241C 312E 1.
             241E 4241 BA
             2420 3D33 =3
             2422 3030 00
             2424 2E44 .D
             2426 413D A=
             2428 372E 7.
             242A 5041 PA
             242C 3D4F =O
             242E 2E45 .E
             2430 4300 C
PRTDEF       2432 0000    ; Default PRINTER: PCB
             2434 0700
             2436 0000
             2438 0008
             243A 0000
             243C 3F00 ?
             243E 0100
             2440 0000
             2442 001C
             2444 5253 RS
             2446 3233 23
             2448 322F 2/
             244A 322E 2.
             244C 4241 BA
             244E 3D39 =9
             2450 3630 60
             2452 302E 0.
             2454 4441 DA
             2456 3D37 =7
             2458 2E50 .P
             245A 413D A=
             245C 4F2E O.
             245E 4543 EC
TPDEF        2460 0000
             2462 0200
             2464 0000
             2466 0008
             2468 0006
             246A 3F00 ?
             246C 0000
             246E 0000
             2470 0004
             2472 5450 TP
             2474 2E45  E
DISKPCB      2476 0000    ; Blocked device Peripheral Control Block initial setup.
             2478 0100
             247A 0000
             247C 000A
             247E 0000
             2480 3F00 ?
             2482 0000
             2484 0000
             2486 0001
             2488 1000
IOPAR1       248A 0000    .WORD 0       ; Control
             248C 0002    .WORD 2       ; Block number
             248E 0040  @ .WORD 64D     ; Byte count
             2490 0000    .WORD 0       ; Buffer index
             2492 2846 (F .WORD TAPEIO4 ; Buffer address
             2494 000F    .WORD 15D     ; Unit number
IOPAR2       2496 0000    .WORD 0       ; Control
             2498 0002    .WORD 2       ; Block number
             249A 0040  @ .WORD 64D     ; Byte count
             249C 0000    .WORD 0       ; Buffer index
             249E 28D8 (  .WORD THRMBUF ; Buffer address
             24A0 0010    .WORD 16D     ; Unit number

; Keyboard layout data
LAYOUT       24A2 3132 12
             24A4 3334 34
             24A6 3536 56
             24A8 3738 78
             24AA 3930 90
```

```
                24AC 7177 qw
                24AE 6572 er
                24B0 7479 ty
                24B2 7569 ui
                24B4 6F70 op
                24B6 2061  a
                24B8 7364 sd
                24BA 6667 fg
                24BC 686A hj
                24BE 6B6C kl
                24C0 FF7A  z
                24C2 7863 xc
                24C4 7662 vb
                24C6 6E6D nm
                24C8 2E0D .
                24CA 2140 !@
                24CC 2324 #$
                24CE 2527 %'
                24D0 262A &*
                24D2 2829 ()
                24D4 B1B2
                24D6 8BB3
                24D8 B43E  >
                24DA 5F2D _-
                24DC 2B22 +"
                24DE 2009
                24E0 0889
                24E2 8384
                24E4 3C5E <^
                24E6 2F3D /=
                24E8 FF8F
                24EA 8A03
                24EC B53F ?
                24EE 3A3B :;
                24F0 2C0D ,
                24F2 7B7D {}
                24F4 8782
                24F6 8E8C
                24F8 8186
                24FA 5B5D []
                24FC 5157 QW
                24FE 4552 ER
                2500 5459 TY
                2502 5549 UI
                2504 4F50 OP
                2506 2041  A
                2508 5344 SD
                250A 4647 FG
                250C 484A HJ
                250E 4B4C KL
                2510 FF5A  Z
                2512 5843 XC
                2514 5642 VB
                2516 4E4D NM
                2518 1B0D
LAYADDR         251A 0D80    .WORD 0D80H   ; Keyboard layout area address in VDP RAM
BUFADDR         251C 251E %  .WORD BUFFER2
BUFFER2         251E 0001    ; Content after first read from OS:
                2520 05FF    ; Data to build operating system segment dictionary
                2522 0008
                2524 0063
                2526 0009
                2528: 04 F4 00 0E 0F 86 00 1E ........
                2530: 08 97 00 27 03 5B 00 2B ...'.[.+
                2538: 01 14 00 2D 02 87 00 30 ...-...0
                2540: 01 06 00 32 02 FE 00 36 ...2...6
                2548: 00 CE 00 37 03 95 00 3B ...7...;
                2550: 01 9D 00 3D 00 99 00 3E ...=...>
                2558: 01 05 00 40 05 83 4B 45 ...@..KE
                2560: 52 4E 45 4C 20 20 50 52 RNEL  PR
                2568: 49 4E 54 45 52 52 49 4E INTERRIN
                2570: 49 54 49 41 4C 49 47 45 ITIALIGE
                2578: 54 43 4D 44 20 20 46 49 TCMD  FI
                2580: 4C 45 4F 50 53 20 50 41 LEOPS PA
                2588: 53 43 41 4C 49 4F 45 58 SCALIOEX
                2590: 54 52 41 49 4F 20 53 4F TRAIO SO
                2598: 46 54 4F 50 53 20 48 45 FTOPS HE
                25A0: 41 50 4F 50 53 20 45 58 APOPS EX
                25A8: 54 52 41 48 45 41 50 45 TRAHEAPE
                25B0: 52 4D 48 45 41 50 53 43 RMHEAPSC
                25B8: 52 45 45 4E 4F 50 53 45 REENOPSE
                25C0: 47 53 43 49 4E 49 43 4F GSCINICO
                25C8: 4E 43 55 52 52 45 4F 53 NCURREOS
                25D0: 55 54 49 4C 20 20 55 53 UTIL  US
```

```
              25D8: 45 52 50 52 4F 47 02 02 ERPROG..
              25E0: 02 03 02 03 02 03 02 02 ........
              25E8: 02 02 02 02 02 02 02 02 ........
              25F0: 02 02 02 02 02 02 02 03 ........
              25F8: 02 02 02 02 02 03 00 00 ........
              2600: 00 00 00 00 00 00 00 00 ........
              2608: 00 00 00 00 00 00 00 00 ........
              2610: 00 00 00 00 00 00 00 00 ........
              2618: 00 00 00 00 00 00 80 01 ........
              2620: 80 07 80 08 80 09 80 02 ........
              2628: 80 02 80 02 80 02 80 02 ........

              271E                        ; Still buffer area

;-----------------------------------------------------------------------------------------------------------
; 80-column screen memory
; 2000-251E used for other purposes during boot time (see above)

              2000 3430 40
              2002 3630 60
              2004 2034  4
              2006 3037 07
              2008 3020 0
              200A 4070 @p
              200C 2020
              200E 2020
              2010 2020
              2012 2020
              2014 2020
              2016 2020
              2018 2020
              201A 2020
              201C 2020
              201E 2020
              2020 2020
              2022 2020
              2024 2020
              2026 2020
              2028 2020
              202A 2020
              202C 2020
              202E 2020
              2030 2020
              2032 2020
              2034 2020
              2036 2020
              2038 2020
              203A 2020
              203C 2020
              203E 2020
              2040 2020
              2042 2020
              2044 2020
              2046 2020
              2048 2020
              204A 2020
              204C 2020
              204E 2020
              2050 3430 40
              2052 3632 62
              2054 2030  0
              2056 3641 6A
              2058 3020 0
              205A 2020
              205C 2042  B
              205E 4C20 L
              2060 2020
              2062 4032 @2
              2064 3030 00
              2066 3420 4
              2068 2020
              206A 2020
              206C 2020
              206E 2020
              2070 2020
              2072 2020
              2074 2020
              2076 2020
              2078 2020
              207A 2020
              207C 2020
              207E 2020

; Cut away a large portion of screen data here, as it contains nothing interesting.
```

```
2640 5374 St
2642 6172 ar
2644 7461 ta
2646 6464 dd
2648 7265 re
264A 7373 ss
264C 2028  (
264E 3420 4
2650 6865 he
2652 7820 x
2654 6469 di
2656 6769 gi
2658 7473 ts
265A 293F )?
265C 2032  2
265E 3030 00
2660 3020 0
2662 2020
2664 2020
2666 2020
2668 2020
266A 2020
266C 2020
266E 2020
2670 2020
2672 2020
2674 2020
2676 2020
2678 2020
267A 2020
267C 2020
267E 2020
2680 2020
2682 2020
2684 2020
2686 2020
2688 2020
268A 2020
268C 2020
268E 2020
2690 2020
2692 2020
2694 2020
2696 2020
2698 2020
269A 2020
269C 2020
269E 2020
26A0 2020
26A2 2020
26A4 2020
26A6 2020
26A8 2020
26AA 2020
26AC 2020
26AE 2020
26B0 2020
26B2 2020
26B4 2020
26B6 2020
26B8 2020
26BA 2020
26BC 2020
26BE 2020
26C0 2020
26C2 2020
26C4 2020
26C6 2020
26C8 2020
26CA 2020
26CC 2020
26CE 2020
26D0 2020
26D2 2020
26D4 2020
26D6 2020
26D8 2020
26DA 2020
26DC 2020
26DE 2020
26E0 5374 St
26E2 6F70 op
26E4 6164 ad
```

```
                26E6 6472 dr
                26E8 6573 es
                26EA 7320 s
                26EC 2028  (
                26EE 3420 4
                26F0 6865 he
                26F2 7820 x
                26F4 6469 di
                26F6 6769 gi
                26F8 7473 ts
                26FA 293F )?
                26FC 2034  4
                26FE 3030 00
                2700 3020 0
                2702 2020
                2704 2020
                2706 2020
                2708 2020
                270A 2020
                270C 2020
                270E 2020
                2710 2020
                2712 2020
                2714 2020
                2716 2020
                2718 2020
                271A 2020
                271C 2020
                271E 2020
                2720 2020
                2722 2020
                2724 2020
                2726 2020
                2728 2020
                272A 2020
                272C 2020
                272E 2020
                2730 2020
                2732 2020
                2734 2020
                2736 2020
                2738 2020
                273A 2020
                273C 2020
                273E 2020
                2740 2020
                2742 2020
                2744 2020
                2746 2020
                2748 2020
                274A 2020
                274C 2020
                274E 2020
                2750 2020
                2752 2020
                2754 2020
                2756 2020
                2758 2020
                275A 2020
                275C 2020
                275E 2020
                2760 2020
                2762 2020
                2764 2020
                2766 2020
                2768 2020
                276A 2020
                276C 2020
                276E 2020
                2770 2020
                2772 2020
                2774 2020
                2776 2020
                2778 2020
                277A 2020
                277C 2020
                277E 2020
; End of 80-column screen

; Pointers to primary code pool in VDP RAM
INTMEM          2780 0DF8    .WORD         ;Start of primary code pool. Init to 0DF8H. Clears from here to
2AA4H on startup.
TOPMEM          2782 3E3C >< .WORD         ;End of primary code pool. Init to 3EDFH. That gives 12519 bytes.
```

```
; Sound data here is in blocks of four words (sometimes bytes), for each of the four sound generators
SOUND1        2784 0001    .WORD         ; Value restored to SOUND8
              2786 0001    .WORD
              2788 0001    .WORD
              278A 0001    .WORD
SOUND8        278C 5975 Yu .WORD         ; Decremented
              278E 595F Y_ .WORD
              2790 5958 YX .WORD
              2792 5943 YC .WORD
SOUND10       2794 0000    .WORD         ; Decremented, compared to zero
              2796 0000    .WORD
              2798 0000    .WORD
              279A 0000    .WORD
SOUND11       279C 0000    .WORD         ; Compared to 1
              279E 0000    .WORD
              27A0 0000    .WORD
              27A2 0000    .WORD
SOUND6        27A4 0000    .WORD         ; Pointer table
              27A6 0000    .WORD
              27A8 0000    .WORD
              27AA 0000    .WORD
SOUND9        27AC 0000    .WORD         ; Compares to 7FFFH (32767)
              27AE 0000    .WORD
              27B0 0000    .WORD
              27B2 0000    .WORD
SOUND14       27B4 00           .BYTE    ; Used for sound processing
              27B5 00      .BYTE
SOUND15       27B6 0000    .WORD         ; Voice+0
              27B8 0000    .WORD
              27BA 0000    .WORD
              27BC 0000    .WORD
              27BE 0000    .WORD         ; Voice+8
              27C0 0000    .WORD
              27C2 0000    .WORD
              27C4 0000    .WORD
              27C6 0000    .WORD         ; Voice+16
              27C8 0000    .WORD
              27CA 0000    .WORD
              27CC 0000    .WORD
              27CE 0000    .WORD         ; Voice+24
              27D0 0000    .WORD
              27D2 0000    .WORD
              27D4 0000    .WORD
              27D6 0000    .WORD
SOUND2        27D8 00           .BYTE
              27D9 00           .BYTE
              27DA 00           .BYTE
              27DB 00           .BYTE
SOUND13       27DC 9F           .BYTE    ; Data sent to sound generator
              27DD BF           .BYTE
              27DE DF           .BYTE
              27DF FF           .BYTE
SOUND5        27E0 0000    .WORD         ; Pointer data replaces SOUND6
              27E2 0000    .WORD
              27E4 0000    .WORD
SOUND17       27E6 0000    .WORD         ; Voice+0
              27E8 0000    .WORD
              27EA 0000    .WORD
              27EC 0000    .WORD
              27EE 0000    .WORD         ; Voice+8
              27F0 0000    .WORD
              27F2 0000    .WORD
              27F4 0000    .WORD
              27F6 0000    .WORD         ; Voice+16
              27F8 0000    .WORD
              27FA 0000    .WORD
              27FC 0000    .WORD
              27FE 0000    .WORD         ; Voice+24
              2800 0000    .WORD
              2802 0000    .WORD
              2804 0000    .WORD
              2806 0000    .WORD

SPRCOIN       2808 7FFF    .WORD         ;Last sprite coincidence counter
SPRBIT1       280A 0000    .WORD         ;Bit map for moving sprites
SPRBIT2       280C 0000    .WORD         ;Bit map for moving sprites
RETSAV3       280E 3348 3H .WORD

; Copy of VDP registers 1-7
; Contains write bit and register address too
VDPCOPY       2810 81F0    .WORD         ; R1 modes
              2812 8202    .WORD         ; R2 Screen image table                (0800H)
              2814 832F  / .WORD         ; R3 Color table                       (0BC0H)
              2816 8400    .WORD         ; R4 Pattern descriptor table          (0000H)
```

```
             2818 8518    .WORD         ; R5 Sprite attribute list (0C00H)
             281A 8600    .WORD         ; R6 Sprite descriptor table             (0000H)
             281C 001F    .WORD         ; R7 Fore- and background color          (here black/white, which
isn't default)

VDPST        281E CB00    .WORD         ; VDP status copy

TAPEIO1      2820 FFFF    .WORD         ; Tape IO data
TAPEIO2      2822 0000    .WORD         ; Tape IO data

; Workspace for BIOS
BIOSWS       2824 0000    .WORD         ; R0
             2826 0000    .WORD
             2828 0000    .WORD
             282A 0000    .WORD
             282C 0000    .WORD
             282E 0000    .WORD
             2830 0000    .WORD
             2832 0000    .WORD
             2834 0000    .WORD
             2836 0000    .WORD
             2838 0000    .WORD
             283A 0000    .WORD
             283C 0000    .WORD
             283E 0000    .WORD
             2840 0000    .WORD
             2442 0000    .WORD         ; R15

TAPEIO3      2844 0000    .WORD         ; Tape IO data
TAPEIO4      2846 0000    .WORD         ; 64 byte data buffer also used at startup
             2848 0000    .WORD
             284A 0000    .WORD
             284C 0000    .WORD
             284E 0000    .WORD
             2850 0000    .WORD
             2852 0000    .WORD
             2854 0000    .WORD
             2856 0000    .WORD
             2858 0000    .WORD
             285A 0000    .WORD
             285C 0000    .WORD
             285E 0000    .WORD
             2860 0000    .WORD
             2862 0000    .WORD
             2864 0000    .WORD
             2866 0000    .WORD
             2868 0000    .WORD
             286A 0000    .WORD
             286C 0000    .WORD
             286E 0000    .WORD
             2870 0000    .WORD
             2872 0000    .WORD
             2874 0000    .WORD
             2876 0000    .WORD
             2878 0000    .WORD
             287A 0000    .WORD
             287C 0000    .WORD
             287E 0000    .WORD
             2880 0000    .WORD
             2882 0000    .WORD
             2884 0000    .WORD

CURFLAG      2886 0000    .WORD         ; Cursor addressing flag
SAVCOL       2888 0000    .WORD         ; Save column on screen

KBUFPNT      288A 0BE0    .WORD         ; Keyboard buffer pointer
KBLAYO       288C 0D80    .WORD         ; Keyboard layout area pointer
SCBLANK      288E 6BA4 k  .WORD         ; Screen blanking timer
STOPFLG      2890 0000    .WORD         ; Stop flag. False=0, true=FFFF
ALOCKFL      2892 0000    .WORD         ; Alpha lock flag
FLUSHFL      2894 0000    .WORD         ; Flush flag

;Subsidiary workspace. Used at boot too
SUB2WS       2896 0000    .WORD         ; R0
             2898 0000    .WORD
             289A 0000    .WORD
             289C 0BB9    .WORD
             289E 29A8 )  .WORD         ; R4
             28A0 5B02 [  .WORD
KBUFMOD      28A2 FF40  @ .WORD         ; R6 key buffer mode flag FF40=empty, FF00=something, FF80=full
KBUFSTO      28A4 000A    .WORD         ; R7 key buffer store pointer
KBUFFTC      28A6 000A    .WORD         ; R8 key buffer fetch pointer
             28A8 0000    .WORD
             28AA 8C02    .WORD         ; R10 VDPWA
```

```
             28AC 41BE A  .WORD
             28AE 2AA6 *  .WORD         ; R12
             28B0 28B6 (  .WORD         ; R13 Was called from SIMIWS
             28B2 3368 3h .WORD NOKEYSC ; R14 That's KEYSCAN in the interrupt service
             28B4 D400    .WORD         ; R15

; Simulated interrupt workspace
SIMIWS       28B6 0000    .WORD         ; R0
             28B8 0004    .WORD
             28BA 0006    .WORD         ; R2 Sprite move vertical | horizontal
             28BC 0000    .WORD         ; R3 Sprite move vertical | horizontal
             28BE 0000    .WORD         ; R4 Sprite countdown
             28C0 0000    .WORD         ; R5 Sprite link
             28C2 0000    .WORD         ; R6 Sprite vertical | horizontal position
             28C4 0000    .WORD         ; R7 Sprite pattern | clock/color
             28C6 0000    .WORD
             28C8 0000    .WORD
             28CA 280C (  .WORD         ; R10
             28CC 3518 5  .WORD
             28CE 0000    .WORD
             28D0 8380    .WORD         ; R13 has been called from Pascal WS
             28D2 51F6 Q  .WORD         ; R14 VDP interrupt call from unit IO handler
             28D4 C000    .WORD         ; R15

THRMIO3      28D6 0000    .WORD
THRMBUF      28D8 0000    .WORD         ; 132 byte output buffer for thermal printer
             28DA 0000    .WORD
             28DC 0000    .WORD
             28DE 0000    .WORD
             28E0 0000    .WORD
             28E2 0000    .WORD
             28E4 0000    .WORD
             28E6 0000    .WORD
             28E8 0000    .WORD
             28EA 0000    .WORD
             28EC 0000    .WORD
             28EE 0000    .WORD
             28F0 0000    .WORD
             28F2 0000    .WORD
             28F4 0000    .WORD
             28F6 0000    .WORD
             28F8 0000    .WORD
             28FA 0000    .WORD
             28FC 0000    .WORD
             28FE 0000    .WORD
             2900 0000    .WORD
             2902 0000    .WORD
             2904 0000    .WORD
             2906 0000    .WORD
             2908 0000    .WORD
             290A 0000    .WORD
             290C 0000    .WORD
             290E 0000    .WORD
             2910 0000    .WORD
             2912 0000    .WORD
             2914 0000    .WORD
             2916 0000    .WORD
             2918 0000    .WORD
             291A 0000    .WORD
             291C 0000    .WORD
             291E 0000    .WORD
             2920 0000    .WORD
             2922 0000    .WORD
             2924 0000    .WORD
             2926 0000    .WORD
             2928 0000    .WORD
             292A 0000    .WORD
             292C 0000    .WORD
             292E 0000    .WORD
             2930 0000    .WORD
             2932 0000    .WORD
             2934 0000    .WORD
             2936 0000    .WORD
             2938 0000    .WORD
             293A 0000    .WORD
             293C 0000    .WORD
             293E 0000    .WORD
             2940 0000    .WORD
             2942 0000    .WORD
             2944 0000    .WORD
             2946 0000    .WORD
             2948 0000    .WORD
             294A 0000    .WORD
             294C 0000    .WORD
```

```
             294E 0000    .WORD
             2950 0000    .WORD
             2952 0000    .WORD
             2954 0000    .WORD
             2956 0000    .WORD
             2958 0000    .WORD
             295A 0000    .WORD         ; End of buffer for thermal printer
THRMIO1      295C 0000    .WORD
THRMIO2      295E 0000    .WORD

PPOINT       2960 3E90 >  .WORD         ; PCB pointer
IOOPCOD      2962 0300    .WORD         ; Opcode to be inserted into PAB

; Workspace for IO
IOWS         2964 0000    .WORD     ; R0
             2966 507A Pz .WORD     ; R1
             2968 C000    .WORD     ; R2
             296A 0000    .WORD     ; R3
             296C 0008    .WORD     ; R4
             296E 83E0    .WORD     ; R5
             2970 3E98 >  .WORD     ; R6
             2972 4174 At .WORD     ; R7
             2974 0700    .WORD     ; R8
             2976 1300    .WORD     ; R9
             2978 0108    .WORD     ; R10
             297A 0000    .WORD     ; R11
             297C 1F00    .WORD     ; R12
             297E 83C0    .WORD     ; R13
             2980 5B08 [  .WORD     ; R14
             2982 C000    .WORD     ; R15

; CRU addresses of cards with interrupt routines.
INTIX        2984 0000    .WORD                      ; Interrupt table index
INTLIST      2986 1400    .WORD                      ;  Clock card
             2988 1300    .WORD                      ;  RS232 card
             298A 0000    .WORD                      ; End of list
             298C 0000    .WORD
             298E 0000    .WORD
             2990 0000    .WORD
             2992 0000    .WORD
             2994 0000    .WORD
             2996 0000    .WORD
             2998 0000    .WORD
             299A 0000    .WORD
             299C 0000    .WORD
             299E 0000    .WORD
             29A0 0000    .WORD
             29A2 0000    .WORD

CRUSAVE      29A4 8300    .WORD                      ; R12 savearea
IOASM        29A6 0000    .WORD                      ; True when unit IO called as assembly procedure
; IO control block. Used for UNIT IO interface into BIOS
IOCTRL       29A8 0002    .WORD                      ; Opcode. Used as index in table at 2C68.
BLKNUM       29AA 0A00    .WORD                      ; Block number (+2)
BYTCNT       29AC 0001    .WORD                      ; Byte count (+4)
BUFADR       29AE ACE6    .WORD                      ; Buffer address (+6)
DRIVENO      29B0 0006    .WORD                      ; Drive number (+8)
CTRLPAR      29B2 0000    .WORD                      ; Control parameter (+10)
IOCOD        29B4 0000    .WORD                      ; IO result code (+12)
             29B6 0000    .WORD                      ; Physical drive number (+14)
; One byte buffers for (serial) devices
; Used to communicate IO result code for all devices
SERBUF       29B8 00      .BYTE         ; Unit #0
             29B9 0D      .BYTE
             29BA 20      .BYTE
             29BB 00      .BYTE
             29BC 00      .BYTE         ; Unit #4
             29BD 00      .BYTE
             29BE 0D      .BYTE
             29BF 00      .BYTE
             29C0 00      .BYTE
             29C1 00      .BYTE         ; Unit #9
             29C2 00      .BYTE
             29C3 00      .BYTE
             29C4 00      .BYTE         ; Unit #12
             20C5 FF      .BYTE
             29C6 00      .BYTE         ; Unit #14
             29C7 FF      .BYTE         ; Unit #15
             29C8 FF      .BYTE
             29C9 FF      .BYTE
             29CA FF      .BYTE
             29CB FF      .BYTE
             29CC FF      .BYTE
             29CD FF      .BYTE
```

```
             29CE FF      .BYTE
             29CF FF      .BYTE
             29D0 FF      .BYTE
             29D1 FF      .BYTE
             29D2 FF      .BYTE
             29D3 FF      .BYTE
             29D4 FF      .BYTE
             29D5 FF      .BYTE
             29D6 FF      .BYTE         ; Unit #30
             29D7 00      .BYTE
             29D8 00      .BYTE
             29D9 00      .BYTE
             29DA 00      .BYTE
             29DB 00      .BYTE         ; Unit #35

; PCB pointer table for volumes. PCBs are stored in VDP RAM amd are extended PABs.
PCBPNT       29DC 0000    .WORD         ; Unit #0
             29DE 0000    .WORD         ; Unit #1 CONSOLE:
             29E0 0000    .WORD         ; Unit #2 KEYBOARD:
             29E2 0000    .WORD         ; Unit #3
DRV4PCB      29E4 3ECD >  .WORD         ; Unit #4 Drive 1
             29E6 3ECD >  .WORD         ; Unit #5 Drive 2
PRTPCB       29E8 3E90 >  .WORD         ; Unit #6 PRINTER:
             29EA 3E53 >S .WORD         ; Unit #7 REMIN:
REMPCB       29EC 3E53 >S .WORD         ; Unit #8 REMOUT:
             29EE 3ECD >  .WORD         ; Unit #9 Drive 3
             29F0 3ECD >  .WORD         ; Unit #10 Drive 4
             29F2 3ECD >  .WORD         ; Unit #11 Drive 5
             29F4 3ECD >  .WORD         ; Unit #12 Drive 6
             29F6 3ECD >  .WORD         ; Unit #13 Assigned to unit loaded from tape
             29F8 3ECD >  .WORD         ; Unit #14 OS:
             29FA 3ECD >  .WORD         ; Unit #15 Implemented in GROM, if it existed
             29FC 3ECD >  .WORD         ; Unit #16
             29FE 3ECD >  .WORD         ; Unit #17
             2A00 3ECD >  .WORD         ; Unit #18
             2A02 3ECD >  .WORD         ; Unit #19
             2A04 3ECD >  .WORD         ; Unit #20
             2A06 3ECD >  .WORD         ; Unit #21
             2A08 3ECD >  .WORD         ; Unit #22
             2A0A 3ECD >  .WORD         ; Unit #23
             2A0C 3ECD >  .WORD         ; Unit #24
             2A0E 3ECD >  .WORD         ; Unit #25
             2A10 3ECD >  .WORD         ; Unit #26
             2A12 3ECD >  .WORD         ; Unit #27
             2A14 3ECD >  .WORD         ; Unit #28
             2A16 3ECD >  .WORD         ; Unit #29
             2A18 3ECD >  .WORD         ; Unit #30 Implemented in GROM, if it existed
TAPEPCB      2A1A 0000    .WORD         ; Unit #31 Tape
TPPCB        2A1C 3E3D >= .WORD         ; Unit #32 Thermal printer
             2A1E 0000    .WORD         ; Unit #33
             2A20 0000    .WORD         ; Unit #34
             2A22 0000    .WORD         ; Unit #35

; Register save area R0-R7.
SAVER0       2A24 5036 P6 .WORD         ; R0
SAVER1       2A26 AD98    .WORD
SAVER2       2A28 4D6E Mn .WORD
             2A2A 0004    .WORD         ; R3
             2A2C 008F    .WORD
             2A2E AD6A  j .WORD
             2A30 AD96    .WORD
             2A32 FB52  R .WORD         ; R7
SAVER11      2A34 4F48 OH .WORD         ; Save area R11

REAL3RD      2A36 0000    .WORD         ; Save area during segment fault, 3rd word for real constant
conversion
REAL1ST      2A38 0000    .WORD         ; Save area during segment fault, 1st word for real constant
conversion
SAVER13      2A3A 0000    .WORD         ; Code read address
SAVER15      2A3C 0000    .WORD         ; Code memory type
SAVER12      2A3E 8300    .WORD         ; PME fetch address
SAVFLIP      2A40 0001
REALSIZ      2A42 0000    .WORD         ; Save area during segment fault, real size for constant conversion
REALPNT      2A44 0000    .WORD         ; Save area during segment fault, constant pointer for real
conversion
EXECERR      2A46 0019    .WORD         ; I exec error, 25D at start
             2A48 FFB4    .WORD         ; E_Rec_P (Environment record pointer for current environment)
Indexed from EXECERR

             2A4A FDD0    .WORD
             2A4C FF04    .WORD
             2A4E A45E  ^ .WORD
             2A50 A3DA    .WORD
             2A52 A406    .WORD
```

```
             2A54 FF88    .WORD
             2A56 FF5C  \ .WORD
             2A58 FF30  0 .WORD
             2A5A A432  2 .WORD
             2A5C A356  V .WORD
             2A5E FE54  T .WORD
             2A60 FED8    .WORD
             2A62 FEAC    .WORD
             2A64 FE28  ( .WORD
             2A66 FD78  x .WORD         ; Kernel environment record pointer
             2A68 FCF4    .WORD
             2A6A FCC8    .WORD
             2A6C FC9C    .WORD
             2A6E FC70  p .WORD
             2A70 FC44  D .WORD
             2A72 FC18    .WORD
             2A74 FBEC    .WORD
             2A76 FBC0    .WORD
             2A78 FB94    .WORD
             2A7A 0011    .WORD         ; Cleared at startup
             2A7C 0000    .WORD         ; Cleared at startup
             2A7E 0000    .WORD
             2A80 0000    .WORD
             2A82 0000    .WORD
             2A84 0000    .WORD
             2A86 0000    .WORD
             2A88 0000    .WORD
             2A8A 0000    .WORD
             2A8C 0000    .WORD
             2A8E 0000    .WORD
             2A90 0000    .WORD
             2A92 0000    .WORD
             2A94 0000    .WORD
             2A96 0000    .WORD
             2A98 0000    .WORD
             2A9A 0000    .WORD
             2A9C 0000    .WORD
             2A9E 0000    .WORD

; Keyscan data for 83C6-83CA.           ; Registers moved here at startup, after clearing
KSCSAVE      2AA0 0100    .WORD
             2AA2 FFFF    .WORD
             2AA4 FF83    .WORD         ; End of area cleared at startup

SYSDATA      2AA6 2AA6 *  .WORD SYSDATA ; Loaded from GROM on second GROM repository transfer (Load address)
             2AA8 0AD8    .WORD 2776D   ; Loaded from GROM on second GROM repository transfer (Byte count)

VDPINT       2AAA 28B6 (  .WORD SIMIWS  ; Vector for simulated VDP "interrupt"
             2AAC 32DC 2  .WORD SIMINT

KEYSCAN      2AAE 2896 (  .WORD SUB2WS  ; Vector for keyboard scan and buffer storage
             2AB0 41BA A  .WORD KSCANX

; Various PME data for task switching and similar
CURSIB       2AB2 AD2A  * .WORD         ; Current SIB pointer                                  CURSIB
ERECP        2AB4 AD20    .WORD         ; Current environment record pointer     E_Rec_P
READYQ       2AB6 A000    .WORD         ; Poiner to TIB at head of ready queue   READYQ
EVECP        2AB8 A8FE    .WORD         ; Current environment vector pointer     E_Vec_P
CURTSK       2ABA 2BD0 +  .WORD         ; Current TIB pointer                                  CURTSK
NATCALL      2ABC 0000    .WORD         ; Native call flag. Also base (0) for processor register access.
SEGCNAB      2ABE B95C  \ .WORD         ; Current segment constant pool pointer (absolute).
SEGCNRL      2AC0 0BC6    .WORD         ; Current segment constant pool pointer (relative).
             2AC2 00      .BYTE         ; Referred to here it's procedure number as a word
CURPRNO      2AC3 01      .BYTE         ; Current procedure number (byte)
FRSTWRD      2AC4 FFFF    .WORD         ; First used word in set
             2AC6 0002    .WORD         ; Not directly referred to
IDIDENT      2AC8 0000    .WORD         ; Identifier found by IDSEARCH
             2ACA 0009    .WORD         ; Not directly referred to

IPCSAVE      2ACC B8C2    .WORD         ; P-code IPC save area. Used by PME during fault recovery

FCALLFL      2ACE 0000    .WORD         ; Formal procedure call flag (data on stack)
LASTWRD      2AD0 FFFF    .WORD         ; Bit manipulation SRS (Subrange set)
NEWSP        2AD2 0000    .WORD         ; Set test
WORDCNT      2AD4 0006    .WORD         ; Bit manipulation status SRS (Subrange set)
MTYPFL       2AD6 0000    .WORD         ; Memory type flag for segment
SMALSET      2AD8 0000    .WORD

RETSAV1      2ADA 5C22 \" .WORD         ; Return address (R11) save area when unit I/O is called as
subroutines.
RETSAV2      2ADC 4D02 M  .WORD         ; Return address (R11) save area

SYMCURS      2ADE 0000    .WORD         ; Pointer to SYMCURSOR in IDSEARCH
```

```
CUREREC     2AE0 AD20    .WORD         ; Current environment record
SEGBASE     2AE2 AD96    .WORD         ; Current code segment base.
DICTPNT     2AE4 BDD2    .WORD         ; Routine dictionary pointer in current code segment
            2AE6 0000    .WORD         ; Not directly referred to

PMEFLIP     2AE8 0001    .WORD         ; PME flip indicator. If zero do flip
FLTCNT      2AEA 0000    .WORD         ; Fault counter
GLOBPNT     2AEC A91C    .WORD         ; Global data pointer when error
IDSRBUF     2AEE 0000    .WORD         ; Buffer for IDSEARCH
STKMISS     2AF0 000C    .WORD         ; Stack fault data missing

SAVPACK     2AF2 BCE2    .WORD         ; Save area (packing, code type)
SAVPAK2     2AF4 0007    .WORD         ; SRS (Subrange set)
            2AF6 0D0D    .ASCII        ; CR,CR ?
            2AF8 5365 Se .ASCII "Segment ________ not found:"
            2AFA 676D gm
            2AFC 656E en
            2AFE 7420 t
SEGNAME     2B00 5F5F __
            2B02 5F5F __
            2B04 5F5F __
            2B06 5F5F __
            2B08 206E  n
            2B0A 6F74 ot
            2B0C 2066  f
            2B0E 6F75 ou
            2B10 6E64 nd
            2B12 3A0D :
            2B14 5075 Pu .ASCII "Put volume _______ in unit #--"
            2B16 7420 t
            2B18 766F vo
            2B1A 6C75 lu
            2B1C 6D65 me
VOLNAME     2B1E 205F  _
            2B20 5F5F __
            2B22 5F5F __
            2B24 5F5F __
            2B26 2069  i
            2B28 6E20 n
            2B2A 756E un
            2B2C 6974 it
            2B2E 2023  # COC
UNITNUM     2B30 2D2D --
            2B32 0D50  P .ASCII "Press spacebar to continue..."
            2B34 7265 re
            2B36 7373 ss
            2B38 2073  s
            2B3A 7061 pa
            2B3C 6365 ce
            2B3E 6261 ba
            2B40 7220 r
            2B42 746F to
            2B44 2063  c
            2B46 6F6E on
            2B48 7469 ti
            2B4A 6E75 nu
            2B4C 652E e.
            2B4E 2E2E ..
BUFFER      2B50 0001    .WORD         ; Used when there's a segment fault and the disk can't be found
            2B52 0000    .WORD
            2B54 0000    .WORD
            2B56 005A  Z .WORD
            2B58 0000    .WORD
            2B5A 2AF6 *  .WORD
            2B5C 0002    .WORD
            2B5E 0000    .WORD
            2B60 0000    .WORD
            2B62 0001    .WORD
            2B64 0000    .WORD
            2B66 2B00 +  .WORD
            2B68 0001    .WORD
BUFCNT      2B6A 001A    .WORD
TEN         2B6C 000A    .WORD 10D

SYSCOM      2B6E 0000    .WORD         ; IO completion code. Start of SYSCOM
FAULCOD     2B70 0000    .WORD         ; System fault code
SYSUNIT     2B72 0004    .WORD         ; Physical unit for startup
            2B74 0000    .WORD
            2B76 0000    .WORD
            2B78 0001    .WORD
            2B7A 0000    .WORD
REALSEM     2B7C 0000    .WORD         ; Real_Sem faulthandler start
            2B7E A10E    .WORD
```

```
FATIB       2B80 2BD0 +  .WORD         ; Fault_TIB
FAEREC      2B82 A432  2 .WORD         ; Fault_EREC
FAWORD      2B84 0000    .WORD         ; Fault_Words
FATYPE      2B86 0080    .WORD         ; Fault_type 80H segment fault, 81H stack fault

SIBPNT      2B88 AD74  t .WORD         ; SIB pointer
            2B8A 0001    .WORD
            2B8C 0000    .WORD
            2B8E 0000    .WORD
            2B90 0000    .WORD
            2B92 0000    .WORD
            2B94 0000    .WORD
            2B96 0000    .WORD
            2B98 0000    .WORD
            2B9A 0000    .WORD
            2B9C 0000    .WORD
            2B9E 8483    C    R3,*R2
            2BA0 F2F1    SOCB *R1+,R11
            2BA2 8493    C    *R3,*R2
TIMELOW     2BA4 C085    .WORD         ; System clock (low)
TIMEHI      2BA6 0000    .WORD         ; System clock (high)
            2BA8 020D    ; Not used?
            2BAA 0000    ; Not used?
            2BAC 8200    ; Not used?
            2BAE 8D8E    ; Not used?
            2BB0 8B89    ; Not used?
            2BB2 0008    ; Not used?
            2BB4 0000    ; Not used?
            2BB6 0000    ; Not used?
            2BB8 0018    .WORD 24D     ; Screen height (24)
            2BBA 0028  ( .WORD 40D     ; Screen width (40)
            2BBC 8A      .BYTE         ; Code for arrow down
            2BBD 8B      .BYTE         ; Code for arrow up
            2BBE 89      .BYTE         ; Code for arrow right
            2BBF 88      .BYTE         ; Code for arrow left
FLUSH       2BC0 87      .BYTE         ; Code for FLUSH
ETXCOD      2BC1 03      .BYTE         ; Code for etx
STOPCOD     2BC2 8E      .BYTE         ; Code for STOP
BREAK       2BC3 11      .BYTE         ; Code for BREAK
            2BC4 3F      .BYTE         ; Code for editor bad character
            2BC5 08 ?    .BYTE         ; Code for backspace
            2BC6 1B      .BYTE         ; Code for escape
            2BC7 8F      .BYTE         ; Code for line delete
            2BC8 03      .BYTE         ; Code for etx
            2BC9 00      .BYTE         ; ?
ALOCK       2BCA 8C      .BYTE         ; Code for alpha lock
            2BCB 00      .BYTE         ; Character mask 7 or 8 bit
            2BCC 0000    .WORD
            2BCE 0050  P .WORD 80D     ; Screen line length?

;TIB for main task
MAINTIB     2BD0 0000    .WORD         ; Wait_Q: TIB_Ptr
            2BD2 00      .BYTE         ; Prior
            28D3 80           .BYTE    ; Flags
            2BD4 D8D4    .WORD         ; SP_Low: Mem_Ptr
            2BD6 0000    .WORD         ; SP_Upr: Mem_Ptr
            2BD8 F840  @ .WORD         ; SP: Mem_Ptr
            2BDA F8F6    .WORD         ; MP: MSCW_Ptr
            2BDC 0000    .WORD         ; Reserved
            2BDE 021D    .WORD         ; IPC
            2BE0 AD20    .WORD         ; Env: ERec_Ptr
            2BE2 00      .BYTE         ; Proc_Num
            2BE3 08      .BYTE         ; TIB_IOresult
            2BE4 0000    .WORD         ; Hang_Ptr: Sem_Ptr
            2BE6 0000    .WORD         ; M_Depend
            2BE8 0001    .WORD         ; Main_Task: boolean
            2BEA FB80    .WORD         ; Start_MSCW: MSCW_Ptr

OPREAD      2BEC 0000    .WORD 0       ; Table with opcode indexes into table at 2C68 device driver entry
points
OPWRIT      2BEE 0002    .WORD 2
OPCLR       2BF0 0004    .WORD 4
OPSTAT      2BF2 0006    .WORD 6

; User defined IO devices
USERDEF     2BF4 007F  □ .WORD 127D    ; Highest device number
UNPNT       2BF6 0000    .WORD         ; Pointer to unit pointer table
PCBPNT      2BF8 0000    .WORD         ; Pointer to PCB pointer table
BUFAPNT     2BFA 0000    .WORD         ; Pointer to buffer address table

; Table with corresponding drive number for blocked units.
DRVNUM      2BFC 00      .BYTE 0
            2BFD 00      .BYTE 0
            2BFE 00      .BYTE 0
```

```
             2BFF 00      .BYTE 0
             2C00 01      .BYTE 1       ; #4:
             2C01 02           .BYTE 2  ; #5:
             2C02 00      .BYTE 0
             2C03 00      .BYTE 0
             2C04 00      .BYTE 0
             2C05 03      .BYTE 3       ; #9:
             2C06 04      .BYTE 4       ; #10:
             2C07 05      .BYTE 5       ; #11:
             2C08 06      .BYTE 6       ; #12:
             2C09 10      .BYTE 16D     ; #13 Loaded from tape
             2C0A 11      .BYTE 17D     ; #14 OS:
             2C0B 12      .BYTE 18D     ; #15:
             2C0C 13      .BYTE 19D
             2C0D 14      .BYTE 20D
             2C0E 15      .BYTE 21D
             2C0F 16      .BYTE 22D
             2C10 17      .BYTE 23D     ; #20
             2C11 18      .BYTE 24D
             2C12 19      .BYTE 25D
             2C13 1A      .BYTE 26D
             2C14 1B      .BYTE 27D
             2C15 1C      .BYTE 28D     ; #25
             2C16 1D      .BYTE 29D
             2C17 1E      .BYTE 30D
             2C18 1F      .BYTE 31D
             2C19 20      .BYTE 32D
             2C1A 21   !  .BYTE 33D     ; #30:
             2C1B 00      .BYTE 0       ; TAPE #31
             2C1C 00      .BYTE 0       ; TP #32
             2C1D 00      .BYTE 0
             2C1E 00      .BYTE 0
             2C1F 00      .BYTE 0
; Unit pointer table
UNITPNT      2C20 0000    .WORD 0                    ; #0 (reserved)
             2C22 2C68 ,h .WORD CONSOLE ; #1 CONSOLE
             2C24 2C68 ,h .WORD CONSOLE ; #2 SYSTERM
             2C26 0000    .WORD 0                    ; #3 (Graphic) Reserved for Terak microcomputers.
             2C28 2C70 ,p .WORD BLOCKED ; #4 Diskette name
             2C2A 2C70 ,p .WORD BLOCKED ; #5 Diskette name
             2C2C 2C80 ,  .WORD SERIAL  ; #6 PRINTER
             2C2E 2C80 ,  .WORD SERIAL  ; #7 REMIN
             2C30 2C80 ,  .WORD SERIAL  ; #8 REMOUT
             2C32 2C70 ,p .WORD BLOCKED ; #9 Diskette name
             2C34 2C70 ,p .WORD BLOCKED ; #10 Diskette name. Normally set to 0. Must be changed to 2C70H
(BLOCKED) for fourth drive to work.
             2C36 0000    .WORD 0                    ; #11 (Diskette name)
             2C38 0000    .WORD 0                    ; #12 (Diskette name)
             2C3A 2C70 ,p .WORD BLOCKED ; #13 Assigned to unit loaded from tape
             2C3C 2C70 ,p .WORD BLOCKED ; #14 OS
             2C3E 2C70 ,p .WORD BLOCKED ; #15 Handled as GROM units at standard GROM base address
             2C40 2C70 ,p .WORD BLOCKED ; #16 GROM library
             2C42 2C70 ,p .WORD BLOCKED ; #17
             2C44 2C70 ,p .WORD BLOCKED ; #18
             2C46 2C70 ,p .WORD BLOCKED ; #19
             2C48 2C70 ,p .WORD BLOCKED ; #20
             2C4A 2C70 ,p .WORD BLOCKED ; #21
             2C4C 2C70 ,p .WORD BLOCKED ; #22
             2C4E 2C70 ,p .WORD BLOCKED ; #23
             2C50 2C70 ,p .WORD BLOCKED ; #24
             2C52 2C70 ,p .WORD BLOCKED ; #25
             2C54 2C70 ,p .WORD BLOCKED ; #26
             2C56 2C70 ,p .WORD BLOCKED ; #27
             2C58 2C70 ,p .WORD BLOCKED ; #28
             2C5A 2C70 ,p .WORD BLOCKED ; #29
             2C5C 2C70 ,p .WORD BLOCKED ; #30 End of GROM library
             2C5E 2C88 ,  .WORD TAPE                 ; #31 TAPE
             2C60 2C78 ,x .WORD THERMAL ; #32 TP
             2C62 0000    .WORD                      ; #33
             2C64 0000    .WORD                      ; #34
             2C66 0000    .WORD                      ; #35

; Addresses in secondary ROM page for different unit IO operations.
; Pascal's unitwait and unitbusy are common for all units, so they need no table entry.
; Console
CONSOLE      2C68 538E S  .WORD CONREAD ; BIOS equivalent of Pascal's unitread
             2C6A 547E T~ .WORD CONWRIT ; BIOS equivalent of Pascal's unitwrite
             2C6C 5270 Rp .WORD CONINIT ; BIOS equivalent of Pascal's unitclear
             2C6E 5360 S` .WORD CONSTAT ; BIOS equivalent of Pascal's unitstatus
; Blocked devices
BLOCKED      2C70 5796 W  .WORD DSKREAD
             2C72 5740 W@ .WORD DSKWRIT
             2C74 568C V  .WORD DSKINIT
             2C76 5652 VR .WORD DSKSTAT
```

```
;Thermal printer
THERMAL       2C78 0000    .WORD 0                    ; Read not supported
              2C7A 59FA Y  .WORD TPWRIT
              2C7C 59CA Y  .WORD TPINIT
              2C7E 0000    .WORD 0                    ; Status not supported
; RS232 card
SERIAL        2C80 5ABE Z  .WORD REMREAD
              2C82 5AD8 Z  .WORD REMWRIT
              2C84 5AB4 Z  .WORD REMINIT
              2C86 0000    .WORD 0                    ; Status not supported
; Tape
TAPE          2C88 5B68 [h .WORD TAPREAD
              2C8A 5B22 [" .WORD TAPWRIT
              2C8C 5B10 [  .WORD TAPINIT
              2C8E 0000    .WORD 0                    ; Status not supported

; Sound data
SOUND4        2C90 80      .BYTE          ; Tone 1 frequency pre-amble
              2C91 A0    C .BYTE          ; Tone 2 frequency pre-amble
              2C92 C0            .BYTE    ; Tone 3 frequency pre-amble
              2C93 E0      .BYTE          ; Noise control
SOUND19       2C94 90            .BYTE
              2C95 B0      .BYTE
              2C96 D0            .BYTE
              2C97 F0      .BYTE
SOUND12       2C98 9F            .BYTE    ; Data sent to sound generator
              2C99 BF      .BYTE
              2C9A DF      .BYTE
              2C9B FF            .BYTE
REMIO         2C9C 01            .BYTE    ; Serial communication data
SOUND16 2C9D C3      .BYTE
              2C9E 0006    .WORD 6
              2CA0 0064  d .WORD 100D

SOUND7        2CA2 C000    .WORD          ; COC value for data via pointer SOUND6
              2CA4 8081    .WORD
              2CA6 C200    .WORD
              2CA8 C100    .WORD
SOUND18       2CAA A0            .BYTE
              2CAB 00            .BYTE

VALDSR        2CAC AA00    .WORD          ; DSR validation code
CLKBIT        2CAE 0080    .WORD          ; Bit mask used by sprite movement
; Copy of VDP registers for standard mode
VDPCOP2       2CB0 81F0    .WORD          ; VDP copy R1
              2CB2 8202    .WORD          ; VDP copy R2
              2CB4 832F  / .WORD          ; VDP copy R3
              2CB6 8400    .WORD          ; VDP copy R4
              2CB8 8518    .WORD          ; VDP copy R5
              2CBA 8600    .WORD          ; VDP copy R6
              2CBC 001F    .WORD          ; VDP copy R7
SCREN80       2CBE 2000    .WORD          ; Pointer 80 column screen
CURLINE       2CC0 0017    .WORD          ; Current line
CURCOL        2CC2 0000    .WORD          ; Current column
CURXY         2CC4 2730 '0 .WORD          ; Current 80 column address
WIDTH         2CC6 0028  ( .WORD 40D      ; Screen width (physical, 32 or 40)
HEIGHT        2CC8 0018    .WORD 24D      ; Screen height
WINDOW        2CCA 0000    .WORD          ; Current screen window
LINE23        2CCC 0730  0 .WORD          ; Number of bytes in 23 lines (for scroll)
              2CCE 0040  @ .WORD          ; Used by tape IO
AUTORPT 2CD0 0200    .WORD                ; Autorepeat counter
KEYLIM        2CD2 0006    .WORD          ; Keyscan limit (key buffering isn't done on every interrupt)
KEYCNT        2CD4 0001    .WORD          ; Kyescan limit counter
              2CD6 0002    .WORD          ; Not referred to
DSRERR        2CD8 0004    .WORD          ; Used in peripheral call
NOVOL         2CDA 0009    .WORD 9        ; IO error code "No such volume"
              2CDC 0008    .WORD          ; Not referred to
LASTGRM       2CDE 17FE    .WORD          ; Last valid GROM data address
GROMGAP       2CE0 0802    .WORD          ; GROM data address gap
GRMADDR       2CE2 0000    .WORD          ; GROM address for GROM volume access
VALIDVO       2CE4 FA00    .WORD          ; Valid volume indicator
SECSIZE       2CE6 0100    .WORD          ; Physical disk sector size
GROMSEC       2CE8 0200    .WORD          ; GROM simulated sector size
DSKBUF        2CEA 3F00 ?  .WORD          ; Address to disk IO buffer in VDP RAM
              2CEC 1100    JLT  2CEE      ; Not referred to, but happens to be disk drive CRU base address.
Coincidence?

; Disk sector access error code table
SECTERR       2CEE 06      .BYTE 6        ; SECTERR[0]
              2CEF 09      .BYTE 9        ; SECTERR[1]
              2CF0 07      .BYTE 7        ; SECTERR[2]
              2CF1 11      .BYTE 17D      ; SECTERR[3]
              2CF2 34      .BYTE 52D      ; SECTERR[4]
              2CF3 10      .BYTE 16D      ; SECTERR[5]
```

```
THBUFPT       2CF4 28D8 (  .WORD THRMBUF ; Thermal printer buffer pointer
WRCODE        2CF6 0300    .WORD 300H                ; Opcode write for IO
RECODE        2CF8 0200    .WORD 200H                ; Opcode read for IO

; Table with code read locations for different code memory types. Goes to R13 of PASCALWS
CODREAD       2CFA 0000    .WORD         ; CPU RAM
              2CFC 8800    .WORD         ; VDP RAM  #13
              2CFE 5BFC [  .WORD         ; P-code GROM  #14
              2D00 9800    .WORD         ; Ready for standard GROM as well!  #15
              2D02 9804    .WORD         ; Ready for standard GROM as well!
              2D04 9808    .WORD         ; Ready for standard GROM as well!
              2D06 980C    .WORD         ; Ready for standard GROM as well!
              2D08 9810    .WORD         ; Ready for standard GROM as well!  #19
              2D0A 9814    .WORD         ; Ready for standard GROM as well!  #20
              2D0C 9818    .WORD         ; Ready for standard GROM as well!
              2D0E 981C    .WORD         ; Ready for standard GROM as well!
              2D10 9820    .WORD         ; Ready for standard GROM as well!
              2D12 9824  $ .WORD         ; Ready for standard GROM as well!
              2D14 9828  ( .WORD         ; Ready for standard GROM as well!  #25
              2D16 982C  , .WORD         ; Ready for standard GROM as well!
              2D18 9830  0 .WORD         ; Ready for standard GROM as well!
              2D1A 9834  4 .WORD         ; Ready for standard GROM as well!
              2D1C 9838  8 .WORD         ; Ready for standard GROM as well!
              2D1E 9810    .WORD         ; Ready for standard GROM as well!  #30 (but this address is wrong.
Ought to be 983C)

; Code type indicator for different code types. Goes to R15 in PASCALWS
CODETYP       2D20 0000    .WORD 0       ; CPU RAM
              2D22 FFFF    .WORD -1D     ; VDP RAM #13
              2D24 0001    .WORD 1       ; GROM #14
              2D26 0001    .WORD 1       ; GROM
              2D28 0001    .WORD 1    ; GROM
              2D2A 0001    .WORD 1    ; GROM
              2D2C 0001    .WORD 1    ; GROM
              2D2E 0001    .WORD 1    ; GROM
              2D30 0001    .WORD 1    ; GROM
              2D32 0001    .WORD 1    ; GROM
              2D34 0001    .WORD 1    ; GROM
              2D36 0001    .WORD 1    ; GROM
              2D38 0001    .WORD 1    ; GROM
              2D3A 0001    .WORD 1    ; GROM
              2D3C 0001    .WORD 1    ; GROM
              2D3E 0001    .WORD 1    ; GROM
              2D40 0001    .WORD 1    ; GROM
              2D42 0001    .WORD 1    ; GROM
              2D44 0001    .WORD 1    ; GROM #30

; Table with address for the PME inner interpreter to load for different code types.
; GROM and VDP have the same. Different address loading is handled by LOADADDR.
; The code pointed to is moved to 8300H (PME FETCH address) for execution there.
; Prepared for standard GROM as well
PMECORE       2D46 3298 2  .WORD  PMERAM
              2D48 3250 2P .WORD  PMEMAP
              2D4A 3250 2P .WORD  PMEMAP
              2D4C 3250 2P .WORD  PMEMAP
              2D4E 3250 2P .WORD  PMEMAP
              2D50 3250 2P .WORD  PMEMAP
              2D52 3250 2P .WORD  PMEMAP
              2D54 3250 2P .WORD  PMEMAP
              2D56 3250 2P .WORD  PMEMAP
              2D58 3250 2P .WORD  PMEMAP
              2D5A 3250 2P .WORD  PMEMAP
              2D5C 3250 2P .WORD  PMEMAP
              2D5E 3250 2P .WORD  PMEMAP
              2D60 3250 2P .WORD  PMEMAP
              2D62 3250 2P .WORD  PMEMAP
              2D64 3250 2P .WORD  PMEMAP
              2D66 3250 2P .WORD  PMEMAP
              2D68 3250 2P .WORD  PMEMAP
              2D6A 3250 2P .WORD  PMEMAP

; Table with address offsets for reading code in different memory types
; P-code is always relative, but GROM in the console is not.
ADDROFS       2D6C 0000    .WORD  0
              2D6E 0000    .WORD  0                  ; #13: Unit loaded from tape. Set to something if
there is one.
              2D70 0000    .WORD  0                  ; OS:
              2D72 6000 `  .WORD  6000H  ; For cartridge space, to avoid console GROM.
              2D74 6000 `  .WORD  6000H
              2D76 6000 `  .WORD  6000H
              2D78 6000 `  .WORD  6000H
              2D7A 6000 `  .WORD  6000H
              2D7C 6000 `  .WORD  6000H
```

```
             2D7E 6000 `  .WORD  6000H
             2D80 6000 `  .WORD  6000H
             2D82 6000 `  .WORD  6000H
             2D84 6000 `  .WORD  6000H
             2D86 6000 `  .WORD  6000H
             2D88 6000 `  .WORD  6000H
             2D8A 6000 `  .WORD  6000H
             2D8C 6000 `  .WORD  6000H
             2D8E 6000 `  .WORD  6000H
             2D90 6000 `  .WORD  6000H  ; #30

; Single bit masks for general use
SBITMSK      2D92 0001    .WORD  1H
             2D94 0002    .WORD  2H
             2D96 0004    .WORD  4H
             2D98 0008    .WORD  8H
             2D9A 0010    .WORD  10H
             2D9C 0020    .WORD  20H
             2D9E 0040  @ .WORD  40H                 ; +12
             2DA0 0080    .WORD  80H
             2DA2 0100    .WORD  100H
             2DA4 0200    .WORD  200H   ; +18
             2DA6 0400    .WORD  400H
             2DA8 0800    .WORD  800H
             2DAA 1000    .WORD  1000H
             2DAC 2000    .WORD  2000H
             2DAE 4000 @  .WORD  4000H  ; +28
             2DB0 8000    .WORD  8000H  ; +30

; Address table for p-code interpreter. Opcode is a .WORD index.
OPCODE       2DB2 4968 .WORD SLDC       ; SLDC0 Short load constant
             2DB4 4968 .WORD SLDC
             2DB6 4968 .WORD SLDC
             2DB8 4968 .WORD SLDC
             2DBA 4968 .WORD SLDC
             2DBC 4968 .WORD SLDC
             2DBE 4968 .WORD SLDC
             2DC0 4968 .WORD SLDC
             2DC2 4968 .WORD SLDC
             2DC4 4968 .WORD SLDC
             2DC6 4968 .WORD SLDC
             2DC8 4968 .WORD SLDC
             2DCA 4968 .WORD SLDC
             2DCC 4968 .WORD SLDC
             2DCE 4968 .WORD SLDC
             2DD0 4968 .WORD SLDC
             2DD2 4968 .WORD SLDC
             2DD4 4968 .WORD SLDC
             2DD6 4968 .WORD SLDC
             2DD8 4968 .WORD SLDC
             2DDA 4968 .WORD SLDC
             2DDC 4968 .WORD SLDC
             2DDE 4968 .WORD SLDC
             2DE0 4968 .WORD SLDC
             2DE2 4968 .WORD SLDC
             2DE4 4968 .WORD SLDC
             2DE6 4968 .WORD SLDC
             2DE8 4968 .WORD SLDC
             2DEA 4968 .WORD SLDC
             2DEC 4968 .WORD SLDC
             2DEE 4968 .WORD SLDC
             2DF0 4968 .WORD SLDC       ; SLDC31
             2DF2 4972 .WORD SLDL       ; SLDL1 Short load local
             2DF4 4972 .WORD SLDL
             2DF6 4972 .WORD SLDL
             2DF8 4972 .WORD SLDL
             2DFA 4972 .WORD SLDL
             2DFC 4972 .WORD SLDL
             2DFE 4972 .WORD SLDL
             2E00 4972 .WORD SLDL
             2E02 4972 .WORD SLDL
             2E04 4972 .WORD SLDL
             2E06 4972 .WORD SLDL
             2E08 4972 .WORD SLDL
             2E0A 4972 .WORD SLDL
             2E0C 4972 .WORD SLDL
             2E0E 4972 .WORD SLDL
             2E10 4972 .WORD SLDL       ; SLDL16
             2E12 497E .WORD SLDO       ; SLDO1 Short load global
             2E14 497E .WORD SLDO
             2E16 497E .WORD SLDO
             2E18 497E .WORD SLDO
             2E1A 497E .WORD SLDO
             2E1C 497E .WORD SLDO
```

```
2E1E 497E .WORD SLDO
2E20 497E .WORD SLDO
2E22 497E .WORD SLDO
2E24 497E .WORD SLDO
2E26 497E .WORD SLDO
2E28 497E .WORD SLDO
2E2A 497E .WORD SLDO
2E2C 497E .WORD SLDO
2E2E 497E .WORD SLDO
2E30 497E .WORD SLDO       ; SLDO16
2E32 48CC .WORD UNUSED     ; Unused, fails
2E34 48CC .WORD UNUSED     ; Unused, fails
2E36 48CC .WORD UNUSED     ; Unused, fails
2E38 48CC .WORD UNUSED     ; Unused, fails
2E3A 48CC .WORD UNUSED     ; Unused, fails
2E3C 48CC .WORD UNUSED     ; Unused, fails
2E3E 48CC .WORD UNUSED     ; Unused, fails
2E40 48CC .WORD UNUSED     ; Unused, fails
2E42 48CC .WORD UNUSED     ; Unused, fails
2E44 48CC .WORD UNUSED     ; Unused, fails
2E46 48CC .WORD UNUSED     ; Unused, fails
2E48 48CC .WORD UNUSED     ; Unused, fails
2E4A 48CC .WORD UNUSED     ; Unused, fails
2E4C 48CC .WORD UNUSED     ; Unused, fails
2E4E 48CC .WORD UNUSED     ; Unused, fails
2E50 48CC .WORD UNUSED     ; Unused, fails
2E52 48CC .WORD UNUSED     ; Unused, fails
2E54 48CC .WORD UNUSED     ; Unused, fails
2E56 48CC .WORD UNUSED     ; Unused, fails
2E58 48CC .WORD UNUSED     ; Unused, fails
2E5A 48CC .WORD UNUSED     ; Unused, fails
2E5C 48CC .WORD UNUSED     ; Unused, fails
2E5E 48CC .WORD UNUSED     ; Unused, fails
2E60 48CC .WORD UNUSED     ; Unused, fails
2E62 48CC .WORD UNUSED     ; Unused, fails
2E64 48CC .WORD UNUSED     ; Unused, fails
2E66 48CC .WORD UNUSED     ; Unused, fails
2E68 48CC .WORD UNUSED     ; Unused, fails
2E6A 48CC .WORD UNUSED     ; Unused, fails
2E6C 48CC .WORD UNUSED     ; Unused, fails
2E6E 48CC .WORD UNUSED     ; Unused, fails
2E70 48CC .WORD UNUSED     ; Unused, fails
2E72 4A08 .WORD SLLA       ; SLLA1 Short load local address
2E74 4A08 .WORD SLLA
2E76 4A08 .WORD SLLA
2E78 4A08 .WORD SLLA
2E7A 4A08 .WORD SLLA
2E7C 4A08 .WORD SLLA
2E7E 4A08 .WORD SLLA
2E80 4A08 .WORD SLLA       ; SLLA8
2E82 4A22 .WORD SSTL       ; SSTL1 Short store local
2E84 4A22 .WORD SSTL
2E86 4A22 .WORD SSTL
2E88 4A22 .WORD SSTL
2E8A 4A22 .WORD SSTL
2E8C 4A22 .WORD SSTL
2E8E 4A22 .WORD SSTL
2E90 4A22 .WORD SSTL       ; SSTL8
2E92 4D2A .WORD SCXG1      ; SCXG1 Short call external global
2E94 4D36 .WORD SCXG
2E96 4D36 .WORD SCXG
2E98 4D36 .WORD SCXG
2E9A 4D36 .WORD SCXG
2E9C 4D36 .WORD SCXG
2E9E 4D36 .WORD SCXG
2EA0 4D36 .WORD SCXG       ; SCXG8
2EA2 498A .WORD SIND       ; SIND0 Short index
2EA4 498A .WORD SIND
2EA6 498A .WORD SIND
2EA8 498A .WORD SIND
2EAA 498A .WORD SIND
2EAC 498A .WORD SIND
2EAE 498A .WORD SIND
2EB0 498A .WORD SIND       ; SIND7
2EB2 4B52 .WORD LDCB       ; Load constant byte
2EB4 49E2 .WORD LDCI       ; Load constant integer
2EB6 4C30 .WORD LCO        ; Load constant offset
2EB8 4BEC .WORD LDC        ; Load constant
2EBA 49FC .WORD LLA        ; Load local address
2EBC 4A30 .WORD LDO        ; Load global
2EBE 4A3E .WORD LAO        ; Load global address
2EC0 49EE .WORD LDL        ; Load local
2EC2 4A80 .WORD LDA        ; Load intermediate address
2EC4 4A6A .WORD LOD        ; Load intermediate
```

```
2EC6 48FE .WORD UJP        ; Unconditional jump
2EC8 4914 .WORD JPL        ; Unconditional jump long
2ECA 524C .WORD MPI        ; Multiply integer
2ECC 521E .WORD DVI        ; Divide integer
2ECE 4B28 .WORD STM        ; Store multiple
2ED0 520A .WORD MODI       ; Modulo integer
2ED2 4CDC .WORD CLP        ; Call local procedure
2ED4 4CE4 .WORD CGP        ; Call global procedure
2ED6 4CC8 .WORD CIP        ; Call intermediate procedure
2ED8 4D22 .WORD CXL        ; Call external local
2EDA 4D4E .WORD CXG        ; Call external global
2EDC 4D06 .WORD CXI        ; Call external intermediate
2EDE 500E .WORD RPU        ; Return from procedure
2EE0 4D80 .WORD CFP        ; Call formal procedure
2EE2 49DA .WORD LDCN       ; Load constant nil
2EE4 4B3E .WORD LSL        ; Load static link
2EE6 4ACA .WORD LDE        ; Load extended
2EE8 4AD8 .WORD LAE        ; Load extended address
2EEA 4996 .WORD NOP        ; No operation
2EEC 4C6A .WORD LPR        ; Load processor register
2EEE 48D4 .WORD BPT        ; Break point
2EF0 51D8 .WORD BNOT       ; Boolean not
2EF2 51CC .WORD LOR        ; Logical or
2EF4 51C4 .WORD LAND       ; Logical and
2EF6 51F8 .WORD ADI        ; Add integer
2EF8 5204 .WORD SBI        ; Subtract integer
2EFA 4A1A .WORD STL        ; Store local
2EFC 4A4E .WORD SRO        ; Store global
2EFE 4A98 .WORD STR        ; Store intermediate
2F00 4998 .WORD LDB        ; Load byte
2F02 48B6 .WORD NAT        ; Enter native code
2F04 48C4 .WORD NAT_INFO   ; Native code information
2F06 48CC .WORD UNUSED     ; Reserved, fails
2F08 50CE .WORD CAP        ; Copy array parameter
2F0A 510A .WORD CSP        ; Copy string parameter
2F0C 4A5A .WORD SLOD1      ; Short load intermediate
2F0E 4A62 .WORD SLOD2      ; Short load intermediate
2F10 48CC .WORD UNUSED     ; Unused, fails
2F12 52E6 .WORD EQUI       ; Equal integer
2F14 52E6 .WORD EQUI       ; Not equal integer
2F16 52E6 .WORD EQUI       ; Less than or equal integer
2F18 52E6 .WORD EQUI       ; Greater than or equal integer
2F1A 52E6 .WORD EQUI       ; Less than or equal unsigned
2F1C 52E6 .WORD EQUI       ; Greater than or equal unsigned
2F1E 5638 .WORD EQPWR      ; Equal set
2F20 5638 .WORD EQPWR      ; Less than or equal set
2F22 5638 .WORD EQPWR      ; Greater than or equal set
2F24 52F4 .WORD EQBYT      ; Equal byte array
2F26 52F4 .WORD EQBYT      ; Less than or equal byte array
2F28 52F4 .WORD EQBYT      ; Greater than or equal byte array
2F2A 5470 .WORD SRS        ; Subrange set
2F2C 4C3E .WORD SWAP       ; Swap
2F2E 5F34 .WORD TNC        ; Truncate real
2F30 5F04 .WORD RND        ; Round real
2F32 5EB0 .WORD ADR        ; Add real
2F34 5EB8 .WORD SBR        ; Subtract real
2F36 5EC8 .WORD MPR        ; Multiply real
2F38 5EC0 .WORD DVR        ; Divide real
2F3A 4AAC .WORD STO        ; Store
2F3C 4B5A .WORD MOV        ; Move
2F3E 4C56 .WORD DUPR       ; Duplicate real
2F40 540C .WORD ADJ        ; Adjust set
2F42 49A6 .WORD STB        ; Store byte
2F44 5150 .WORD LDP        ; Load packed
2F46 5170 .WORD STP        ; Store packed
2F48 5256 .WORD CHK        ; Check subrange bounds
2F4A 5F24 .WORD FLT        ; Float
2F4C 5EDC .WORD EQREAL     ; Equal real
2F4E 5EDC .WORD EQREAL     ; Less than or equal real
2F50 5EDC .WORD EQREAL     ; Greater than or equal real
2F52 4AF4 .WORD LDM        ; Load multiple
2F54 4C88 .WORD SPR        ; Store processor reqister
2F56 4904 .WORD EFJ        ; Equal false jump
2F58 490C .WORD NFJ        ; Not equal false jump
2F5A 48E6 .WORD FJP        ; False jump
2F5C 4932 .WORD FJPL       ; False long jump
2F5E 493C .WORD XJP        ; Case jump
2F60 49CC .WORD IXA        ; Index array
2F62 51AA .WORD IXP        ; Index packed array
2F64 4AE8 .WORD STE        ; Store extended
2F66 554E .WORD INN        ; Set membership
2F68 55EE .WORD UNI        ; Set union
2F6A 55FE .WORD INT        ; Set intersection
2F6C 5622 .WORD DIF        ; Set difference
```

```
             2F6E 56B4 .WORD SIGNAL     ; Signal
             2F70 5720 .WORD WAIT       ; Wait
             2F72 51F2 .WORD ABI        ; Absolute value integer
             2F74 51FE .WORD NGI        ; Negate integer
             2F76 4C4C .WORD DUPI       ; Duplicate one word
             2F78 5ED0 .WORD ABR        ; Absolute value real
             2F7A 5ED6 .WORD NGR        ; Negate real
             2F7C 51D2 .WORD LNOT       ; Logical not
             2F7E 49BA .WORD IND        ; Index
             2F80 49C4 .WORD INC        ; Increment
             2F82 533C .WORD EQSTR      ; Equal string
             2F84 533C .WORD EQSTR      ; Less than or equal string
             2F86 533C .WORD EQSTR      ; Greater than or equal string
             2F88 53A0 .WORD ASTR       ; Assign string
             2F8A 53EC .WORD CSTR       ; Check string index
             2F8C 51E6 .WORD INCI       ; Increment integer
             2F8E 51EC .WORD DECI       ; Decrement integer
             2F90 4CAA .WORD SCPI1      ; Short call intermediate procedure
             2F92 4CB2 .WORD SCPI2      ; Short call intermediate procedure
             2F94 48DC .WORD TJP        ; True jump
             2F96 4BCC .WORD LDCRL      ; Load constant real
             2F98 5E8A .WORD LDRL       ; Load real
             2F9A 5E9E .WORD STRL       ; Store real
             2F9C 5E88 .WORD UNUSED1    ; Unused, does nothing
             2F9E 5E88 .WORD UNUSED1    ; Unused, does nothing
             2FA0 48CC .WORD UNUSED     ; Unused, fails
             2FA2 48CC .WORD UNUSED     ; Unused, fails
             2FA4 48CC .WORD UNUSED     ; Unused, fails
             2FA6 48CC .WORD UNUSED     ; Reserved, fails
             2FA8 48CC .WORD UNUSED     ; Reserved, fails
             2FAA 48CC .WORD UNUSED     ; Reserved, fails
             2FAC 48CC .WORD UNUSED     ; Reserved, fails
             2FAE 48CC .WORD UNUSED     ; Reserved, fails
             2FB0 4996 .WORD RESERVE6   ; Reserved, does nothing

; Address table for Kernel procedures
KERNPROC 2FB2 0000 .WORD                ; Reserved 1
             2FB4 0000 .WORD                        ; Reserved 2
             2FB6 0000 .WORD                        ; Reserved 3
             2FB8 354C .WORD RLOCSEG                ; RELOCSEG 4
             2FBA 0000 .WORD                        ; Reserved 5
             2FBC 0000 .WORD                        ; Reserved 6
             2FBE 0000 .WORD                        ; Reserved 7
             2FC0 0000 .WORD                        ; Reserved 8
             2FC2 0000 .WORD                        ; Reserved 9
             2FC4 0000 .WORD                        ; Reserved 10
             2FC6 0000 .WORD                        ; Reserved 11
             2FC8 0000 .WORD                        ; Reserved 12
             2FCA 0000 .WORD                        ; Reserved 13
             2FCC 0000 .WORD                                ; MOVESEG 14
             2FCE 5856 .WORD MOVELEFT   ; MOVELEFT 15
             2FD0 5884 .WORD MOVERIGHT  ; MOVERIGHT 16
             2FD2 0000 .WORD                                ; Reserved 17
             2FD4 34B6 .WORD UNITRDN                ; UNITREAD 18
             2FD6 34CE .WORD UNITWRN                ; UNITWRITE 19
             2FD8 58D6 .WORD TIME                   ; TIME 20
             2FDA 58A6 .WORD            FILLCHR             ; FILLCHAR 21
             2FDC 58E4 .WORD SCAN                   ; SCAN 22
             2FDE 580E .WORD IOCHECK                ; IOCHECK 23
             2FE0 581C .WORD GETPOOL                ; GETPOOLBYTES 24
             2FE2 582E .WORD PUTPOOL                ; PUTPOOLBYTES 25
             2FE4 2FFE .WORD FLIPSEG                ; FLIPSEGBYTES 26
             2FE6 3018 .WORD QUIET                  ; QUIET 27
             2FE8 0000 .WORD ENABLE                 ; ENABLE 28 (Not implemented)
             2FEA 56AE .WORD ATTACH                 ; ATTACH 29
             2FEC 592A .WORD IORES                  ; IORESULT 30
             2FEE 3500 .WORD UNITBU                 ; UNITBUSY 31
             2FF0 5F5C .WORD PWRTEN                 ; POWEROFTEN 32
             2FF2 350A .WORD UNITWA                 ; UNITWAIT 33
             2FF4 34E6 .WORD UNITCL                 ; UNITCLEAR 34
             2FF6 5930 .WORD HALTBRP                ; Reserved 35 (System fault breakpoint)
             2FF8 34F8 .WORD UNITSTN                ; UNITSTATUS 36
             2FFA 593C .WORD IDSCR                  ; IDSEARCH 37
             2FFC 5A16 .WORD            TREESRC             ; TREESEARCH 38
                                                    ; READSEG  No space for this one?

; Kernel procedure FLIPSEG.BYTES (?) Does a software reset (?)
; Definitely does not flip .WORDs in a segment
FLIPSEG      2FFE 020C    LI   R12,PCODE; P-code card
             3000 1F00
             3002 1E00    SBZ  0        ; Disable card
             3004 04CC    CLR  R12      ; Reset sequence for PSI
             3006 1D00    SBO  CONTROL  ; Timer mode
             3008 1E0F    SBZ  INTREQ   ; Timer interrupt
```

```
             300A 1E00    SBZ  CONTROL
             300C 04E0    CLR  @83C2
             300E 83C2
             3010 04E0    CLR  @83C4    ; User interrupt link
             3012 83C4
             3014 0420    BLWP @0000    ; Warm reset vector
             3016 0000

; Kernel procedure QUIET
; Normally disables all events. Here aren't any, so it resets the console.
QUIET        3018 06A0    BL   @PAGE2
             301A 352C 5,
             301C 06A0    BL   @RESCON
             301E 52B2 R
             3020 06A0    BL   @PAGE1
             3022 353C 5<
             3024 045C  \ B    *R12                  ; PME fetch

; Table with pointers to some of the alphabetically sorted key.WORDs in the next table. Each pointer is
followed
; by the number of entries beginning with the letter in question. @@@@@@@@ means no entry beginning with that
letter.
; Used by IDSEARCH
IDPOINT      3026 3098 0  .WORD IDAND   ; AND 2
IDCOUNT      3028 0002    .WORD 2
             302A 30AC 0  .WORD IDBEGIN ; BEGIN 1
             302C 0001    .WORD 1
             302E 30B6 0  .WORD IDCASE  ; CASE 2
             3030 0002    .WORD 2
             3032 30CA 0  .WORD IDDEFIN ; DEFINITI 4
             3034 0004    .WORD 4
             3036 30F2 0  .WORD IDELSE  ; ELSE 3
             3038 0003    .WORD 3
             303A 3110 1  .WORD IDFOR   ; FOR 4
             303C 0004    .WORD 4
             303E 3138 18 .WORD IDGOTO  ; GOTO 1
             3040 0001    .WORD 1
             3042 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 H
             3044 0001    .WORD 1
             3046 3142 1B .WORD IDIF                 ; IF 4
             3048 0004    .WORD 4
             304A 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 J
             304C 0001    .WORD 1
             304E 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 K
             3050 0001    .WORD 1
             3052 316A 1j .WORD IDLABEL ; LABEL 1
             3054 0001    .WORD 1
             3056 3174 1t .WORD IDMOD   ; MOD 1
             3058 0001    .WORD 1
             305A 317E 1~ .WORD IDNOT   ; NOT 1
             305C 0001    .WORD 1
             305E 3188 1  .WORD IDOF                 ; OF 2
             3060 0002    .WORD 2
             3062 319C 1  .WORD IDPACK  ; PACKED 4
             3064 0004    .WORD 4
             3066 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 Q
             3068 0001    .WORD 1
             306A 31C4 1  .WORD IDREC   ; RECORD 2
             306C 0002    .WORD 2
             306E 31D8 1  .WORD IDSEPRT ; SEPARATE 3
             3070 0003    .WORD 3
             3072 31F6 1  .WORD IDTHEN  ; THEN 3
             3074 0003    .WORD 3
             3076 3214 2  .WORD IDUNIT  ; UNIT 3
             3078 0003    .WORD 3
             307A 3232 22 .WORD IDVAR   ; VAR 1
             307C 0001    .WORD 1
             307E 323C 2< .WORD IDWHILE ; WHILE 2
             3080 0002    .WORD 2
             3082 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 X
             3084 0001    .WORD 1
             3086 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 Y
             3088 0001    .WORD 1
             308A 308E 0  .WORD ID@@@@  ; @@@@@@@@ 1 Z
             308C 0001    .WORD 1

; Reserved .WORDs, data for IDSEARCH
ID@@@@       308E 4040 @@ ; @@@@@@@@ 000F
             3090 4040 @@
             3092 4040 @@
             3094 4040 @@
             3096 000F
IDAND        3098 414E AN ; AND 2702
             309A 4420 D
```

```
            309C 2020
            309E 2020
            30A0 2702 '
            30A2 4152 AR ; ARRAY 2C0F
            30A4 5241 RA
            30A6 5920 Y  4)
            30A8 2020
            30AA 2C0F ,
IDBEGIN     30AC 4245 BE ; BEGIN 130F
            30AE 4749 GI
            30B0 4E20 N
            30B2 2020
            30B4 130F
IDCASE      30B6 4341 CA ; CASE 150F
            30B8 5345 SE
            30BA 2020
            30BC 2020
            30BE 150F
            30C0 434F CO ; CONST 1C0F
            30C2 4E53 NS
            30C4 5420 T
            30C6 2020
            30C8 1C0F
IDDEFIN     30CA 4445 DE ; DEFINITI 000F
            30CC 4649 FI
            30CE 4E49 NI
            30D0 5449 TI
            30D2 000F
            30D4 4449 DI ; DIV 2703
            30D6 5620 V
            30D8 2020
            30DA 2020
            30DC 2703 '
            30DE 444F DO ; DO 060F
            30E0 2020
            30E2 2020
            30E4 2020
            30E6 060F
            30E8 444F DO ; DOWNTO 080F
            30EA 574E WN
            30EC 544F TO
            30EE 2020
            30F0 080F
IDELSE      30F2 454C EL ; ELSE 0D0F
            30F4 5345 SE
            30F6 2020
            30F8 2020
            30FA 0D0F
            30FC 454E EN ; END 090F
            30FE 4420 D
            3100 2020
            3102 2020
            3104 090F
            3106 4558 EX ; EXTERNAL 350F
            3108 5445 TE
            310A 524E RN
            310C 414C AL
            310E 350F 5
IDFOR       3110 464F FO ; FOR 180F
            3112 5220 R
            3114 2020
            3116 2020
            3118 180F
            311A 4649 FI ; FILE 2E0F
            311C 4C45 LE
            311E 2020
            3120 2020
            3122 2E0F .
            3124 464F FO ; FORWARD 220F
            3126 5257 RW
            3128 4152 AR
            312A 4420 D
            312C 220F "
            312E 4655 FU ; FUNCTION 200F
            3130 4E43 NC
            3132 5449 TI
            3134 4F4E ON
            3136 200F
IDGOTO      3138 474F GO ; GOTO 1A0F
            313A 544F TO
            313C 2020
            313E 2020
            3140 1A0F
IDIF        3142 4946 IF ; IF 140F
```

```
            3144 2020
            3146 2020
            3148 2020
            314A 140F
            314C 494D IM ; IMPLEMEN 340F
            314E 504C PL
            3150 454D EM
            3152 454E EN
            3154 340F 4
            3156 494E IN ; IN 290E
            3158 2020
            315A 2020
            315C 2020
            315E 290E )
            3160 494E IN ; INTERFAC 330F
            3162 5445 TE
            3164 5246 RF
            3166 4143 AC
            3168 330F 3
IDLABEL     316A 4C41 LA ; LABEL 1B0F
            316C 4245 BE
            316E 4C20 L
            3170 2020
            3172 1B0F
IDMOD       3174 4D4F MO ; MOD 2704
            3176 4420 D
            3178 2020
            317A 2020
            317C 2704 '
IDNOT       317E 4E4F NO ; NOT 260F
            3180 5420 T
            3182 2020
            3184 2020
            3186 260F &
IDOF        3188 4F46 OF ; OF 0B0F
            318A 2020
            318C 2020
            318E 2020
            3190 0B0F
            3192 4F52 OR ; OR 2807
            3194 2020
            3196 2020
            3198 2020
            319A 2807 (
IDPACK      319C 5041 PA ; PACKED 2B0F
            319E 434B CK
            31A0 4544 ED
            31A2 2020
            31A4 2B0F +
            31A6 5052 PR ; PROCEDUR 1F0F
            31A8 4F43 OC
            31AA 4544 ED
            31AC 5552 UR
            31AE 1F0F
            31B0 5052 PR ; PROCESS 380F
            31B2 4F43 OC
            31B4 4553 ES
            31B6 5320 S
            31B8 380F 8
            31BA 5052 PR ; PROGRAM 210F
            31BC 4F47 OG
            31BE 5241 RA
            31C0 4D20 M
            31C2 210F !
IDREC       31C4 5245 RE ; RECORD 2D0F
            31C6 434F CO
            31C8 5244 RD
            31CA 2020
            31CC 2D0F -
            31CE 5245 RE ; REPEAT 160F
            31D0 5045 PE
            31D2 4154 AT
            31D4 2020
            31D6 160F
IDSEPRT     31D8 5345 SE ; SEPARATE 360F
            31DA 5041 PA
            31DC 5241 RA
            31DE 5445 TE
            31E0 360F 6
            31E2 5345 SE ; SET 2A0F
            31E4 5420 T
            31E6 2020
            31E8 2020
            31EA 2A0F *
```

```
              31EC 5345 SE ; SEGMENT 210F
              31EE 474D GM
              31F0 454E EN
              31F2 5420 T
              31F4 210F !
IDTHEN        31F6 5448 TH ; THEN 0C0F
              31F8 454E EN
              31FA 2020
              31FC 2020
              31FE 0C0F
              3200 544F TO ; TO 070F
              3202 2020
              3204 2020
              3206 2020
              3208 070F
              320A 5459 TY ; TYPE 1D0F
              320C 5045 PE
              320E 2020
              3210 2020
              3212 1D0F
IDUNIT        3214 554E UN ; UNIT 320F
              3216 4954 IT
              3218 2020
              321A 2020
              321C 320F 2
              321E 554E UN ; UNTIL 0A0F
              3220 5449 TI
              3222 4C20 L
              3224 2020
              3226 0A0F
              3228 5553 US ; USES 310F
              322A 4553 ES
              322C 2020
              322E 2020
              3230 310F 1
IDVAR         3232 5641 VA ; VAR 1E0F
              3234 5220 R
              3236 2020
              3238 2020
              323A 1E0F
IDWHILE       323C 5748 WH ; WHILE 170F
              323E 494C IL
              3240 4520 E
              3242 2020
              3244 170F
              3246 5749 WI ; WITH 190F
              3248 5448 TH
              324A 2020
              324C 2020
              324E 190F

; These two procedures are transferred to RAM PAD at 8300H when used, for speed reasons.
; Code for PME instruction decode. Used when p-code is in mapped memory, i.e. GROM or VDP
PMEMAP        3250 0588    INC  R8
              3252 D05D  ] MOVB *R13,R1  ; Fetch op-code
              3254 0971  q SRL  R1,7
              3256 C0A1    MOV  @OPCODE(R1),R2         ; Op-code table
              3258 2DB2 -
              325A C032  2 MOV  *R2+,R0  ; Branch address
              325C 0450  P B    *R0      ; Branch to instruction, or parameter fetch
              325E 0588    INC  R8
              3260 04C4    CLR  R4
              3262 D81D    MOVB *R13,@PASCALWS+9       ; Byte parameter to R4
              3264 8389
              3266 0588    INC  R8
              3268 04C3    CLR  R3
              326A D81D    MOVB *R13,@PASCALWS+7       ; Byte parameter to R3
              326C 8387
              326E 0452  R B    *R2      ; Interpret instruction
              3270 0588    INC  R8
              3272 04C5    CLR  R5
              3274 D81D    MOVB *R13,@PASCALWS+11      ; Byte parameter to R5
              3276 838B
              3278 0588    INC  R8
              327A 04C4    CLR  R4
              327C D81D    MOVB *R13,@PASCALWS+9       ; Byte parameter to R4
              327E 8389
              3280 04C3    CLR  R3
              3282 D0DD    MOVB *R13,R3  ; Byte or word parameter to R3
              3284 1103    JLT  328C
              3286 06C3    SWPB R3
              3288 0588    INC  R8
              328A 0452  R B    *R2      ; Interpret instruction if in 0..127 range
              328C 0243  C ANDI R3,7F00
```

```
             328E 7F00 
             3290 D81D    MOVB *R13,@PASCALWS+7       ; Expand to 0..32767 range
             3292 8387
             3294 05C8    INCT R8
             3296 0452  R B    *R2      ; Interpret instruction

; Code for PME instruction decode. Used when p-code is in CPU RAM
PMERAM       3298 D078  x MOVB *R8+,R1  ; Fetch op-code
             329A 0971  q SRL  R1,7
             329C C0A1    MOV  @OPCODE(R1),R2         ; Op-code table
             329E 2DB2 -
             32A0 C032  2 MOV  *R2+,R0  ; Branch address
             32A2 0450  P B    *R0      ; Branch to instruction, or parameter fetch
             32A4 1000    NOP
             32A6 04C4    CLR  R4
             32A8 D838  8 MOVB *R8+,@PASCALWS+9       ; Byte parameter to R4
             32AA 8389
             32AC 1000    NOP
             32AE 04C3    CLR  R3
             32B0 D838  8 MOVB *R8+,@PASCALWS+7       ; Byte parameter to R3
             32B2 8387
             32B4 0452  R B    *R2      ; Interpret instruction
             32B6 1000    NOP
             32B8 04C5    CLR  R5
             32BA D838  8 MOVB *R8+,@PASCALWS+11      ; Byte parameter to R5
             32BC 838B
             32BE 1000    NOP
             32C0 04C4    CLR  R4
             32C2 D838  8 MOVB *R8+,@PASCALWS+9       ; Byte parameter to R4
             32C4 8389
             32C6 1000    NOP
             32C8 04C3    CLR  R3
             32CA D0F8    MOVB *R8+,R3  ; Byte or word parameter to R3
             32CC 1102    JLT  32D2
             32CE 06C3    SWPB R3
             32D0 0452  R B    *R2      ; Interpret instruction if in 0..127 range
             32D2 0243  C ANDI R3,7F00
             32D4 7F00 
             32D6 D838  8 MOVB *R8+,@PASCALWS+7       ; Expand to 0..32767 range
             32D8 8387
             32DA 0452  R B    *R2      ; Interpret instruction

; Code for simulated interrupts
; The p-system always runs with interrupts disabled. Interrupt service is provided by reading the
; interrupt request as an ordinary IO-bit. If it's active, the code is executed.
; The IO-bit is tested after every taken (p-code) JUMP instruction and also during various output tasks.
; Both vertical blanking and external interrupts are serviced.
SIMINT       32DC 1D02    SBO  2
             32DE D820    MOVB @VSTATUS,@VDPST
             32E0 8802
             32E2 281E (
             32E4 05A0    INC  @TIMELOW ; Increment TIME values
             32E6 2BA4 +
             32E8 1702    JNC  NOROLL2  ; Count carry?
             32EA 05A0    INC  @TIMEHI
             32EC 2BA6 +
NOROLL2      32EE 05E0    INCT @SCBLANK
             32F0 288E (
             32F2 1609    JNE  NOBLANK
             32F4 4820 H  SZC  @SBITMSK+12,@VDPCOPY   ; Contains 40H
             32F6 2D9E -
             32F8 2810 (
             32FA C260  ` MOV  @VDPCOPY,R9
             32FC 2810 (
             32FE C2A0    MOV  @SUBWIN+20,R10         ; Get VDPWA from SUBWIN R10
             3300 83B4
             3302 06A0    BL   @VDPWAR  ; Screen blanking timeout
             3304 47C8 G
NOBLANK      3306 04CC    CLR  R12
             3308 1F01    TB   1                      ; External interrupt
             330A 131C    JEQ  3344
             330C 1D01    SBO  EXTMASK  ; External interrupt service (DSR cards)
             330E 020C    LI   R12,PCODE
             3310 1F00
             3312 1E00    SBZ  0
             3314 04E0    CLR  @INTIX   ; Interrupt table index
             3316 2984 )
INTSRCH      3318 0202    LI   R2,INTLIST             ; List of cards with interrupt service
             331A 2986 )
             331C A0A0    A    @INTIX,R2
             331E 2984 )
             3320 C312    MOV  *R2,R12  ; Get CRU address of card
             3322 130D    JEQ  INTEND   ; End of list?
             3324 05E0    INCT @INTIX
```

```
             3326 2984 )
             3328 1D00    SBO  0                     ; Enable card
             332A C0A0    MOV  @INTLINK,R2           ; Interrupt link
             332C 400C @
INTRPT       332E C002    MOV  R2,R0
             3330 C0A2    MOV  @2(R2),R2; Interrupt entry point
             3332 0002
             3334 0692    BL   *R2                   ; Call interrupt service
             3336 C090    MOV  *R0,R2
             3338 16FA    JNE  INTRPT   ; More interrupt services on that card?
             333A 1E00    SBZ  0                     ; Disable card
             333C 10ED    JMP  INTSRCH
INTEND       333E 020C    LI   R12,PCODE
             3340 1F00
             3342 1D00    SBO  0                     ; Turn on the p-code card again
             3344 06A0    BL   @SPRMOVE ; Sprite movement
             3346 43FA C
             3348 04C1    CLR  R1                    ; Sound generator number
LOOP38       334A 06A0    BL   @SNDPLAY ; Sound list processing, 0..3
             334C 4556 EV
             334E 0581    INC  R1
             3350 0281    CI   R1,4                  ; Four sound generators
             3352 0004
             3354 16FA    JNE  LOOP38
             3356 0620    DEC  @KEYCNT  ; Keyboard scan counter
             3358 2CD4 ,                             ; Keyboard is not scanned at every interrupt
             335A 1301    JEQ  KEYBRD
             335C 1005    JMP  NOKEYSC
KEYBRD       335E C820    MOV  @KEYLIM,@KEYCNT
             3360 2CD2 ,
             3362 2CD4 ,
             3364 0420    BLWP @KEYSCAN ; Scan the keyboard for the keypress buffer
             3366 2AAE *
NOKEYSC      3368 C2E0    MOV  @PASCALWS+22,R11      ; Save the return link here
             336A 8396
             336C 02E0    LWPI PASCALWS
             336E 8380
             3370 06A0    BL   @LOADAD1 ; Set p-code address pointer
             3372 4144 AD
             3374 02E0    LWPI SIMIWS
             3376 28B6 (
             3378 C80B    MOV  R11,@PASCALWS+22      ; Restore return link
             337A 8396
             337C 0380    RTWP

; Code used to find CRU base and entry point of a device. The p-system finds the DSR for each device
; in advance and stores information about where to find it in the Peripheral Control Block.
DSRSRCH      337E 06CA    SWPB R10                   ; Same as ANDI R10,FFH
             3380 098A    SRL  R10,8
             3382 020C    LI   R12,1000H; Start CRU address
             3384 1000
LOOP39       3386 1D00    SBO  0
             3388 0203    LI   R3,4000H ; DSR address
             338A 4000 @
             338C 9813    CB   *R3,@VALDSR           ; Any DSR?
             338E 2CAC ,
             3390 1309    JEQ  FOUND
AGAIN        3392 1E00    SBZ  0
             3394 022C  , AI   R12,0100H; DSR card increment
             3396 0100
             3398 028C    CI   R12,2000H; End of card CRU space?
             339A 2000
             339C 16F4    JNE  LOOP39
             339E 04C7    CLR  R7
             33A0 04C9    CLR  R9
             33A2 101F    JMP  DSRRET
FOUND        33A4 C10A    MOV  R10,R4   ; Found a valid DSR
             33A6 06C4    SWPB R4                    ; Same as ANDI R10,FFH
             33A8 0984    SRL  R4,8
             33AA A0C4    A    R4,R3
NEXT         33AC C0D3    MOV  *R3,R3   ; Fetch pointer to DSR subprogram link
             33AE 13F1    JEQ  AGAIN                 ; Nothing here?
             33B0 C103    MOV  R3,R4
             33B2 05C4    INCT R4                    ; Point to address
             33B4 C1F4    MOV  *R4+,R7  ; Address to routine
             33B6 9D08    CB   R8,*R4+  ; Length byte
             33B8 16F9    JNE  NEXT                  ; Try the next one
             33BA C146  F MOV  R6,R5                 ; R6 is the PAB pointer
             33BC 0225  % AI   R5,10D   ; Device name address pointer
             33BE 000A
             33C0 0209    LI   R9,VDPWA
             33C2 8C02
             33C4 06C5    SWPB R5                    ; R5 to VDPWA
             33C6 D645  E MOVB R5,*R9
```

```
             33C8 06C5    SWPB R5
             33CA D645  E MOVB R5,*R9
             33CC C148  H MOV  R8,R5
             33CE 0985    SRL  R5,8                  ; Right justify name length counter
LOOP40       33D0 9D29  ) CB   @-WAOFFS(R9),*R4+     ; VDPRD
             33D2 FBFE
             33D4 16EB    JNE  NEXT
             33D6 0605    DEC  R5                    ; Name length counter
             33D8 16FB    JNE  LOOP40
             33DA 1E00    SBZ  0
             33DC C24C  L MOV  R12,R9
             33DE 022A  * AI   R10,0100
             33E0 0100
DSRRET       33E2 020C    LI   R12,PCODE
             33E4 1F00
             33E6 1D00    SBO  0                     ; Turn on p-code
             33E8 0420    BLWP @VMBW
             33EA 478E G
             33EC 2972 )r .WORD IOWS+14 ; Source pointer
             33EE 2960 )` .WORD PPOINT  ; Destination pointer
             33F0 0008    .WORD 8                    ; Byte count
             33F2 1E00    SBZ  0                     ; Turn off card again
             33F4 045B  [ B    *R11

; Peripheral device call. Goes directly to the correct CRU base and entry address, if known.
; Otherwise the code above is called first, to search for the device.
PERCALL      33F6 2964 )d .WORD IOWS
             33F8 33FA 3  .WORD PERCALX

PERCALX      33FA C1A0    MOV  @PPOINT,R6            ; PCB pointer
             33FC 2960 )`
             33FE 0226  & AI   R6,8                  ; PAB pointer
             3400 0008
             3402 0420    BLWP @VMBR                 ; Fetch PAB
             3404 4774 Gt
             3406 2960 )` .WORD PPOINT  ; PCB pointer
             3408 2972 )r .WORD IOWS+14 ; R7
             340A 000A    .WORD 10D
             340C 024B  K ANDI R11,001F ; Clear opcode and error code in loaded PAB
             340E 001F
             3410 A2E0    A    @IOOPCOD,R11          ; Insert opcode
             3412 2962 )b
             3414 0420    BLWP @VMBW
             3416 478E G
             3418 297A )z .WORD IOWS+22 ; R11
             341A 2970 )p .WORD IOWS+12 ; R6
             341C 0002    .WORD 2
             341E 020C    LI   R12,PCODE; Turn off p-code card
             3420 1F00
             3422 1E00    SBZ  0
             3424 C1C7    MOV  R7,R7                 ; Entry address known?
             3426 1602    JNE  HASENT
             3428 06A0    BL   @DSRSRCH ; Figure out entry address
             342A 337E 3~
HASENT       342C C249  I MOV  R9,R9                 ; CRU base known?
             342E 1602    JNE  HASCRU
             3430 0209    LI   R9,PCODE
             3432 1F00
HASCRU       3434 0205    LI   R5,INTWS+26           ; R13
             3436 83DA
             3438 C035  5 MOV  *R5+,R0  ; Save R13-R15 of INTWS
             343A C075  u MOV  *R5+,R1
             343C C0B5    MOV  *R5+,R2
LOOP54       343E C148  H MOV  R8,R5                 ; Name length device base name
             3440 0985    SRL  R5,8                  ; Make word
             3442 C805    MOV  R5,@NAMLEN
             3444 8354  T
             3446 A146  F A    R6,R5                 ; Add to PAB pointer
             3448 0225  % AI   R5,10                 ; Pass the PAB
             344A 000A
             344C C805    MOV  R5,@PERIOD
             344E 8356  V
             3450 C2CA    MOV  R10,R11
             3452 098B    SRL  R11,8
             3454 C80B    MOV  R11,@GPLWS+2
             3456 83E2
             3458 C807    MOV  R7,@GPLWS+4           ; Entry address
             345A 83E4
             345C 131E    JEQ  UNKVOL   ; Volume unknown?
             345E C809    MOV  R9,@GPLWS+24          ; CRU base address to GPLWS R12
             3460 83F8
             3462 02E0    LWPI GPLWS
             3464 83E0
             3466 1D00    SBO  0                     ; Enable card
```

```
             3468 0692    BL   *R2                    ; Execute DSR
             346A 101A    JMP  NEWSRC
             346C 1E00    SBZ  0                      ; Disable card
             346E 02E0    LWPI IOWS
             3470 2964 )d
             3472 0205    LI   R5,83DA
             3474 83DA
             3476 CD40  @ MOV  R0,*R5+  ; Restore registers
             3478 CD41  A MOV  R1,*R5+
             347A CD42  B MOV  R2,*R5+
             347C 04C0    CLR  R0                    ; Error indicator
PCALOUT      347E 020C    LI   R12,PCODE
             3480 1F00
             3482 1D00    SBO  0                      ; Enable p-code card
             3484 0420    BLWP @VMBR                 ; Fetch first word of PAB to R11
             3486 4774 Gt
             3488 2970 )p .WORD IOWS+12
             348A 297A )z .WORD IOWS+22
             348C 0002    .WORD 2
             348E 024B  K ANDI R11,00E0 ; Mask out error code
             3490 00E0
             3492 1302    JEQ  RETPCAL
             3494 C020    MOV  @DSRERR,R0
             3496 2CD8 ,
RETPCAL      3498 0380    RTWP                       ; Return from peripheral call

UNKVOL       349A C020    MOV  @NOVOL,R0; Code for "no such volume"
             349C 2CDA ,
             349E 10EF    JMP  PCALOUT  ; Leave with an error

NEWSRC       34A0 02E0    LWPI IOWS
             34A2 2964 )d
             34A4 C309    MOV  R9,R12   ; Restore
             34A6 06A0    BL   @AGAIN
             34A8 3392 3
             34AA 10C9    JMP  LOOP54

; The Kernel UNIT-procedures have different entry points.
; One is used when called from assembly, the other when called from p-code.
; Kernel procedure UNITREAD
UNITRD       34AC C820    MOV  @CONTRUE,@IOASM       ; Assembly entry
             34AE 40B6 @
             34B0 29A6 )
             34B2 C80B    MOV  R11,@RETSAV1
             34B4 2ADA *
UNITRDN      34B6 C820    MOV  @OPREAD,@IOCTRL       ; Normal p-code entry
             34B8 2BEC +
             34BA 29A8 )
             34BC 06A0    BL   @PAGE2
             34BE 352C 5,
             34C0 0460  ` B    @UNITIO
             34C2 509C P

; Kernel procedure UNITWRITE
UNITWR       34C4 C820    MOV  @CONTRUE,@IOASM       ; Assembly entry
             34C6 40B6 @
             34C8 29A6 )
             34CA C80B    MOV  R11,@RETSAV1
             34CC 2ADA *
UNITWRN      34CE C820    MOV  @OPWRIT,@IOCTRL       ; Normal p-code entry
             34D0 2BEE +
             34D2 29A8 )
             34D4 06A0    BL   @PAGE2
             34D6 352C 5,
             34D8 0460  ` B    @UNITIO
             34DA 509C P

; Kernel procedure UNITCLEAR
UNITCL       34DC C820    MOV  @CONTRUE,@IOASM       ; Assembly entry
             34DE 40B6 @
             34E0 29A6 )
             34E2 C80B    MOV  R11,@RETSAV1
             34E4 2ADA *
UNITCLN      34E6 06A0    BL   @PAGE2   ; Normal p-code entry
             34E8 352C 5,
             34EA 0460  ` B    @UNCLR
             34EC 5196 Q

; Kernel procedure UNITSTATUS
UNITST       34EE C820    MOV  @CONTRUE,@IOASM       ; Assembly entry
             34F0 40B6 @
             34F2 29A6 )
             34F4 C80B    MOV  R11,@RETSAV1
             34F6 2ADA *
```

```
UNITSTN       34F8 06A0    BL   @PAGE2    ; Normal p-code entry
              34FA 352C 5,
              34FC 0460   ` B    @UNSTAT
              34FE 51BC Q

; Kernel procedure UNITBUSY. Always return false.
UNITBU        3500 05CA    INCT SP
              3502 C6A0    MOV  @CONFALS,*SP
              3504 40A8 @
              3506 0460   ` B    @LOADADDR; Load address to p-code and execute
              3508 4142 AB

; Kernel procedure UNITWAIT. Always returns immediately.
UNITWA        350A 05CA    INCT SP
              350C 0460   ` B    @LOADADDR; Load address to p-code and execute
              350E 4142 AB

; Code to return from a UNIT-procedure call.
; Turns on normal page and finds out how to return.
UNITRET       3510 06A0    BL   @PAGE1
              3512 353C 5<
              3514 06A0    BL   @LOADAD1
              3516 4144 AD
              3518 8820    C    @IOASM,@CONTRUE        ; Called from assembly level?
              351A 29A6 )
              351C 40B6 @
              351E 1301    JEQ  ISASM                  ; Yes, return to assembly
              3520 045C   \ B    *R12      ; PME fetch   ; No, return to p-code
ISASM         3522 04E0    CLR  @IOASM
              3524 29A6 )
              3526 C2E0    MOV  @RETSAV1,R11
              3528 2ADA *
              352A 045B   [ B    *R11

; Code which switches in alternate p-code ROM page.
PAGE2         352C C80C    MOV  R12,@CRUSAVE
              352E 29A4 )
              3530 020C    LI   R12,PCODE2
              3532 1F80
              3534 1D00    SBO  0
              3536 C320    MOV  @CRUSAVE,R12
              3538 29A4 )
              353A 045B   [ B    *R11

; Code which turns on normal p-code ROM page. Sometimes entered at PAGE1B.
PAGE1         353C C80C    MOV  R12,@CRUSAVE
              353E 29A4 )
PAGE1B        3540 020C    LI   R12,PCODE2
              3542 1F80
              3544 1E00    SBZ  0
              3546 C320    MOV  @CRUSAVE,R12
              3548 29A4 )
              354A 045B   [ B    *R11

; Kernel procedure RELOCSEG
RLOCSEG       354C 06A0    BL   @PAGE2
              354E 352C 5,
              3550 020C    LI   R12,RLOCRT
              3552 3558 5X
              3554 0460   ` B    @RLOCEXE
              3556 5F5E _^
RLOCRT        3558 06A0    BL   @PAGE1B
              355A 3540 5@
              355C 045C   \ B    *R12      ; PME fetch

; Code enabling normal p-code ROM page and then issues the run-time error "unknown instruction".
UNKINS        355E 06A0    BL   @PAGE1B
              3560 3540 5@
              3562 06A0    BL   @FAULGEN
              3564 484A HJ
              3566 000B    .WORD 11       ; Unimplemented instruction

; Code erasing the 80 column screen memory. Called after startup
CLRSCR        3568 C2E0    MOV  @PCONT,R11             ; Holds address to PME. Loaded at startup
              356A 23F8 #
              356C C060   ` MOV  @SCREN80,R1
              356E 2CBE ,
              3570 0202    LI   R2,24*80
              3572 0780
LOOP55        3574 CC60   ` MOV @SPACES,*R1+            ; Fill screen with spaces
              3576 40A2 @
              3578 0642    B DECT R2
              357A 16FC    JNE  LOOP55
              357C 045B   [ B    *R11
```

```
             357E 045B  [ B    *R11
; End of what's loaded on the second GROM -> ROM transfer at startup.

; Kernel global data area.
KRNGLOB      3580 3580 5  .WORD KRNGLOB
             3582 0000    .WORD
             3584 0000    .WORD
             3586 0000    .WORD
             3588 0000    .WORD
             358A 2B6E +n .WORD SYSCOM  ; syscom: ^integer;   NOTE: From interface text in dummy KERNEL
module included in SYSTEM.LINKER
             358C A7B8    .WORD                      ; gfiles: array[0..5] of integer; Global files,
0=input, 1=output
             358E A7FA    .WORD
             3590 A83C  < .WORD
             3592 0000    .WORD
             3594 0000    .WORD
             3596 0000    .WORD
             3598 A8BE    .WORD         ; Not referred to by Linker (filler: array[0..4] of integer;)
             359A A87E  ~ .WORD
             359C 0000    .WORD
             359E 0000    .WORD
             35A0 0000    .WORD
             35A2 0000    .WORD         ; stupid: boolean;
             35A4 0000    .WORD         ; slowterm: boolean;
             35A6 001B    .WORD         ; altmode: char; (chr(0), chr(27))
             35A8 0000    .WORD         ; gotcode: boolean;
             35AA 0000    .WORD         ; gotsym: boolean;
             35AC 0650  P               ; Code file volume ID codevid: string[7];
             35AE 4153 AS
             35B0 5359 SY
             35B2 535B S[
             35B4 0650  P               ; Symbol file volume ID symvid: string[7];
             35B6 4153 AS
             35B8 5359 SY
             35BA 535B S[
             35BC 0650  P               ; Work file volume ID workvid: string[7];
             35BE 4153 AS
             35C0 5359 SY
             35C2 535B S[

             35C4 0000    .WORD 0       ; Code file name codetid: string[15];
             35C6 FFFF    .WORD -1
             35C8 FFFF    .WORD -1
             35CA FFFF    .WORD -1
             35CC FFFF    .WORD -1
             35CE FFFF    .WORD -1
             35D0 FFFF    .WORD -1
             35D2 FFFF    .WORD -1
             35D4 0000    .WORD 0       ; Symbol file name symtid: string[15];
             35D6 FFFF    .WORD -1
             35D8 FFFF    .WORD -1
             35DA FFFF    .WORD -1
             35DC FFFF    .WORD -1
             35DE FFFF    .WORD -1
             35E0 FFFF    .WORD -1
             35E2 FFFF    .WORD -1
             35E4 0000    .WORD 0       ; Work file name worktid: string[15];
             35E6 FFFF    .WORD -1
             35E8 FFFF    .WORD -1
             35EA FFFF    .WORD -1
             35EC FFFF    .WORD -1
             35EE FFFF    .WORD -1
             35F0 FFFF    .WORD -1
             35F2 FFFF    .WORD -1

             35F4 BDE6    .WORD                      ; Not referred to by Linker (filler: array[0..5] of
integer;)
             35F6 2BD0 +  .WORD MAINTIB
             35F8 A778  x .WORD
             35FA A83C  < .WORD
             35FC A7FA    .WORD
             35FE A7B8  A .WORD
             3600 0223  #  ; Prefix volume name (dkvid: string[7])
             3602 355C 5\
             3604 5359 SY
             3606 535B S[
             3608 0650  P               ; Root volume name (syvid: string[7])
             360A 4153 AS
             360C 5359 SY
             360E 535B S[
THEDATE      3610 33E3 3   ; System date.
                           ; packed record
                           ;   month: 1..12; 4 bits
```

```
                                        ;   date:  1..31; 5 bits
                                        ;   year:  0..100; 7 bits
                                        ; end;
             3612 0005    .WORD         ; cmdstate: integer; (for
getcommand)(HALTINIT,DEBUGCALL,UPROGNOU,UPROGUOK,SYSPROG,COMPONLY,COMPANDGO,COMPDEBUG,LINKANDGO,LINKDEBUG);
             3614 0001    .WORD         ; Heap_Info.lock    :semaphore;
             3616 0000    .WORD
             3618 BDF2    .WORD         ; Heap_Info.HeapTop :MemPtr;
             361A BDE6    .WORD         ; Heap_Info.TopMark :MemPtr;
             361C 0000    .WORD         ; Task_Info.TaskDone:semaphore;
             361E 0000    .WORD
             3620 0001    .WORD         ; Task_Info.Lock    :semaphore;
             3622 0000    .WORD
             3624 0000    .WORD         ; Task_Info.N_Tasks :integer;
; Decimal constants.
IPOT         3626 0001    .WORD 1       ; Integer powers of ten
             3628 000A    .WORD 10
             362A 0064  d .WORD 100
             362C 03E8    .WORD 1000
             362E 2710 '  .WORD 10000

; This area (102 bytes) is not referred to from assembly. But it's used as an environment record when running
the Filer.
             3630 0000    .WORD
             3632 0000    .WORD
             3634 0001    .WORD 1
             3636 0020    .WORD 20H
             3638 0000    .WORD
             363A 0000    .WORD
             363C 0000    .WORD
             363E 0000    .WORD
             3640 0000    .WORD
             3642 03FF    .WORD 3FFH (1023D)
             3644 FFFF    .WORD -1
             3646 FFFF    .WORD -1
             3648 FFFF    .WORD -1
             364A FFFF    .WORD -1
             364C FFFF    .WORD -1
             364E FFFF    .WORD -1
             3650 FFFF    .WORD -1
             3652 FFFF    .WORD -1
             3654 FFFF    .WORD -1
             3656 FFFF    .WORD -1
             3658 FFFF    .WORD -1
             365A FFFF    .WORD -1
             365C FFFF    .WORD -1
             365E FFFF    .WORD -1
             3660 FFFF    .WORD -1
             3662 FFFF    .WORD -1
             3664 FFFF    .WORD -1
             3666 FFFF    .WORD -1
             3668 FFFF    .WORD -1
             366A FFFF    .WORD -1
             366C FFFF    .WORD -1
             366E FFFF    .WORD -1
             3670 FFFF    .WORD -1
             3672 FFFF    .WORD -1
             3674 FFFF    .WORD -1
             3676 FFFF    .WORD -1
             3678 FFFF    .WORD -1
             367A FFFF    .WORD -1
             367C FFFF    .WORD -1
             367E FFFF    .WORD -1
             3680 0000    .WORD 0
             3682 0000    .WORD 0
             3684 0000    .WORD 0
             3686 0000    .WORD 0
             3688 0000    .WORD 0
             368A 0080    .WORD 0
             368C 0000    .WORD 0
             368E 0000    .WORD 0
             3690 0000    .WORD 0
             3692 0000    .WORD 0
             3694 0000    .WORD 0

; Table with name and type of units.
; array [0..32] of record
;   volume_id   : string[7];
;   is_blocked  : boolean;
;   no_of_blocks: integer; (* maxint if not blocked *)
; end;

UNITABLE     3696 0000    ; #0
             3698 0008
```

```
369A 0000
369C 0000
369E 0000
36A0 7FFF
36A2 0743  C ; #1
36A4 4F4E ON
36A6 534F SO
36A8 4C45 LE
36AA 0000
36AC 7FFF
36AE 0753  S ; #2
36B0 5953 YS
36B2 5445 TE
36B4 524D RM
36B6 0000
36B8 7FFF
36BA 0000    ; #3
36BC 0014
36BE 0000
36C0 FFFF
36C2 0000
36C4 7FFF
36C6 0650  P ; #4
36C8 4153 AS
36CA 5359 SY
36CC 535B S[
36CE 0001
36D0 02D0
36D2 054D  M ; #5
36D4 554C UL
36D6 5449 TI
36D8 2D46 -F
36DA 0001
36DC 02D0
36DE 0750  P ; #6
36E0 5249 RI
36E2 4E54 NT
36E4 4552 ER
36E6 0000
36E8 7FFF
36EA 0552  R ; #7
36EC 454D EM
36EE 494E IN
36F0 FFFF
36F2 0000
36F4 7FFF
36F6 0652  R ; #8
36F8 454D EM
36FA 4F55 OU
36FC 5452 TR
36FE 0000
3700 7FFF
3702 0649  I ; #9
3704 4E56 NV
3706 454E EN
3708 5404 T
370A 0001
370C 02D0
370E 0750  P ; #10
3710 534F SO
3712 5552 UR
3714 4345 CE
3716 0001
3718 02D0
371A 0000    ; #11
371C 0000
371E 0000
3720 0000
3722 0001
3724 7FFF
3726 0000    ; #12
3728 0000
372A 0001
372C 0000
372E 0001
3730 7FFF
3732 0000    ; #13
3734 0000
3736 0000
3738 0000
373A 0001
373C 7FFF
373E 024F  O ; #14
3740 5300 S
```

```
3742 FFFF
3744 FFFF
3746 0001
3748 0068  h
374A 0000    ; #15
374C FFFF
374E FFFF
3750 FFFF
3752 0001
3754 7FFF
3756 0000    ; #16
3758 FFFF
375A FFFF
375C FFFF
375E 0001
3760 7FFF
3762 0000    ; #17
3764 FFFF
3766 FFFF
3768 FFFF
376A 0001
376C 7FFF
376E 0000    ; #18
3770 FFFF
3772 FFFF
3774 FFFF
3776 0001
3778 7FFF
377A 0000    ; #19
377C FFFF
377E FFFF
3780 0000
3782 0001
3784 7FFF
3786 0000    ; #20
3788 0000
378A 0000
378C 0000
378E 0001
3790 7FFF
3792 0000    ; #21
3794 0000
3796 0000
3798 0000
379A 0001
379C 7FFF
379E 0000    ; #22
37A0 0000
37A2 0000
37A4 0000
37A6 0001
37A8 7FFF
37AA 0000    ; #23
37AC 0000
37AE 0000
37B0 0000
37B2 0001
37B4 7FFF
37B6 0000    ; #24
37B8 0000
37BA 0000
37BC 0000
37BE 0001
37C0 7FFF
37C2 0000    ; #25
37C4 FFFF
37C6 FFFF
37C8 FFFF
37CA 0001
37CC 7FFF
37CE 0000    ; #26
37D0 FFFF
37D2 FFFF
37D4 FFFF
37D6 0001
37D8 7FFF
37DA 0000    ; #27
37DC FFFF
37DE FFFF
37E0 FFFF
37E2 0001
37E4 7FFF
37E6 0000    ; #28
37E8 FFFF
```

```
              37EA FFFF
              37EC FFFF
              37EE 0001
              37F0 7FFF
              37F2 0000    ; #29
              37F4 FFFF
              37F6 FFFF
              37F8 FFFF
              37FA 0001
              37FC 7FFF
              37FE 0000    ; #30
              3800 0000
              3802 0000
              3804 0000
              3806 0001
              3808 7FFF
              380A 0454  T ; #31
              380C 4150 AP
              380E 4555 EU
              3810 0000
              3812 0000
              3814 7FFF
              3816 0000    ; #32
              3818 0000
              381A 0000
              381C 0000
              381E 0000
              3820 7FFF

; These are the files the system will look for when you press A, C, E, F or L.
; The system will look for these files. If they are not found, it will look on all drives and
; then update the data here, if they are found elsewhere.

; File name for the assembler, string[23]
ASMNAME       3822 1650  P
              3824 4153 AS
              3826 5359 SY
              3828 533A S:
              382A 5359 SY
              382C 5354 ST
              382E 454D EM
              3830 2E41 .A
              3832 5353 SS
              3834 4D42 MB
              3836 4C45 LE
              3838 52BD R

; File name for the compiler, string[23]
CMPNAME       383A 1650  P
              383C 4153 AS
              383E 5359 SY
              3840 533A S:
              3842 5359 SY
              3844 5354 ST
              3846 454D EM
              3848 2E43 .C
              384A 4F4D OM
              384C 5049 PI
              384E 4C45 LE
              3850 52BD R

; File name for the editor, string[23]
EDTNAME       3852 1450  P
              3854 4153 AS
              3856 5359 SY
              3858 533A S:
              385A 5359 SY
              385C 5354 ST
              385E 454D EM
              3860 2E45 .E
              3862 4449 DI
              3864 544F TO
              3866 5245 RE
              3868 FFFF

; File name for the filer, string[23]
FILNAME       386A 1350  P
              386C 4153 AS
              386E 5359 SY
              3870 533A S:
              3872 5359 SY
              3874 5354 ST
              3876 454D EM
              3878 2E46 .F
```

```
             387A 494C IL
             387C 4552 ER
             387E FFFF
             3880 0000

; File name for the linker,string[23]
LNKNAME      3882 1450  P
             3884 4153 AS
             3886 5359 SY
             3888 533A S:
             388A 5359 SY
             388C 5354 ST
             388E 454D EM
             3890 2E4C .L
             3892 494E IN
             3894 4B45 KE
             3896 5245 RE

; Not used from assembly
             3898 0000    .WORD 0
             389A 0000    .WORD 0
             389C 0000    .WORD 0
             389E 0000    .WORD 0
             38A0 0000    .WORD 0
             38A2 0000    .WORD 0
             38A4 0020    .WORD
             38A6 0000    .WORD 0
             38A8 0000    .WORD 0
             38AA 0001    .WORD
             38AC A360  ` .WORD
             38AE AD6A  j .WORD
             38B0 2A46 *F .WORD EXECERR
             38B2 A8FE    .WORD
             38B4 0001    .WORD
             38B6 0000    .WORD
             38B8 0000    .WORD
             38BA A360  ` .WORD
             38BC 0000    .WORD
             38BE 0000    .WORD
             38C0 0000    .WORD
             38C2 0000    .WORD
             38C4 0000    .WORD
             38C6 0000    .WORD
             38C8 0000    .WORD

; File name for the library text file, usually '*USERLIB.TEXT',string[23]
             38CA 0D2A  *
             38CC 5553 US
             38CE 4552 ER
             38D0 4C49 LI
             38D2 422E B.
             38D4 5445 TE
             38D6 5854 XT
             38D8 FFFF
             38DA FFFF
             38DC FFFF
             38DE FFFF
             38E0 FFFF

; Not used from assembly
             38E2 A45E  ^ A    *R14,*R1
             38E4 C94C  L MOV  R12,@0(R5)
             38E6 0000
             38E8 0000    .WORD
             38EA 0000    .WORD
             38EC 0000    .WORD
             38EE 0000    .WORD
             38F0 0000    .WORD
             38F2 0000    .WORD
             38F4 000A    .WORD
             38F6 0000    .WORD
             38F8 0000    .WORD

OKGO         38FA 3939 99 .ASCII "99"   ; "99" when running
                                        ; "GO" when stopped because e.g. stack overflow
                                        ; "NO" when stopped by Halt command
             38FC 2F34 /4 .ASCII "/4"   ; "/4" when running
             38FE 0000    .WORD

; STRTBUF is cleared during startup
; The rest does not seem to have any meaningful use when in operation
STRTBUF      3900 0000
             3902 0000
             3904 2BD0 +  .WORD MAINTIB
```

```
3906 0000
3908 0000
390A 0012
390C 0000
390E 0000
3910 0000
3912 0000
3914 0000
3916 0000
3918 0000
391A 0000
391C 0000
391E 0000
3920 0000
3922 0800
3924 0000
3926 0030  0
3928 0000
392A 0000
392C 0000
392E 0000
3930 0000
3932 0000
3934 0001
3936 0000
3938 0000
393A 0000
393C 0000
393E 0000
3940 FFFF
3942 FFFF
3944 FFFF
3946 FFFF
3948 FFFF
394A FFFF
394C FFFF
394E FFFF
3950 FFFF
3952 FFFF
3954 FFFF
3956 FFFF
3958 FFFF
395A FFFF
395C FFFF
395E FFFF
3960 FFFF
3962 FFFF
3964 FFFF
3966 FFFF
3968 FFFF
396A FFFF
396C FFFF
396E FFFF
3970 FFFF
3972 FFFF
3974 FFFF
3976 FFFF
3978 FFFF
397A FFFF
397C FFEF
397E FFFF
3980 0000
3982 0000
3984 0000
3986 0000
3988 0000
398A 0000
398C 0000
398E 0000
3990 0000
3992 0000
3994 0000
3996 0000
3998 0000
399A 0000
399C 0080
399E 0040  @
39A0 0000
39A2 0000
39A4 0000
39A6 0000
39A8 0002
39AA 0040  @
39AC 0004
```

```
39AE 0040  @
39B0 0000
39B2 0000
39B4 0000
39B6 0000
39B8 0000
39BA 0000
39BC 0000
39BE 0000
39C0 FFFF
39C2 FFFF
39C4 FFFF
39C6 FFFF
39C8 FFFF
39CA FFFF
39CC FFFF
39CE FFFF
39D0 FFFF
39D2 FFFF
39D4 FFFF
39D6 FFFF
39D8 FFFF
39DA FFFF
39DC FFFF
39DE FFFF
39E0 FFFF
39E2 FFFF
39E4 FFFF
39E6 FFFF
39E8 FFFF
39EA FFFF
39EC FFFF
39EE FFFF
39F0 FFFF
39F2 FFFF
39F4 FFFF
39F6 FFFF
39F8 FFFF
39FA FFFF
39FC FFFF
39FE FFFF
3A00 0000
3A02 0000
3A04 0000
3A06 0000
3A08 0000
3A0A 0000
3A0C 0000
3A0E 0020
3A10 0000
3A12 0000
3A14 0000
3A16 0000
3A18 0000
3A1A 0080
3A1C 0081
3A1E 0004
3A20 0000
3A22 0000
3A24 0001
3A26 0020
3A28 0000
3A2A 0000
3A2C 0000
3A2E 0004
3A30 0000
3A32 0000
3A34 0040  @
3A36 0000
3A38 0000
3A3A 0000
3A3C 0000
3A3E 0000
3A40 FFFF
3A42 FFFF
3A44 FFFF
3446 FFFF
3A48 FFFF
3A4A FFFF
3A4C FFFF
3A4E FFFF
3A50 FFFF
3A52 FFFF
3A54 FFFF
```

3A56 FFFF
3A58 FFFF
3A5A FFFF
3A5C FFFF
3A5E FFFF
3A60 FFFF
3A62 FFFF
3A64 FFFE
3A66 FFFF
3A68 FFFF
3A6A FFFF
3A6C FFFF
3A6E FFFF
3A70 FFFF
3A72 FFFF
3A74 FFFF
3A76 FFFF
3A78 FFFF
3A7A FFFF
3A7C FFFF
3A7E FFFF
3A80 0000
3A82 0000
3A84 0000
3A86 0000
3A88 0000
3A8A 0000
3A8C 0000
3A8E 0000
3A90 0000
3A92 0000
3A94 0010
3A96 0000
3A98 0000
3A9A 0000
3A9C 0020
3A9E 0000
3AA0 0000
3AA2 0000
3AA4 0000
3AA6 0000
3AA8 0000
3AAA 0000
3AAC 0000
3AAE 0000
3AB0 0000
3AB2 0000
3AB4 0000
3AB6 0000
3AB8 0000
3ABA 0000
3ABC 0000
3ABE 0000
3AC0 FFFF
3AC2 FFFF
3AC4 FFFF
3AC6 FFFF
3AC8 FFFF
3ACA FFFF
3ACC FFFF
3ACE FFFF
3AD0 FFFF
3AD2 FFFF
3AD4 FFFF
3AD6 FFFF
3AD8 FFFF
3ADA FFFF
3ADC FFFF
3ADE FFFF
3AE0 FFFF
3AE2 FFFF
3AE4 FFFF
3AE6 FFFF
3AE8 FFFF
3AEA FFFF
3AEC FFFF
3AEE FFFF
3AF0 FFFF
3AF2 FFFF
3AF4 FFFF
3AF6 FFFF
3AF8 FFFF
3AFA FFFF
3AFC FFFF

3AFE FFFF
3B00 0000
3B02 0000
3B04 0000
3B06 0000
3B08 0008
3B0A 0010
3B0C 0000
3B0E 0000
3B10 0000
3B12 0000
3B14 0008
3B16 0000
3B18 0000
3B1A 0004
3B1C 0002
3B1E 0000
3B20 0000
3B22 0000
3B24 0000
3B26 0000
3B28 0000
3B2A 0000
3B2C 0004
3B2E 0000
3B30 0000
3B32 0000
3B34 0000
3B36 0000
3B38 0000
3B3A 0000
3B3C 0000
3B3E 0000
3B40 FFFF
3B42 FFFF
3B44 FFFF
3B46 FFFF
3B48 FFFF
3B4A FFFF
3B4C FFFF
3B4E FFFF
3B50 FFFF
3B52 FFFF
3B54 FFFF
3B56 FFFF
3B58 FFFF
3B5A FFFF
3B5C FFFF
3B5E FFFF
3B60 FFFF
3B62 FFFF
3B64 FFFF
3B66 FFFF
3B68 FFFF
3B6A FFFF
3B6C FFFF
3B6E FFFF
3B70 FFFF
3B72 FFFF
3B74 FFFF
3B76 FFFF
3B78 FFFF
3B7A FFFF
3B7C FFFF
3B7E FFFF
3B80 0000
3B82 0000
3B84 0000
3B86 0000
3B88 0000
3B8A 0000
3B8C 0000
3B8E 0000
3B90 0000
3B92 0000
3B94 0000
3B96 0080
3B98 0000
3B9A 0000
3B9C 0000
3B9E 0000
3BA0 0000
3BA2 0000
3BA4 0000

```
3BA6 0000
3BA8 0000
3BAA 0004
3BAC 0000
3BAE 0000
3BB0 0000
3BB2 0000
3BB4 0000
3BB6 0000
3BB8 0000
3BBA 0000
3BBC 0000
3BBE 0000
3BC0 FFFF
3BC2 FFFF
3BC4 FFFF
3BC6 FFFF
3BC8 FFFF
3BCA FFF7
3BCC FFFF
3BCE FFFF
3BD0 FFFF
3BD2 FFFF
3BD4 FFFF
3BD6 FFFF
3BD8 FFFF
3BDA FFFF
3BDC FFFF
3BDE FFFF
3BE0 FFFF
3BE2 FFFF
3BE4 FFFF
3BE6 FFFF
3BE8 FFFF
3BEA FFFF
3BEC FFFF
3BEE FFFF
3BF0 FFFF
3BF2 FFFF
3BF4 FFFF
3BF6 FFFF
3BF8 FFFF
3BFA FFFF
3BFC FFFF
3BFE FFFF
3C00 0000
3C02 0000
3C04 0000
3C06 0000
3C08 0001
3C0A 0000
3C0C 0000
3C0E 0000
3C10 0000
3C12 0000
3C14 0002
3C16 0000
3C18 0000
3C1A 0000
3C1C 0000
3C1E 0000
3C20 0000
3C22 0000
3C24 0000
3C26 0002
3C28 0000
3C2A 0000
3C2C 0000
3C2E 0000
3C30 0000
3C32 0000
3C34 0000
3C36 0040  @
3C38 0000
3C3A 0000
3C3C 0000
3C3E 0000
3C40 FFFF
3C42 FFFF
3C44 FFFF
3C46 FFFF
3C48 FFFF
3C4A FFFF
3C4C FFFF
```

3C4E FFFF
3C50 FFFF
3C52 FFFF
3C54 FFFF
3C56 FFFF
3C58 FFFF
3C5A FFFF
3C5C FFFF
3C5E FFFF
3C60 FFFF
3C62 FFFF
3C64 FFFF
3C66 FFFF
3C68 FFFF
3C6A FFFF
3C6C FFFF
3C6E FFFF
3C70 FFFF
3C72 FFFF
3C74 FFFF
3C76 FFFF
3C78 FFFF
3C7A FFFF
3C7C FFFF
3C7E FFFF
3C80 0000
3C82 0000
3C84 0000
3C86 0000
3C88 0000
3C8A 0000
3C8C 0000
3C8E 0000
3C90 0000
3C92 0000
3C94 0000
3C96 0000
3C98 0000
3C9A 0000
3C9C 0000
3C9E 0000
3CA0 0000
3CA2 0000
3CA4 0000
3CA6 0000
3CA8 0000
3CAA 0000
3CAC 0000
3CAE 0000
3CB0 0000
3CB2 0000
3CB4 0000
3CB6 0000
3CB8 0000
3CBA 0000
3CBC 0000
3CBE 0000
3CC0 FFFF
3CC2 FFFF
3CC4 FFFF
3CC6 FFFF
3CC8 FFFF
3CCA FFFF
3CCC FFFF
3CCE FFFF
3CD0 FFFF
3CD2 FFFF
3CD4 FFFF
3CD6 FFFF
3CD8 FFFF
3CDA FFFF
3CDC FFFF
3CDE FFFF
3CE0 FFFF
3CE2 FFFF
3CE4 FFFF
3CE6 FFFF
3CE8 FFFF
3CEA FFFF
3CEC FFFF
3CEE FFFF
3CF0 FFFF
3CF2 FFFF
3CF4 FFFF

```
3CF6 FFFF
3CF8 FFFF
3CFA FFFF
3CFC FFFF
3CFE FFFF
3D00 0000
3D02 0000
3D04 0000
3D06 0004
3D08 0000
3D0A 0000
3D0C 0020
3D0E 0000
3D10 0000
3D12 0001
3D14 0000
3D16 0000
3D18 0000
3D1A 0000
3D1C 0000
3D1E 00C0
3D20 0000
3D22 0000
3D24 0003
3D26 0000
3D28 0000
3D2A 0000
3D2C 0000
3D2E 0000
3D30 0000
3D32 0000
3D34 0000
3D36 0000
3D38 0000
3D3A 0000
3D3C 0001
3D3E 0000
3D40 FFFF
3D42 FFFF
3D44 FFFF
3D46 FFFF
3D48 FFFF
3D4A FFFF
3D4C FFFF
3D4E FFFF
3D50 FFFF
3D52 FFFF
3D54 FFFF
3D56 FFFF
3D58 FFFF
3D5A FFFF
3D5C FFFF
3D5E FFFF
3D60 FFFF
3D62 FFFF
3D64 FFFF
3D66 FFFF
3D68 FFFF
3D6A FFFF
3D6C FFFF
3D6E FFFF
3D70 FFFF
3D72 FFFF
3D74 FFFF
3D76 FFFF
3D78 FFFF
3D7A FFFF
3D7C FFFF
3D7E FFFF
3D80 0000
3D82 0000
3D84 0000
3D86 0000
3D88 0028  (
3D8A 0000
3D8C 0000
3D8E 0000
3D90 0000
3D92 0000
3D94 0000
3D96 0000
3D98 0000
3D9A 0000
3D9C 0000
```

```
3D9E 0000
3DA0 0000
3DA2 0000
3DA4 0000
3DA6 0000
3DA8 0000
3DAA 0000
3DAC 0000
3DAE 0004
3DB0 0000
3DB2 0000
3DB4 0000
3DB6 0000
3DB8 0000
3DBA 0000
3DBC 0000
3DBE 0000
3DC0 FFFF
3DC2 FFFF
3DC4 FFFF
3DC6 FFFF
3DC8 FFFF
3DCA FFFF
3DCC FFFF
3DCE FFFF
3DD0 FFFF
3DD2 FFFF
3DD4 FFFF
3DD6 FFFF
3DD8 FFFF
3DDA FFFF
3DDC FFFF
3DDE FFFF
3DE0 FFFF
3DE2 FFFF
3DE4 FFFF
3DE6 FFFF
3DE8 FFFF
3DEA FFFF
3DEC FFFF
3DEE FFFF
3DF0 FFFF
3DF2 FFFF
3DF4 FFFF
3DF6 FFFF
3DF8 FFFF
3DFA FFFF
3DFC FFFF
3DFE FFFF
3E00 0000
3E02 0000
3E04 0000
3E06 0000
3E08 0000
3E0A 0000
3E0C 0000
3E0E 0000
3E10 0000
3E12 0000
3E14 0000
3E16 0000
3E18 0010
3E1A 0000
3E1C 0000
3E1E 0000
3E20 0010
3E22 0000
3E24 0040  @
3E26 0000
3E28 0000
3E2A 0004
3E2C 000C
3E2E 0000
3E30 0020
3E32 0000
3E34 0000
3E36 0000
3E38 0000
3E3A 0000
3E3C 0000
3E3E 0000
3E40 FFFF
3E42 FFFF
3E44 FFFF
```

3E46 FFFF
3E48 FFFF
3E4A FFFF
3E4C FFFF
3E4E FFFF
3E50 FFFF
3E52 FFFF
3E54 FFFF
3E56 FFFF
3E58 FFFF
3E5A FFFF
3E5C FFFF
3E5E FFFF
3E60 FFFF
3E62 FFFF
3E64 FFFF
3E66 FFFF
3E68 FFFF
3E6A FFFF
3E6C FFFF
3E6E FFFF
3E70 FFFF
3E72 FFFF
3E74 FFFF
3E76 FFFF
3E78 FFFF
3E7A FFFF
3E7C FFFF
3E7E FFFF
3E80 0000
3E82 0000
3E84 0000
3E86 0000
3E88 0000
3E8A 0000
3E8C 0000
3E8E 0000
3E90 0000
3E92 0000
3E94 0000
3E96 0000
3E98 0000
3E9A 0000
3E9C 0000
3E9E 0000
3EA0 0010
3EA2 0000
3EA4 0000
3EA6 0000
3EA8 0000
3EAA 0000
3EAC 0000
3EAE 0000
3EB0 0000
3EB2 0000
3EB4 0000
3EB6 0000
3EB8 0000
3EBA 0000
3EBC 0000
3EBE 0000
3EC0 FFFF
3EC2 FFFF
3EC4 FFFF
3EC6 FFFF
3EC8 FFFF
3ECA FFFF
3ECC FFFF
3ECE FFFF
3ED0 FFFF
3ED2 FFFF
3ED4 FFFF
3ED6 FFFF
3ED8 FFFF
3EDA FFFF
3EDC FFFF
3EDE FFFF
3EE0 FFFF
3EE2 FFFF
3EE4 FFFF
3EE6 FFFF
3EE8 FFFF
3EEA FFFF
3EEC FFFF

```
3EEE FFFF
3EF0 FFFF
3EF2 FFFF
3EF4 FFFF
3EF6 FFFF
3EF8 FFFF
3EFA FFFF
3EFC FFFF
3EFE FFFF
3F00 0000
3F02 0000
3F04 0000
3F06 0000
3F08 0000
3F0A 0000
3F0C 0000
3F0E 0000
3F10 0000
3F12 0000
3F14 0000
3F16 0000
3F18 0000
3F1A 0000
3F1C 0000
3F1E 0000
3F20 0000
3F22 0000
3F24 0000
3F26 0000
3F28 0000
3F2A 0000
3F2C 0000
3F2E 0008
3F30 0020
3F32 0020
3F34 0040  @
3F36 0000
3F38 0000
3F3A 0000
3F3C 0000
3F3E 0000
3F40 FFFF
3F42 FFFF
3F44 FFFF
3F46 FFFF
3F48 FFFF
3F4A FFFF
3F4C FFFF
3F4E FFFF
3F50 FFFF
3F52 FFFF
3F54 FFFF
3F56 FFFF
3F58 FFFF
3F5A FFFF
3F5C FFFF
3F5E FFFF
3F60 FFFF
3F62 FFFF
3F64 FFFF
3F66 FFFF
3F68 FFFF
3F6A FFFF
3F6C FFFF
3F6E FFFF
3F70 FFFF
3F72 FFFF
3F74 FFFF
3F76 FFFF
3F78 FFFF
3F7A FFFF
3F7C FFFF
3F7E FFFF
3F80 0000
3F82 0000
3F84 0000
3F86 0000
3F88 0000
3F8A 0000
3F8C 0000
3F8E 0000
3F90 0000
3F92 0000
3F94 0000
```

```
              3F96 0000
              3F98 0000
              3F9A 0000
              3F9C 0000
              3F9E 0000
              3FA0 0000
              3FA2 0000
              3FA4 0000
              3FA6 0000
              3FA8 0000
              3FAA 0002
              3FAC 0000
              3FAE 0000
              3FB0 0000
              3FB2 0000
              3FB4 0010
              3FB6 0008
              3FB8 0000
              3FBA 0000
              3FBC 0000
              3FBE 0000
              3FC0 FFFF
              3FC2 FFFF
              3FC4 FFFF
              3FC6 FFFF
              3FC8 FFFF
              3FCA FFFF
              3FCC FFFF
              3FCE FFFF
              3FD0 FFFF
              3FD2 FFFF
              3FD4 FFFF
              3FD6 FFFF
              3FD8 FFFF
              3FDA FFFF
              3FDC FFFF
              3FDE FFFF
              3FE0 FFFF
              3FE2 FFFF
              3FE4 FFFF
              3FE6 FFFF
              3FE8 FFFF
              3FEA FFFF
              3FEC FFFF
              3FEE FFFF
              3FF0 FFFF
              3FF2 FFFF
              3FF4 FFFF
              3FF6 FFFF
              3FF8 FFFF
              3FFA FFFD
              3FFC FFFF
              3FFE FFFF

; Here starts the p-code card's ROM --------------------------------------

CARDVAL       4000 AA01    .WORD AA01H   ; Header, version
              4002 0000    .WORD 0                     ; Number of programs, reserved
              4004 4010 @  .WORD PWRUP   ; Power up link
              4006 0000    .WORD 0                     ; First user program link
              4008 0000    .WORD 0                     ; DSR link
              400A 0000    .WORD 0                     ; Subprogram link
INTLINK       400C 0000    .WORD 0                     ; Interrupt link
              400E 0000    .WORD 0                     ; BASIC subprogram link
PWRUP         4010 0000    .WORD 0                     ; Linkage
              4012 4038 @8 .WORD PWRBEG  ; Power up routine
              4014 0000    .WORD 0

INTERP        4016 FFE0    .WORD                       ; Stack base. Also base point for interpreter-related
relocation.
              4018 0006    .WORD                       ; Used at startup. Could have been the time stamp
increment
              401A 484A HJ .WORD FAULGEN ; Runtime error
              401C 2D6E -n .WORD ADDROFS ; Table with address offsets for different code spaces
              401E 2B6E +n .WORD SYSCOM  ; SYSCOM
              4020 2AAA *  .WORD VDPINT  ; "Vector" for interrupt
              4022 2DB2 -  .WORD OPCODE  ; Opcode table
              4024 34DC 4  .WORD UNITCL  ; Address for UNITCLEAR
              4026 34C4 4  .WORD UNITWR  ; Address for UNITWRITE
              4028 34AC 4  .WORD UNITRD  ; Address for UNITREAD
              402A 34EE 4  .WORD UNITST  ; Address for UNITSTATUS
              402C 2BF4 +  .WORD USERDEF ; User defined IO devices
              402E 2C20 ,  .WORD UNITPNT ; Table with pointer to IO unit's function blocks
              4030 29DC )  .WORD PCBPNT  ; Table PCB pointers
```

```
             4032 2BFC +  .WORD DRVNUM  ; Table blocked device drive numbers
             4034 47D8 G  .WORD CURWIN  ; Show current window on screen (vector)
             4036 2780 '  .WORD INTMEM  ; First available low memory after 80 col screen

; Beginning of power up routine
PWRBEG       4038 0201    LI   R1,KRNGLOB            ; Kernel global data area
             403A 3580 5
             403C 8860  ` C    @TXTNO,@037AH(R1)      ; Check if 'NO' @38FA, then don't start
             403E 406C @l
             4040 037A  z
             4042 1327  ' JEQ  NOSTART
             4044 02E0    LWPI PASCALWS
             4046 8380
             4048 0204    LI   R4,GROMWA; P-code card GROM write address
             404A 5FFE _
             404C D0A4    MOVB @-WAOFFS(R4),R2        ; From GROMRD
             404E FBFE
             4050 04E0    CLR  @STRTBUF
             4052 3900 9
             4054 0202    LI   R2,GROMCOD             ; Code repository in GROM. At E804H are 1310 bytes
which are transferred to 2000H.
             4056 E804
             4058 D502    MOVB R2,*R4   ; To GROMWA
             405A 06C2    SWPB R2
             405C D502    MOVB R2,*R4
             405E 06A0    BL   @GROMLOD ; Fetch code from GROM repository, 1310 bytes
             4060 4070 @p
             4062 06A0    BL   @INITCOD ; Branch to that temporary setup code to start the system
             4064 2004
PMERUN       4066 020C    LI   R12,FETCH; PME fetch address
             4068 8300
             406A 045C  \ B    *R12     ; PME fetch. Now the p-system is started and executes p-code

TXTNO        406C 4E4F NO .ASCII "NO"
TXTGO        406E 474F GO .ASCII "GO"

; Load data from GROM (code repository) to RAM
; First fetch start and end destination address
; Then fetch data from GROM
GROMLOD      4070 0203    LI   R3,4     ; Byte in two addresses
             4072 0004
             4074 0201    LI   R1,PASCALWS+10         ; R5 Pascal WS
             4076 838A
LOOP01       4078 DC64  d MOVB @-WAOFFS(R4),*R1+      ; From GROM data (402H less)
             407A FBFE
             407C 0603    DEC  R3
             407E 16FC    JNE  LOOP01
             4080 6185 a  S    R5,R6
             4082 CD45  E MOV  R5,*R5+  ; R5 pointer
             4084 0646  F DECT R6                     ; R6 counter
             4086 CD46  F MOV  R6,*R5+
             4088 0646  F DECT R6
LOOP02       408A DD64  d MOVB @-WAOFFS(R4),*R5+      ; From GROM data
             408C FBFE
             408E 0606    DEC  R6
             4090 16FC    JNE  LOOP02
NOSTART      4092 069B    BL   *R11     ; Returns to console if not p-code start, otherwise returns to p-
system boot sequence

UCASE        4094 415A AZ .ASCII "AZ"
NUMERIC      4096 3039 09 .ASCII "09"
LCASE        4098 617A az .ASCII "az"
USCORE       409A 5F24 _$ .ASCII "_$"
SPCR         409C 200D    .WORD 200DH   ; Space CR
BYLF         409E 0A      .BYTE AH                    ; LF
BYDLE        409F 10           .BYTE 10H                          ; DLE
FORMFED      40A0 0C      .BYTE CH                    ; FF
             40A1 00           .BYTE 0                ; NUL
SPACES       40A2 2020    .ASCII "  "   ; Used as .WORD to clear 80-column screen
LINEFED      40A4 000A    .WORD AH                    ; NUL LF
             40A6 0800    .WORD 800H                  ; BS NUL
CONFALS      40A8 0000    .WORD 0       ; Constant FALSE
CONNIL       40AA 0000    .WORD 0       ; Constant NIL
ZERO         40AC 0000    .WORD 0
MINUS1       40AE FFFF    .WORD -1
CON001       40B0 0001    .WORD 1
CON200H      40B2 0200    .WORD 200H
STAMP        40B4 0006    .WORD 6
CONTRUE      40B6 0001    .WORD 1       ; Byte sex current segment, TRUE
ALLFF        40B8 FFFF    .WORD FFFFH
CON016       40BA 0010    .WORD 16D     ; Constant 16
BUFIX        40BC 0000    .WORD         ; Buffer offset index in IOJ

; Table with bit masks used to handle packed fields
```

```
MASKTAB       40BE 0000    .WORD 0H
              40C0 0001    .WORD 1H
              40C2 0003    .WORD 3H
              40C4 0007    .WORD 7H
              40C6 000F    .WORD FH
              40C8 001F    .WORD 1FH
              40CA 003F    .WORD 3FH
              40CC 007F    .WORD 7FH
              40CE 00FF    .WORD FFH
              40D0 01FF    .WORD 1FFH
              40D2 03FF    .WORD 3FFH
              40D4 07FF    .WORD 7FFH
              40D6 0FFF    .WORD FFFH
              40D8 1FFF    .WORD 1FFFH
              40DA 3FFF    .WORD 3FFFH
              40DC 7FFF    .WORD 7FFFH
              40DE FFFF    .WORD FFFFH

; Tables with shift instructions for handling packed fields
SHLEFT        40E0 1000    NOP
              40E2 0A11    SLA  R1,1
              40E4 0A21  ! SLA  R1,2
              40E6 0A31  1 SLA  R1,3
              40E8 0A41  A SLA  R1,4
              40EA 0A51  Q SLA  R1,5
              40EC 0A61  a SLA  R1,6
              40EE 0A71  q SLA  R1,7
              40F0 0A81    SLA  R1,8
              40F2 0A91    SLA  R1,9
              40F4 0AA1    SLA  R1,10
              40F6 0AB1    SLA  R1,11
              40F8 0AC1    SLA  R1,12
              40FA 0AD1    SLA  R1,13
              40FC 0AE1    SLA  R1,14
              40FE 0AF1    SLA  R1,15
SHRIGHT       4100 1000    NOP
              4102 0913    SRL  R3,1
              4104 0923  # SRL  R3,2
              4106 0933  3 SRL  R3,3
              4108 0943  C SRL  R3,4
              410A 0953  S SRL  R3,5
              410C 0963  c SRL  R3,6
              410E 0973  s SRL  R3,7
              4110 0983    SRL  R3,8
              4112 0993    SRL  R3,9
              4114 09A3    SRL  R3,10
              4116 09B3    SRL  R3,11
              4118 09C3    SRL  R3,12
              411A 09D3    SRL  R3,13
              411C 09E3    SRL  R3,14
              411E 09F3    SRL  R3,15
SHCIRC        4120 1000    NOP
              4122 0BF0    SRC  R0,15
              4124 0BE0    SRC  R0,14
              4126 0BD0    SRC  R0,13
              4128 0BC0    SRC  R0,12
              412A 0BB0    SRC  R0,11
              412C 0BA0    SRC  R0,10
              412E 0B90    SRC  R0,9
              4130 0B80    SRC  R0,8
              4132 0B70  p SRC  R0,7
              4134 0B60  ` SRC  R0,6
              4136 0B50  P SRC  R0,5
              4138 0B40  @ SRC  R0,4
              413A 0B30  0 SRC  R0,3
              413C 0B20    SRC  R0,2
              413E 0B10    SRC  R0,1
              4140 1000    NOP

; Load p-code address pointer
; Enter at LOADADDR if p-code execution should continue at loaded address.
; Enter at LOADAD1 if address should be loaded and then return to caller.
LOADADDR 4142 C2CC   MOV  R12,R11        ; PME fetch
LOADAD1       4144 C3CF    MOV  R15,R15  ; PME memory type
              4146 1508    JGT  GROMCODE ; 4158 In GROM?
              4148 1306    JEQ  CPUCODE  ; 4156 In CPU RAM
; Code in VDP RAM
              414A DB60  ` MOVB @8391,@WAOFFS(R13)     ; Lsby R8 IPC
              414C 8391
              414E 0402
              4150 1000    NOP
              4152 DB48  H MOVB R8,@WAOFFS(R13)
              4154 0402
CPUCODE       4156 045B  [ B    *R11                   ; Done, return to PME
```

```
GROMCODE 4158 DB48  H MOVB R8,@WAOFFS(R13)
              415A 0402
              415C 1000    NOP
              415E DB60  ` MOVB @8391,@WAOFFS(R13)
              4160 8391
              4162 0402
              4164 045B  [ B    *R11                  ; Done, return to PME

; Fetch data and flip bytes if needed. Return in R2
GETDAT        4166 C3CF    MOV  R15,R15
              4168 160B    JNE  NOTCPU
              416A C0B1    MOV  *R1+,R2
              416C C820    MOV  @PMEFLIP,@PMEFLIP      ; 2AE8
              416E 2AE8 *
              4170 2AE8 *
              4172 1601    JNE  NOFLIP   ; 4176
              4174 06C2    SWPB R2
NOFLIP        4176 045B  [ B    *R11

; Fetch data from *R1, no flipping if in CPU RAM. Return in R2
GETDAT2       4178 C3CF    MOV  R15,R15  ; Code type
              417A 1602    JNE  NOTCPU   ; Not CPU RAM
              417C C0B1    MOV  *R1+,R2
              417E 045B  [ B    *R11
NOTCPU        4180 1507    JGT  INGROM
              4182 DB60  ` MOVB @PASCALWS+3,@WAOFFS(R13)            ; Load address to VDP RAM
              4184 8383
              4186 0402
              4188 1000    NOP
              418A DB41  A MOVB R1,@WAOFFS(R13)
              418C 0402
              418E 1006    JMP  CHKFLIP
INGROM        4190 DB41  A MOVB R1,@WAOFFS(R13)                     ; Load address to GROM
              4192 0402
              4194 1000    NOP
              4196 DB60  ` MOVB @PASCALWS+3,@WAOFFS(R13)            ; Load address to GROM
              4198 8383
              419A 0402
CHKFLIP       419C C0A0    MOV  @PMEFLIP,R2
              419E 2AE8 *
              41A0 1605    JNE  NOTFLIP
              41A2 D81D    MOVB *R13,@PASCALWS+5       ; Flipped code data to R2 Lsby
              41A4 8385
              41A6 05C1    INCT R1
              41A8 D09D    MOVB *R13,R2  ; Code data
              41AA 045B  [ B    *R11
NOTFLIP       41AC D09D    MOVB *R13,R2  ; Code data
              41AE 05C1    INCT R1
              41B0 D81D    MOVB *R13,@PASCALWS+5       ; Code data to R2 Lsby
              41B2 8385
              41B4 045B  [ B    *R11

; Keyboard scanning
KSCANV        41B6 2896 (  .WORD SUB2WS
              41B8 41BA A  .WORD KSCANX
KSCANX        41BA 06A0    BL   @KSCAN   ; Do keyscan
              41BC 42C2 B
              41BE 04C0    CLR  R0
              41C0 D020    MOVB @KEYCODE,R0
              41C2 8375  u
              41C4 9800    CB   R0,@ALLFF; No key
              41C6 40B8 @
              41C8 1360  ` JEQ  CHKSTOP
              41CA D060  ` MOVB @GPLST,R1
              41CC 837C  |
              41CE 1304    JEQ  KEYDOWN  ; Same as last time?
              41D0 C820    MOV  @SBITMSK+18,@AUTORPT   ; Contains 200H
              41D2 2DA4 -
              41D4 2CD0 ,
              41D6 1006    JMP  NORPT

KEYDOWN       41D8 0620    DEC  @AUTORPT
              41DA 2CD0 ,
              41DC 1656  V JNE  CHKSTOP
              41DE C820    MOV  @SBITMSK+12,@AUTORPT   ; Contains 40H
              41E0 2D9E -
              41E2 2CD0 ,
NORPT         41E4 04E0    CLR  @SCBLANK
              41E6 288E (
              41E8 E820    SOC  @SBITMSK+12,@VDPCOPY   ; Contains 40H
              41EA 2D9E -
              41EC 2810 (
              41EE C260  ` MOV  @VDPCOPY,R9
              41F0 2810 (
```

```
                41F2 06A0    BL   @VDPWAW
                41F4 47C4 G
                41F6 0280    CI   R0,8100  ; Screen left
                41F8 8100
                41FA 135A  Z JEQ  SCRLEFT
                41FC 0280    CI   R0,8600  ; Screen right
                41FE 8600
                4200 1348  H JEQ  SCRIGHT
                4202 9020    CB   @FLUSH,R0; Flush key
                4204 2BC0 +
                4206 1605    JNE  NOFLUSH
                4208 0560  ` INV  @FLUSHFL ; Flush := not flush
                420A 2894 (
                420C 04E0    CLR  @STOPFLG ; Stop := false
                420E 2890 (
                4210 103C  < JMP  CHKSTOP

NOFLUSH         4212 9020    CB   @STOPCOD,R0
                4214 2BC2 +
                4216 1603    JNE  NOSTOP
                4218 0560  ` INV  @STOPFLG ; Stop := not stop
                421A 2890 (
                421C 1036  6 JMP  CHKSTOP

NOSTOP          421E 9020    CB   @BREAK,R0; Check BREAK key
                4220 2BC3 +
                4222 160C    JNE  NOBREAK
                4224 04E0    CLR  @FLUSHFL ; BREAK key
                4226 2894 (
                4228 04E0    CLR  @STOPFLG
                422A 2890 (
                422C C820    MOV  @SBITMSK+6,@FAULCOD    ; Contains 8
                422E 2D98 -
                4230 2B70 +p
                4232 0201    LI   R1,DOBREAK
                4234 4840 H@
                4236 C801    MOV  R1,@PASCALWS+24        ; F12 (branch address)
                4238 8398
                423A 1027  ' JMP  CHKSTOP

NOBREAK         423C 9020    CB   @ALOCK,R0; Check soft alphalock key
                423E 2BCA +
                4240 1603    JNE  NOALOCK
                4242 0560  ` INV  @ALOCKFL
                4244 2892 (
                4246 1021  ! JMP  CHKSTOP

NOALOCK         4248 21A0 !  COC  @SBITMSK+14,R6         ; Contains 80H. Key buffer full?
                424A 2DA0 -
                424C 1603    JNE  NOTFULL                ; Buffer not full
                424E 06A0    BL   @BEEP
                4250 480E H
                4252 101B    JMP  CHKSTOP

NOTFULL         4254 C260  ` MOV  @ALOCKFL,R9            ; Make upper case?
                4256 2892 (
                4258 1308    JEQ  NOUCASE
                425A 9800    CB   R0,@LCASE; Check if keyboard input is within 'a'..'z'
                425C 4098 @
                425E 1A05    JL   NOUCASE
                4260 9800    CB   R0,@LCASE+1
                4262 4099 @
                4264 1B02    JH   NOUCASE
                4266 7020 p  SB   @SPCR,R0 ; Convert to upper case
                4268 409C @
NOUCASE         426A C247  G MOV  R7,R9                  ; Key buffer index
                426C A260  ` A    @KBUFPNT,R9            ; Add position of key buffer area
                426E 288A (
                4270 06A0    BL   @VDPWAW  ; Set VDP address
                4272 47C4 G
                4274 DA80    MOVB R0,@-2(R10)           ; To VDPWD, store key in buffer
                4276 FFFE
                4278 0587    INC  R7                    ; Point to next position in buffer
                427A 0247  G ANDI R7,001F  ; Circular buffer 32 bytes long
                427C 001F
                427E 8207    C    R7,R8                 ; Compare to fetch pointer
                4280 1602    JNE  SOMEBUF  ; Something in buffer, but not full?
                4282 E1A0    SOC  @SBITMSK+14,R6        ; Contains 80H             ; Buffer full flag
                4284 2DA0 -
SOMEBUF         4286 41A0 A  SZC  @SBITMSK+12,R6        ; Contains 40H             ; Something is in buffer
flag
                4288 2D9E -
CHKSTOP         428A C020    MOV  @STOPFLG,R0           ; If stopflag is set, only keep checking the keyboard
                428C 2890 (
```

```
             428E 1695    JNE  KSCANX
             4290 0380    RTWP

; Move the visible screen window to the right. Check against current physical width
SCRIGHT      4292 0202    LI   R2,80                 ; Calculate highest window number of 32/40 columns
             4294 0050  P
             4296 60A0 `  S    @WIDTH,R2
             4298 2CC6 ,
             429A 04C1    CLR  R1
             429C C020    MOV  @WIDTH,R0
             429E 2CC6 ,
             42A0 0910    SRL  R0,1
             42A2 3C40 <@ DIV  R0,R1
             42A4 8801    C    R1,@WINDOW
             42A6 2CCA ,
             42A8 1308    JEQ  DISPWIN  ; Already max to the right?
             42AA 05A0    INC  @WINDOW
             42AC 2CCA ,
             42AE 1005    JMP  DISPWIN

; Move the visible screen window left.
SCRLEFT      42B0 C020    MOV  @WINDOW,R0
             42B2 2CCA ,
             42B4 1302    JEQ  DISPWIN
             42B6 0620    DEC  @WINDOW
             42B8 2CCA ,
DISPWIN      42BA 06A0    BL   @DISPCUR
             42BC 47D2 G
             42BE 0460  ` B    KSCANX   ; Read the keyboard again
             42C0 41BA A

; Calling console's keyscan code. A lot of things moved around to adapt to it.
KSCAN        42C2 D820    MOVB @SPACES,@GPLST
             42C4 40A2 @
             42C6 837C  |
             42C8 D260  ` MOVB @KBD,R9  ; Keyboard code
             42CA 8374  t
             42CC 1628  ( JNE  PASKBD
KSCAN2       42CE 0200    LI   R0,INTWS+6             ; Save some data before keyscan
             42D0 83C6
             42D2 020C    LI   R12,KSCSAVE
             42D4 2AA0 *
             42D6 C050  P MOV  *R0,R1
             42D8 CC3C  < MOV  *R12+,*R0+
             42DA C090    MOV  *R0,R2
             42DC CC3C  < MOV  *R12+,*R0+
             42DE C0D0    MOV  *R0,R3
             42E0 CC3C  < MOV  *R12+,*R0+
             42E2 0200    LI   R0,INTWS+20
             42E4 83D4
             42E6 C130  0 MOV  *R0+,R4
             42E8 C170  p MOV  *R0+,R5
             42EA C270  p MOV  *R0+,R9
             42EC D820    MOVB @VDPCOPY+1,@INTWS+20
             42EE 2811 (
             42F0 83D4
             42F2 02E0    LWPI GPLWS
             42F4 83E0
             42F6 06A0    BL   @CONSCAN               ; Console keyboard scan
             42F8 000E
             42FA 02E0    LWPI SUB2WS
             42FC 2896 (
             42FE 0200    LI   R0,INTWS+6             ; Restore data after keyscan
             4300 83C6
             4302 020C    LI   R12,KSCSAVE
             4304 2AA0 *
             4306 CF10    MOV  *R0,*R12+
             4308 CC01    MOV  R1,*R0+
             430A CF10    MOV  *R0,*R12+
             430C CC02    MOV  R2,*R0+
             430E CF10    MOV  *R0,*R12+
             4310 CC03    MOV  R3,*R0+
             4312 0200    LI   R0,83D4
             4314 83D4
             4316 CC04    MOV  R4,*R0+
             4318 CC05    MOV  R5,*R0+
             431A CC09    MOV  R9,*R0+
KSCANRT      431C 045B  [ B    *R11

; Executed when keyboard (@KBD) is not zero. Reading Pascal keyboard layout here
PASKBD       431E 04C9    CLR  R9
             4320 04C0    CLR  R0
LOOP98       4322 0701    SETO R1
             4324 020C    LI   R12,0024H; CRU keyboard select
```

```
          4326 0024  $
          4328 06C9    SWPB R9
          432A 30C9 0  LDCR R9,3
          432C 06C9    SWPB R9
          432E 020C    LI   R12,0006 ; Joystick fire (column 1)
          4330 0006
          4332 3601 6  STCR R1,8
          4334 06C1    SWPB R1
          4336 0541  A INV  R1
          4338 1310    JEQ  COL2
          433A C249  I MOV  R9,R9
          433C 162C  , JNE  4396
          433E 2060  ` COC  SBITMSK+10,R1          ; Contains 20H
          4340 2D9C -
          4342 1604    JNE  434C
          4344 05C0    INCT R0
          4346 4060 @` SZC  SBITMSK+10,R1          ; Contains 20H
          4348 2D9C -
          434A 1004    JMP  4354

          434C 2060  ` COC  SBITMSK+8,R1           ; Contains 10H
          434E 2D9A -
          4350 1622  " JNE  4396
COL2      4352 0580    INC  R0
          4354 4060 @` SZC  SBITMSK+8,R1           ; Contains 10H
          4356 2D9A -
          4358 161E    JNE  4396
          435A 0589    INC  R9                     ; Next column
          435C 0289    CI   R9,5
          435E 0005
          4360 11E0    JLT  LOOP98
          4362 D1A0    MOVB @ALLFF,R6
          4364 40B8 @
          4366 D806    MOVB R6,@KEYCODE
          4368 8375  u
          436A 0640  @ DECT R0
          436C 1608    JNE  437E
          436E 25A0 %  CZC  SBITMSK+8,R6           ; Contains 10H
          4370 2D9A -
          4372 1602    JNE  4378
          4374 41A0 A  SZC  SBITMSK+10,R6          ; Contains 20H
          4376 2D9C -
          4378 E1A0    SOC  SBITMSK+8,R6           ; Contains 10H
          437A 2D9A -
          437C 10CF    JMP  KSCANRT

          437E 21A0 !  COC  SBITMSK+8,R6           ; Contains 10H
          4380 2D9A -
          4382 16CC    JNE  KSCANRT
          4384 41A0 A  SZC  SBITMSK+8,R6           ; Contains 10H
          4386 2D9A -
          4388 25A0 %  CZC  SBITMSK+10,R6          ; Contains 20H
          438A 2D9C -
          438C 16C7    JNE  KSCANRT
          438E D820    MOVB @SPACES,@KEYCODE
          4390 40A2 @
          4392 8375  u
          4394 045B  [ B    *R11

          4396 06C1    SWPB R1
          4398 070C    SETO R12      ; -1
LOOP57    439A 058C    INC  R12
          439C 0A11    SLA  R1,1
          439E 17FD    JNC  LOOP57
          43A0 028C    CI   R12,4
          43A2 0004
          43A4 1105    JLT  43B0
          43A6 0509    NEG  R9
          43A8 0229  ) AI   R9,9
          43AA 0009
          43AC 022C  , AI   R12,-4
          43AE FFFC
          43B0 0201    LI   R1,10D
          43B2 000A
          43B4 384C 8L MPY  R12,R1
          43B6 A242  B A    R2,R9
          43B8 06C9    SWPB R9
          43BA 9189    CB   R9,R6
          43BC 1607    JNE  43CC
          43BE D820    MOVB @BUFIX,@GPLST
          43C0 40BC @
          43C2 837C  |
          43C4 C006    MOV  R6,R0
          43C6 0240  @ ANDI R0,0003
```

```
             43C8 0003
             43CA 1004    JMP  43D4

             43CC 0246  F ANDI R6,FFFC
             43CE FFFC
             43D0 E180    SOC  R0,R6
             43D2 D189    MOVB R9,R6
             43D4 06C9    SWPB R9
             43D6 0201    LI   R1,40D
             43D8 0028  (
             43DA 3840 8@ MPY  R0,R1
             43DC A242  B A    R2,R9
             43DE A260  ` A    @KBLAYO,R9            ; Keyboard layout
             43E0 288C (
             43E2 06C9    SWPB R9
             43E4 D689    MOVB R9,*R10  ; VDPWA
             43E6 06C9    SWPB R9
             43E8 D689    MOVB R9,*R10  ; VDPWA
             43EA 0640  @ DECT R0
             43EC 1602    JNE  43F2
             43EE 0266  f ORI  R6,0030
             43F0 0030  0
             43F2 D82A  * MOVB @-WAOFFS(R10),@KEYCODE ; 0402H offset backwards, VDPRD
             43F4 FBFE
             43F6 8375  u
             43F8 045B  [ B    *R11

; Sprite movement, executed at simulated VDP interrupt
; Sprite_change_list is as defined by the compiler manual.

; type
;   link = ^sprite_change_list;
;   sprite_change_list = packed record
;                         packet: set of
(spr_pattern,spr_color;spr_clock,spr_y_pos,spr_x_pos,spr_y_vel,spr_x_vel);
;                         pattern_number: integer;
;                         color: integer;
;                         clock: integer;
;                         y_pos: integer;
;                         x_pos: integer;
;                         y_vel: integer;
;                         x_vel: integer;
;                         countdown: integer;
;                         link: ^sprite_change_list;
;                         auto_dispose: boolean;
;                       end;

SPRMOVE      43FA C020    MOV  @SPRCOIN,R0
             43FC 2808 (
             43FE 1304    JEQ  ZROCOIN
             4400 0280    CI   R0,7FFF  ; 32767
             4402 7FFF
             4404 1308    JEQ  COINOVF  ; Coincidence counter at ceiling
             4406 1005    JMP  CONT07

ZROCOIN      4408 C020    MOV  @VDPST,R0
             440A VDPST (
             440C 0240  @ ANDI R0,2000  ; Mask out coincidence bit
             440E 2000
             4410 1302    JEQ  COINOVF  ; No coincidence now
CONT07       4412 05A0    INC  @SPRCOIN
             4414 2808 (
COINOVF      4416 C80B    MOV  R11,@RETSAV3
             4418 280E (
             441A 04CC    CLR  R12
             441C 0201    LI   R1,1
             441E 0001
             4420 020A    LI   R10,SPRBIT1           ; Sprite bitmap
             4422 280A (
             4424 C69A    MOV  *R10,*R10; All zero?
             4426 130A    JEQ  NOSPR1
             4428 100E    JMP  CONT08

NXTSPR       442A 058C    INC  R12
             442C 0A11    SLA  R1,1
             442E 170B    JNC  CONT08
             4430 024C  L ANDI R12,001F ; Becomes zero after counting past last sprite
             4432 001F
             4434 1603    JNE  NOSPR1   ; Still possible sprites
SPRRET       4436 C2E0    MOV  @RETSAV3,R11
             4438 280E (
             443A 045B  [ B    *R11                  ; Return to general interrupt processing

NOSPR1       443C 05CA    INCT R10                   ; Advance to next sprite bitmap
```

```
              443E C69A    MOV  *R10,*R10; All zero?
              4440 13FA    JEQ  SPRRET   ; Nothing there?
              4442 0201    LI   R1,1
              4444 0001
CONT08        4446 C09A    MOV  *R10,R2  ; Fetch the sprite bitmap
              4448 2081    COC  R1,R2
              444A 16EF    JNE  NXTSPR   ; Nothing here, try next
              444C C00C    MOV  R12,R0   ; Sprite number
              444E 0A30  0 SLA  R0,3                   ; Multiply by 8
              4450 0220    AI   R0,SPRMOT; Sprite motion data, right after attribute list
              4452 0C80
              4454 0420    BLWP @VMBR
              4456 4774 Gt
              4458 28B6 (  .WORD SIMIWS  ; SIMIWS R0
              445A 28BA (  .WORD SIMIWS+4; SIMIWS R2
              445C 0008    .WORD 8                     ; Byte count
              445E C00C    MOV  R12,R0   ; Sprite number
              4460 0A20    SLA  R0,2
              4462 0220    AI   R0,SPRATT; Sprite attribute list
              4464 0C00
              4466 0420    BLWP @VMBR
              4468 4774 Gt
              446A 28B6 (  .WORD SIMIWS  ; SIMIWS R0
              446C 28C2 (  .WORD SIMIWS+12D            ; SIMIWS R6
              446E 0004    .WORD 4                     ; Byte count
              4470 06A0    BL   @SPRUPD  ; Update sprite position in R6, R7
              4472 4538 E8
              4474 0286    CI   R6,C0FF  ; Bytes 192 | 255 (max vertical, horizontal)
              4476 C0FF
              4478 120A    JLE  ONSCRN
              447A 0286    CI   R6,E000  ; Bytes 224 | 0
              447C E000
              447E 1B07    JH   ONSCRN
              4480 D0C3    MOVB R3,R3
              4482 1305    JEQ  ONSCRN
              4484 1502    JGT  448A
              4486 0226  & AI   R6,C000H ; Bytes 192 | 0            Adds a total of 224 | 0
              4488 C000
              448A 0226  & AI   R6,2000H ; Bytes 32 | 0
              448C 2000
ONSCRN        448E 06C2    SWPB R2                     ; Now flip in horizontal
              4490 06C3    SWPB R3
              4492 06C6    SWPB R6
              4494 06A0    BL   @SPRUPD  ; Update sprite position in R6
              4496 4538 E8
              4498 06C2    SWPB R2                     ; Now flip back to vertical | horizontal
              449A 06C3    SWPB R3
              449C 06C6    SWPB R6
              449E C104    MOV  R4,R4                  ; Sprite countdown
              44A0 133B  ; JEQ  LISTUPD
              44A2 0604    DEC  R4
              44A4 1639  9 JNE  LISTUPD
              44A6 C145  E MOV  R5,R5                  ; Sprite link pointer
              44A8 1604    JNE  SPRLIVE  ; Sprie has new motion data
              44AA 4681 F  SZC  R1,*R10  ; If nil, clear sprite from bitmap
              44AC 04C6    CLR  R6                     ; Sprite data
              44AE 04C7    CLR  R7
              44B0 1033  3 JMP  LISTUPD

; Here R5 is a pointer to sprite_change_list. Compiler manual page 144
; Packing of data, also in SIMIWS, causes a lot of maneouvering here
SPRLIVE       44B2 C215    MOV  *R5,R8   ; Fetch sprite packet set
              44B4 0B18    SRC  R8,1
              44B6 1702    JNC  NOPAT
              44B8 D1E5    MOVB @3(R5),R7; Pattern number
              44BA 0003
NOPAT         44BC 0B18    SRC  R8,1
              44BE 1708    JNC  NOCOL
              44C0 C247  G MOV  R7,R9
              44C2 0249  I ANDI R9,0080
              44C4 0080
              44C6 06C7    SWPB R7
              44C8 D1E5    MOVB @5(R5),R7; Pattern color
              44CA 0005
              44CC 06C7    SWPB R7
              44CE E1C9    SOC  R9,R7
NOCOL         44D0 0B18    SRC  R8,1
              44D2 1708    JNC  NOCLK
              44D4 C265  e MOV  @6(R5),R9; Sprite clock
              44D6 0006
              44D8 1303    JEQ  CLRCLK
              44DA E1E0    SOC  @CLKBIT,R7             ; Set clock bit
              44DC 2CAE ,
              44DE 1002    JMP  NOCLK
```

```
CLRCLK      44E0 41E0 A  SZC  @CLKBIT,R7            ; Clear clock bit
            44E2 2CAE ,
NOCLK       44E4 0B18    SRC  R8,1
            44E6 1702    JNC  NOYPOS
            44E8 D1A5    MOVB @9(R5),R6; Sprite y_pos
            44EA 0009
NOYPOS      44EC 0B18    SRC  R8,1
            44EE 1704    JNC  44F8
            44F0 06C6    SWPB R6
            44F2 D1A5    MOVB @11D(R5),R6           ; Sprite x_pos
            44F4 000B
            44F6 06C6    SWPB R6
NOXPOS      44F8 0B18    SRC  R8,1
            44FA 1702    JNC  NOYVEL
            44FC D0E5    MOVB @13D(R5),R3           ; Sprite y_vel
            44FE 000D
NOYVEL      4500 0B18    SRC  R8,1
            4502 1704    JNC  NOXVEL
            4504 06C3    SWPB R3
            4506 D0E5    MOVB @15D(R5),R3           ; Sprite x_vel
            4508 000F
            450A 06C3    SWPB R3
NOXVEL      450C C125  % MOV  @16D(R5),R4           ; Sprite countdown
            450E 0010
            4510 C265  e MOV  @20D(R5),R9           ; Sprite auto_dispose
            4512 0014
            4514 C165  e MOV  @18D(R5),R5           ; Sprite link
            4516 0012
LISTUPD     4518 0420    BLWP @VMBW                 ; Update sprite attribute list
            451A 478E G
            451C 28C2 (  .WORD SIMIWS+12            ; SIMIWS R6 (data)
            451E 28B6 (  .WORD SIMIWS  ; SIMIWS R0 (VDP pointer)
            4520 0004    .WORD 4                    ; Byte count
            4522 C00C    MOV  R12,R0
            4524 0A30  0 SLA  R0,3
            4526 0220    AI   R0,SPRMOT; Point into sprite motion table
            4528 0C80
            452A 0420    BLWP @VMBW                 ; Update sprite motion/linkage list
            452C 478E G
            452E 28BA (  .WORD SIMIWS+4; SIMIWS R2 (data)
            4530 28B6 (  .WORD SIMIWS  ; SIMIWS R0 (VDP pointer)
            4532 0008    .WORD 8                    ; Byte count
            4534 0460  ` B    @NXTSPR
            4536 442A D*

; Update sprite positions
; According to the compiler manual, movement is 1/32 pixel per interrupt.
SPRUPD      4538 04C9    CLR  R9
            453A 04C8    CLR  R8
            453C D242  B MOVB R2,R9
            453E 0849  I SRA  R9,4
            4540 D203    MOVB R3,R8
            4542 0848  H SRA  R8,4
            4544 A248  H A    R8,R9                 ; R9 := R2/16+R3/16
            4546 04C8    CLR  R8
            4548 D206    MOVB R6,R8                 ; Sprite vertical
            454A A209    A    R9,R8                 ; Sprite horizontal
            454C D188    MOVB R8,R6                 ; Vertical := vertical+R9
            454E 06C8    SWPB R8
            4550 0948  H SRL  R8,4
            4552 D088    MOVB R8,R2                 ; Return reminder of division by 16 to R2
            4554 045B  [ B    *R11

; Sound player for interrupt service.
; Data definition from the interface text of unit sound:

; type
;   sndlstptr = ^sndlstrec;
;   sndlstrec = record
;                 listsize: integer;
;                 curoffset: integer;
;                 packet: packed array [0..1] of 0..255; A list head where the list is as long as needed?
;               end;

SNDPLAY     4556 C081    MOV  R1,R2                 ; R1 is 0 to 3 (sound generators)
            4558 0A12    SLA  R2,1                  ; R2 is word pointer
            455A 0622  " DEC  @SOUND8(R2)
            455C 278C '
            455E 160A    JNE  ENDTONE
            4560 C022  " MOV  @SOUND9(R2),R0
            4562 27AC '
            4564 0280    CI   R0,7FFF  ; Max duration of a BEAT
            4566 7FFF
```

```
                4568 1302    JEQ  ENDBEAT
                456A 05A2    INC  @SOUND9(R2)            ; Increment BEAT
                456C 27AC '
ENDBEAT         456E C8A2    MOV  @SOUND1(R2),@SOUND8(R2)             ; Comes here if SOUND9 = 32767
                4570 2784 '
                4572 278C '
ENDTONE         4574 C022   " MOV  @SOUND10(R2),R0       ; Comes here if SOUND8 not decremented to zero
                4576 2794 '
                4578 130C    JEQ  4592
                457A 0622   " DEC  @SOUND10(R2)
                457C 2794 '
                457E 160C    JNE  ENDSERV  ; End sound play service for this voice
                4580 C022   " MOV  @SOUND11(R2),R0
                4582 279C '
                4584 0280    CI   R0,1
                4586 0001
                4588 130C    JEQ  45A2
                458A D821   ! MOVB @SOUND12(R1),@SNDGEN   ; Play SOUND12 if SOUND11 <> 1
                458C 2C98 ,
                458E 8400
                4590 1008    JMP  45A2

                4592 C022   " MOV  @SOUND11(R2),R0       ; Comes here if SOUND10 = 0
                4594 279C '
                4596 1605    JNE  45A2
ENDSERV         4598 045B   [ B    *R11

                459A D821   ! MOVB @SOUND13(R1),@SNDGEN
                459C 27DC '
                459E 8400
                45A0 045B   [ B    *R11

                45A2 0622   " DEC  @SOUND11(R2)                       ; Comes here if SOUND10 decremented to 0
now
                45A4 279C '
                45A6 16F8    JNE  ENDSERV  ; End sound player for this voice
NEWPNT          45A8 C0E2    MOV  @SOUND6(R2),R3         ; Fetch data pointer
                45AA 27A4 '
ENDSND          45AC D033   3 MOVB *R3+,R0
                45AE 2020    COC  @SOUND7,R0
                45B0 2CA2 ,
                45B2 1602    JNE  45B8
                45B4 0460   ` B    @46BE
                45B6 46BE F

                45B8 2020    COC  @SBITMSK+30,R0         ; 8000H  Comes here if SOUND7 bits not matching
                45BA 2DB0 -
                45BC 1344   D JEQ  4646
                45BE 2020    COC  @SBITMSK+28,R0         ; 4000H
                45C0 2DAE -
                45C2 1326   & JEQ  4610
                45C4 06C0    SWPB R0                     ; Must be most significant frequency byte (max 16383)
                45C6 D033   3 MOVB *R3+,R0  ; Fetch one more byte
                45C8 06C0    SWPB R0
                45CA 0281    CI   R1,3                   ; Last voice
                45CC 0003
                45CE 1317    JEQ  ISNOISE
                45D0 0204    LI   R4,1                   ; Convert frequency to count value
                45D2 0001
                45D4 C140   @ MOV  R0,R5
                45D6 0915    SRL  R5,1
                45D8 0225   % AI   R5,B503
                45DA B503
                45DC 1701    JNC  NOCARRY
                45DE 0584    INC  R4
NOCARRY         45E0 3D00 =  DIV  R0,R4
                45E2 C144   D MOV  R4,R5
                45E4 0244   D ANDI R4,03FF  ; Max frequency value
                45E6 03FF
                45E8 0A44   D SLA  R4,4                   ; Six frequency bits in second byte
                45EA 06C5    SWPB R5
                45EC 0245   E ANDI R5,0F00  ; Four frequency bits in first byte
                45EE 0F00
                45F0 B161   a AB   @SOUND4(R1),R5         ; Add generator pre-amble
                45F2 2C90 ,
                45F4 D805    MOVB R5,@SNDGEN             ; First byte frequency
                45F6 8400
                45F8 D804    MOVB R4,@SNDGEN             ; Second byte frequency
                45FA 8400
                45FC 10D7    JMP  ENDSND

ISNOISE         45FE 0280    CI   R0,7                   ; Max noise control value
                4600 0007
                4602 1BD4    JH   ENDSND   ; Invalid noise value
```

```
4604 06C0    SWPB R0
4606 B021  ! AB   @SOUND4(R1),R0         ; Add operation code
4608 2C90 ,
460A D800    MOVB R0,@SNDGEN
460C 8400
460E 10CE    JMP  ENDSND

4610 C883    MOV  R3,@SOUND6(R2)
4612 27A4 '
4614 C100    MOV  R0,R4
4616 06C4    SWPB R4
4618 0244  D ANDI R4,001F
461A 001F
461C 1602    JNE  4622
461E 0204    LI   R4,32D
4620 0020
4622 3922 9" MPY  @SOUND1(R2),R4
4624 2784 '
4626 C105    MOV  R5,R4
4628 2020    COC  @SBITMSK+26,R0         ; 2000H
462A 2DAC -
462C 160A    JNE  4642
462E 0225  % AI   R5,4
4630 0004
4632 0935  5 SRL  R5,3
4634 6105 a  S    R5,R4
4636 C884    MOV  R4,@SOUND10(R2)
4638 2794 '
463A 0585    INC  R5
463C C885    MOV  R5,@SOUND11(R2)
463E 279C '
4640 10AC    JMP  459A

4642 04C5    CLR  R5
4644 10F8    JMP  4636

4646 9020    CB   @SOUND7+3,R0
4648 2CA5 ,
464A 1316    JEQ  4678
464C 2020    COC  @SBITMSK+26,R0         ; 2000H
464E 2DAC -
4650 1307    JEQ  4660
4652 2020    COC  @SBITMSK+24,R0         ; 1000H
4654 2DAA -
4656 132D  - JEQ  46B2
4658 9020    CB   @SOUND7+2,R0
465A 2CA4 ,
465C 1310    JEQ  467E
465E 10A6    JMP  ENDSND

4660 D133  3 MOVB *R3+,R4
4662 9020    CB   @SOUND18,R0
4664 2CAA ,
4666 16A2    JNE  ENDSND
4668 0544  D INV  R4
466A 0244  D ANDI R4,0F00
466C 0F00
466E B121  ! AB   @SOUND19(R1),R4
4670 2C94 ,
4672 D844  D MOVB R4,@SOUND13(R1)
4674 27DC '
4676 109A    JMP  ENDSND

4678 D0E1    MOVB @SOUND14(R1),R3
467A 27B4 '
467C 1609    JNE  4690
467E D861  a MOVB @SOUND12(R1),@SOUND13(R1)
4680 2C98 ,
4682 27DC '
4684 04E2    CLR  @SOUND5(R2)
4686 27E0 '
4688 D862  b MOVB @SOUND5(R2),@SOUND14(R1)
468A 27E0 '
468C 27B4 '
468E 1085    JMP  459A

4690 0983    SRL  R3,8
4692 C103    MOV  R3,R4
4694 0A22  " SLA  R2,2
4696 A0C2    A    R2,R3
4698 0922  " SRL  R2,2
469A C8A3    MOV  @SOUND15(R3),@SOUND6(R2)
469C 27B6 '
469E 27A4 '
```

```
46A0 C8A3    MOV  @SOUND17(R3),@SOUND5(R2)
46A2 27E6 '
46A4 27E0 '
46A6 0644  D DECT R4
46A8 06C4    SWPB R4
46AA D844  D MOVB R4,@SOUND14(R1)
46AC 27B4 '
46AE 0460  ` B    @NEWPNT
46B0 45A8 E

46B2 0240  @ ANDI R0,0F00
46B4 0F00
46B6 D840  @ MOVB R0,@SOUND2(R1)
46B8 27D8 '
46BA 0460  ` B    @ENDSND
46BC 45AC E

46BE 2020    COC  @SBITMSK+26,R0         ; 2000H Comes here if SOUND7 bits matched fetched via
SOUND6 pointer
46C0 2DAC -
46C2 1353  S JEQ  476A
46C4 9020    CB   @SOUND7,R0
46C6 2CA2 ,
46C8 160D    JNE  46E4
46CA D033  3 MOVB *R3+,R0  ; R3 sound list pointer?
46CC 06C0    SWPB R0
46CE D033  3 MOVB *R3+,R0
46D0 06C0    SWPB R0
46D2 C8A2    MOV  @SOUND5(R2),@SOUND6(R2)             ; SOUND5 -> SOUND6
46D4 27E0 '
46D6 27A4 '
46D8 A8A0    A    @CON004,@SOUND6(R2)    ; Advance in list?
46DA 4772 Gr
46DC 27A4 '
46DE A880    A    R0,@SOUND6(R2)         ; Add a jump?
46E0 27A4 '
46E2 1041  A JMP  4766

46E4 9020    CB   @SOUND7+6,R0
46E6 2CA8 ,
46E8 160F    JNE  4708
46EA D033  3 MOVB *R3+,R0  ; R3 sound list pointer?
46EC 06C0    SWPB R0
46EE D033  3 MOVB *R3+,R0
46F0 06C0    SWPB R0
46F2 C8A2    MOV  @SOUND5(R2),@SOUND6(R2)             ; SOUND5 -> SOUND6
46F4 27E0 '
46F6 27A4 '
46F8 A8A0    A    @CON004,@SOUND6(R2)    ; Advance in list?
46FA 4772 Gr
46FC 27A4 '
46FE A880    A    R0,@SOUND6(R2)         ; Add a jump?
4700 27A4 '
4702 C022  " MOV  @SOUND5(R2),R0
4704 27E0 '
4706 101D    JMP  4742

4708 9020    CB   @SOUND7+4,R0
470A 2CA6 ,
470C 160B    JNE  4724
470E D8B3    MOVB *R3+,@SOUND6(R2)       ; Fetch new pointer
4710 27A4 '
4712 D8B3    MOVB *R3+,@SOUND6+1(R2)
4714 27A5 '
4716 C8A2    MOV  @SOUND6(R2),@SOUND5(R2)             ; SOUND6 -> SOUND5  Opposite to above
4718 27A4 '
471A 27E0 '
471C A8A0    A    @CON004,@SOUND6(R2)    ; Advance in list
471E 4772 Gr
4720 27A4 '
4722 1021  ! JMP  GONEW

4724 9020    CB   @SOUND16,R0            ; SOUND16 is SOUND7-5
4726 2C9D ,
4728 1621  ! JNE  PT2
472A D8B3    MOVB *R3+,@SOUND6(R2)       ; Fetch new pointer
472C 27A4 '
472E D8B3    MOVB *R3+,@SOUND6+1(R2)
4730 27A5 '
4732 C022  " MOV  @SOUND5(R2),R0
4734 27E0 '
4736 C8A2    MOV  @SOUND6(R2),@SOUND5(R2)             ; SOUND6 -> SOUND5  Opposite to above
4738 27A4 '
473A 27E0 '
```

```
             473C A8A0    A    @CON004,@SOUND6(R2)    ; Advance in list
             473E 4772 Gr
             4740 27A4 '
             4742 D121  ! MOVB @SOUND14(R1),R4
             4744 27B4 '
             4746 0984    SRL  R4,8
             4748 0284    CI   R4,8
             474A 0008
             474C 1301    JEQ  IS8
             474E 05C4    INCT R4
IS8          4750 0A22  " SLA  R2,2                   ; Now voice *8
             4752 C144  D MOV  R4,R5
             4754 A102    A    R2,R4                  ; Add voice*8 to value read from SOUND14
             4756 0922  " SRL  R2,2                   ; Restore voice*2
             4758 C903    MOV  R3,@SOUND15(R4)
             475A 27B6 '
             475C C900    MOV  R0,@SOUND17(R4)
             475E 27E6 '
             4760 06C5    SWPB R5
             4762 D845  E MOVB R5,@SOUND14(R1)
             4764 27B4 '
GONEW        4766 0460  ` B    @NEWPNT
             4768 45A8 E

PT3          476A 0583    INC  R3                     ; Increment pointer by 3
PT2          476C 05C3    INCT R3                     ; Increment pointer by 2
             476E 0460  ` B    @ENDSND
             4770 45AC E

CON004       4772 0004    .WORD 4

; Video multiple byte read
;            Pointer to pointer to address to read from
;            Pointer to address to write to
;            Byte count
; R10 in SUBWIN contains VDPWA

VMBR         4774 83A0    .WORD SUBWIN
             4776 4778 Gx .WORD VMBRX
VMBRX        4778 C07E  ~ MOV  *R14+,R1
             477A C0BE    MOV  *R14+,R2
             477C C0FE    MOV  *R14+,R3
             477E C251  Q MOV  *R1,R9
             4780 06A0    BL   @VDPWAR
             4782 47C8 G
LOOP31       4784 DCAA    MOVB @-WAOFFS(R10),*R2+     ; From VDPRD
             4786 FBFE
             4788 0603    DEC  R3
             478A 16FC    JNE  LOOP31
             478C 0380    RTWP

; Video multiple byte write
;            Pointer to address to read from
;            Pointer to pointer to address to write to
;            Byte count
; R10 in SUBWIN contains VDPWA

VMBW         478E 83A0    .WORD SUBWIN
             4790 4792 G  .WORD VMBWX
VMBWX        4792 C07E  ~ MOV  *R14+,R1 ; Pointer to source data
VMBW3        4794 C0BE    MOV  *R14+,R2 ; Pointer to VDP address
             4796 C0FE    MOV  *R14+,R3 ; Byte count
VMBW2        4798 C252  R MOV  *R2,R9   ; R2 points to address to write to
             479A 06A0    BL   @VDPWAW  ; VDP address for write
             479C 47C4 G
LOOP32       479E DAB1    MOVB *R1+,@-2(R10)                      ; R1 points to data block to write to
VDPWD
             47A0 FFFE
             47A2 0603    DEC  R3
             47A4 16FC    JNE  LOOP32
RET02        47A6 0380    RTWP

; Indirect video multiple byte write
;            Pointer to pointer to source as first data
IVMBW        47A8 83A0    .WORD SUBWIN
             47AA 47AC G  .WORD IVMBWX
IVMBWX       47AC C07E  ~ MOV  *R14+,R1
             47AE C051  Q MOV  *R1,R1
             47B0 10F1    JMP  VMBW3

; Double indirect video multiple byte write
;            Third word is pointer to byte count
I2VMBW       47B2 83A0    .WORD SUBWIN
             47B4 47B6 G  .WORD I2VMBWX
```

```
I2VMBWX       47B6 C07E  ~ MOV  *R14+,R1 ; Pointer to pointer (to data)
              47B8 C0BE    MOV  *R14+,R2 ; Pointer to address to write to
              47BA C0FE    MOV  *R14+,R3 ; Pointer to byte count
              47BC C051  Q MOV  *R1,R1   ; Pointer to data
              47BE C0D3    MOV  *R3,R3   ; Zero bytes?
              47C0 13F2    JEQ  RET02                 ; RTWP
              47C2 10EA    JMP  VMBW2

; Load VDP address from R9 to address in R10
VDPWAW        47C4 0269  i ORI  R9,4000  ; For writing
              47C6 4000 @
VDPWAR        47C8 06C9    SWPB R9                    ; For reading
              47CA D689    MOVB R9,*R10
              47CC 06C9    SWPB R9
              47CE D689    MOVB R9,*R10
              47D0 045B  [ B    *R11

; Display current window (number in WINDOW), called with BL
DISPCUR       47D2 0420    BLWP @CURWIN
              47D4 47D8 G
              47D6 045B  [ B    *R11

; Display current 40 col window (number in WINDOW) on screen
CURWIN        47D8 83A0    .WORD SUBWS
              47DA 47DC G  .WORD CURWINX
CURWINX       47DC C0E0    MOV  @WIDTH,R3
              47DE 2CC6 ,
              47E0 0913    SRL  R3,1
              47E2 38E0 8  MPY  @WINDOW,R3            ; Figure out starting column of current window
              47E4 2CCA ,
              47E6 A120    A    @SCREN80,R4           ; 80 column screen start in RAM
              47E8 2CBE ,
              47EA 0209    LI   R9,0800  ; Physical screen start in VDP RAM
              47EC 0800
              47EE 06A0    BL   @VDPWAW  ; Content of R9 to *R10 (VDPWA)
              47F0 47C4 G
              47F2 C020    MOV  @HEIGHT,R0            ; Number of screen lines to copy
              47F4 2CC8 ,
LOOP30        47F6 C084    MOV  R4,R2
              47F8 C0E0    MOV  @WIDTH,R3; One row of physical screen width at a time
              47FA 2CC6 ,
LOOP29        47FC DAB2    MOVB *R2+,@-2(R10)         ; Effectively VDPWD
              47FE FFFE
              4800 0603    DEC  R3
              4802 16FC    JNE  LOOP29
              4804 0224  $ AI   R4,80    ; Screen width
              4806 0050  P
              4808 0600    DEC  R0                    ; Count the rows (height)
              480A 16F5    JNE  LOOP30
              480C 0380    RTWP

; Beep. Executes when chr(7) is output.
BEEP          480E 0203    LI   R3,SOUND3
              4810 4834 H4
LOOP36        4812 D133  3 MOVB *R3+,R4
              4814 0984    SRL  R4,8
LOOP33        4816 D833  3 MOVB *R3+,@SNDGEN
              4818 8400
              481A 0604    DEC  R4                    ; Count sound bytes
              481C 16FC    JNE  LOOP33
              481E D0B3    MOVB *R3+,R2  ; Durations
              4820 1308    JEQ  BEEPRT
              4822 0982    SRL  R2,8
LOOP35        4824 0201    LI   R1,8000  ; One duration unit
              4826 8000
LOOP34        4828 0601    DEC  R1
              482A 16FE    JNE  LOOP34
              482C 0602    DEC  R2                    ; Number of durations
              482E 16FA    JNE  LOOP35
              4830 10F0    JMP  LOOP36

BEEPRT        4832 045B  [ B    *R11

; Sound data for beep. Number of bytes first
SOUND3        4834 06BF    .BYTE 6,8FH
              4836 DFFF    .BYTE DFH,FFH
              4838 8005    .BYTE 80H,5
              483A 9201    .BYTE 92H,1   ; Ends with duration value
              483C 019F    .WORD                      ; Not referred to
              483E 0000    .WORD                      ; Not referred to

; When BREAK key is pressed
DOBREAK       4840 020C    LI   R12,FETCH; PME fetch address
              4842 8300
```

```
             4844 C060  ` MOV  @FAULCOD,R1           ; Get fault code
             4846 2B70 +p
             4848 1003    JMP  FAUL2

; Fault handling
FAULGEN      484A C05B  [ MOV  *R11,R1  ; Data after call instruction
             484C 04E0    CLR  @MTYPFL
             484E 2AD6 *
FAUL2        4850 0281    CI   R1,0080  ; Segment fault code
             4852 0080
             4854 1605    JNE  NOSEGFLT
             4856 C805    MOV  R5,@FAEREC
             4858 2B82 +
             485A 04E0    CLR  @FAWORD  ; No words missing for stack
             485C 2B84 +
             485E 100B    JMP  CONT01

NOSEGFLT 4860 0281    CI   R1,0081       ; Stack fault
             4862 0081
             4864 1614    JNE  NOSTKFLT
             4866 C820    MOV  @CUREREC,@FAEREC       ; Faulting environment record
             4868 2AE0 *
             486A 2B82 +
             486C C0A0    MOV  @STKMISS,R2            ; Missing bytes (?) on stack
             486E 2AF0 *
             4870 0912    SRL  R2,1
             4872 C802    MOV  R2,@FAWORD             ; Words missing for stack
             4874 2B84 +

CONT01       4876 C820    MOV  @CURTSK,@FATIB         ; TIB of faulting task
             4878 2ABA *
             487A 2B80 +
             487C C801    MOV  R1,@FATYPE                          ; Type of fault
             487E 2B86 +
             4880 0201    LI   R1,REALSEM
             4882 2B7C +|
             4884 064A  J DECT SP
             4886 C681    MOV  R1,*SP
             4888 0201    LI   R1,SIGNAL+2            ; Signal fault semaphore
             488A 56B6 V
             488C 0451  Q B    *R1

NOSTKFLT 488E 064A  J DECT SP ; Not a stack fault
             4890 04DA    CLR  *SP                    ; Push zero
             4892 064A  J DECT SP
             4894 04DA    CLR  *SP                    ; Push zero
             4896 05A0    INC  @FLTCNT
             4898 2AEA *
             489A 064A  J DECT SP
             489C C681    MOV  R1,*SP   ; Push fault code
             489E C80E    MOV  R14,@GLOBPNT
             48A0 2AEC *
             48A2 0201    LI   R1,EXECERR             ; When the p-system starts EXECERR is only 25?
             48A4 2A46 *F
             48A6 C161  a MOV  @2(R1),R5; Get pointer to environment record (E_Rec_P)
             48A8 0002
             48AA 0203    LI   R3,2
             48AC 0002
             48AE 020B    LI   R11,CALX4
             48B0 4D6E Mn
             48B2 0460  ` B    @GETSEGB ; Find base of segment via E_Rec_P
             48B4 4ECC N

; Executing native code:
; Memory can start with NAT | NOP or p-code | NAT
; Then assembly code follows
; Return to PME with BL *R11 to get address where to continue correct

; Enter native code
NAT          48B6 48B8    .WORD NAT+2
             48B8 0588    INC  R8                     ; Advance to next even address
             48BA 0248  H ANDI R8,FFFE  ; Make even
             48BC FFFE
             48BE 0698    BL   *R8                    ; Enter assembly
             48C0 C20B    MOV  R11,R8   ; Update IPC
             48C2 045C  \ B    *R12     ; PME Fetch

; Native code information
NAT_INFO     48C4 8316    .WORD LDUB
             48C6 A203    A    R3,R8                  ; Jump over native information
             48C8 0460  ` B    @LOADADDR; Load address to p-code and execute
             48CA 4142 AB

; Unused opcodes
```

```
UNUSED        48CC 48CE   .WORD UNUSED+2
              48CE 06A0   BL   @FAULGEN
              48D0 484A HJ
              48D2 000B    .WORD 11      ; Unimplemented instruction
; Breakpoint
BPT           48D4 48D6    .WORD BPT+2
              48D6 06A0    BL   @FAULGEN
              48D8 484A HJ
              48DA 0010    .WORD 16      ; Breakpoint code

; True jump
TJP           48DC 8316    .WORD LDUB
              48DE C07A  z MOV  *SP+,R1
              48E0 0911    SRL  R1,1
              48E2 1806    JOC  DOJUMP
              48E4 045C  \ B    *R12     ; PME fetch

; False jump
FJP           48E6 8316    .WORD LDUB
              48E8 C07A  z MOV  *SP+,R1
              48EA 0911    SRL  R1,1
              48EC 1701    JNC  DOJUMP
              48EE 045C  \ B    *R12     ; PME fetch

; Short jump, check which direction
DOJUMP        48F0 0283    CI   R3,0080H
              48F2 0080
              48F4 1102    JLT  JMPFWD
JMPREV        48F6 0263  c ORI  R3,FF00  ; Sign extension
              48F8 FF00
JMPFWD        48FA A203    A    R3,R8
              48FC 100F    JMP  POLLINT

; Unconditional jump
UJP           48FE 8316    .WORD LDUB
              4900 11FA    JLT  JMPREV
              4902 10FB    JMP  JMPFWD

; Equal false jump
EFJ           4904 8316    .WORD LDUB
              4906 6EBA n  S    *SP+,*SP+
              4908 16F3    JNE  DOJUMP
              490A 045C  \ B    *R12     ; PME fetch

; Not equal false jump
NFJ           490C 8316    .WORD LDUB
              490E 6EBA n  S    *SP+,*SP+
              4910 13EF    JEQ  DOJUMP
              4912 045C  \ B    *R12                  ; PME fetch

; Unconditional long jump
; Interrupt request is polled when long jumps are done
JPL           4914 830E    .WORD LDUBB   ; Get immediate value
              4916 D120    MOVB @PASCALWS+7,R4        ; Make offset -> word
              4918 8387
              491A A204    A    R4,R8                 ; Take jump
; Interrupt poll
POLLINT       491C C18C    MOV  R12,R6   ; Save fetch address
              491E 04CC    CLR  R12
              4920 1F02    TB   VDPIRQ   ; VDP interrupt request check
              4922 1304    JEQ  NOVPINT  ; 492C
              4924 0420    BLWP @VDPINT  ; 2AAA
              4926 2AAA *
              4928 C30C    MOV  R12,R12  ; Fetch or CRU value?
              492A 1601    JNE  NORSTOR  ; Don't restore the address
NOVPINT       492C C306    MOV  R6,R12   ; Restore fetch address
NORSTOR       492E 0460  ` B    @LOADADDR; Load address to p-code and execute
              4930 4142 AB

; False long jump
FJPL          4932 830E    .WORD LDUBB   ; Get immediate value
              4934 C07A  z MOV  *SP+,R1  ; Pop boolean
              4936 0911    SRL  R1,1                 ; Move bit to carry
              4938 17EE    JNC  JPL+2                ; False?
              493A 045C  \ B    *R12                  ; PME fetch

; Case jump
; Parameter is base of jump table. Table starts with min and max index, then the jump addresses
XJP           493C 8330    .WORD LDBIG
              493E 0A13    SLA  R3,1
              4940 A0E0    A    @SEGCNAB,R3           ; Current segment constant table absolute
              4942 2ABE *
              4944 C043  C MOV  R3,R1
              4946 06A0    BL   @GETDAT  ; Get data from *R1 (with flip) -> R2
```

```
             4948 4166 Af
             494A C002    MOV  R2,R0
             494C 06A0    BL   @GETDAT  ; Get data from *R1 (with flip) -> R2
             494E 4166 Af
             4950 C0FA    MOV  *SP+,R3  ; Get index
             4952 8083    C    R3,R2                ; Check if index in range
             4954 1507    JGT  NOCASE
             4956 60C0 `  S    R0,R3
             4958 1105    JLT  NOCASE
             495A 0A13    SLA  R3,1
             495C A043  C A    R3,R1
             495E 06A0    BL   @GETDAT  ; Get data from *R1 (with flip) -> R2
             4960 4166 Af
             4962 A202    A    R2,R8                ; Add the case table jump offset to IPC
NOCASE       4964 0460  ` B    @LOADADDR; Load address to p-code and execute
             4966 4142 AB

; Short load word constant
SLDC         4968 496A    .WORD SLDCX
SLDCX        496A 0911    SRL  R1,1                 ; Restore original opcode
             496C 064A  J DECT SP
             496E C681    MOV  R1,*SP   ; Push the opcode
             4970 045C  \ B    *R12                 ; PME fetch

; Short load local
SLDL         4972 4974    .WORD SLDL+2
             4974 A049  I A    R9,R1     ; Frame pointer added to opcode
             4976 064A  J DECT SP
             4978 C6A1    MOV  @-54(R1),*SP          ; Compensate for opcode and MSCW size
             497A FFCA
             497C 045C  \ B    *R12      ; PME fetch

; Short load global
SLDO         497E 4980    .WORD SLDO+2
             4980 A04E  N A   R14,R1     ; Global frame pointer added to opcode
             4982 064A  J DECT SP
             4984 C6A1    MOV  @-86(R1),*SP          ; Opcode starts at 96
             4986 FFAA
             4988 045C  \ B    *R12      ; PME fetch

; Short index
SIND         498A 498C    .WORD SIND+2
             498C 0221  ! AI   R1,-240
             498E FF10
             4990 A05A  Z A    *SP,R1
             4992 C691    MOV  *R1,*SP
             4994 045C  \ B    *R12      ; PME fetch

; Reserved
RESERVE6     4996 8300    .WORD FETCH
NOP          .EQU          RESERVE6

; Load byte
LDB          4998 499A    .WORD LDB+2
             499A C07A  z MOV  *SP+,R1  ; Byte-address (two words)
             499C A05A  Z A    *SP,R1
             499E 04DA    CLR  *SP
             49A0 DA91    MOVB *R1,@1(SP)           ; The byte to load
             49A2 0001
             49A4 045C  \ B    *R12      ; PME fetch

; Store byte
STB          49A6 49A8    .WORD STB+2
             49A8 C06A  j MOV  @2(SP),R1; Address is given as a combination of two words (byte-pointer)
             49AA 0002
             49AC A06A  j A    @4(SP),R1
             49AE 0004
             49B0 D46A  j MOVB @1(SP),*R1           ; The byte to store
             49B2 0001
             49B4 022A  * AI   SP,6                 ; Pop three words
             49B6 0006
             49B8 045C  \ B    *R12      ; PME fetch

; Index into record and fetch value
IND          49BA 8330  0 .WORD LDBIG   ; Fetch index (word count)
             49BC 0A13    SLA  R3,1
             49BE A0DA    A    *SP,R3    ; Record base address
             49C0 C693    MOV  *R3,*SP
             49C2 045C  \ B    *R12      ; PME fetch

; Increment pointer
INC          49C4 8330  0 .WORD LDBIG   ; Pointer index (word count)
             49C6 0A13    SLA  R3,1
             49C8 A683    A    R3,*SP    ; Add base address
```

```
             49CA 045C  \ B    *R12     ; PME fetch

; Index Array
IXA          49CC 8330  0 .WORD LDBIG   ; Array element size (words)
             49CE C03A  : MOV  *SP+,R0  ; Array index
             49D0 1303    JEQ  ZEROIX   ; Index is zero
             49D2 3803 8  MPY  R3,R0                ; Element size*index number
             49D4 0A11    SLA  R1,1                 ; Expand to byte
             49D6 A681    A    R1,*SP   ; Add to base pointer
ZEROIX       49D8 045C  \ B    *R12     ; PME fetch

; Load constant NIL
LDCN         49DA 49DC I  .WORD LDCN+2
             49DC 064A  J DECT SP
             49DE 04DA    CLR  *SP      ; NIL = zero
             49E0 045C  \ B    *R12     ; PME fetch

; Load constant integer
LDCI         49E2 830E    .WORD LDUBB
             49E4 D120    MOVB @PASCALWS+7,R4         ; Assemble two bytes to one word
             49E6 8387
             49E8 064A  J DECT SP
             49EA C684    MOV  R4,*SP
             49EC 045C  \ B    *R12                   ; PME fetch

; Load local
LDL          49EE 8330  .WORD LDBIG                   ; Word offset
             49F0 0A13    SLA  R3,1                   ; Make byte address
             49F2 A0C9    A    R9,R3                  ; Frame pointer
             49F4 064A  J DECT SP
             49F6 C6A3    MOV  @8(R3),*SP             ; Fetch from local activation record
             49F8 0008
             49FA 045C  \ B    *R12     ; PME fetch

; Load local address
LLA          49FC 8330  0 .WORD LDBIG   ; Get offset (word)
             49FE 0A13    SLA  R3,1                   ; Make byte
             4A00 0223  # AI   R3,0008  ; Point to data in local activation record
             4A02 0008
             4A04 C043  C MOV  R3,R1
             4A06 1005    JMP  ADRLOAD

; Short load local address
SLLA         4A08 4A0A    .WORD SLLA+2
             4A0A 0221  ! AI   R1,-182  ; Opcode *2 is 192 for SLLA1
             4A0C FF4A  J
             4A0E 0241  A ANDI R1,FFFE  ; Make address even
             4A10 FFFE
ADRLOAD      4A12 A049  I A    R9,R1    ; Add frame pointer
             4A14 064A  J DECT SP
             4A16 C681    MOV  R1,*SP
             4A18 045C  \ B    *R12     ; PME fetch

; Store local
STL          4A1A 8330  0 .WORD LDBIG
             4A1C 0A13    SLA  R3,1
             4A1E C043  C MOV  R3,R1
             4A20 1003    JMP  STOLOC

; Short store local word
SSTL         4A22 4A24    .WORD SSTL+2
             4A24 0221    LI              R1,-206
             4A26 FF32  2
STOLOC       4A28 A049  I A    R9,R1    ; Frame pointer
             4A2A C87A  z MOV  *SP+,@8(R1)
             4A2C 0008
             4A2E 045C  \ B    *R12     ; PME fetch

; Load global word
LDO          4A30 8330    .WORD LDIBG
             4A32 0A13    SLA  R3,1
             4A34 A0CE    A    R14,R3   ; Global frame pointer
             4A36 064A  J DECT SP
             4A38 C6A3    MOV  @8(R3),*SP
             4A3A 0008
             4A3C 045C  \ B    *R12     ; PME fetch

; Load address global
LAO          4A3E 8330    .WORD LDBIG
             4A40 0A13    SLA  R3,1
             4A42 A0CE    A    R14,R3   ; Global data frame pointer
             4A44 0223  # AI   R3,8
             4A46 0008
             4A48 064A  J DECT SP
```

```
             4A4A C683    MOV  R3,*SP
             4A4C 045C  \ B    *R12     ; PME fetch

; Store global
SRO          4A4E 8330    .WORD LDBIG
             4A50 0A13    SLA  R3,1
             4A52 A0CE    A    R14,R3   ; Global data frame pointer
             4A54 C8FA    MOV  *SP+,@8(R3)
             4A56 0008
             4A58 045C  \ B    *R12     ; PME fetch

; Short load intermediate 1
SLOD1        4A5A 8330    .WORD LDBIG   ; Get offset to R3
             4A5C 0204    LI   R4,1     ; Parent activation record
             4A5E 0001
             4A60 1005    JMP  LOD+2

; Short load intermediate 2
SLOD2        4A62 8330    .WORD LDBIG   ; Get offset to R3
             4A64 0204    LI   R4,2     ; Grandparent activation record
             4A66 0002
             4A68 1001    JMP  LOD+2

; Load intermediate word
LOD          4A6A 8328    .WORD LDUBIG  ; Get link count and offset
             4A6C C089    MOV  R9,R2                ; Local frame pointer
LOOP71       4A6E C092    MOV  *R2,R2   ; MSSTAT link from MSCW
             4A70 0604    DEC  R4
             4A72 15FD    JGT  LOOP71   ; Traverse activation records
             4A74 0A13    SLA  R3,1                 ; Offset
             4A76 A0C2    A    R2,R3
             4A78 064A  J DECT SP
             4A7A C6A3    MOV  @8(R3),*SP           ; Push word from activation record X levels up
             4A7C 0008
             4A7E 045C  \ B    *R12     ; PME fetch

; Load intermediate address
LDA          4A80 8328    .WORD LDUBIG  ; Get link count and offset
             4A82 C089    MOV  R9,R2                ; Local frame pointer
LOOP06       4A84 C092    MOV  *R2,R2   ; Traverse list of activation records
             4A86 0604    DEC  R4
             4A88 15FD    JGT  LOOP06
             4A8A 0A13    SLA  R3,1
             4A8C 0223  # AI   R3,8                 ; Offset in activation record
             4A8E 0008
             4A90 A0C2    A    R2,R3
             4A92 064A  J DECT SP
             4A94 C683    MOV  R3,*SP   ; Push address
             4A96 045C  \ B    *R12     ; PME fetch

; Store intermediate
STR          4A98 8328    .WORD LDUBIG
             4A9A C089    MOV  R9,R2                ; Local frame
LOOP05       4A9C C092    MOV  *R2,R2   ; Traverse list of activation records
             4A9E 0604    DEC  R4
             4AA0 15FD    JGT  LOOP05
             4AA2 0A13    SLA  R3,1
             4AA4 A0C2    A    R2,R3
             4AA6 C8FA    MOV  *SP+,@8(R3)          ; Into activation record data area
             4AA8 0008
             4AAA 045C  \ B    *R12     ; PME fetch

; Store
STO          4AAC 4AAE    .WORD STO+2
             4AAE C06A  j MOV  @2(SP),R1
             4AB0 0002
             4AB2 C47A  z MOV  *SP+,*R1
             4AB4 05CA    INCT SP
             4AB6 045C  \ B    *R12                ; PME fetch

; Find address with offset R3 in global activation record of segment R4
FNDOFFS      4AB8 0A14    SLA  R4,1                 ; Segment number to word
             4ABA A120    A    @EVECP,R4; Point into Environment vector
             4ABC 2AB8 *
             4ABE C114    MOV  *R4,R4   ; Global data pointer for segment
             4AC0 C124  $ MOV  @0(R4),R4
             4AC2 0000
             4AC4 0A13    SLA  R3,1
             4AC6 A103    A    R3,R4                ; Add offset
             4AC8 045B  [ B    *R11

; Load extended word
LDE          4ACA 8328    .WORD LDUBIG
             4ACC 06A0    BL   @FNDOFFS
```

```
             4ACE 4AB8 J
             4AD0 064A  J DECT SP
             4AD2 C6A4    MOV  @8(R4),*SP            ; Push global data from other segment
             4AD4 0008
             4AD6 045C  \ B    *R12     ; PME fetch

; Load extended address
LAE          4AD8 8328  ( .WORD LDUBIG
             4ADA 06A0    BL   @FNDOFFS
             4ADC 4AB8 J
             4ADE 0224  $ AI   R4,8                  ; MSCW offset
             4AE0 0008
             4AE2 064A  J DECT SP
             4AE4 C684    MOV  R4,*SP
             4AE6 045C  \ B    *R12                  ; PME fetch

; Store extended
STE          4AE8 8328    .WORD LDUBIG
             4AEA 06A0    BL   @FNDOFFS
             4AEC 4AB8 J
             4AEE C93A  : MOV  *SP+,@8(R4)
             4AF0 0008
             4AF2 045C  \ B    *R12                  ; PME fetch

; Load multiple words
LDM          4AF4 8316    .WORD LDUB
             4AF6 C808    MOV  R8,@IPCSAVE
             4AF8 2ACC *
             4AFA 0660  ` DECT @IPCSAVE
             4AFC 2ACC *
             4AFE 0A13    SLA  R3,1
             4B00 C803    MOV  R3,@STKMISS
             4B02 2AF0 *
             4B04 C820    MOV  @ERECP,@CUREREC
             4B06 2AB4 *
             4B08 2AE0 *
             4B0A 06A0    BL   @CHKSTK
             4B0C 4E98 N
             4B0E C0BA    MOV  *SP+,R2
             4B10 A0A0    A    @STKMISS,R2
             4B12 2AF0 *
             4B14 0642  B DECT R2
             4B16 0283    CI   R3,0000
             4B18 0000
             4B1A 1305    JEQ  EXLDM
LOOP19       4B1C 064A  J DECT SP
             4B1E C692    MOV  *R2,*SP
             4B20 0642  B DECT R2
             4B22 0643  C DECT R3
             4B24 15FB    JGT  LOOP19
EXLDM        4B26 045C  \ B    *R12     ; PME fetch

; Store multiple words
STM          4B28 8316    .WORD LDUB
             4B2A 1307    JEQ  ZCNT01   ; Zero count
             4B2C C04A  J MOV  SP,R1
             4B2E A043  C A    R3,R1
             4B30 A043  C A    R3,R1
             4B32 C051  Q MOV  *R1,R1
LOOP07       4B34 CC7A  z MOV  *SP+,*R1+
             4B36 0603    DEC  R3
             4B38 15FD    JGT  LOOP07
ZCNT01       4B3A 05CA    INCT SP
             4B3C 045C  \ B    *R12     ; PME fetch

; Load static link
LSL          4B3E 8316    .WORD LDUB
             4B40 C089    MOV  R9,R2    ; Local frame pointer
             4B42 C0C3    MOV  R3,R3
             4B44 1303    JEQ  DONE01
LOOP08       4B46 C092    MOV  *R2,R2
             4B48 0603    DEC  R3
             4B4A 15FD    JGT  LOOP08
DONE01       4B4C 064A  J DECT SP
             4B4E C682    MOV  R2,*SP
             4B50 045C  \ B    *R12     ; PME fetch

; Load constant byte
LDCB         4B52 8316    .WORD LDUB
             4B54 064A  J DECT SP
             4B56 C683    MOV  R3,*SP
             4B58 045C  \ B    *R12                  ; PME fetch

; Move
```

```
MOV          4B5A 8328    .WORD LDUBIG
             4B5C C043  C MOV  R3,R1                 ; Words to move
             4B5E 1317    JEQ  ZCNT02
             4B60 C0FA    MOV  *SP+,R3  ; Address or offset
             4B62 C0BA    MOV  *SP+,R2  ; Address
             4B64 C144  D MOV  R4,R5                 ; Type
             4B66 1303    JEQ  BLOCK                 ; Block address
             4B68 A0E0    A    @SEGBASE,R3           ; Constant block in current segment
             4B6A 2AE2 *
             4B6C 1011    JMP  LDCCON4

BLOCK        4B6E 8820    C    @PMEFLIP,@CONTRUE     ; Check flip indicator for current segment
             4B70 2AE8 *
             4B72 40B6 @
             4B74 1309    JEQ  LOOP09   ; Same sex, just continue
             4B76 0285    CI   R5,2                  ; Byte flipping if mode=2
             4B78 0002
             4B7A 1606    JNE  LOOP09   ; No byte filpping requested
LOOP10       4B7C CCB3    MOV  *R3+,*R2+; Opposite sex, do byte flipping
             4B7E 06E2    SWPB @-2(R2)  ; Flip "behind the back"
             4B80 FFFE
             4B82 0601    DEC  R1
             4B84 15FB    JGT  LOOP10
             4B86 045C  \ B    *R12     ; PME fetch
LOOP09       4B88 CCB3    MOV  *R3+,*R2+; Just move, no flipping
             4B8A 0601    DEC  R1
             4B8C 15FD    JGT  LOOP09
ZCNT02       4B8E 045C  \ B    *R12     ; PME fetch

LDCCON4      4B90 C3CF    MOV  R15,R15  ; Code type flag
             4B92 13ED    JEQ  BLOCK                 ; CPU RAM
             4B94 C108    MOV  R8,R4
             4B96 C203    MOV  R3,R8
             4B98 06A0    BL   @LOADAD1 ; Set code address
             4B9A 4144 AD
             4B9C C020    MOV  @PMEFLIP,R0
             4B9E 2AE8 *
             4BA0 160C    JNE  LOOP04
             4BA2 0285    CI   R5,2                  ; Flip request mode
             4BA4 0002
             4BA6 1609    JNE  LOOP04   ; No flipping
LOOP03       4BA8 D01D    MOVB *R13,R0  ; Code read data, byte flipping
             4BAA 06C0    SWPB R0
             4BAC D01D    MOVB *R13,R0  ; Code read data
             4BAE CC80    MOV  R0,*R2+
             4BB0 0601    DEC  R1
             4BB2 15FA    JGT  LOOP03
LDCCON5      4BB4 C204    MOV  R4,R8                 ; Restore IPC
             4BB6 0460  ` B    @LOADADDR; Load address to p-code and execute
             4BB8 4142 AB

LOOP04       4BBA D01D    MOVB *R13,R0  ; Code read data, no byte flipping
             4BBC 1000    NOP
             4BBE D81D    MOVB *R13,@PASCALWS+1      ; Code read data
             4BC0 8381
             4BC2 CC80    MOV  R0,*R2+
             4BC4 0601    DEC  R1
             4BC6 15F9    JGT  LOOP04
             4BC8 10F5    JMP  LDCCON5

CON008       4BCA 0008    .WORD 8

; Load constant real
LDCRL        4BCC 4BCE K  .WORD LDCRL+2
             4BCE C808    MOV  R8,@IPCSAVE           ; Set up for stack fault check
             4BD0 2ACC *
             4BD2 0620    DEC  @IPCSAVE
             4BD4 2ACC *
             4BD6 0202    LI   R2,CONT02
             4BD8 4BDE K
             4BDA 0460  ` B    @LDBIG   ; -> R3
             4BDC 8330  0
CONT02       4BDE 0201    LI   R1,4                  ; A real is four words long
             4BE0 0004
             4BE2 04C5    CLR  R5
             4BE4 C820    MOV  @CON008,@STKMISS      ; Eight bytes
             4BE6 4BCA K
             4BE8 2AF0 *
             4BEA 1016    JMP  LDCON3

; Load multiple word constant
LDC          4BEC 4BEE    .WORD LDC+2
             4BEE C808    MOV  R8,@IPCSAVE           ; Save in case of stack fault
             4BF0 2ACC *
```

```
             4BF2 0620    DEC  @IPCSAVE
             4BF4 2ACC *
             4BF6 0202    LI   R2,LDCCON1
             4BF8 4BFE K
             4BFA 0460  ` B    @LDUBIG  ; Mode and word offset to R4, R3
             4BFC 8328  (
LDCCON1      4BFE C144  D MOV  R4,R5                 ; Mode
             4C00 C103    MOV  R3,R4                 ; Offset in constant pool of current segment
             4C02 0202    LI   R2,LDCCON2
             4C04 4C0A L
             4C06 0460  ` B    @LDUB                 ; Word count->R3
             4C08 8316
LDCCON2      4C0A C043  C MOV  R3,R1                 ; Save word count
             4C0C 1601    JNE  LDCCON6  ; Word count not 0?
             4C0E 045C  \ B    *R12                  ; PME fetch
LDCCON6      4C10 0A13    SLA  R3,1                  ; Make into bytes
             4C12 C803    MOV  R3,@STKMISS           ; To stack fault data missing
             4C14 2AF0 *
             4C16 C0C4    MOV  R4,R3                 ; Constant pool offset
LDCON3       4C18 C820    MOV  @ERECP,@CURERE        ; E_Rec_P to current environment
             4C1A 2AB4 *
             4C1C 2AE0 *
             4C1E 06A0    BL   @CHKSTK
             4C20 4E98 N
             4C22 6281 b  S    R1,SP    ; Subtract word count*2
             4C24 6281 b  S    R1,SP
             4C26 C08A    MOV  SP,R2
             4C28 0A13    SLA  R3,1                  ; Make offset in bytes
             4C2A A0E0    A    @SEGCNAB,R3           ; Add current segment constant pool pointer (absolute)
             4C2C 2ABE *
             4C2E 10B0    JMP  LDCCON4

; Load constant offset
LCO          4C30 8330    .WORD LDBIG
             4C32 0A13    SLA  R3,1
             4C34 A0E0    A    @SEGCNRL,R3           ; Current segment constant pool pointer (relative)
             4C36 2AC0 *
             4C38 064A  J DECT SP
             4C3A C683    MOV  R3,*SP
             4C3C 045C  \ B    *R12     ; PME fetch

; Swap at top of stack
SWAP         4C3E 4C40    .WORD SWAP+2
             4C40 C05A  Z MOV  *SP,R1
             4C42 C6AA    MOV  @2(SP),*SP
             4C44 0002
             4C46 CA81    MOV  R1,@2(SP)
             4C48 0002
             4C4A 045C  \ B    *R12     ; PME fetch

; Duplicate one word
DUPI         4C4C 4C4E    .WORD DUPI+2
             4C4E 064A  J DECT SP
             4C50 C6AA    MOV  @2(SP),*SP
             4C52 0002
             4C54 045C  \ B    *R12     ; PME fetch

; Duplicate real
DUPR         4C56 4C58    .WORD DUPR+2
             4C58 C04A  J MOV  SP,R1
             4C5A 022A  * AI   SP,-8
             4C5C FFF8
             4C5E C08A    MOV  SP,R2
             4C60 CCB1    MOV  *R1+,*R2+
             4C62 CCB1    MOV  *R1+,*R2+
             4C64 CCB1    MOV  *R1+,*R2+
             4C66 C491    MOV  *R1,*R2
             4C68 045C  \ B    *R12     ; PME fetch

; Load processor register
; Loads/stores in following registers
; -1 CURTASK
; -2 EVEC
; -3 READYQ
; If top of stack isn't negative access is into TIB

LPR          4C6A 4C6C    .WORD LPR+2
             4C6C C07A  z MOV  *SP+,R1
             4C6E 0A11    SLA  R1,1
             4C70 1107    JLT  REGNEG1  ; If negative just fetch register
             4C72 A060  ` A    @CURTSK,R1
             4C74 2ABA *
             4C76 06A0    BL   @UPDTIB  ; Update register copies in Task Info Block
             4C78 579A W
```

```
             4C7A 064A  J DECT SP
             4C7C C691    MOV  *R1,*SP
             4C7E 045C  \ B    *R12     ; PME fetch
REGNEG1      4C80 064A  J DECT SP
             4C82 C6A1    MOV  @NATCALL(R1),*SP
             4C84 2ABC *
             4C86 045C  \ B    *R12     ; PME fetch

; Store processor register
SPR          4C88 4C8A    .WORD SPR+2
             4C8A C03A  : MOV  *SP+,R0
             4C8C C07A  z MOV  *SP+,R1
             4C8E 0A11    SLA  R1,1
             4C90 1109    JLT  REGNEG2
             4C92 06A0    BL   @UPDTIB  ; Update register copies in Task Info Block
             4C94 579A W
             4C96 A060  ` A    @CURTSK,R1
             4C98 2ABA *
             4C9A C440  @ MOV  R0,*R1
             4C9C 020B    LI   R11,LOADADDR          ; Fake return address
             4C9E 4142 AB
             4CA0 0460  ` B    @STRTTSK
             4CA2 57BE W
REGNEG2      4CA4 C840  @ MOV  R0,@NATCALL(R1)
             4CA6 2ABC *
             4CA8 045C  \ B    *R12     ; PME fetch

; Short call intermediate procedure
SCPI1        4CAA 8316    .WORD LDUB
             4CAC 0204    LI   R4,1
             4CAE 0001
             4CB0 1003    JMP  SCPIX

SCPI2        4CB2 8316    .WORD LDUB
             4CB4 0204    LI   R4,2
             4CB6 0002
SCPIX        4CB8 C808    MOV  R8,@IPCSAVE
             4CBA 2ACC *
             4CBC 0660  ` DECT @IPCSAVE
             4CBE 2ACC *
             4CC0 020B    LI   R11,CLP02
             4CC2 4CF2 L
             4CC4 0460  ` B    @TRAVLOC ; Traverse activation records
             4CC6 4DB4 M

; Call intermediate procedure
CIP          4CC8 830E    .WORD LDUBB
             4CCA C088    MOV  R8,R2
             4CCC 0222  " AI   R2,-3
             4CCE FFFD
             4CD0 C802    MOV  R2,@IPCSAVE
             4CD2 2ACC *
             4CD4 020B    LI   R11,CLP02
             4CD6 4CF2 L
             4CD8 0460  ` B    @TRAVLOC ; Traverse activation records
             4CDA 4DB4 M

; Call local procedure
CLP          4CDC 8316    .WORD LDUB
             4CDE C809    MOV  R9,@GLOBPNT           ; Local frame pointer
             4CE0 2AEC *
             4CE2 1003    JMP  CLP01

; Call global procedure
CGP          4CE4 8316    .WORD LDUB                 ; Procedure number -> R3
             4CE6 C80E    MOV  R14,@GLOBPNT          ; Global frame pointer
             4CE8 2AEC *
CLP01        4CEA C808    MOV  R8,@IPCSAVE           ; Save IPC
             4CEC 2ACC *
             4CEE 0660  ` DECT @IPCSAVE ; Back two bytes (CGP, UB)
             4CF0 2ACC *
CLP02        4CF2 C120    MOV  @DICTPNT,R4           ; Dictionary pointer
             4CF4 2AE4 *
             4CF6 C160  ` MOV  @ERECP,R5
             4CF8 2AB4 *
             4CFA C1A0    MOV  @SEGBASE,R6
             4CFC 2AE2 *
             4CFE 06A0    BL   @LOADSEG ; Find the address (load segment if necessary)
             4D00 4DC6 M
             4D02 0460  ` B    @LOADADDR; Load address to p-code and execute
             4D04 4142 AB

; Call external intermediate
CXI          4D06 8316    .WORD LDUB
```

```
             4D08 C808    MOV  R8,@IPCSAVE
             4D0A 2ACC *
             4D0C 0660  ` DECT @IPCSAVE
             4D0E 2ACC *
             4D10 C143  C MOV  R3,R5
             4D12 0202    LI   R2,CONT03
             4D14 4D1A M
             4D16 0460  ` B    @LDUBB
             4D18 830E
CONT03       4D1A 020B    LI   R11,CALX1
             4D1C 4D6A Mj
             4D1E 0460  ` B    @TRAVLOC ; Traverse activation records
             4D20 4DB4 M

; Call external local
CXL          4D22 830E    .WORD LDUBB
             4D24 C809    MOV  R9,@GLOBPNT            ; Local frame pointer
             4D26 2AEC *
             4D28 101A    JMP  CALX2

; Short call external global 1. Segment 1 is Kernel.
SCXG1        4D2A 8316    .WORD LDUB                  ; Procedure number -> R3
             4D2C 06A0    BL   @CALLKRN
             4D2E 4DA0 M
             4D30 C160  ` MOV  @CON001,R5             ; Segment number
             4D32 40B0 @
             4D34 1005    JMP  CALX3

; Short call external global
SCXG         4D36 8316    .WORD LDUB                  ; Procedure number -> R3
             4D38 0911    SRL  R1,1
             4D3A 0221  ! AI   R1,-111  ; To get the number from the opcode
             4D3C FF91
             4D3E C141  A MOV  R1,R5
CALX3        4D40 C808    MOV  R8,@IPCSAVE            ; Prepare for possible segment fault
             4D42 2ACC *
             4D44 0660  ` DECT @IPCSAVE
             4D46 2ACC *
             4D48 C80E    MOV  R14,@GLOBPNT
             4D4A 2AEC *
             4D4C 100E    JMP  CALX1

; Call external global
CXG          4D4E 830E    .WORD LDUBB
             4D50 0284    CI   R4,1                   ; R4 Segment number, R3 global procedure number
             4D52 0001
             4D54 1602    JNE  NOKRN
             4D56 06A0    BL   @CALLKRN ; Segment #1 is kernel procedures
             4D58 4DA0 M
NOKRN        4D5A C80E    MOV  R14,@GLOBPNT           ; Global frame pointer
             4D5C 2AEC *
CALX2        4D5E C088    MOV  R8,R2                  ; IPC Prepare for possible stack fault
             4D60 0222  " AI   R2,-3
             4D62 FFFD
             4D64 C802    MOV  R2,@IPCSAVE            ; Back IPC 3 bytes (length of CXG UB1, UB2)
             4D66 2ACC *
             4D68 C144  D MOV  R4,R5                  ; Segment number -> R5
CALX1        4D6A 06A0    BL   @VIASEGT ; Find segment via Environment vector
             4D6C 4EC4 N

; Now the segment with the procedure to call is determined. Find it, load if needed and then execute
CALX4        4D6E C184    MOV  R4,R6                  ; Segment position in code pool
             4D70 06A0    BL   @SETPNT  ; Set pointers to segment information
             4D72 4FCA O
             4D74 06A0    BL   @LOADSEG ; Find the address (load segment if necessary)
             4D76 4DC6 M
             4D78 06A0    BL   @TIBUPD  ; Update TIB for procedure called
             4D7A 4E6C Nl
             4D7C 0460  ` B    @LOADADDR; Load address of code to execute
             4D7E 4142 AB

; Call formal procedure
CFP          4D80 4D82    .WORD CFP+2
             4D82 C808    MOV  R8,@IPCSAVE            ; Prepare for fault
             4D84 2ACC *
             4D86 0620    DEC  @IPCSAVE ; Point to same p-code again
             4D88 2ACC *
             4D8A 05A0    INC  @FCALLFL
             4D8C 2ACE *
             4D8E C16A  j MOV  @2(SP),R5; Environment record pointer
             4D90 0002
             4D92 06A0    BL   @GETSEGB ; Find base of segment via E_Rec_P
             4D94 4ECC N
             4D96 C0DA    MOV  *SP,R3   ; Procedure number
```

```
             4D98 C82A  * MOV  @MSIPC(SP),@GLOBPNT    ; Static link field in new MSCW
             4D9A 0004
             4D9C 2AEC *
             4D9E 10E7    JMP  CALX4                 ; Locate, load and call the procedure

; Call a Kernel procedure with the number in R3
CALLKRN      4DA0 0283    CI   R3,38                 ; Number of Kernel procedures ; R3 is procedure number
             4DA2 0026  &
             4DA4 1506    JGT  RET01
             4DA6 0A13    SLA  R3,1                  ; Make word
             4DA8 C0A3    MOV  @KERNPROC-2(R3),R2    ; Kernel procedure table index
             4DAA 2FB0 /
             4DAC 1301    JEQ  NOLINK   ; Not implemented?
             4DAE 0452  R B    *R2                   ; Call Kernel procedure
NOLINK       4DB0 0813    SRA  R3,1                  ; Back to byte
RET01        4DB2 045B  [ B    *R11

; Traverse stack activation records
TRAVLOC      4DB4 C089    MOV  R9,R2                 ; Local frame pointer
             4DB6 C104    MOV  R4,R4                 ; Level count
             4DB8 1303    JEQ  ENDLEV
LOOP11       4DBA C092    MOV  *R2,R2
             4DBC 0604    DEC  R4                    ; Count the levels
             4DBE 15FD    JGT  LOOP11
ENDLEV       4DC0 C802    MOV  R2,@GLOBPNT
             4DC2 2AEC *
             4DC4 045B  [ B    *R11

; Called when Calling a procedure to load a segment when known which one is desired
LOADSEG      4DC6 C80B    MOV  R11,@RETSAV2
             4DC8 2ADC *
             4DCA C043  C MOV  R3,R1                 ; Procedure number to call
             4DCC 0A11    SLA  R1,1                  ; Make byte
             4DCE 0501    NEG  R1                    ; Procedure dictionary is stored backwards
             4DD0 A044  D A    R4,R1                 ; Start of procedure dictionary
             4DD2 06A0    BL   @GETDAT2 ; Get pointer to procedure to call
             4DD4 4178 Ax
             4DD6 0A12    SLA  R2,1                  ; Make byte
             4DD8 A086    A    R6,R2                 ; Add segment's position in code pool
             4DDA C042  B MOV  R2,R1                 ; Pointer to procedure -> R1
             4DDC 06A0    BL   @GETDAT2 ; Get procedure's data size
             4DDE 4178 Ax
             4DE0 C801    MOV  R1,@SAVPACK
             4DE2 2AF2 *
             4DE4 04E0    CLR  @NATCALL ; Native call flag
             4DE6 2ABC *
             4DE8 0A12    SLA  R2,1                  ; Make datasize to bytes
             4DEA 1504    JGT  DATPOS
             4DEC 1303    JEQ  DATPOS
             4DEE 0502    NEG  R2
             4DF0 05A0    INC  @NATCALL ; Indicate data size negative, native call
             4DF2 2ABC *
DATPOS       4DF4 0222  " AI   R2,10                 ; Add space required for MSCW
             4DF6 000A
             4DF8 C020    MOV  @FLTCNT,R0            ; Fault counter
             4DFA 2AEA *
             4DFC 1606    JNE  HASFLT   ; Fault counter>0
             4DFE C802    MOV  R2,@STKMISS           ; Number of words needed on stack
             4E00 2AF0 *
             4E02 C805    MOV  R5,@CUREREC           ; E_Rec of called segment
             4E04 2AE0 *
             4E06 06A0    BL   @CHKSTK  ; Is there room?
             4E08 4E98 N
HASFLT       4E0A 04E0    CLR  @FLTCNT
             4E0C 2AEA *
             4E0E C020    MOV  @FCALLFL,R0           ; Formal procedure call has three arguments on stack
             4E10 2ACE *
             4E12 1304    JEQ  NOFORM
             4E14 022A  * AI   SP,6                  ; Pop three words used by CFP
             4E16 0006
             4E18 04E0    CLR  @FCALLFL
             4E1A 2ACE *
NOFORM       4E1C C020    MOV  @NATCALL,R0
             4E1E 2ABC *
             4E20 130B    JEQ  PCODCAL
             4E22 04E0    CLR  @NATCALL
             4E24 2ABC *
             4E26 0691    BL   *R1      ; Call external procedure as assembly (native call)
             4E28 C160  ` MOV  @ERECP,R5
             4E2A 2AB4 *
             4E2C 06A0    BL   @GETSEGB ; Find base of segment via E_Rec_P
             4E2E 4ECC N
             4E30 06A0    BL   @SETPNT  ; Set pointers to segment information
             4E32 4FCA O
```

```
             4E34 0460  ` B    @LOADADDR; Load address to p-code and execute
             4E36 4142 AB

PCODCAL      4E38 6282 b  S    R2,SP                  ; Allocate activation record
             4E3A 6220 b  S    @SEGBASE,R8            ; IPC should be segment relative in MSCW
             4E3C 2AE2 *
             4E3E CAA0    MOV  @CURPRNO-1,@MSPROC(SP) ; Procedure number of caller -> MSPROC
             4E40 2AC2 *
             4E42 0008
             4E44 CAA0    MOV  @ERECP,@MSENV(SP)                   ; Caller's E_Rec_Pointer -> MSENV
             4E46 2AB4 *
             4E48 0006
             4E4A CA88    MOV  R8,@MSIPC(SP)          ; Program counter at caller; -> MSIPC
             4E4C 0004
             4E4E CA89    MOV  R9,@MSDYN(SP)          ; Activation record of caller  -> MSDYN
             4E50 0002
             4E52 CAA0    MOV  @GLOBPNT,@MSSTAT(SP)   ; Global data pointer of lexical parent  -> MSSTAT
             4E54 2AEC *
             4E56 0000
             4E58 C201    MOV  R1,R8                  ; Procedure dictionary
             4E5A C803    MOV  R3,@CURPRNO-1          ; Procedure number to call
             4E5C 2AC2 *
             4E5E C24A  J MOV  SP,R9
             4E60 064A  J DECT SP
             4E62 C6A0    MOV  @SAVPACK,*SP           ; Pointer to procedure to call
             4E64 2AF2 *
             4E66 C2E0    MOV  @RETSAV2,R11
             4E68 2ADC *
             4E6A 045B  [ B    *R11

; Updated Task Information Block on procedure call
TIBUPD       4E6C C805    MOV  R5,@ERECP; Update E_Rec_P
             4E6E 2AB4 *
             4E70 C825  % MOV  @2(R5),@EVECP          ; Update E_Vec_P
             4E72 0002
             4E74 2AB8 *
             4E76 C3A5    MOV  @0(R5),R14             ; Global data frame pointer
             4E78 0000
             4E7A C165  e MOV  @4(R5),R5; Update SIB_P
             4E7C 0004
             4E7E C805    MOV  R5,@CURSIB
             4E80 2AB2 *
             4E82 C806    MOV  R6,@SEGBASE            ; Segment's position in code pool
             4E84 2AE2 *
             4E86 C0A0    MOV  @CURSIB,R2
             4E88 2AB2 *
             4E8A A8A0    A    @STAMP,@4(R2)          ; Time stamp increment in SIB
             4E8C 40B4 @
             4E8E 0004
             4E90 1902    JNO  NOROL2
             4E92 0460  ` B    @ROLLOVR ; Handle time stamp rollover
             4E94 5A60 Z`
NOROL2       4E96 045B  [ B    *R11

; Stack fault check
CHKSTK       4E98 C120    MOV  @CURTSK,R4             ; Current TIB
             4E9A 2ABA *
             4E9C C124  $ MOV  @4(R4),R4; SP_Low
             4E9E 0004
             4EA0 0224  $ AI   R4,80D   ; 80 bytes, 40 words
             4EA2 0050  P
             4EA4 1805    JOC  STKFAUL
             4EA6 A120    A    @STKMISS,R4            ; Add missing words
             4EA8 2AF0 *
             4EAA 1802    JOC  STKFAUL
             4EAC 8284    C    R4,SP                  ; What about standard stack?
             4EAE 1A09    JL   CHKRET
STKFAUL      4EB0 0204    LI   R4,80D   ; 80 bytes, 40 words
             4EB2 0050  P
             4EB4 A804    A    R4,@STKMISS            ; Bytes needed plus 80
             4EB6 2AF0 *
             4EB8 C220    MOV  @IPCSAVE,R8            ; The instruction that started it all
             4EBA 2ACC *
             4EBC 06A0    BL   @FAULGEN
             4EBE 484A HJ
             4EC0 0081    .WORD 81H     ; Data stack fault
CHKRET       4EC2 045B  [ B    *R11

; Entry when finding the environment record via the environment vector (segment table)
; Like when called from SIGNAL or WAIT.

VIASEGT      4EC4 0A15    SLA  R5,1                   ; Segment number to word
             4EC6 A160  ` A    @EVECP,R5; Point to array of E_Rec_P
             4EC8 2AB8 *
```

```
             4ECA C155  U MOV  *R5,R5   ; Fetch address to E_Rec for called segment -> R5

; Finds the segment base
GETSEGB      4ECC C00F    MOV  R15,R0   ; Code type
             4ECE C065  e MOV  @4(R5),R1; Called segment's SIB_P -> R1
             4ED0 0004
             4ED2 C111    MOV  *R1,R4   ; Segment's position in code pool
             4ED4 1302    JEQ  NOTMEM   ; Not in memory?
             4ED6 04C2    CLR  R2                    ; Segment is in memory
             4ED8 105D  ] JMP  SINMEM

; Required segment is not in memory
NOTMEM       4EDA C0A1    MOV  @32D(R1),R2           ; M_Type -> R2
             4EDC 0020
             4EDE 1305    JEQ  PSEUDO   ; Pseudo? Because if it is, it may be one that can run from memory
without loading it.
SEGFAUL      4EE0 C220    MOV  @IPCSAVE,R8
             4EE2 2ACC *
             4EE4 06A0    BL   @FAULGEN ; Create a segment fault to get it loaded
             4EE6 484A HJ
             4EE8 0080    .WORD 80H                  ; Segment fault

PSEUDO       4EEA C121  ! MOV  @22D(R1),R4           ; VIP (Volume information pointer) Disk drive info.
             4EEC 0016
             4EEE C114    MOV  *R4,R4   ; Volume number
             4EF0 0284    CI   R4,13D   ; Compare to 13
             4EF2 000D
             4EF4 1103    JLT  STDDSK   ; Standard disk?
             4EF6 0284    CI   R4,31D
             4EF8 001F
             4EFA 112A  * JLT  NOTSTD   ; Volume # <31?

; Volume number not in [13..30]
STDDSK       4EFC C121  ! MOV  @2(R1),R4; Number of active calls to segment
             4EFE 0002
             4F00 1624  $ JNE  HASCALL
             4F02 C0A0    MOV  @CURSIB,R2            ; No active calls
             4F04 2AB2 *
             4F06 C803    MOV  R3,@SAVPACK
             4F08 2AF2 *
             4F0A 04C4    CLR  R4
             4F0C C0E2    MOV  @2(R2),R3; Active calls current SIB
             4F0E 0002
             4F10 1302    JEQ  4F16
             4F12 C122  " MOV  @18D(R2),R4           ; Segment length current segment (words)
             4F14 0012
             4F16 C0A0    MOV  @SIBPNT,R2            ; SIB pointer in SYSCOM
             4F18 2B88 +
             4F1A 1308    JEQ  4F2C
LOOP56       4F1C C0E2    MOV  @8(R2),R3; Residency
             4F1E 0008
             4F20 1302    JEQ  CANSWAP  ; Swappable if zero
             4F22 A122  " A    @18D(R2),R4           ; Make segment length bytes by adding to itself
             4F24 0012
CANSWAP      4F26 C0A2    MOV  @28D(R2),R2           ; Next SIB in list
             4F28 001C
             4F2A 16F8    JNE  LOOP56   ; There is another linked?
             4F2C C0E0    MOV  @SAVPACK,R3
             4F2E 2AF2 *
             4F30 A121  ! A    @18D(R1),R4           ; Segment length (words)
             4F32 0012
             4F34 C0A0    MOV  @TOPMEM,R2            ; Calculate space in primary code pool
             4F36 2782 '
             4F38 60A0 `  S    @INTMEM,R2            ; R2 is now topmem-intmem (bytes)
             4F3A 2780 '
             4F3C 0912    SRL  R2,1                  ; Make words
             4F3E 8084    C    R4,R2                 ; More space needed than available?
             4F40 15CF    JGT  SEGFAUL
             4F42 C08B    MOV  R11,R2   ; Save return address
             4F44 06A0    BL   @SEGLOAD ; Load segment

             4F46 5A9E Z
             4F48 C2C2    MOV  R2,R11   ; Reload return address
HASCALL      4F4A 0202    LI   R2,2                  ; User's program segment number
             4F4C 0002
             4F4E 1022  " JMP  SINMEM

; Not in standard disk unit, so [13..30]
; #13 is assigned to volume loaded from tape.
; #14 is OS:, GROM-disk on p-code card
; #15 - #30 are prepared to be p-code in GROM on cartrigdes starting at address 6000H
NOTSTD       4F50 0224  $ AI   R4,-12   ; Base index set to 0
             4F52 FFF4
             4F54 0A14    SLA  R4,1                  ; Make byte
```

```
              4F56 C084    MOV  R4,R2
              4F58 C121  ! MOV  @20D(R1),R4           ; Segment's disk address (word)
              4F5A 0014
              4F5C 05C4    INCT R4                    ; Pass something (two words)
              4F5E 0A14    SLA  R4,1                  ; Make byte
              4F60 A122  " A    @ADDROFS(R2),R4
              4F62 2D6C -l
              4F64 C062  b MOV  @CODREAD(R2),R1
              4F66 2CFA ,
              4F68 C822  " MOV  @CODETYP(R2),@SAVPACK
              4F6A 2D20 -
              4F6C 2AF2 *
              4F6E 1107    JLT  VDPADD   ; -1 means VDP
              4F70 D844  D MOVB R4,@WAOFFS(R1)         ; Load address GROM style
              4F72 0402
              4F74 1000    NOP
              4F76 D860  ` MOVB @PASCALWS+9,@WAOFFS(R1)
              4F78 8389
              4F7A 0402
              4F7C 1007    JMP  GETDATA

VDPADD        4F7E D860  ` MOVB @PASCALWS+9,@WAOFFS(R1)             ; Load address VDP style
              4F80 8389
              4F82 0402
              4F84 1000    NOP
              4F86 D844  D MOVB R4,@WAOFFS(R1)
              4F88 0402
              4F8A 1000    NOP
GETDATA       4F8C D111    MOVB *R1,R4   ; Read word to R4, no flipping
              4F8E 1000    NOP
              4F90 D811    MOVB *R1,@PASCALWS+9
              4F92 8389

; Segment is in memory
SINMEM        4F94 C060  ` MOV  @MTYPFL,R1            ; Memory type flag for segment
              4F96 2AD6 *
              4F98 1307    JEQ  MEMCHG
              4F9A C062  b MOV  @CODREAD(R2),R1       ; New code read address
              4F9C 2CFA ,
              4F9E C022  " MOV  @CODETYP(R2),R0       ; New code memory type
              4FA0 2D20 -
              4FA2 04E0    CLR  @MTYPFL  ; Memory type flag
              4FA4 2AD6 *
              4FA6 045B  [ B    *R11

; Change code memory type for the code.
; Will exchange PME core in RAM PAD if needed

MEMCHG        4FA8 C362  b MOV  @CODREAD(R2),R13      ; Code read address table
              4FAA 2CFA ,
              4FAC C3E2    MOV  @CODETYP(R2),R15      ; Code type table
              4FAE 2D20 -
              4FB0 280F (  XOR  R15,R0                ; Is the PME core equivalent
              4FB2 0910    SRL  R0,1
              4FB4 1709    JNC  MEMEQV                ; Yes
              4FB6 C062  b MOV  @PMECORE(R2),R1       ; No, exchange it
              4FB8 2D46 -F
              4FBA 0202    LI   R2,FETCH
              4FBC 8300
              4FBE 0200    LI   R0,0025H ; Size of core
              4FC0 0025  %
LOOP27        4FC2 CCB1    MOV  *R1+,*R2+
              4FC4 0600    DEC  R0
              4FC6 15FD    JGT  LOOP27
MEMEQV        4FC8 045B  [ B    *R11

; Sets pointers to constant pool and dictionary on segment switch
SETPNT        4FCA C044  D MOV  R4,R1                 ; Segment base
              4FCC 0221  ! AI   R1,12D   ; Code sex indicator (executable code segment format)
              4FCE 000C
              4FD0 05A0    INC  @PMEFLIP ; Set to no flipping
              4FD2 2AE8 *
              4FD4 C00B    MOV  R11,R0
              4FD6 06A0    BL   @GETDAT2 ; Read code segment byte sex to R2
              4FD8 4178 Ax
              4FDA 04E0    CLR  @PMEFLIP ; Assume flipping
              4FDC 2AE8 *
              4FDE 0282    CI   R2,0001  ; Byte sex native?
              4FE0 0001
              4FE2 1602    JNE  DOFLIP   ; No, flipping needed
              4FE4 05A0    INC  @PMEFLIP ; No flipping
              4FE6 2AE8 *
DOFLIP        4FE8 06A0    BL   @GETDAT2 ; Read constant pool pointer to R2
              4FEA 4178 Ax
```

```
             4FEC 0A12    SLA  R2,1                  ; To byte
             4FEE C802    MOV  R2,@SEGCNRL           ; Segment constant pool pointer relative
             4FF0 2AC0 *
             4FF2 A084    A    R4,R2                 ; Add start of segment in code pool
             4FF4 C802    MOV  R2,@SEGCNAB           ; Segment constant pool pointer absolute
             4FF6 2ABE *
             4FF8 C044  D MOV  R4,R1                 ; Set address to beginning of segment
             4FFA 1002    JMP  PAGE2A
             4FFC 0000    .WORD
             4FFE AE03    .WORD

; ---------------------------------------------------------------------------
; Beginning of the paged ROM. This is standard page (when CRU bit 1F80 is cleared).

PAGE2A       5000 06A0    BL   @GETDAT2 ; Read procedure dictionary pointer to R2
             5002 4178 Ax
             5004 0A12    SLA  R2,1                  ; Make byte
             5006 A102    A    R2,R4                 ; Add to segment base
             5008 C804    MOV  R4,@DICTPNT           ; Load procedure dictionary pointer for segment
             500A 2AE4 *
             500C 0450  P B    *R0                   ; Saved return address

; Return from procedure
RPU          500E 5010    .WORD RPU+2
             5010 C808    MOV  R8,@IPCSAVE           ; Because if the segment to return to is no longer in
memory, there may be a fault.
             5012 2ACC *
             5014 0620    DEC  @IPCSAVE
             5016 2ACC *
             5018 0202    LI   R2,CONT04
             501A 5020 P
             501C 0460  ` B    @LDBIG   ; More words to remove from stack, if needed.
             501E 8330  0
CONT04       5020 8829  ) C    @MSENV(R9),@ERECP      ; MSENV, returning within the same segment?
             5022 0006
             5024 2AB4 *
             5026 1309    JEQ  SAMESEG
             5028 C169  i MOV  @MSENV(R9),R5          ; MSENV -> R5
             502A 0006
             502C 06A0    BL   @GETSEGB ; Find base of segment to return to via E_Rec_P
             502E 4ECC N
             5030 C184    MOV  R4,R6
             5032 06A0    BL   @SETPNT  ; Set pointers to segment information
             5034 4FCA O
             5036 06A0    BL   @TIBUPD  ; Update TIB for procedure called
             5038 4E6C Nl
SAMESEG      503A C289    MOV  R9,SP                 ; Local environment pointer to spack pointer
             503C C82A  * MOV  @MSPROC(SP),@CURPRNO-1 ;Procedure number
             503E 0008
             5040 2AC2 *
             5042 1103    JLT  NEGPROC
             5044 C22A  * MOV  @MSIPC(SP),R8          ; Restore IPC from data in MSCW
             5046 0004
             5048 1011    JMP  NORMRET

NEGPROC      504A C060  ` MOV  @CURPRNO-1,R1          ; Return via address in Exit_IC
             504C 2AC2 *
             504E 0520    NEG  @CURPRNO-1            ; Make the stored procedure number positive
             5050 2AC2 *
             5052 0A11    SLA  R1,1                  ; Make the negative procedure number to a byte pointer
             5054 A060  ` A    @DICTPNT,R1           ; Index into the procedure dictionary, which is stored
backwards
             5056 2AE4 *
             5058 06A0    BL   @GETDAT2 ; Find pointer to Datasize value
             505A 4178 Ax
             505C 0A12    SLA  R2,1                  ; Word -> byte
             505E A0A0    A    @SEGBASE,R2           ; Segment relative pointer
             5060 2AE2 *
             5062 C042  B MOV  R2,R1
             5064 0641  A DECT R1                    ; Exit_IC is one word above Datasize
             5066 06A0    BL   @GETDAT2 ; *R1+ -> R2
             5068 4178 Ax
             506A C202    MOV  R2,R8                 ; Finally got the IPC to return to.
NORMRET      506C C26A  j MOV  @MSDYN(SP),R9          ; MSDYN, pointer to activation record of caller
             506E 0002
             5070 A220    A    @SEGBASE,R8           ; Restored IPC is segment relative. Make it absolute.
             5072 2AE2 *
             5074 022A  * AI   SP,10                 ; Pop MSCW
             5076 000A
             5078 0A13    SLA  R3,1                  ; Make words to pop into bytes
             507A A283    A    R3,SP                 ; Pop according to B parameter to RPU
             507C 0460  ` B    @LOADADDR; Load address to p-code and execute
             507E 4142 AB
```

```
; Set up address for copying memory blocks
; Called by copy array/string parameter
ADDRCOP       5080 C1DA    MOV  *SP,R7   ; Address of E_Rec, if any -> R7
              5082 C067  g MOV  @2(R7),R1; Destination to copy to -> R1
              5084 0002
              5086 8817    C    *R7,@CONNIL           ; Simple address?
              5088 40AA @
              508A 1604    JNE  VIAEREC  ; No, via E_Rec
              508C 05CA    INCT SP
              508E C0BA    MOV  *SP+,R2  ; Get destination address
              5090 04C0    CLR  R0                    ; Code type flag set to CPU RAM
              5092 045B  [ B    *R11

VIAEREC       5094 C157  W MOV  *R7,R5   ; Global data pointer
              5096 C1C1    MOV  R1,R7                 ; Segment offset for data
              5098 C18B    MOV  R11,R6
              509A 05A0    INC  @MTYPFL
              509C 2AD6 *
              509E 06A0    BL   @GETSEGB ; Find base of segment via E_Rec_P
              50A0 4ECC N
              50A2 A107    A    R7,R4
              50A4 05CA    INCT SP
              50A6 C0BA    MOV  *SP+,R2
              50A8 C000    MOV  R0,R0                 ; Code type flag
              50AA 1503    JGT  GROM1
              50AC 1109    JLT  VDP1
              50AE C044  D MOV  R4,R1                 ; CPU RAM address
              50B0 0456  V B    *R6                   ; Saved return address
GROM1         50B2 D844  D MOVB R4,@WAOFFS(R1)        ; Write address
              50B4 0402
              50B6 1000    NOP
              50B8 D860  ` MOVB @PASCALWS+9,@WAOFFS(R1)
              50BA 8389
              50BC 0402
              50BE 0456  V B    *R6                   ; Saved return address
VDP1          50C0 D860  ` MOVB @PASCALWS+9,@WAOFFS(R1)            ; Write address
              50C2 8389
              50C4 0402
              50C6 1000    NOP
              50C8 D844  D MOVB R4,@WAOFFS(R1)
              50CA 0402
              50CC 0456  V B    *R6                   ; Saved return address

; Copy array parameter
CAP           50CE 50D0    .WORD CAP+2
              50D0 C808    MOV  R8,@IPCSAVE           ; Prepare for stack overflow
              50D2 2ACC *
              50D4 0620    DEC  @IPCSAVE ; One byte back (CAP)
              50D6 2ACC *
              50D8 0202    LI   R2,CONT05
              50DA 50E0 P
              50DC 0460  ` B    @LDBIG   ; Array size (words) -> R3
              50DE 8330  0
CONT05        50E0 C0C3    MOV  R3,R3                 ; Zero size array?
              50E2 1603    JNE  DOCOPY   ; No
              50E4 022A  * AI   SP,4                  ; Pop arguments
              50E6 0004
              50E8 045C  \ B    *R12                  ; PME fetch
DOCOPY        50EA 06A0    BL   @ADDRCOP
              50EC 5080 P
              50EE C000    MOV  R0,R0
              50F0 1307    JEQ  CRAMCOP  ; CPU RAM copy?
MRAMCOP       50F2 DC91    MOVB *R1,*R2+ ; Memory mapped copy
              50F4 1000    NOP
              50F6 DC91    MOVB *R1,*R2+
              50F8 0603    DEC  R3
              50FA 15FB    JGT  MRAMCOP
              50FC 0460  ` B    @LOADADDR; Load address to p-code and execute
              50FE 4142 AB
CRAMCOP       5100 CCB1    MOV  *R1+,*R2+
              5102 0603    DEC  R3
              5104 15FD    JGT  CRAMCOP
              5106 0460  ` B    @LOADADDR; Load address to p-code and execute
              5108 4142 AB

; Copy string parameter
CSP           510A 8316    .WORD LDUB
              510C C808    MOV  R8,@IPCSAVE           ; Prepare for string overflow
              510E 2ACC *
              5110 0660  ` DECT @IPCSAVE
              5112 2ACC *
              5114 06A0    BL   @ADDRCOP ; Setup addresses for copying
              5116 5080 P
              5118 C000    MOV  R0,R0                 ; Code type flag
```

```
             511A 130F    JEQ  CSP01                 ; CPU RAM
             511C D011    MOVB *R1,R0
             511E 9800    CB   R0,@PASCALWS+7
             5120 8387
             5122 1B08    JH   STRBIG
             5124 DC80    MOVB R0,*R2+
             5126 1304    JEQ  CSP02
             5128 0980    SRL  R0,8
LOOP12       512A DC91    MOVB *R1,*R2+
             512C 0600    DEC  R0
             512E 15FD    JGT  LOOP12
CSP02        5130 0460  ` B    @LOADADDR; Load address to p-code and execute
             5132 4142 AB

STRBIG       5134 06A0    BL   @FAULGEN
             5136 484A HJ
             5138 000D    .WORD 13      ; String overflow

CSP01        513A D011    MOVB *R1,R0
             513C 9800    CB   R0,@PASCALWS+7
             513E 8387
             5140 1BF9    JH   STRBIG
             5142 0990    SRL  R0,9
             5144 0580    INC  R0
LOOP72       5146 CCB1    MOV  *R1+,*R2+
             5148 0600    DEC  R0
             514A 15FD    JGT  LOOP72
             514C 0460  ` B    @LOADADDR; Load address to p-code and execute
             514E 4142 AB

; Load packed
; A packed field pointer is on the stack
;  Rightmost bit in the field
;  Number of bits in the field
;  Address of the word the field is in
LDP          5150 5152    .WORD LDP+2
             5152 C07A  z MOV  *SP+,R1  ; pack-ptr.right_bit
             5154 0A11    SLA  R1,1                 ; Adjust to word index into table
             5156 C0BA    MOV  *SP+,R2  ; pack-ptr.field_width
             5158 0A12    SLA  R2,1                 ; Adjust to word index into table
             515A C0FA    MOV  *SP+,R3  ; pack-ptr.addr
             515C C0D3    MOV  *R3,R3   ; Fetch word
             515E 04A1    X    @SHRIGHT(R1)         ; Shift R3 to right-align
             5160 4100 A
             5162 C122  " MOV  @MASKTAB(R2),R4      ; Mask for the number of bits
             5164 40BE @
             5166 0544  D INV  R4
             5168 40C4 @  SZC  R4,R3                ; Clear bits not in the field
             516A 064A  J DECT SP
             516C C683    MOV  R3,*SP   ; Extracted packed data to stack top
             516E 045C  \ B    *R12     ; PME fetch

; Store packed
STP          5170 5172    .WORD STP+2
             5172 C07A  z MOV  *SP+,R1  ; The word to store
             5174 C0BA    MOV  *SP+,R2  ; pack-ptr.right_bit
             5176 0A12    SLA  R2,1                 ; Adjust to word index into table
             5178 C0FA    MOV  *SP+,R3  ; pack-ptr.field_width
             517A 0A13    SLA  R3,1                 ; Adjust to word index into table
             517C C123  # MOV  @MASKTAB(R3),R4
             517E 40BE @
             5180 0544  D INV  R4
             5182 4044 @D SZC  R4,R1                ; Clear bits not in field
             5184 C001    MOV  R1,R0
             5186 04A2    X    @SHLEFT(R2)          ; Shift R1 leftwards to bit position
             5188 40E0 @
             518A C801    MOV  R1,@SAVPACK
             518C 2AF2 *
             518E C063  c MOV  @MASKTAB(R3),R1
             5190 40BE @
             5192 0541  A INV  R1
             5194 E001    SOC  R1,R0                ; Keep bits outside mask field
             5196 0541  A INV  R1
             5198 4001 @  SZC  R1,R0                ; Clear bits inside
             519A 04A2    X    @SHCIRC(R2)          ; Circulate R0 mask to proper position
             519C 4120 A
             519E C07A  z MOV  *SP+,R1  ; pack-ptr.addr
             51A0 0540  @ INV  R0
             51A2 4440 D@ SZC  R0,*R1   ; Clear bit field in destination
             51A4 E460  ` SOC  @SAVPACK,*R1         ; Set bits in destinaitons bit field
             51A6 2AF2 *
             51A8 045C  \ B    *R12     ; PME fetch

; Index packed array
```

```
; pack-ptr.right_bit:= (TOS mod UB1) * UB2
; pack-ptr.field_width:= UB2
; pack-ptr.addr:= TOS□1 + (TOS div UB1) * x
IXP           51AA 830E    .WORD LDUBB   ; UB1 elements/word -> R4, UB2 field width -> R3
              51AC 04C0    CLR  R0                     ; For DIV
              51AE C07A  z MOV  *SP+,R1  ; Index
              51B0 3C04 <  DIV  R4,R0                  ; Index/(elements per word)
              51B2 0A10    SLA  R0,1                   ; Bytes
              51B4 A680    A    R0,*SP   ; Add to array base (pack-ptr.addr)
              51B6 064A  J DECT SP
              51B8 C683    MOV  R3,*SP   ; Field width (pack-ptr.field_width)
              51BA C001    MOV  R1,R0
              51BC 3803 8  MPY  R3,R0
              51BE 064A  J DECT SP
              51C0 C681    MOV  R1,*SP   ; pack-ptr.right_bit
              51C2 045C  \ B    *R12     ; PME fetch

; Logical instructions work on all bits
; Logical AND
LAND          51C4 51C6 Q  .WORD LAND+2
              51C6 055A  Z INV  *SP
              51C8 46BA F  SZC  *SP+,*SP
              51CA 045C  \ B    *R12     ; PME fetch

; Logical OR
LOR           51CC 51CE Q  .WORD LOR+2
              51CE E6BA    SOC  *SP+,*SP
              51D0 045C  \ B    *R12     ; PME fetch

; Logical NOT
LNOT          51D2 51D4    .WORD LNOT+2
              51D4 055A  Z INV  *SP
              51D6 045C  \ B    *R12     ; PME fetch

; Boolean NOT
BNOT          51D8 51DA Q  .WORD BNOT+2
              51DA C05A  Z MOV  *SP,R1
              51DC 0541  A INV  R1
              51DE 0241  A ANDI R1,0001H ; The value TRUE is 0001H
              51E0 0001
              51E2 C681    MOV  R1,*SP
              51E4 045C  \ B    *R12     ; PME fetch

; Increment integer
INCI          51E6 51E8 Q  .WORD INCI+2
              51E8 059A    INC  *SP
              S51EA 045C  \ B    *R12    ; PME fetch

; Decrement integer
DECI          51EC 51EE Q  .WORD DECI+2
              51EE 061A    DEC *SP
              51F0 045C  \ B    *R12     ; PME fetch

; Absolute value integer
ABI           51F2 51F4 Q  .WORD ABI+2
              51F4 075A  Z ABS  *SP
              51F6 045C  \ B    *R12     ; PME fetch

; Add integer
ADI           51F8 51FA Q  .WORD ADI+2
              51FA A6BA    A    *SP+,*SP
              51FC 045C  \ B    *R12     ; PME fetch

; Negate integer
NGI           51FE 5200 R  .WORD NGI+2
              5200 051A    NEG  *SP
              5202 045C  \ B    *R12     ; PME fetch

; Subtract integer
SBI           5204 5206 R  .WORD SBI+2
              5206 66BA f  S    *SP+,*SP
              5208 045C  \ B    *R12     ; PME fetch

; Modulo integer
MODI          520A 520C R  .WORD MODI+2
              520C C0FA    MOV  *SP+,R3
              520E 131B    JEQ  FAUDIVZ  ; Fault, divide by zero
              5210 0743  C ABS  R3
              5212 C09A    MOV  *SP,R2
              5214 0742  B ABS  R2
              5216 04C1    CLR  R1
              5218 3C43 <C DIV  R3,R1
              521A C682    MOV  R2,*SP   ; Reminder
              521C 045C  \ B    *R12     ; PME fetch
```

```
; Divide integer
DVI           521E 5220 R  .WORD DVI+2
              5220 04C0    CLR  R0                    ; Flags
              5222 04C4    CLR  R4
              5224 C0FA    MOV  *SP+,R3
              5226 130F    JEQ  FAUDIVZ
              5228 1502    JGT  POS1
              522A 0704    SETO R4                    ; Flag negative
              522C 0743  C ABS  R3
POS1          522E C09A    MOV  *SP,R2
              5230 1309    JEQ  ZERORES  ; And zero is already on the stack here
              5232 1502    JGT  POS2
              5234 0700    SETO R0                    ; Flag negative
              5236 0742  B ABS  R2
POS2          5238 04C1    CLR  R1
              523A 3C43 <C DIV  R3,R1
              523C 2900 )  XOR  R0,R4                 ; One negative makes it negative
              523E 1301    JEQ  POSRES
              5240 0501    NEG  R1
POSRES        5242 C681    MOV  R1,*SP
ZERORES       5244 045C  \ B    *R12     ; PME fetch

FAUDIVZ       5246 06A0    BL   @FAULGEN ; Create a fault
              5248 484A HJ
              524A 0006    .WORD 6                    ; Divide by zero

; Multiply integer. Returns low 16 bits which implies a signed multiply
MPI           524C 524E RN .WORD MPI+2
              524E C07A  z MOV  *SP+,R1
              5250 385A 8Z MPY  *SP,R1
              5252 C682    MOV  R2,*SP
              5254 045C  \ B    *R12     ; PME fetch

; Check subrange limit
CHK           5256 5258    .WORD CHK+2
              5258 C07A  z MOV  *SP+,R1   ; High limit
              525A C0BA    MOV  *SP+,R2   ; Low limit
              525C 805A  Z C    *SP,R1    ; Check high
              525E 1503    JGT  RGFAUL
              5260 809A    C    *SP,R2    ; Check low
              5262 1101    JLT  RGFAUL
              5264 045C  \ B    *R12     ; PME fetch

RGFAUL        5266 06A0    BL   @FAULGEN
              5268 484A HJ
              526A 0001    .WORD 1        ; Value range error, invalid index

; Compare byte arrays
COMPARR       526C 8106    C    R6,R4                 ; Which is the larger one
              526E 1502    JGT  GT1                   ; First is larger?
              5270 C0C6    MOV  R6,R3                 ; Smallest value goes to R3 to set the length to
compare
              5272 1001    JMP  GT2

GT1           5274 C0C4    MOV  R4,R3
GT2           5276 1325  % JEQ  CTTEST
              5278 C165  e MOV  @TSTTAB(R5),R5        ; According to memory mode
              527A 527E R~
              527C 0455  U B    *R5

; Table with code segments for different memory types when array is in a code segment.
; CPU means CPU RAM, MAP means mapped memory (VDP RAM or GROM).
; Compares the arrays until a difference is found, or the short one ends.
; Then goes to testing for =, <= or >=.
TSTTAB        527E 5286    .WORD CPUCPU
              5280 5290    .WORD MAPCPU
              5282 529A    .WORD CPUMAP
              5284 52A4    .WORD MAPMAP

; Both arrays in CPU memory
CPUCPU        5286 9C32  2 CB   *R2+,*R0+
              5288 161D    JNE  TESTS
              528A 0603    DEC  R3
              528C 15FC    JGT  CPUCPU
              528E 1019    JMP  CTTEST

; First in mapped segment, second in CPU memory
MAPCPU        5290 9772  r CB   *R2+,*R13
              5292 1618    JNE  TESTS
              5294 0603    DEC  R3
              5296 15FC    JGT  MAPCPU
              5298 1014    JMP  CTTEST
```

```
; First in CPU memory, second in mapped
CPUMAP        529A 9C1D    CB   *R13,*R0+
              529C 1613    JNE  TESTS
              529E 0603    DEC  R3
              52A0 15FC    JGT  CPUMAP
              52A2 100F    JMP  CTTEST

; Both in mapped memory. Address has to be reloaded all the time, in case both are in GROM.
MAPMAP        52A4 C148  H MOV  R8,R5
              52A6 C202    MOV  R2,R8
              52A8 06A0    BL   @LOADAD1
              52AA 4144 AD
              52AC 0582    INC  R2
              52AE D1DD    MOVB *R13,R7
              52B0 C200    MOV  R0,R8
              52B2 06A0    BL   @LOADAD1
              52B4 4144 AD
              52B6 0580    INC  R0
              52B8 C205    MOV  R5,R8
              52BA 9747  G CB   R7,*R13
              52BC 1603    JNE  TESTS
              52BE 0603    DEC  R3
              52C0 15F1    JGT  MAPMAP
CTTEST        52C2 8106    C    R6,R4                  ; Compare, then test

TESTS         52C4 0491    X    *R1                    ; Jumps to 52CE if test is true (condition false)
              52C6 C6A0    MOV  @CONTRUE,*SP           ; Condition is true
              52C8 40B6 @
              52CA 0460  ` B    @LOADADDR; Load address to p-code and execute
              52CC 4142 AB
              52CE 04DA    CLR  *SP                    ; Condition is false
              52D0 0460  ` B    @LOADADDR; Load address to p-code and execute
              52D2 4142 AB

; Instruction table for single value test instructions. Four words ahead jump.
; They are inverted compared to the p-code meaning due to the design above.
SINGTST       52D4 1604    JNE  52DE      ; EQUI
              52D6 1304    JEQ  52E0      ; NEQI
              52D8 1504    JGT  52E2      ; LEQI
              52DA 1104    JLT  52E4      ; GEQI
              52DC 1B04    JH   52E6      ; LEUSW
              52DE 1A04    JL   52E8      ; GEUSW

; Instruction table for tests on multiple bytes (strings, real)
ARRYTST       52E0 1604    JNE  52EA      ; EQbytes
              52E2 1504    JGT  52EC      ; LEbytes
              52E4 1104    JLT  52EE      ; GEbytes

; Common entry for
; integer =, <>, <=, >= and unsigned <= and >=
; Opcodes B0 to B5
EQUI          52E6 52E8    .WORD EQUI+2
              52E8 0221  ! AI   R1,SINGTST-(2*176)     ; To make 2*opcode point to the right entry in the
table above
              52EA 5174 Qt
              52EC 8EAA    C    @2(SP),*SP+
              52EE 0002
              52F0 0460  ` B    @TESTS
              52F2 52C4 R

; Equal byte array, less than or equal or greater than or equal byte array. Opcodes B9-BB
EQBYT         52F4 8320    .WORD LDUBBG
              52F6 0221  ! AI   R1,ARRYTST-(2*185)     ; Opcode EQBYT is B9 (185)
              52F8 516E Qn
              52FA C183    MOV  R3,R6                 ; Array size
              52FC 04C3    CLR  R3                    ; Memory mode flag CPU/CPU
              52FE C03A  : MOV  *SP+,R0  ; Address/offset array 1
              5300 C09A    MOV  *SP,R2   ; Address/offset array 2
              5302 C145  E MOV  R5,R5                 ; Mode flag array 1
              5304 130A    JEQ  ARR1DIR  ; Array 1 direct address
              5306 A020    A    @SEGBASE,R0           ; Add segment base address to offset
              5308 2AE2 *
              530A C3CF    MOV  R15,R15  ; Code type flag
              530C 1306    JEQ  ARR1DIR  ; In CPU memory with direct access?
              530E 05C3    INCT R3                    ; Memory mode flag MAP/CPU
              5310 C1C8    MOV  R8,R7                 ; Save IPC
              5312 C200    MOV  R0,R8                 ; Address in mapped memory
              5314 06A0    BL   @LOADAD1 ; Load address but don't execute code there
              5316 4144 AD
              5318 C207    MOV  R7,R8                 ; Restore IPC
ARR1DIR       531A C104    MOV  R4,R4                 ; Mode flag array 2
              531C 130B    JEQ  ARR2DIR
              531E A0A0    A    @SEGBASE,R2
              5320 2AE2 *
```

```
             5322 C3CF    MOV  R15,R15
             5324 1307    JEQ  ARR2DIR  ; Direct memory access?
             5326 0223  # AI   R3,4                   ; Memory mode flag XXX/MAP
             5328 0004
             532A C1C8    MOV  R8,R7                  ; Save IPC
             532C C202    MOV  R2,R8                  ; Address in mapped memory
             532E 06A0    BL   @LOADAD1 ; Load it
             5330 4144 AD
             5332 C207    MOV  R7,R8                  ; Restore IPC
ARR2DIR      5334 C143  C MOV  R3,R5                  ; Memory mode flag
             5336 C106    MOV  R6,R4                  ; Array size
             5338 0460  ` B    @COMPARR
             533A 526C Rl

; Equal, less than or equal and greater than or equal string
EQSTR        533C 830E    .WORD LDUBB   ; Mode flags (string pointer or offset in code segment)
             533E 0221  ! AI   R1,ARRYTST-(2*E8H)     ; Opcode EQSTR is E8 (232) *2 => 52E0
             5340 5110 Q
             5342 04C5    CLR  R5                     ; Index in code pointer table
             5344 C03A  : MOV  *SP+,R0  ; First string
             5346 C09A    MOV  *SP,R2   ; Second string
             5348 C104    MOV  R4,R4                  ; Mode string 1
             534A 130F    JEQ  CRAMST1  ; Direct address
             534C A020    A    @SEGBASE,R0            ; In code segment, add base pointer
             534E 2AE2 *
             5350 C3CF    MOV  R15,R15  ; Code type
             5352 130B    JEQ  CRAMST1  ; CPU RAM
             5354 05C5    INCT R5                     ; Index in code pointer table
             5356 C1C8    MOV  R8,R7                  ; IPC copy
             5358 C200    MOV  R0,R8                  ; First string address
SOUND20      535A 06A0    BL   @LOADAD1 ; Load address to string. The byte @SOUND20+1 just happens to be
good for sound processing
             535C 4144 AD
             535E 04C4    CLR  R4
             5360 D81D    MOVB *R13,@PASCALWS+9       ; Length -> LSBy R4
             5362 8389
             5364 0580    INC  R0                     ; Point to next byte in string
             5366 C207    MOV  R7,R8                  ; Restore IPC
             5368 1003    JMP  EQSTR01

CRAMST1      536A 04C4    CLR  R4
             536C D830  0 MOVB *R0+,@PASCALWS+9       ; Length -> LSBy R4
             536E 8389
EQSTR01      5370 C0C3    MOV  R3,R3                  ; Mode string 2
             5372 1311    JEQ  CRAMST2
             5374 A0A0    A    @SEGBASE,R2            ; In code segment, add base pointer
             5376 2AE2 *
             5378 C3CF    MOV  R15,R15  ; Code type
             537A 130D    JEQ  CRAMST2  ; CPU RAM
             537C 0225  % AI   R5,4                   ; Index in code pointer table
             537E 0004
             5380 C1C8    MOV  R8,R7                  ; Save IPC
             5382 C202    MOV  R2,R8                  ; Second string address
             5384 06A0    BL   @LOADAD1 ; Load address to string
             5386 4144 AD
             5388 04C6    CLR  R6
             538A D81D    MOVB *R13,@PASCALWS+13      ; Length -> LSBy R6
             538C 838D
             538E 0582    INC  R2                     ; Point to next byte in string
             5390 C207    MOV  R7,R8                  ; Restore IPC
             5392 0460  ` B    @COMPARR
             5394 526C Rl

CRAMST2      5396 04C6    CLR  R6
             5398 D832  2 MOVB *R2+,@PASCALWS+13      ; Length -> LSBy R6
             539A 838D
             539C 0460  ` B    @COMPARR
             539E 526C Rl

; Assign string
ASTR         53A0 830E    .WORD LDUBB   ; R3 dest size, R4 source mode (=0 is variable, <>0 is const pool
offset)
             53A2 C07A  z MOV  *SP+,R1  ; Source
             53A4 C0BA    MOV  *SP+,R2  ; Destination
             53A6 04C5    CLR  R5                     ; Source size
             53A8 C104    MOV  R4,R4                  ; Source type
             53AA 1314    JEQ  VARSRC   ; In variable
             53AC A060  ` A    @SEGBASE,R1            ; In code segment
             53AE 2AE2 *
             53B0 C3CF    MOV  R15,R15  ; Code type
             53B2 1310    JEQ  VARSRC   ; CPU RAM
             53B4 C108    MOV  R8,R4                  ; Save IPC
             53B6 C201    MOV  R1,R8
             53B8 06A0    BL   @LOADAD1 ; Address to code segment
```

```
             53BA 4144 AD
             53BC C204    MOV  R4,R8                 ; Restore IPC
             53BE D81D    MOVB *R13,@PASCALWS+11     ; Read string size -> LSBy R5
             53C0 838B
             53C2 80C5    C    R5,R3                 ; Compare actual source length to declared destination
             53C4 1510    JGT  ERRSTR   ; No room in destination?
             53C6 DCA0    MOVB @PASCALWS+11,*R2+     ; Copy length to destination
             53C8 838B
LOOP25       53CA DC9D    MOVB *R13,*R2+; Read bytes from source in segment
             53CC 0605    DEC  R5
             53CE 15FD    JGT  LOOP25
             53D0 0460  ` B    @LOADADDR; Load address to p-code and execute
             53D2 4142 AB

VARSRC       53D4 D811    MOVB *R1,@PASCALWS+11
             53D6 838B
             53D8 80C5    C    R5,R3                 ; Compare actual source length to declared destination

             53DA 1505    JGT  ERRSTR   ; No room in destination?
             53DC 0585    INC  R5
LOOP26       53DE CCB1    MOV  *R1+,*R2+; Copy source to destination
             53E0 0645  E DECT R5
             53E2 15FD    JGT  LOOP26
             53E4 045C  \ B    *R12     ; PME fetch

ERRSTR       53E6 06A0    BL   @FAULGEN
             53E8 484A HJ
             53EA 000D    .WORD 13      ; String overflow

; Check string index, 1 <= index <= current length of string is OK
CSTR         53EC 53EE    .WORD CSTR+2
             53EE C09A    MOV  *SP,R2   ; Index value
             53F0 C06A  j MOV  @2(SP),R1; String variable address
             53F2 0002
             53F4 04C3    CLR  R3
             53F6 D811    MOVB *R1,@PASCALWS+7       ; R3 LSBy
             53F8 8387
             53FA 0282    CI   R2,1
             53FC 0001
             53FE 1103    JLT  ERRCSTR  ; Index < 1?
             5400 80C2    C    R2,R3
             5402 1501    JGT  ERRCSTR  ; Index > dynamic length?
             5404 045C  \ B    *R12     ; PME fetch

ERRCSTR      5406 06A0    BL   @FAULGEN
             5408 484A HJ
             540A 0001    .WORD 1       ; Value range error, invalid index

; Adjust set (modify size of set, make it smaller or larger)
ADJ          540C 8316    .WORD LDUB                 ; New length -> R3
             540E C05A  Z MOV  *SP,R1   ; Current length
             5410 80C1    C    R1,R3
             5412 1103    JLT  EXPAND   ; Expand set
             5414 151D    JGT  SHRINK   ; Shrink set
             5416 05CA    INCT SP                    ; Same size, so done
             5418 045C  \ B    *R12     ; PME fetch

EXPAND       541A 60C1 `  S    R1,R3                 ; Calulate expansion needed
             541C C003    MOV  R3,R0
             541E 0A13    SLA  R3,1                  ; Size is in words
             5420 C808    MOV  R8,@IPCSAVE           ; Prepare for stack overflow
             5422 2ACC *
             5424 0660  ` DECT @IPCSAVE
             5426 2ACC *
             5428 C820    MOV  @ERECP,@CUREREC
             542A 2AB4 *
             542C 2AE0 *
             542E C803    MOV  R3,@STKMISS           ; Number of words to expand
             5430 2AF0 *
             5432 06A0    BL   @CHKSTK  ; Check for stack overflow
             5434 4E98 N
             5436 05CA    INCT SP                    ; Point to set data
             5438 C08A    MOV  SP,R2
             543A 6283 b  S    R3,SP                 ; Allocate the desired number of bytes
             543C C0CA    MOV  SP,R3
             543E C041  A MOV  R1,R1                 ; If current length is zero there's nothing to copy
             5440 1303    JEQ  LOOP17
LOOP16       5442 CCF2    MOV  *R2+,*R3+; Copy top of set to new beginning
             5444 0601    DEC  R1
             5446 15FD    JGT  LOOP16
LOOP17       5448 04F3    CLR  *R3+                  ; Clear expanded part of set
             544A 0600    DEC  R0
             544C 15FD    JGT  LOOP17
             544E 045C  \ B    *R12     ; PME fetch
```

```
SHRINK      5450 05CA    INCT SP                    ; Point to set data
            5452 C083    MOV  R3,R2
            5454 0A13    SLA  R3,1                  ; Number of bytes to shrink
            5456 A0CA    A    SP,R3                 ; Pop words no longer needed
            5458 0643  C DECT R3
            545A 0A11    SLA  R1,1                  ; Current length to bytes
            545C A04A  J A    SP,R1
            545E 0641  A DECT R1
LOOP18      5460 C453  S MOV  *R3,*R1  ; Copy old start of set to new start position
            5462 0643  C DECT R3
            5464 0641  A DECT R1
            5466 0602    DEC  R2
            5468 15FB    JGT  LOOP18
            546A C281    MOV  R1,SP
            546C 05CA    INCT SP
            546E 045C  \ B    *R12     ; PME fetch

; Subrange set (Create a set within given range, inclusive)
SRS         5470 5472    .WORD SRS+2
            5472 C05A  Z MOV  *SP,R1   ; Last value
            5474 C0EA    MOV  @2(SP),R3; First value
            5476 0002
            5478 1167  g JLT  EMPTY                 ; First value <0
            547A 8043  C C    R3,R1
            547C 1565  e JGT  EMPTY                 ; TOS-1>TOS
            547E 04C2    CLR  R2
            5480 3CA0 <  DIV  @CON016,R2            ; Sets are stored as bits in words
            5482 40BA @
            5484 C802    MOV  R2,@SAVPAK2           ; Word for starting bit
            5486 2AF4 *
            5488 0A13    SLA  R3,1                  ; Reminder index
            548A 0204    LI   R4,FFFFH
            548C FFFF
            548E 2923 )# XOR  @MASKTAB(R3),R4
            5490 40BE @
            5492 C804    MOV  R4,@FRSTWRD           ; SAVWR1 save first word with odd bits
            5494 2AC4 *
            5496 C820    MOV  @CONFALS,@SMALSET     ; Assume status multiple words
            5498 40A8 @
            549A 2AD8 *
            549C 0582    INC  R2
            549E 0A42  B SLA  R2,4                  ; *16 Calculate last bit in same word
            54A0 8081    C    R1,R2                 ; Last value to last bit in word already used+1
            54A2 1506    JGT  MULTISE  ; More than one word needed
            54A4 1305    JEQ  MULTISE
            54A6 C820    MOV  @CONTRUE,@SMALSET     ; Status single word
            54A8 40B6 @
            54AA 2AD8 *
            54AC 0222  " AI   R2,-16D  ; Revert bit number by one word
            54AE FFF0
MULTISE     54B0 6042 `B S    R2,R1                 ; Last bit - bit not included in word
            54B2 04C0    CLR  R0
            54B4 3C20 <  DIV  @CON016,R0
            54B6 40BA @
            54B8 C800    MOV  R0,@WORDCNT           ; Save number of words
            54BA 2AD4 *
            54BC 0581    INC  R1                    ; Increment to include bit
            54BE 0A11    SLA  R1,1
            54C0 04C3    CLR  R3
            54C2 28E1 (  XOR  @MASKTAB(R1),R3       ;Set bits needed
            54C4 40BE @
            54C6 C803    MOV  R3,@LASTWRD           ; Save last word with odd bits
            54C8 2AD0 *
            54CA C0A0    MOV  @WORDCNT,R2           ; Number of words
            54CC 2AD4 *
            54CE A0A0    A    @SAVPAK2,R2           ; Word for first bit
            54D0 2AF4 *
            54D2 8820    C    @SMALSET,@CONTRUE     ; Check if small set
            54D4 2AD8 *
            54D6 40B6 @
            54D8 1301    JEQ  SINGLWR  ; Single word?
            54DA 0582    INC  R2
SINGLWR     54DC 0582    INC  R2
            54DE C142  B MOV  R2,R5                 ; Save number of words
            54E0 0A12    SLA  R2,1
            54E2 C802    MOV  R2,@STKMISS           ; Number of bytes needed on stack
            54E4 2AF0 *
            54E6 C808    MOV  R8,@IPCSAVE           ; Prepare for stack oveflow
            54E8 2ACC *
            54EA 0620    DEC  @IPCSAVE
            54EC 2ACC *
            54EE C820    MOV  @ERECP,@CUREREC
            54F0 2AB4 *
```

```
                54F2 2AE0 *
                54F4 06A0    BL   @CHKSTK
                54F6 4E98 N
                54F8 022A  * AI   SP,4                   ; Pop two arguments
                54FA 0004
                54FC 8820    C    @SMALSET,@CONFALS
                54FE 2AD8 *
                5500 40A8 @
                5502 1308    JEQ  MULTIWR  ; Multiple words
                5504 C0A0    MOV  @FRSTWRD,R2            ; Combine single words
                5506 2AC4 *
                5508 28A0 (  XOR  @LASTWRD,R2
                550A 2AD0 *
                550C 0542   B INV  R2
                550E 064A   J DECT SP                    ; Push set word
                5510 C682    MOV  R2,*SP
                5512 100F    JMP  SRSWRAP  ; Wrap it up

MULTIWR         5514 064A   J DECT SP
                5516 C6A0    MOV  @LASTWRD,*SP           ; Last word
                5518 2AD0 *
                551A C060   ` MOV  @WORDCNT,R1           ; Number of words
                551C 2AD4 *
                551E 1306    JEQ  NOMID                  ; No middle words?
                5520 0203    LI   R3,FFFF  ; Full set
                5522 FFFF
LOOP73          5524 064A   J DECT SP
                5526 C683    MOV  R3,*SP   ; Fill middle words
                5528 0601    DEC  R1
                552A 15FC    JGT  LOOP73
NOMID           552C 064A   J DECT SP
                552E C6A0    MOV  @FRSTWRD,*SP           ; First word
                5530 2AC4 *
                5532 04C2    CLR  R2
                5534 C060   ` MOV  @SAVPAK2,R1           ; Word for starting bit
                5536 2AF4 *
                5538 1304    JEQ  NOCLR                  ; No clear part of set
LOOP74          553A 064A   J DECT SP
                553C C682    MOV  R2,*SP   ; Clear up to starting word
                553E 0601    DEC  R1
                5540 15FC    JGT  LOOP74
NOCLR           5542 064A   J DECT SP
                5544 C685    MOV  R5,*SP   ; Number of words
                5546 045C   \ B    *R12                  ; PME fetch

EMPTY           5548 05CA    INCT SP
                554A 04DA    CLR  *SP
                554C 045C   \ B    *R12     ; PME fetch

; Set membership (Check if TOS-1 is a part of the set at TOS. (This is listed in the wrong order in most
IAG!))
; Or are the values upside down in the stack diagram?
INN             554E 5550    .WORD INN+2
                5550 C03A   : MOV  *SP+,R0  ; Size of set
                5552 C04A   J MOV  SP,R1                 ; First word in set
                5554 A040   @ A    R0,R1
                5556 A040   @ A    R0,R1                 ; Points to value to look for
                5558 C0D1    MOV  *R1,R3
                555A 111B    JLT  NOTINN   ; Negative values not allowed in sets
                555C 04C2    CLR  R2
                555E 3CA0 <  DIV  @CON016,R2             ; Find out which word to look in
                5560 40BA @
                5562 8080    C    R0,R2
                5564 1216    JLE  NOTINN   ; Set smaller than value?
                5566 0A12    SLA  R2,1                   ; Convert to bytes
                5568 A08A    A    SP,R2                  ; Add to SP pointing at beginning of set
                556A C812    MOV  *R2,@SAVPACK           ; Save the interesting set word
                556C 2AF2 *
                556E C0D1    MOV  *R1,R3   ; Fetch value to find again
                5570 04C2    CLR  R2
                5572 3CA0 <  DIV  @CON016,R2
                5574 40BA @
                5576 0A13    SLA  R3,1                   ; Make reminder index into table
                5578 C123   # MOV  @SBITMSK(R3),R4       ; Single bit mask
                557A 2D92 -
                557C 0544   D INV  R4
                557E C0A0    MOV  @SAVPACK,R2            ; Fetch the word to check
                5580 2AF2 *
                5582 4084 @  SZC  R4,R2                  ; Clear all bits but the interesting one
                5584 1306    JEQ  NOTINN
                5586 C281    MOV  R1,SP                  ; Restore stack pointer
                5588 05CA    INCT SP                     ; A bit redundant...
                558A 064A   J DECT SP
                558C C6A0    MOV  @CONTRUE,*SP
```

```
             558E 40B6 @
             5590 045C  \ B    *R12     ; PME fetch

NOTINN       5592 C281    MOV  R1,SP                 ; Restore stack pointer
             5594 05CA    INCT SP                    ; A bit redundant...
             5596 064A  J DECT SP
             5598 C6A0    MOV  @CONFALS,*SP
             559A 40A8 @
             559C 045C  \ B    *R12     ; PME fetch

; Prepare for manipulation of two sets
; Returns with SP pointing to top set, R2 pointing to second.
; Expands set if second one is smaller
PREPSET      559E C07A  z MOV  *SP+,R1  ; Top set size (words)
             55A0 C08A    MOV  SP,R2
             55A2 A081    A    R1,R2                 ; Add the size of the top set (in bytes)
             55A4 A081    A    R1,R2
             55A6 8072  r C    *R2+,R1  ; Compare set sizes
             55A8 1320    JEQ  EXPREP   ; Equal?
             55AA 151F    JGT  EXPREP   ; Second one greater?
             55AC 064A  J DECT SP                    ; Back stack pointer to top of first set again
             55AE C05A  Z MOV  *SP,R1   ; Get size of first set
             55B0 0642  B DECT R2                    ; Back R2 to point to top of second set
             55B2 C812    MOV  *R2,@SAVPACK          ; Size of set two
             55B4 2AF2 *
             55B6 C481    MOV  R1,*R2   ; Put size of first into second set
             55B8 C081    MOV  R1,R2
             55BA 60A0 `  S    @SAVPACK,R2           ; Size difference between first and second set
             55BC 2AF2 *
             55BE C802    MOV  R2,@SAVPAK2           ; Size difference
             55C0 2AF4 *
             55C2 A060  ` A    @SAVPACK,R1           ; Add size of first to itself (bytes)
             55C4 2AF2 *
             55C6 05C1    INCT R1                    ; Include size word
             55C8 C80A    MOV  SP,@SAVPACK           ; SP pointing at first set (original position)
             55CA 2AF2 *
             55CC 0A12    SLA  R2,1                  ; Make size difference bytes instead
             55CE 6282 b  S    R2,SP                 ; Create space on stack for difference
             55D0 C08A    MOV  SP,R2                 ; Save that stack pointer
             55D2 C0E0    MOV  @SAVPACK,R3           ; Original location of first set
             55D4 2AF2 *
LOOP13       55D6 CCB3    MOV  *R3+,*R2+; Move first set into new location
             55D8 0601    DEC  R1
             55DA 15FD    JGT  LOOP13
             55DC C060  ` MOV  @SAVPAK2,R1           ; Size difference
             55DE 2AF4 *
LOOP14       55E0 04F2    CLR  *R2+                  ; Clear overflow part of expanded set
             55E2 0601    DEC  R1
             55E4 15FD    JGT  LOOP14
             55E6 0460  ` B    @PREPSET
             55E8 559E U

EXPREP       55EA C041  A MOV  R1,R1                 ; Size of top set
             55EC 045B  [ B    *R11

; Set union (OR 5587between sets)
UNI          55EE 55F0    .WORD UNI+2
             55F0 06A0    BL   @PREPSET ; Get SP pointing to TOS, R2 to TOS-1
             55F2 559E U
             55F4 1303    JEQ  EXUNI                 ; Top set size = 0?
LOOP20       55F6 ECBA    SOC  *SP+,*R2+; OR function
             55F8 0601    DEC  R1
             55FA 15FD    JGT  LOOP20
EXUNI        55FC 045C  \ B    *R12     ; PME fetch

; Set intersection (AND between sets)
INT          55FE 5600    .WORD INT+2
             5600 06A0    BL   @PREPSET ; Get SP pointing to TOS, R2 to TOS-1
             5602 559E U
             5604 C801    MOV  R1,@SAVPACK           ; Top set size
             5606 2AF2 *
             5608 1304    JEQ  5612                  ; Set size zero?
LOOP22       560A 055A  Z INV  *SP                   ; AND function
             560C 4CBA L  SZC  *SP+,*R2+
             560E 0601    DEC  R1
             5610 15FC    JGT  LOOP22
             5612 C05A  Z MOV  *SP,R1
             5614 6060 `` S    @SAVPACK,R1
             5616 2AF2 *
             5618 1303    JEQ  EXINT
LOOP21       561A 04F2    CLR  *R2+
             561C 0601    DEC  R1
             561E 15FD    JGT  LOOP21
EXINT        5620 045C  \ B    *R12     ; PME fetch
```

```
; Set difference (TOS-1 AND NOT TOS)
DIF           5622 5624    .WORD DIF+2
              5624 06A0    BL   @PREPSET ; Get SP pointing to TOS, R2 to TOS-1
              5626 559E U
              5628 1303    JEQ  EXDIF                 ; Top set size zero?
LOOP23        562A 4CBA L  SZC  *SP+,*R2+; NOT AND function
              562C 0601    DEC  R1
              562E 15FD    JGT  LOOP23
EXDIF         5630 045C  \ B    *R12     ; PME fetch

SETTEST       5632 565A VZ .WORD EQSET
              5634 568E V  .WORD LESET
              5636 56A0 V  .WORD GESET

; Equal set, less than or equal set and greater than or equal set
EQPWR         5638 563A    .WORD EQPWR+2
              563A C141  A MOV  R1,R5                 ; Opcode is B6 for EQPWR. B6*2=364D
              563C 06A0    BL   @PREPSET ; Get SP pointing to TOS, R2 to TOS-1
              563E 559E U
              5640 0642  B DECT R2
              5642 C132  2 MOV  *R2+,R4
              5644 0A14    SLA  R4,1
              5646 A102    A    R2,R4
              5648 C804    MOV  R4,@NEWSP
              564A 2AD2 *
              564C 0225  % AI   R5,-364  ; Subtract opcode*2
              564E FE94
              5650 C125  % MOV  @SETTEST(R5),R4       ; Test code to execute from table
              5652 5632 V2
              5654 0642  B DECT R2
              5656 C0F2    MOV  *R2+,R3
              5658 0454  T B    *R4                   ; Execute correct test sequence

; Equal set testing
EQSET         565A C041  A MOV  R1,R1
              565C 1305    JEQ  5668
              565E 8CBA    C    *SP+,*R2+
              5660 160A    JNE  TSTFALS
              5662 0603    DEC  R3
              5664 0601    DEC  R1
              5666 15FB    JGT  565E
              5668 C0C3    MOV  R3,R3
              566A 130B    JEQ  TSTTRUE
LOOP15        566C C072  r MOV  *R2+,R1
              566E 1603    JNE  TSTFALS
              5670 0603    DEC  R3
              5672 15FC    JGT  LOOP15
              5674 1006    JMP  TSTTRUE

TSTFALS       5676 C2A0    MOV  @NEWSP,SP; Test came out false
              5678 2AD2 *
              567A 064A  J DECT SP
              567C C6A0    MOV  @CONFALS,*SP
              567E 40A8 @
              5680 045C  \ B    *R12     ; PME fetch

TSTTRUE       5682 C2A0    MOV  @NEWSP,SP; Test came out true
              5684 2AD2 *
              5686 064A  J DECT SP
              5688 C6A0    MOV  @CONTRUE,*SP
              568A 40B6 @
              568C 045C  \ B    *R12     ; PME fetch

; Less than or equal set testing
LESET         568E C041  A MOV  R1,R1
              5690 1305    JEQ  569C
LOOP24        5692 4CBA L  SZC  *SP+,*R2+
              5694 16F0    JNE  TSTFALS
              5696 0603    DEC  R3
              5698 0601    DEC  R1
              569A 15FB    JGT  LOOP24
              569C 0460  ` B    @5668
              569E 5668 Vh

; Greater than or equal set testing
GESET         56A0 C041  A MOV  R1,R1
              56A2 13EF    JEQ  TSTTRUE
LOOP28        56A4 4EB2 N  SZC  *R2+,*SP+
              56A6 16E7    JNE  TSTFALS
              56A8 0601    DEC  R1
              56AA 15FC    JGT  LOOP28
              56AC 10EA    JMP  TSTTRUE
```

```
; Kernel procedure ATTACH
; Does nothing, just pops arguments
; In systems where it's implemented it will connect a hardware interrupt with a sempahore.
ATTACH        56AE 022A  * AI   SP,4
              56B0 0004
              56B2 045C  \ B    *R12     ; PME fetch

; Signal semaphore
SIGNAL        56B4 56B6    .WORD SIGNAL+2
              56B6 C07A  z MOV  *SP+,R1  ; Semaphore address
              56B8 8821  ! C    @2(R1),@CONNIL        ; Queue pointer nil or some process waiting?
              56BA 0002
              56BC 40AA @
              56BE 1603    JNE  ISQ
              56C0 05A1    INC  @0(R1)   ; Increment semaphore event (nothing is waiting)
              56C2 0000
              56C4 102B  + JMP  SIGRET   ; Return to p-code

ISQ           56C6 C121  ! MOV  @2(R1),R4; Fetch queue address (something is waiting)
              56C8 0002
              56CA 06A0    BL   @WAITQR4 ; Fetch TIB
              56CC 578C W
              56CE C844  D MOV  R4,@2(R1); To semaphore.queue (next one waiting)
              56D0 0002
              56D2 C960  ` MOV  @CONNIL,@20D(R5)       ; TIB_Hang_Ptr := nil
              56D4 40AA @
              56D6 0014
              56D8 C120    MOV  @READYQ,R4
              56DA 2AB6 *
              56DC 06A0    BL   @TASKINS               ; Insert signalled task in Ready_Queue
              56DE 575C W\
              56E0 C804    MOV  R4,@READYQ                          ; Update Ready_Q
              56E2 2AB6 *
              56E4 C060  ` MOV  @CURTSK,R1
              56E6 2ABA *
              56E8 9865  e CB   @3(R5),@3(R1)                       ; Fetched TIB.priority vs.
Curtsk.priority
              56EA 0003
              56EC 0003
              56EE 1A16    JL   SIGJUMP  ; Return if fetched task has lower priority
              56F0 C060  ` MOV  @CURSIB,R1             ; Perform the task switch
              56F2 2AB2 *
              56F4 A860  ` A    @STAMP,@4(R1)          ; Increment SIB.TimeStamp
              56F6 40B4 @
              56F8 0004
              56FA 1902    JNO  NOROLL   ; Rollover?
              56FC 06A0    BL   @ROLLOVR ; Handle time stamp rollover
              56FE 5A60 Z`
NOROLL        5700 06A0    BL   @UPDTIB  ; Saves task info in current TIB (R5 points to task)
              5702 579A W
              5704 C120    MOV  @READYQ,R4
              5706 2AB6 *
              5708 06A0    BL   @TASKINS ; Insert task at head of Ready_Q
              570A 575C W\
              570C 06A0    BL   @WAITQR4 ; Fetch TIB. Returns pointer to first ready task in Reqdy_Q in R4.
              570E 578C W
              5710 C804    MOV  R4,@READYQ
              5712 2AB6 *
              5714 C805    MOV  R5,@CURTSK             ; Make task to run the current one
              5716 2ABA *
              5718 06A0    BL   @STRTTSK ; Start task
              571A 57BE W
SIGRET        571C 0460  ` B    @LOADADDR; Load address to p-code and execute
              571E 4142 AB

; Wait for semaphore
WAIT          5720 5722    .WORD WAIT+2
              5722 C07A  z MOV  *SP+,R1  ; Semaphore address
              5724 C0A1    MOV  @0(R1),R2; Semaphore count
              5726 0000
              5728 1303    JEQ  DOWAIT   ; Is zero, must wait?
              572A 0621  ! DEC  @0(R1)   ; Decrement semaphore and continue
              572C 0000
              572E 045C  \ B    *R12     ; PME fetch

DOWAIT        5730 06A0    BL   @UPDTIB  ; Save task info in current TIB
              5732 579A W
              5734 C121  ! MOV  @2(R1),R4; Pointer to waiting task
              5736 0002
              5738 06A0    BL   @TASKINS ; Place task in queue waiting for this semaphore.
              573A 575C W\
              573C C844  D MOV  R4,@2(R1); Semaphore pointer to first task in queue
              573E 0002
```

```
             5740 C941  A MOV  R1,@20D(R5)           ; Update Hang_Ptr to know what this task is waiting
for
             5742 0014
             5744 C120    MOV  @READYQ,R4            ; Get the task at head of Ready_Q
             5746 2AB6 *
             5748 06A0    BL   @WAITQR4 ; Take out the first task from Ready_Q.
             574A 578C W
             574C C804    MOV  R4,@READYQ            ; Make it the running task.
             574E 2AB6 *
             5750 C805    MOV  R5,@CURTSK
             5752 2ABA *
             5754 06A0    BL   @STRTTSK ; Start that task
             5756 57BE W
             5758 0460   ` B    @LOADADDR; Load address to p-code and execute
             575A 4142 AB

; Inserts task in queue pointed to by R4. Returns new Ready_Queue in R4.
TASKINS      575C C184    MOV  R4,R6                 ; Ready_Q.
             575E C1E0    MOV  @CONNIL,R7
             5760 40AA @
NXTTIB       5762 8806    C    R6,@CONNIL            ; Ready_Q = nil?
             5764 40AA @
             5766 1308    JEQ  LOWPRIO               ; Nothing in Ready_Q.
             5768 9966  f CB   @3(R6),@3(R5)         ; Compare task priorities
             576A 0003
             576C 0003
             576E 1A04    JL   LOWPRIO  ; First task in Ready_Q has lower priority
             5770 C1C6    MOV  R6,R7                 ; Ready_Q to R7
             5772 C1A6    MOV  @0(R6),R6; Pointer to next TIB in Ready_Q
             5774 0000
             5776 10F5    JMP  NXTTIB   ; Continue to find where to put this task.

LOWPRIO      5778 C946  F MOV  R6,@0(R5); READYQ -> Semaphore.queue^.WaitQ
             577A 0000
             577C 8807    C    R7,@CONNIL            ; Pointer to new task, if any.
             577E 40AA @
             5780 1303    JEQ  NONEW                 ; Nothing new to run?
             5782 C9C5    MOV  R5,@0(R7); Task pointer to new one
             5784 0000
             5786 045B  [ B    *R11
NONEW        5788 C105    MOV  R5,R4                 ; Nothing to run, semaphore.queue^ is new task.
             578A 045B  [ B    *R11

; TIB_Wait_Q to R4
WAITQR4      578C C144  D MOV  R4,R5
             578E C125  % MOV  @0(R5),R4; TIB pointer of the TIB waiting for the semaphore
             5790 0000
             5792 C960  ` MOV  @CONNIL,@0(R5)        ; Nothing waiting for this one
             5794 40AA @
             5796 0000
             5798 045B  [ B    *R11

;Update TIB with current register data
UPDTIB       579A C160  ` MOV  @CURTSK,R5
             579C 2ABA *
             579E C949  I MOV  R9,@10D(R5)           ;Store frame pointer in TIB (MSCW_Ptr)
             57A0 000A
             57A2 C948  H MOV  R8,@14D(R5)           ;Store IPC in TIB
             57A4 000E
             57A6 6960 i` S    @SEGBASE,@14D(R5)     ; Segment relative IPC
             57A8 2AE2 *
             57AA 000E
             57AC C960  ` MOV  @ERECP,@16D(R5)       ; Environment record pointer
             57AE 2AB4 *
             57B0 0010
             57B2 C94A  J MOV  SP,@8(R5); Stack pointer
             57B4 0008
             57B6 D960  ` MOVB @CURPRNO,@19D(R5)     ; Current procedure number
             57B8 2AC3 *
             57BA 0013
             57BC 045B  [ B    *R11

; Update current registers from TIB
STRTTSK      57BE C80B    MOV  R11,@RETSAV2
             57C0 2ADC *
             57C2 C160  ` MOV  @CURTSK,R5            ; Current task
             57C4 2ABA *
             57C6 C265  e MOV  @10D(R5),R9           ; Local frame pointer
             57C8 000A
             57CA C225  % MOV  @14D(R5),R8           ; IPC
             57CC 000E
             57CE C2A5    MOV  @8(R5),SP; Stack pointer
             57D0 0008
             57D2 04E0    CLR  @CURPRNO-1
```

```
             57D4 2AC2 *
             57D6 D825  % MOVB @19D(R5),@CURPRNO      ; Procedure number
             57D8 0013
             57DA 2AC3 *
             57DC C165  e MOV  @16D(R5),R5            ; E_Rec_P of new task
             57DE 0010
             57E0 C805    MOV  R5,@ERECP; Current environment record pointer
             57E2 2AB4 *
             57E4 C825  % MOV  @2(R5),@EVECP          ; Current environment vector pointer
             57E6 0002
             57E8 2AB8 *
             57EA C825  % MOV  @4(R5),@CURSIB
             57EC 0004
             57EE 2AB2 *
             57F0 C3A5    MOV  @0(R5),R14             ; Global frame pointer
             57F2 0000
             57F4 04E0    CLR  @SEGBASE
             57F6 2AE2 *
             57F8 C808    MOV  R8,@IPCSAVE
             57FA 2ACC *
             57FC 06A0    BL   @GETSEGB ; Find base of segment via E_Rec_P
             57FE 4ECC N
             5800 C804    MOV  R4,@SEGBASE
             5802 2AE2 *
             5804 A204    A    R4,R8                  ; IPC is segment relative
             5806 C2E0    MOV  @RETSAV2,R11
             5808 2ADC *
             580A 0460  ` B    @SETPNT  ; Set pointers to segment information
             580C 4FCA O

; Kernel procedure IOCHECK
IOCHECK      580E C060  ` MOV  @SYSCOM,R1
             5810 2B6E +n
             5812 1601    JNE  IOERROR
             5814 045C  \ B    *R12                   ; PME fetch

IOERROR      5816 06A0    BL   @FAULGEN
             5818 484A HJ
             581A 000A    .WORD 10      ; User IO error

; Kernel procedure GETPOOLBYTES
GETPOOL      581C C17A  z MOV  *SP+,R5  ; E_Rec_P for segment
             581E C808    MOV  R8,@IPCSAVE            ; Prepare for segment fault
             5820 2ACC *
             5822 05A0    INC  @MTYPFL
             5824 2AD6 *
             5826 06A0    BL   @GETSEGB ; Find base of segment via E_Rec_P
             5828 4ECC N
             582A 0460  ` B    @LOADADDR; Load address to p-code and execute
             582C 4142 AB

; Kernel procedure PUTPOOLBYTES
; Returns available memory in main code pool minus space required by non-swappable segments
PUTPOOL      582E 04C4    CLR  R4                     ; Total word count
             5830 C0A0    MOV  @SIBPNT,R2             ; Address to Segment information block
             5832 2B88 +
             5834 1308    JEQ  NOSIB
MORESIB      5836 C0E2    MOV  @8(R2),R3; Segment residency
             5838 0008
             583A 1302    JEQ  CANSWAP  ; Swapable?
             583C A122  " A    @18D(R2),R4            ; Segment length (words)
             583E 0012
CANSWAP      5840 C0A2    MOV  @28D(R2),R2            ; Next SIB in list
             5842 001C
             5844 16F8    JNE  MORESIB
NOSIB        5846 C0A0    MOV  @TOPMEM,R2
             5848 2782 '
             584A 60A0 `  S    @INTMEM,R2             ; Topmem - intmem -> R2
             584C 2780 '
             584E 0912    SRL  R2,1                   ; Make words
             5850 6084 `  S    R4,R2                  ; Available - requested
             5852 C682    MOV  R2,*SP
             5854 045C  \ B    *R12                   ; PME fetch

; Kernel procedure MOVELEFT
; Figures out if MOV can be used (all numbers even), otherwise uses MOVB
; Source and destination are given as some offset+base
MOVLEFT      5856 C0FA    MOV  *SP+,R3  ; Byte count
             5858 1112    JLT  MOVEXIT
             585A 1311    JEQ  MOVEXIT
             585C C07A  z MOV  *SP+,R1  ; Destination
             585E A07A  z A    *SP+,R1
             5860 C0BA    MOV  *SP+,R2  ; Source
             5862 A0BA    A    *SP+,R2
```

```
             5864 C101    MOV  R1,R4                 ; Check if any number is odd
             5866 E102    SOC  R2,R4
             5868 E103    SOC  R3,R4
             586A 0914    SRL  R4,1
             586C 1804    JOC  BYTLMOV  ; If so move bytewise
WRDLMOV      586E CC72  r MOV  *R2+,*R1+; Move wordwise
             5870 0643  C DECT R3
             5872 15FD    JGT  WRDLMOV
             5874 045C  \ B    *R12                  ; PME fetch
BYTLMOV      5876 DC72  r MOVB *R2+,*R1+
             5878 0603    DEC  R3
             587A 15FD    JGT  BYTLMOV
             587C 045C  \ B    *R12                  ; PME fetch
MOVEXIT      587E 022A  * AI   SP,8                  ; Pop remaining arguments
             5880 0008
             5882 045C  \ B    *R12                  ; PME fetch

; Kernel procedure MOVERIGHT
; Will always move bytewise
MOVRIGT      5884 C0FA    MOV  *SP+,R3  ; Byte count
             5886 11FB    JLT  MOVEXIT
             5888 13FA    JEQ  MOVEXIT
             588A C07A  z MOV  *SP+,R1  ; Destination
             588C A07A  z A    *SP+,R1
             588E C0BA    MOV  *SP+,R2  ; Source
             5890 A0BA    A    *SP+,R2
             5892 C103    MOV  R3,R4
             5894 0604    DEC  R4
             5896 A044  D A    R4,R1                 ; Adjust pointers to point at end of data
             5898 A084    A    R4,R2
BYTRMOV      589A D452  R MOVB *R2,*R1
             589C 0602    DEC  R2
             589E 0601    DEC  R1
             58A0 0603    DEC  R3
             58A2 15FB    JGT  BYTRMOV
             58A4 045C  \ B    *R12                  ; PME fetch

; Kernel procecdure FILLCHAR
; Will optimize with word MOV if address and byte count are even
FILLCHR      58A6 C0FA    MOV  *SP+,R3  ; Character to fill
             58A8 06C3    SWPB R3
             58AA C13A  : MOV  *SP+,R4  ; Number of characters
             58AC 1111    JLT  FILEXIT
             58AE 1310    JEQ  FILEXIT
             58B0 C07A  z MOV  *SP+,R1  ; Destination offset+base
             58B2 A07A  z A    *SP+,R1
             58B4 C001    MOV  R1,R0
             58B6 E004    SOC  R4,R0
             58B8 0910    SRL  R0,1
             58BA 1806    JOC  FILBYTE
             58BC D803    MOVB R3,@PASCALWS+7         ; Duplicate byte to fill with
             58BE 8387
FILWORD      58C0 CC43  C MOV  R3,*R1+
             58C2 0644  D DECT R4
             58C4 15FD    JGT  FILWORD
             58C6 045C  \ B    *R12                  ; PME fetch
FILBYTE      58C8 DC43  C MOVB R3,*R1+
             58CA 0604    DEC  R4
             58CC 15FD    JGT  FILBYTE
             58CE 045C  \ B    *R12                  ; PME fetch
FILEXIT      58D0 022A  * AI   SP,4                  ; Pop remaining arguments
             58D2 0004
             S58D4 045C  \ B    *R12                 ; PME fetch

; Kernel procedure TIME
TIME         58D6 C07A  z MOV  *SP+,R1  ; Get pointer. Arguments are var-parameters
             58D8 C460  ` MOV  @TIMELOW,*R1
             58DA 2BA4 +
             58DC C07A  z MOV  *SP+,R1
             58DE C460  ` MOV  @TIMEHI,*R1
             58E0 2BA6 +
             58E2 045C  \ B    *R12     ; PME fetch

; Kernel procedure SCAN
; The parameters aren't what they look like at Pascal level
SCAN         58E4 04C0    CLR  R0                    ; Position in byte array
             58E6 05CA    INCT SP                    ; Skip unused mask field parameter
             58E8 C07A  z MOV  *SP+,R1  ; Byte-pointer to array
             58EA A07A  z A    *SP+,R1  ; Add base
             58EC C0BA    MOV  *SP+,R2  ; Character to look for
             58EE 06C2    SWPB R2                    ; Put in byte position
             58F0 C0FA    MOV  *SP+,R3  ; Scan type (= or <>)
             58F2 C13A  : MOV  *SP+,R4  ; Scan length
             58F4 1318    JEQ  ENDSCAN
```

```
             58F6 110B    JLT  GOLEFT   ; Scans to the left
LOOP58       58F8 90B1    CB   *R1+,R2
             58FA 1306    JEQ  ISEQUAL
             58FC C0C3    MOV  R3,R3
             58FE 1613    JNE  ENDSCAN  ; If we jump here bytes were not equal and that's what we are
looking for
LOOP59       5900 0580    INC  R0                    ; Increase position
             5902 0604    DEC  R4                    ; Byte count
             5904 15F9    JGT  LOOP58
             5906 100F    JMP  ENDSCAN  ; Ran out of byte count

ISEQUAL      5908 C0C3    MOV  R3,R3
             590A 130D    JEQ  ENDSCAN  ; Should be equal
             590C 10F9    JMP  LOOP59

GOLEFT       590E 04C0    CLR  R0                    ; Position in array
LOOP60       5910 9091    CB   *R1,R2
             5912 1307    JEQ  LEFTEQ   ; They are equal?
             5914 C0C3    MOV  R3,R3
             5916 1607    JNE  ENDSCAN  ; If we jump here bytes were not equal and that's what we are
looking for
LOOP61       5918 0600    DEC  R0                    ; Distance will be negative when searching backwards
             591A 0601    DEC  R1
             591C 0584    INC  R4
             591E 11F8    JLT  LOOP60   ; More bytes to look at
             5920 1002    JMP  ENDSCAN

LEFTEQ       5922 C0C3    MOV  R3,R3
             5924 16F9    JNE  LOOP61   ; Should not be equal
ENDSCAN      5926 C680    MOV  R0,*SP
             5928 045C  \ B    *R1

; Kernel procedure IORESULT
IORES        592A C6A0    MOV  @SYSCOM,*SP
             592C 2B6E +n
             592E 045C  \ B    *R12     ; PME fetch

; Kernel procedure 35, reserved, causes breakpoint halt
HALTBRP      5930 06A0    BL   @FAULGEN
             5932 484A HJ
             5934 000E    .WORD 14      ; Halt, breakpoint

; Unimplemented instruction fault, but seems not to be called from any place
             5936 06A0    BL   @FAULGEN
             5938 484A HJ
             593A 000B    .WORD BH      ; Unimplemented instruction

; Compiler support - Kernel procedure IDSEARCH
IDSRC        593C C07A  z MOV  *SP+,R1  ; Buffer
             593E C801    MOV  R1,@IDSRBUF           ; Buffer origin
             5940 2AEE *
             5942 C0BA    MOV  *SP+,R2  ; Record pointer to SYMCURSOR
             5944 A052  R A    *R2,R1   ; Add cursor pointer to buffer base pointer
             5946 C802    MOV  R2,@SYMCURS           ; Save pointer to SYMCURSOR
             5948 2ADE *
             594A C102    MOV  R2,R4
             594C 05C2    INCT R2                    ; Point to SY
             594E 0224  $ AI   R4,6                  ; Point to ID
             5950 0006
             5952 0203    LI   R3,4
             5954 0004
             5956 C144  D MOV  R4,R5                 ; ID pointer
SPFILL       5958 CD20    MOV @SPACES,*R4+           ; Space filler
             595A 40A2 @
             595C 0603    DEC  R3                    ; Eight spaces
             595E 15FC    JGT  SPFILL
             5960 C105    MOV  R5,R4
             5962 0203    LI   R3,8                  ; Eight characters to get
             5964 0008
SKIPUSC      5966 D1B1    MOVB *R1+,R6
             5968 9806    CB   R6,@USCORE            ; '_'
             596A 409A @
             596C 13FC    JEQ  SKIPUSC
             596E 9806    CB   R6,@NUMERIC+1         ; '9'
             5970 4097 @
             5972 120C    JLE  LOWNUM
             5974 0246  F ANDI R6,DF00  ; Make lower case
             5976 DF00
             5978 9806    CB   R6,@UCASE; 'A'
             597A 4094 @
             597C 1A0D    JL   NOLETT   ; Not a letter
             597E 9806    CB   R6,@UCASE+1           ; 'Z'
             5980 4095 @
             5982 1B0A    JH   NOLETT   ; Not a letter
```

```
ISNUM       5984 0603    DEC  R3
            5986 11EF    JLT  SKIPUSC
            5988 DD06    MOVB R6,*R4+
            598A 10ED    JMP  SKIPUSC

LOWNUM      598C 9806    CB   R6,@NUMERIC           ; '0'
            598E 4096 @
            5990 14F9    JHE  ISNUM
            5992 9806    CB   R6,@USCORE+1          ; Dollar
            5994 409B @
            5996 13F6    JEQ  ISNUM
NOLETT      5998 0641  A DECT R1
            599A 6060 `` S    @IDSRBUF,R1
            599C 2AEE *
            599E C801    MOV  R1,@IDIDENT           ; Identifier
            59A0 2AC8 *
            59A2 C105    MOV  R5,R4                 ; Pointer to retreived identifier
            59A4 04C1    CLR  R1
            59A6 D054  T MOVB *R4,R1   ; Fetch first byte
            59A8 06C1    SWPB R1
            59AA 0221  ! AI   R1,-65D  ; Ascii 'A'
            59AC FFBF
            59AE 0A21  ! SLA  R1,2                  ; Make word index
            59B0 C0E1    MOV  @IDPOINT(R1),R3       ; ID name pointer
            59B2 3026 0&
            59B4 C061  a MOV  @IDCOUNT(R1),R1       ; Number of identifiers with same first letter
            59B6 3028 0(
            59B8 0460  ` B    @IDCONT
            59BA 59BC Y

IDCONT      59BC 0200    LI   R0,4                  ; Word count for identifier
            59BE 0004
            59C0 C803    MOV  R3,@SAVPACK           ; ID name pointer
            59C2 2AF2 *
LOOP65      59C4 8CF4    C    *R4+,*R3+
            59C6 1615    JNE  59F2
            59C8 0600    DEC  R0                    ; End of identifier
            59CA 15FC    JGT  LOOP65
            59CC C060  ` MOV  @SYMCURS,R1           ; Identifier is found
            59CE 2ADE *
            59D0 C460  ` MOV  @IDIDENT,*R1
            59D2 2AC8 *
            59D4 04D2    CLR  *R2                   ; R2 points to SY
            59D6 D8B3    MOVB *R3+,@1(R2)           ; Store SY value
            59D8 0001
            59DA 05C2    INCT R2                    ; R2 points to OP
            59DC 04D2    CLR  *R2
            59DE D8B3    MOVB *R3+,@0001(R2)        ; Store OP value
            59E0 0001
            59E2 05C2    INCT R2                    ; Point to ID field after OP
            59E4 0204    LI   R4,4
            59E6 0004
LOOP66      59E8 CCA0    MOV @SPACES,*R2+           ; Space filling
            59EA 40A2 @
            59EC 0604    DEC  R4
            59EE 15FC    JGT  LOOP66
            59F0 045C  \ B    *R12                  ; PME fetch

            59F2 0601    DEC  R1       ; Identifier count with same first letter
            59F4 1306    JEQ  RANOUT   ; Ran out of entries in table?
            59F6 C0E0    MOV  @SAVPACK,R3           ; Saved ID name pointer
            59F8 2AF2 *
            59FA A0E0    A    @LINEFED,R3           ; Same as AI R3,10. Distance to next entry in table
            59FC 40A4 @
            59FE C105    MOV  R5,R4                 ; Pointer to retreived identifier
            5A00 10DD    JMP  IDCONT

RANOUT      5A02 C0A0    MOV  @SYMCURS,R2           ; Recall saved pointer
            5A04 2ADE *
            5A06 C4A0    MOV  @IDIDENT,*R2          ; Recall saved identifier
            5A08 2AC8 *
            5A0A 05C2    INCT R2
            5A0C 04F2    CLR  *R2+                  ; SY value for "not found" is zero
            5A0E 0201    LI   R1,15                 ; OP value for "not found"
            5A10 000F
            5A12 C481    MOV  R1,*R2
            5A14 045C  \ B    *R12                  ; PME fetch

; Compiler support - Kernel procedure TREESEARCH
TREESRC     5A16 C07A  z MOV  *SP+,R1  ; R1 target pointer (packed array[0..7] of char
            5A18 C0AA    MOV  @2(SP),R2; R2 root pointer
            5A1A 0002
NEWTRY      5A1C 8491    C    *R1,*R2  ; Compare char 0..1
            5A1E 160E    JNE  NOTSAME
```

```
              5A20 88A1    C    @2(R1),@2(R2)          ; Compare char 2..3
              5A22 0002
              5A24 0002
              5A26 160A    JNE  NOTSAME
              5A28 88A1    C    @4(R1),@4(R2)          ; Compare char 4..5
              5A2A 0004
              5A2C 0004
              5A2E 1606    JNE  NOTSAME
              5A30 88A1    C    @6(R1),@6(R2)          ; Compare char 6..7
              5A32 0006
              5A34 0006
              5A36 1602    JNE  NOTSAME
              5A38 04C1    CLR  R1       ; Return 0, target found
              5A3A 100C    JMP  RETTRS

NOTSAME       5A3C 1B06    JH   RIGHT
              5A3E C122   " MOV  @A(R2),R4; R4 left link
              5A40 000A
              5A42 160C    JNE  NEWTRY1  ; Already occupied?
              5A44 0201    LI   R1,-1    ; Return -1, target to the left
              5A46 FFFF
              5A48 1005    JMP  RETTRS

RIGHT         5A4A C122   " MOV  @8(R2),R4; R4 right link
              5A4C 0008
              5A4E 1606    JNE  NEWTRY1  ; Already occupied?
              5A50 0201    LI   R1,1     ; Return 1, target to the right
              5A52 0001
RETTRS        5A54 C13A   : MOV  *SP+,R4  ; Tree root
              5A56 C502    MOV  R2,*R4   ; Result pointer
              5A58 C681    MOV  R1,*SP   ; Return status indicator
              5A5A 045C   \ B    *R12                  ; PME fetch
NEWTRY1       5A5C C084    MOV  R4,R2
              5A5E 10DE    JMP  NEWTRY

; Routine for SIB time stamp rollover
ROLLOVR       5A60 C801    MOV  R1,@SAVER1
              5A62 2A26 *&
              5A64 C800    MOV  R0,@SAVER0
              5A66 2A24 *$
              5A68 C060   ` MOV  @SIBPNT,R1
              5A6A 2B88 +
              5A6C 1308    JEQ  ENDROLL  ; Pointer = nil?
LOOP37        5A6E C021   ! MOV  @4(R1),R0; Time stamp in SIB
              5A70 0004
              5A72 0910    SRL  R0,1
              5A74 C840   @ MOV  R0,@4(R1)
              5A76 0004
              5A78 C061   a MOV  @1CH(R1),R1            ; Link to next SIB
              5A7A 001C
              5A7C 16F8    JNE  LOOP37
ENDROLL       5A7E C060   ` MOV  @SAVER1,R1
              5A80 2A26 *&
              5A82 C020    MOV  @SAVER0,R0
              5A84 2A24 *$
              5A86 045B   [ B    *R11

REAL10K       5A88 4201 B  .WORD 4201H   ; Real value 10000 header

PUSH10K       5A8A 064A   J DECT SP                     ; Places real value 10000.0 on stack
              5A8C 04DA    CLR  *SP
              5A8E 064A   J DECT SP
              5A90 04DA    CLR  *SP
              5A92 064A   J DECT SP
              5A94 04DA    CLR  *SP
              5A96 064A   J DECT SP
              5A98 C6A0    MOV  @REAL10K,*SP           ; 4201H (1+100^2)
              5A9A 5A88 Z
              5A9C 045B   [ B    *R11

SEGLOAD       5A9E C80B    MOV  R11,@SAVER11
              5AA0 2A34 *4
              5AA2 020B    LI   R11,SAVER0             ; Register savearea
              5AA4 2A24 *$
              5AA6 CEC0    MOV  R0,*R11+ ; Save all working registers
              5AA8 CEC1    MOV  R1,*R11+
              5AAA CEC2    MOV  R2,*R11+
              5AAC CEC3    MOV  R3,*R11+
              5AAE CEC4    MOV  R4,*R11+
              5AB0 CEC5    MOV  R5,*R11+
              5AB2 CEC6    MOV  R6,*R11+
              5AB4 CEC7    MOV  R7,*R11+
              5AB6 C121   ! MOV  @18D(R1),R4            ; R1 is called segments SIB_P. Segment length -> R4
              5AB8 0012
```

```
RESTART       5ABA C020    MOV  @INTMEM,R0            ; Interpreters memory pointer in primary code pool
(VDP RAM)
              5ABC 2780 '
              5ABE 04C1    CLR  R1
              5AC0 04C2    CLR  R2                   ; Some accumulated space...
              5AC2 0203    LI   R3,STAMP ; To start comparing age to
              5AC4 40B4 @
              5AC6 C1A0    MOV  @SIBPNT,R6           ; Current SIB
              5AC8 2B88 +
              5ACA 131C    JEQ  SIBNIL1
NEXTSIB       5ACC 88E6    C    @4(R6),@4(R3)        ; Compare time stamps. @4(STAMP) is -1
              5ACE 0004
              5AD0 0004
              5AD2 1B07    JH   NOSWAP
              5AD4 8806    C    R6,@CURSIB
              5AD6 2AB2 *
              5AD8 1304    JEQ  NOSWAP   ; Current segment?
              5ADA C166  f MOV  @8(R6),R5; Residency
              5ADC 0008
              5ADE 1601    JNE  NOSWAP   ; Not swappable
              5AE0 C0C6    MOV  R6,R3                ; Move current to the one to compare with
NOSWAP        5AE2 C166  f MOV  @2(R6),R5; Offset in segment's pool
              5AE4 0002
              5AE6 6140 a@ S    R0,R5                ; Subtract INTMEM
              5AE8 0915    SRL  R5,1                 ; Word.
              5AEA 8105    C    R5,R4                ; Compare to new segment length (words)
              5AEC 1560  ` JGT  SEGFITS
              5AEE A085    A    R5,R2                ; Accumulate space...
              5AF0 C046  F MOV  R6,R1                ; Current SIB
              5AF2 C021  ! MOV  @2(R1),R0; Byte offset in code pool for segment
              5AF4 0002
              5AF6 A021  ! A    @18D(R1),R0          ; Words in segment
              5AF8 0012
              5AFA A021  ! A    @18D(R1),R0          ; Words in segment (now it's bytes)
              5AFC 0012
              5AFE C1A1    MOV  @28D(R1),R6          ; Next SIB in list
              5B00 001C
              5B02 16E4    JNE  NEXTSIB
SIBNIL1       5B04 C160  ` MOV  @TOPMEM,R5
              5B06 2782 '
              5B08 6140 a@ S    R0,R5                ; R5 := topmem-intmem or bytes in segment
              5B0A 0915    SRL  R5,1                 ; To words
              5B0C 8105    C    R5,R4                ; Compare to new segment length (words)
              5B0E 154F  O JGT  SEGFITS
              5B10 A085    A    R5,R2                ; Add size to R2
              5B12 8102    C    R2,R4
              5B14 1511    JGT  ENOUGH   ; Enough to fit new segment?
              5B16 04E3    CLR  @2(R3)   ; Byte offset in code pool
              5B18 0002
              5B1A C0A3    MOV  @28D(R3),R2          ; Next SIB in list
              5B1C 001C
              5B1E C063  c MOV  @26D(R3),R1          ; Previous SIB in list
              5B20 001A
              5B22 1303    JEQ  PREVNIL
              5B24 C842  B MOV  R2,@28D(R1)          ; Next SIB in list to current
              5B26 001C
              5B28 1002    JMP  HASPREV

PREVNIL       5B2A C802    MOV  R2,@SIBPNT
              5B2C 2B88 +
HASPREV       5B2E C082    MOV  R2,R2
              5B30 13C4    JEQ  RESTART
              5B32 C881    MOV  R1,@26D(R2)          ; Previous SIB in list
              5B34 001A
              5B36 10C1    JMP  RESTART

ENOUGH        5B38 C1A0    MOV  @SIBPNT,R6
              5B3A 2B88 +
              5B3C C020    MOV  @INTMEM,R0
              5B3E 2780 '
LINKSIB       5B40 C166  f MOV  @18D(R6),R5          ; Segment length
              5B42 0012
              5B44 0A15    SLA  R5,1
              5B46 C0A6    MOV  @2(R6),R2; Byte offset
              5B48 0002
              5B4A 8002    C    R2,R0
              5B4C 1602    JNE  5B52
              5B4E A005    A    R5,R0
              5B50 102A  * JMP  5BA6

              5B52 C980    MOV  R0,@2(R6); Byte offset
              5B54 0002
              5B56 0204    LI   R4,8800  ; VDPRD
              5B58 8800
```

```
             5B5A D920    MOVB @PASCALWS+5,@WAOFFS(R4)             ; LSBy PascalWS R2 to VDPWA
             5B5C 8385
             5B5E 0402
             5B60 0203    LI   R3,FAC   ; Used as buffer
             5B62 834A  J
             5B64 D902    MOVB R2,@WAOFFS(R4)         ; To VDPWA
             5B66 0402
             5B68 0207    LI   R7,35D   ; sizeof(SIB), or so...
             5B6A 0023  #
             5B6C 81C5    C    R5,R7
             5B6E 1A02    JL   5B74                   ;
             5B70 6147 aG S    R7,R5
             5B72 1002    JMP  5B78

             5B74 C1C5    MOV  R5,R7
             5B76 04C5    CLR  R5
             5B78 C047  G MOV  R7,R1
LOOP67       5B7A DCD4    MOVB *R4,*R3+ ; From VDPRD to FAC
             5B7C 0607    DEC  R7
             5B7E 15FD    JGT  LOOP67
             5B80 0204    LI   R4,VDPWA
             5B82 8C02
             5B84 C1C0    MOV  R0,R7
             5B86 0267  g ORI  R7,4000  ; Address in write mode
             5B88 4000 @
             5B8A D520    MOVB @PASCALWS+15,*R4       ; LSBy R7
             5B8C 838F
             5B8E 0203    LI   R3,FAC   ; Used as buffer
             5B90 834A  J
             5B92 D507    MOVB R7,*R4
             5B94 C1C1    MOV  R1,R7
             5B96 A087    A    R7,R2
             5B98 A007    A    R7,R0
             5B9A 0644  D DECT R4
LOOP68       5B9C D533  3 MOVB *R3+,*R4
             5B9E 0607    DEC  R7
             5BA0 15FD    JGT  LOOP68
             5BA2 C145  E MOV  R5,R5
             5BA4 16D8    JNE  5B56
             5BA6 C046  F MOV  R6,R1
             5BA8 C1A1    MOV  @28D(R1),R6            ; Next_SIB
             5BAA 001C
             5BAC 16C9    JNE  LINKSIB
SEGFITS      5BAE C0A0    MOV  @SAVER1,R2
             5BB0 2A26 *&
             5BB2 C041  A MOV  R1,R1                  ; SIB pointer
             5BB4 130A    JEQ  LASTSIB
             5BB6 C021  ! MOV  @18D(R1),R0            ; Words in segment. Calculate position to load segment
             5BB8 0012
             5BBA 0A10    SLA  R0,1                   ; Bytes
             5BBC A021  ! A    @2(R1),R0; Offset in segment (bytes)
             5BBE 0002
             5BC0 C0E1    MOV  @28D(R1),R3            ; Next_SIB
             5BC2 001C
             5BC4 C842  B MOV  R2,28D(R1)             ; Update Next_SIB
             5BC6 001C
             5BC8 1006    JMP  SAMESIB  ; Keep the same SIB

LASTSIB      5BCA C020    MOV  @INTMEM,R0             ; Buffer pointer
             5BCC 2780 '
             5BCE C0E0    MOV  @SIBPNT,R3             ; Save previous SIB pointer
             5BD0 2B88 +
             5BD2 C802    MOV  R2,@SIBPNT             ; Put new segment in SIB pointer
             5BD4 2B88 +
SAMESIB      5BD6 C880    MOV  R0,@2(R2); Byte offset within code pool
             5BD8 0002
             5BDA 04E2    CLR  @4(R2)   ; Time_Stamp
             5BDC 0004
             5BDE C881    MOV  R1,@26D(R2)            ; Prev_SIB in new segment
             5BE0 001A
             5BE2 C883    MOV  R3,@28D(R2)            ; Next_SIB in new segment
             5BE4 001C
             5BE6 1302    JEQ  DISKSEG
             5BE8 C8C2    MOV  R2,@26D(R3)            ; Previous SIB in list to what was in SIBPNT before
             5BEA 001A
DISKSEG      5BEC C122  " MOV  @22D(R2),R4            ; Pointer to disk drive information
             5BEE 0016
             5BF0 064A  J DECT SP
             5BF2 C694    MOV  *R4,*SP  ; Unit number
             5BF4 064A  J DECT SP
             5BF6 C680    MOV  R0,*SP   ; Buffer pointer (current INTMEM)
             5BF8 1003    JMP  JPGROM1  ; Skip GROM access area

             5BFA 0000    .WORD 0
```

```
GROMRD      5BFC 0006    ;GROM read data
GROMRA      5BFE 6906 i  ;GROM read address

JPGROM1     5C00 064A  J DECT SP
            5C02 C6A0    MOV  @BUFIX,*SP            ; Index
            5C04 40BC @
            5C06 C122  " MOV  @18D(R2),R4           ; Segment length (word)
            5C08 0012
            5C0A 0A14    SLA  R4,1                  ; To bytes
            5C0C 064A  J DECT SP
            5C0E C684    MOV  R4,*SP   ; Byte count
            5C10 064A  J DECT SP
            5C12 C6A2    MOV  @20D(R2),*SP          ; Disk address of segment -> Block number
            5C14 0014
            5C16 0204    LI   R4,4000H ; Control word
            5C18 4000 @
            5C1A 064A  J DECT SP
            5C1C C684    MOV  R4,*SP   ; Control
            5C1E 06A0    BL   @UNITRD  ; Read from drive
            5C20 34AC 4
            5C22 C80C    MOV  R12,@SAVER12          ; PME fetch address
            5C24 2A3E *>
            5C26 C80D    MOV  R13,@SAVER13          ; Code read address
            5C28 2A3A *:
            5C2A C80F    MOV  R15,@SAVER15          ; Code type
            5C2C 2A3C *<
            5C2E C820    MOV  @PMEFLIP,@SAVFLIP     ; Current data flip status
            5C30 2AE8 *
            5C32 2A40 *@
            5C34 05A0    INC  @PMEFLIP
            5C36 2AE8 *
            5C38 020F    LI   R15,FFFF ; Code type VDP RAM
            5C3A FFFF
            5C3C 020D    LI   R13,8800 ; VDPRD
            5C3E 8800
            5C40 C160  ` MOV  @SAVER1,R5
            5C42 2A26 *&
            5C44 C020    MOV  @SYSCOM,R0            ; IO-result
            5C46 2B6E +n
            5C48 1612    JNE  SEGFAIL  ; Failed finding segment on disk
            5C4A C0C5    MOV  R5,R3
            5C4C C023  # MOV  @2(R3),R0
            5C4E 0002
            5C50 C040  @ MOV  R0,R1                 ; Code segment pointer
            5C52 0221  ! AI   R1,4                  ; Segment name is four bytes down
            5C54 0004
            5C56 0223  # AI   R3,10D   ; Segment name offset in SIB
            5C58 000A
            5C5A 0204    LI   R4,4                  ; Length of segment name (in words)
            5C5C 0004
LOOP69      5C5E 06A0    BL   @GETDAT2 ; *R1+ -> R2
            5C60 4178 Ax
            5C62 8CC2    C    R2,*R3+  ; Compare segment name
            5C64 1604    JNE  SEGFAIL
            5C66 0604    DEC  R4
            5C68 15FA    JGT  LOOP69
            5C6A C040  @ MOV  R0,R1                 ; Code segment pointer
            5C6C 1046  F JMP  SEGFOUN

SEGFAIL     5C6E C085    MOV  R5,R2                 ; Pointer to code SIB
            5C70 0222  " AI   R2,10D   ; Segment name string offset
            5C72 000A
            5C74 0203    LI   R3,SEGNAME            ; Space for segment name in string
            5C76 2B00 +
            5C78 CCF2    MOV  *R2+,*R3+; Move segment name into string 'Segment XXXXXXXX not found'
            5C7A CCF2    MOV  *R2+,*R3+
            5C7C CCF2    MOV  *R2+,*R3+
            5C7E C4D2    MOV  *R2,*R3
            5C80 C065  e MOV  @22D(R5),R1           ; Display unit number
            5C82 0016
            5C84 C0F1    MOV  *R1+,R3
            5C86 04C2    CLR  R2
            5C88 3CA0 <  DIV  @TEN,R2  ; Convert to two decimal digits
            5C8A 2B6C +l
            5C8C 06C2    SWPB R2
            5C8E D0C2    MOVB R2,R3                 ; Combine quotient and reminder
            5C90 0223  # AI   R3,'00'  ; '00' Convert digits to text
            5C92 3030 00
            5C94 C803    MOV  R3,@UNITNUM           ; Unit number in text string 'Put volume VVVVVVV in
unit Y'
            5C96 2B30 +0
            5C98 0204    LI   R4,VOLNAME+1          ; Volume name
            5C9A 2B1F +
```

```
             5C9C 0203    LI   R3,7                  ; Seven characters
             5C9E 0007
             5CA0 D0B1    MOVB *R1+,R2  ; Length byte
             5CA2 1305    JEQ  NAMEND   ; End of name?
             5CA4 0982    SRL  R2,8                  ; Make word
             5CA6 60C2 `  S    R2,R3                 ; Calculate trailing spaces
LOOP76       5CA8 DD31  1 MOVB *R1+,*R4+; Move characters in name
             5CAA 0602    DEC  R2
             5CAC 15FD    JGT  LOOP76
NAMEND       5CAE 0201    LI   R1,2000  ; Space character
             5CB0 2000
             5CB2 C0C3    MOV  R3,R3
             5CB4 1303    JEQ  NOTRAIL  ; No trailing spaces?
LOOP77       5CB6 DD01    MOVB R1,*R4+  ; Pad with spaces
             5CB8 0603    DEC  R3
             5CBA 15FD    JGT  LOOP77
NOTRAIL      5CBC 0201    LI   R1,BUFFER; Buffer
             5CBE 2B50 +P
             5CC0 C0E0    MOV  @BUFCNT,R3            ; Byte count
             5CC2 2B6A +j
             5CC4 6283 b  S    R3,SP
             5CC6 C08A    MOV  SP,R2
LOOP70       5CC8 CCB1    MOV  *R1+,*R2+; Push buffer to stack
             5CCA 0643  C DECT R3
             5CCC 15FD    JGT  LOOP70
             5CCE 04CF    CLR  R15
             5CD0 06A0    BL   @UNITCL  ; Clear display

             5CD2 34DC 4
             5CD4 06A0    BL   @UNITWR  ; Write instructions
             5CD6 34C4 4
             5CD8 06A0    BL   @UNITRD  ; Wait for user
             5CDA 34AC 4
             5CDC C320    MOV  @SAVER12,R12          ; PME fetch address
             5CDE 2A3E *>
             5CE0 C360  ` MOV  @SAVER13,R13          ; Code read address
             5CE2 2A3A *:
             5CE4 C3E0    MOV  @SAVER15,R15          ; Code type flag
             5CE6 2A3C *<
             5CE8 C820    MOV  @SAVFLIP,@PMEFLIP     ; Segment data flip status
             5CEA 2A40 *@
             5CEC 2AE8 *
             5CEE C0A0    MOV  @SAVER1,R2
             5CF0 2A26 *&
             5CF2 C022  " MOV  @2(R2),R0
             5CF4 0002
             5CF6 0460  ` B    @DISKSEG
             5CF8 5BEC [

; Code segment found and loaded.

; Conversions from canonical constants to system's native real data format is done here, if needed.
; Canonical 64-bit constants are four to six words long. Three to six may be used by the constant, but it's
padded to
; at least four words to make space for the native format.
; First word is always the signed exponent. The comes mantissa words as four digits, 0-9999. The first
mantissa word
; is negative if the constant is negative. A negative value after that is a terminator.
; The terminator may come already in word 3, but may not come at all, if all words are used.

SEGFOUN      5CFA C1C1    MOV  R1,R7                 ; Code segment pointer
             5CFC 0221  ! AI   R1,12D   ; Byte sex indicator offset
             5CFE 000C
             5D00 06A0    BL   @GETDAT2 ; *R1+ -> R2
             5D02 4178 Ax
             5D04 04E0    CLR  @PMEFLIP
             5D06 2AE8 *
             5D08 0282    CI   R2,1                  ; If 1 then no flipping
             5D0A 0001
             5D0C 1602    JNE  NOFLIP2
             5D0E 05A0    INC  @PMEFLIP
             5D10 2AE8 *
NOFLIP2      5D12 06A0    BL   @GETDAT2 ; *R1+ -> R2  Constant pool pointer (segment relative word pointer)
             5D14 4178 Ax
             5D16 C082    MOV  R2,R2
             5D18 136F  o JEQ  NOCONST  ; Has no constant pool?
             5D1A 0A12    SLA  R2,1                  ; Make byte pointer
             5D1C A087    A    R7,R2                 ; Add segment base
             5D1E C042  B MOV  R2,R1                 ; Point to constant pool
             5D20 06A0    BL   @GETDAT2 ; *R1+ -> R2  Real pool pointer
             5D22 4178 Ax
             5D24 C082    MOV  R2,R2
             5D26 1368  h JEQ  NOCONST  ; No real constant pool?
             5D28 0641  A DECT R1                    ; Back to real pool pointer
```

```
                5D2A 0A12    SLA  R2,1                  ; Make offset pointer byte
                5D2C A042  B A    R2,R1                 ; Add real pool start
                5D2E 06A0    BL   @GETDAT2 ; *R1+ -> R2  Number of real constants
                5D30 4178 Ax
                5D32 C182    MOV  R2,R6                 ; Number of real constants
                5D34 1361  a JEQ  NOCONST
                5D36 C1C1    MOV  R1,R7                 ; Pointer to first real constant
NEXTCON         5D38 C047  G MOV  R7,R1
                5D3A C801    MOV  R1,@REALPNT           ; Real constant pointer
                5D3C 2A44 *D
                5D3E 06A0    BL   @GETDAT2 ; *R1+ -> R2  First word in real constant
                5D40 4178 Ax
                5D42 C802    MOV  R2,@REAL1ST           ; Save first word
                5D44 2A38 *8
                5D46 06A0    BL   @GETDAT2 ; *R1+ -> R2  Second word in real constant
                5D48 4178 Ax
                5D4A C1C1    MOV  R1,R7                 ; Save address in R7 (Now incremented by 4)
                5D4C 064A  J DECT SP
                5D4E C682    MOV  R2,*SP
                5D50 C820    MOV  @SBITMSK+4,@REALSIZ   ; Max four additional mantissa words
                5D52 2D96 -
                5D54 2A42 *B
                5D56 020C    LI   R12,CONT09            ; Return address for real math
                5D58 5D5E ]^
                5D5A 0460  ` B    @FLT+2   ; Float top of stack
                5D5C 5F26 _&
CONT09          5D5E C047  G MOV  R7,R1                 ; Returns here, get constant data pointer back
                5D60 06A0    BL   @GETDAT2 ; *R1+ -> R2  Third word in real constant
                5D62 4178 Ax
                5D64 C1C1    MOV  R1,R7                 ; Save address (incremented by 6 first time)
                5D66 C082    MOV  R2,R2
                5D68 111A    JLT  NEG3RD   ; Third word negative means end of list
                5D6A C0DA    MOV  *SP,R3   ; Second word again
                5D6C 1502    JGT  POS2ND   ; Second word positive?
                5D6E 1301    JEQ  POS2ND
                5D70 0502    NEG  R2                    ; Nope, negate third
POS2ND          5D72 C802    MOV  R2,@REAL3RD
                5D74 2A36 *6
                5D76 06A0    BL   @PUSH10K ; Push real constant 10000.0
                5D78 5A8A Z
                5D7A 020C    LI   R12,CONT10            ; Return address for real math
                5D7C 5D82 ]
                5D7E 0460  ` B    @MPR+2   ; Multiply real values on stack
                5D80 5ECA ^
CONT10          5D82 064A  J DECT SP                    ; Push on stack
                5D84 C6A0    MOV  @REAL3RD,*SP          ; Third word
                5D86 2A36 *6
                5D88 020C    LI   R12,CONT15            ; Return address for real math
                5D8A 5D90 ]
                5D8C 0460  ` B    @FLT+2   ; Float top of stack
                5D8E 5F26 _&
CONT15          5D90 020C    LI   R12,CONT11            ; Return address for real math
                5D92 5D98 ]
                5D94 0460  ` B    @ADR+2   ; Add real values on stack
                5D96 5EB2 ^
CONT11          5D98 0620    DEC  @REALSIZ ; Counter, starts at 4
                5D9A 2A42 *B
                5D9C 15E0    JGT  CONT09
NEG3RD          5D9E 022A  * AI   SP,-10D  ; Push on stack
                5DA0 FFF6
                5DA2 C6A0    MOV  @REAL1ST,*SP          ; First word
                5DA4 2A38 *8
                5DA6 1109    JLT  EXPNEG   ; Negative exponent?
                5DA8 020C    LI   R12,CONT12            ; Return address for real math
                5DAA 5DB0 ]
                5DAC 0460  ` B    @PWROFTEN; Expand exponent
                5DAE 5F5C _\
CONT12          5DB0 020C    LI   R12,CONT16
                5DB2 5DB8 ]
                5DB4 0460  ` B    @MPR+2   ; Multiply real
                5DB6 5ECA ^
CONT16          5DB8 1009    JMP  CONT14

EXPNEG          5DBA 051A    NEG  *SP
                5DBC 020C    LI   R12,CONT13            ; Return address for real math
                5DBE 5DC4 ]
                5DC0 0460  ` B    @PWROFTEN; Expand the negative exponent.
                5DC2 5F5C _\
CONT13          5DC4 020C    LI   R12,CONT14            ; Return address for real math
                5DC6 5DCC ]
                5DC8 0460  ` B    @DVR+2   ; Divide real
                5DCA 5EC2 ^
CONT14          5DCC 0203    LI   R3,VDPWA ; Converted constant written back
                5DCE 8C02
```

```
              5DD0 C020    MOV  @REALPNT,R0           ; Constant pointer
              5DD2 2A44 *D
              5DD4 0260  ` ORI  R0,4000  ; Set to write
              5DD6 4000 @
              5DD8 D4E0    MOVB @PASCALWS+1,*R3       ; LSBy R0
              5DDA 8381
              5DDC 0202    LI   R2,8                  ; Size of a real, in bytes
              5DDE 0008
              5DE0 D4C0    MOVB R0,*R3   ; Address set to constant in VDP RAM
              5DE2 0643  C DECT R3                    ; Change to VDPWD
              5DE4 8820    C    @SBITMSK+4,@REALSIZ
              5DE6 2D96 -
              5DE8 2A42 *B
              5DEA 1601    JNE  LOOP93
              5DEC 05C7    INCT R7                    ; Skip fourth word (never fetched)
LOOP93        5DEE D4FA    MOVB *SP+,*R3 ; Write native constant format into constant pool in code segment
              5DF0 0602    DEC  R2                    ; Real size counter
              5DF2 15FD    JGT  LOOP93
              5DF4 0606    DEC  R6                    ; Number of real constants
              5DF6 15A0    JGT  NEXTCON
NOCONST       5DF8 C360  ` MOV  @SAVER13,R13          ; Code read data address
              5DFA 2A3A *:
              5DFC C3E0    MOV  @SAVER15,R15          ; Code type
              5DFE 2A3C *<
              5E00 C320    MOV  @SAVER12,R12          ; Fetch address
              5E02 2A3E *>
              5E04 C820    MOV  @SAVFLIP,@PMEFLIP
              5E06 2A40 *@
              5E08 2AE8 *
              5E0A 020B    LI   R11,SAVER0            ; Restore registers
              5E0C 2A24 *$
              5E0E C03B  ; MOV  *R11+,R0
              5E10 C07B  { MOV  *R11+,R1
              5E12 C0BB    MOV  *R11+,R2
              5E14 C0FB    MOV  *R11+,R3
              5E16 C13B  ; MOV  *R11+,R4
              5E18 C17B  { MOV  *R11+,R5
              5E1A C1BB    MOV  *R11+,R6
              5E1C C1FB    MOV  *R11+,R7
              5E1E C2E0    MOV  @SAVER11,R11
              5E20 2A34 *4
              5E22 C121  ! MOV  @2(R1),R4; Segment base
              5E24 0002
              5E26 045B  [ B    *R11

; Pop real value on stack to FARG (floating point argument)
POPFARG       5E28 0202    LI   R2,FARG
              5E2A 835C  \
              5E2C 1002    JMP  MOVFLT

; Pop real value on stack to FAC
POPFAC        5E2E 0202    LI   R2,FAC
              5E30 834A  J
MOVFLT        5E32 CCBA    MOV  *SP+,*R2+
              5E34 CCBA    MOV  *SP+,*R2+
              5E36 CCBA    MOV  *SP+,*R2+
              5E38 C4BA    MOV  *SP+,*R2
              5E3A 045B  [ B    *R11

; Push content of FAC (real number) on stack
FACPUSH       5E3C 0201    LI   R1,FAC
              5E3E 834A  J
              5E40 022A  * AI   SP,-8
              5E42 FFF8
              5E44 C08A    MOV  SP,R2
              5E46 CCB1    MOV  *R1+,*R2+
              5E48 CCB1    MOV  *R1+,*R2+
              5E4A CCB1    MOV  *R1+,*R2+
              5E4C C491    MOV  *R1,*R2
              5E4E 045B  [ B    *R11

; Console floating point routine call
FLTCONS       5E50 04E0    CLR  @NAMLEN  ; Floating point status return byte
              5E52 8354  T
              5E54 C81B    MOV  *R11,@GPLWS+4         ; GPLWS R2 given address to console routine
              5E56 83E4
              5E58 06A0    BL   @POPFAC  ; Place arguments where console's real arithmetic expects them to be
              5E5A 5E2E ^.
              5E5C 06A0    BL   @POPFARG
              5E5E 5E28 ^(
              5E60 02E0    LWPI GPLWS
              5E62 83E0
              5E64 0692    BL   *R2                   ; Execute console real arithmetic
              5E66 02E0    LWPI PASCALWS
```

```
             5E68 8380
             5E6A 06A0    BL   @FACPUSH ; Retreive result to stack
             5E6C 5E3C ^<
FLTEXIT      5E6E D020    MOVB @NAMLEN,R0
             5E70 8354  T
             5E72 1601    JNE  FLERR                 ; Floating point error?
             5E74 045C  \ B    *R12                  ; PME fetch

FLERR        5E76 0280    CI   R0,2
             5E78 0002
             5E7A 1603    JNE  NODIV0
             5E7C 06A0    BL   @FAULGEN ; Divide by zero error
             5E7E 484A HJ
             5E80 0006    .WORD 6       ; Divide by zero
NODIV0       5E82 06A0    BL   @FAULGEN
             5E84 484A HJ
             5E86 000C    .WORD 12      ; Floating point math error

UNUSED1      5E88 8300    .WORD FETCH

; Load real
LDRL         5E8A 5E8C ^  .WORD LDRL+2
             5E8C C07A  z MOV  *SP+,R1  ;Address
             5E8E 022A  * AI   SP,-8
             5E90 FFF8
             5E92 C08A    MOV  SP,R2
             5E94 CCB1    MOV  *R1+,*R2+
             5E96 CCB1    MOV  *R1+,*R2+
             5E98 CCB1    MOV  *R1+,*R2+
             5E9A C491    MOV  *R1,*R2
             5E9C 045C  \ B    *R12     ; PME fetch

; Store real
STRL         5E9E 5EA0 ^  .WORD STRL+2
             5EA0 C06A  j MOV  @8(SP),R1;Address
             5EA2 0008
             5EA4 CC7A  z MOV  *SP+,*R1+
             5EA6 CC7A  z MOV  *SP+,*R1+
             5EA8 CC7A  z MOV  *SP+,*R1+
             5EAA C47A  z MOV  *SP+,*R1
             5EAC 05CA    INCT SP                    ;Pop address
             5EAE 045C  \ B    *R12     ; PME fetch

; Add real. Uses console math.
ADR          5EB0 5EB2    .WORD ADR+2
             5EB2 06A0    BL   @FLTCONS ; Console floating point
             5EB4 5E50 ^P
             5EB6 0D80    .WORD 0D80H   ; Console Floating ADD

; Subtract real. Uses console math.
SBR          5EB8 5EBA    .WORD SDR+2
             5EBA 06A0    BL   @FLTCONS ; Console floating point
             5EBC 5E50 ^P
             5EBE 0D7C    .WORD 0D7CH   ; Console Floating SUB

; Divide real. Uses console math.
DVR          5EC0 5EC2    .WORD DVR+2
             5EC2 06A0    BL   @FLTCONS ; Console floating point
             5EC4 5E50 ^P
             5EC6 0FF4    .WORD 0FF4H   ; Console Floating DIV

; Multiply real. Uses console math.
MPR          5EC8 5ECA    .WORD MPR+2
             5ECA 06A0    BL   @FLTCONS ; Console floating point
             5ECC 5E50 ^P
             5ECE 0E88    .WORD 0E88H   ; Console Floating MUL

; Absolute value real (identical to absolute integer)
ABR          5ED0 5ED2    .WORD ARB+2
             5ED2 075A  Z ABS  *SP
             5ED4 045C  \ B    *R12     ; PME fetch

; Negate real (identical to negate integer)
NGR          5ED6 5ED8    .WORD NGR+2
             5ED8 051A    NEG  *SP
             5EDA 045C  \ B    *R12     ; PME fetch

; Common entry for equal, less than or equal and greater than or equal real. Uses console compare.
EQREAL       5EDC 5EDE    .WORD EQREAL+2
             5EDE 0221  ! AI   R1,5146  ; Opcode is CDH (205) *2 => 52E0
             5EE0 5146 QF
             5EE2 04E0    CLR  @NAMLEN  ; Floating point status
             5EE4 8354  T
             5EE6 06A0    BL   @POPFAC  ; One argument to FAC...
```

```
             5EE8 5E2E ^.
             5EEA 06A0    BL   @POPFARG ; ... the other to ARG.
             5EEC 5E28 ^(
             5EEE 064A  J DECT SP
             5EF0 02E0    LWPI GPLWS
             5EF2 83E0
             5EF4 0203    LI   R3,EQRCONT            ; Pre-load return address from SCOMP
             5EF6 5EFC ^
             5EF8 0460  ` B    @0D50                 ; Console floating point compare (SCOMP)
             5EFA 0D50  P
EQRCONT      5EFC 02E0    LWPI PASCALWS
             5EFE 8380
             5F00 0460  ` B    @TESTS
             5F02 52C4 R

; Round real number. Uses console routine
RND          5F04 5F06    .WORD RND+2
             5F06 04E0    CLR  @NAMLEN
             5F08 8354  T
             5F0A 06A0    BL   @POPFAC
             5F0C 5E2E ^.
             5F0E 02E0    LWPI GPLWS
             5F10 83E0
             5F12 06A0    BL   @12B8                 ; Console convert floating to integer (CFI)
             5F14 12B8
             5F16 02E0    LWPI PASCALWS
             5F18 8380
             5F1A 064A  J DECT SP
             5F1C C6A0    MOV  @FAC,*SP
             5F1E 834A  J
             5F20 0460  ` B    @FLTEXIT
             5F22 5E6E ^n

; Float
FLT          5F24 5F26    .WORD FLT+2
             5F26 C83A  : MOV  *SP+,@FAC
             5F28 834A  J
             5F2A 06A0    BL   @FLOAT
             5F2C 5F94 _
             5F2E 06A0    BL   @FACPUSH
             5F30 5E3C ^<
             5F32 045C  \ B    *R12     ; PME fetch

; Truncate real
TNC          5F34 5F36    .WORD TNC+2
             5F36 C01A    MOV  *SP,R0
             5F38 075A  Z ABS  *SP
             5F3A 04C1    CLR  R1
             5F3C D81A    MOVB *SP,@PASCALWS+3       ; To LSBy R1
             5F3E 8383
             5F40 0221  ! AI   R1,-63D  ; Exponent bias
             5F42 FFC1
             5F44 1107    JLT  5F54     ; Too small for an integer?
             5F46 0281    CI   R1,6
             5F48 0006
             5F4A 1504    JGT  5F54     ; Too big for an integer
             5F4C C08A    MOV  SP,R2
             5F4E A081    A    R1,R2
             5F50 0582    INC  R2
             5F52 D481    MOVB R1,*R2
             5F54 0540  @ INV  R0
             5F56 11D7    JLT  RND+2    ; If positive end with "round real number"
             5F58 051A    NEG  *SP
             5F5A 10D5    JMP  RND+2    ; Final action is round real number

; Kernel procedure POWEROFTEN
; Floating point number format in 99/4A has exponent by 100, not ten.
PWRTEN       5F5C C03A  : MOV  *SP+,R0
             5F5E 0280    CI   R0,-127D ; Check limit for real number's exponent of ten
             5F60 FF81
             5F62 1114    JLT  EXPFAUL
             5F64 0280    CI   R0,127D
             5F66 007F
             5F68 1511    JGT  EXPFAUL
             5F6A 0910    SRL  R0,1                  ; Odd or even exponent of ten?
             5F6C 1803    JOC  EXPODD
             5F6E 0201    LI   R1,1
             5F70 0001
             5F72 1002    JMP  EXPEVEN

EXPODD       5F74 0201    LI   R1,10D
             5F76 000A
EXPEVEN      5F78 0220    AI   R0,64D   ; Exponent bias
             5F7A 0040  @
```

```
             5F7C D060  ` MOVB @PASCALWS+1,R1        ; LSBy of R0
             5F7E 8381
             5F80 C08A    MOV  SP,R2
             5F82 CC81    MOV  R1,*R2+  ; Load 1 or 10 and exponent
             5F84 04F2    CLR  *R2+
             5F86 04F2    CLR  *R2+
             5F88 04D2    CLR  *R2
             5F8A 045C  \ B    *R12     ; PME fetch

EXPFAUL      5F8C 06A0    BL   @FAULGEN ; Exponent is out of range
             5F8E 484A HJ
             5F90 000C    .WORD 12      ; Floating point math error

CON100       5F92 0064    .WORD 100     ; Constant 100

; Integer in FAC to floating point value in FAC
FLOAT        5F94 0204    LI   R4,FAC   ; Address to floating point accumulator
             5F96 834A  J
             5F98 C014    MOV  *R4,R0   ; Save integer there in R0
             5F9A C084    MOV  R4,R2                ; Copy to pointer to use
             5F9C 04F2    CLR  *R2+                 ; Clear FAC
             5F9E 04F2    CLR  *R2+
             5FA0 04F2    CLR  *R2+
             5FA2 04D2    CLR  *R2
             5FA4 C140  @ MOV  R0,R5                ; Save integer -> R5 (negative flag)
             5FA6 1321  ! JEQ  FLOTRET  ; Integer in FAC=0?
             5FA8 0740  @ ABS  R0
             5FAA 0203    LI   R3,64D   ; Zero exponent (biased by 64)
             5FAC 0040  @
             5FAE 0280    CI   R0,100D  ; Radix 100 compare
             5FB0 0064  d
             5FB2 1A13    JL   LTHUNDR  ; Less than 100?
             5FB4 0280    CI   R0,10000D
             5FB6 2710 '
             5FB8 1A08    JL   LTTENTH  ; Less than 10000?
             5FBA 0583    INC  R3                   ; Increment exponent
             5FBC C040  @ MOV  R0,R1
             5FBE 04C0    CLR  R0
             5FC0 3C20 <  DIV  @CON100,R0           ; Divide mantissa by 100
             5FC2 5F92 _
             5FC4 D920    MOVB @PASCALWS+3,@3(R4)   ; PASCALWS R1 LSBy, reminder after DIV
             5FC6 8383
             5FC8 0003
LTTENTH      5FCA 0583    INC  R3                   ; Increment exponent
             5FCC C040  @ MOV  R0,R1
             5FCE 04C0    CLR  R0
             5FD0 3C20 <  DIV  @CON100,R0           ; Divide mantissa by 100 again
             5FD2 5F92 _
             5FD4 D920    MOVB @PASCALWS+3,@2(R4)   ; PASCALWS R1 LSBy, reminder after DIV
             5FD6 8383
             5FD8 0002
LTHUNDR      5FDA D920    MOVB @PASCALWS+1,@1(R4)   ; PASCALWS R0 LSBy
             5FDC 8381
             5FDE 0001
             5FE0 D520    MOVB @PASCALWS+7,*R4      ; PASCALWS R3 LSBy
             5FE2 8387
             5FE4 0545  E INV  R5
             5FE6 1101    JLT  FLOTRET  ; If it's negative now, then the number is not
             5FE8 0514    NEG  *R4
FLOTRET      5FEA 045B  [ B    *R11

             5FEC 0000    .WORD 0
             5FEE 0000    .WORD 0
             5FF0 0000    .WORD 0
             5FF2 0000    .WORD 0
             5FF4 0000    .WORD 0
             5FF6 0000    .WORD 0
             5FF8 0000    .WORD 0
             5FFA 0000    .WORD 0

GROMWD       5FFC 0606    ;GROM write data (never used)
GROMWA       5FFE 0606    ;GROM write address

; Standard page ends here.
; ------------------------------------------------------------------------------------------------------
; P-code card second page (when CRU bit 1F80H is set)

; This page implements BIOS functions, as part of UCSD p-system's RSP
; (Run-time Support Package)
; At Pascal level, the most basic IO procedures are unitread and similar. They can be called from Pascal
level.
; Once invoked, they'll forward the call to the BIOS IO functions, which do the actual work.
; These BIOS functions are implemented on the second page on the p-code card.
```

```
; Device write for non-blocked devices
PAGE2B      5000 C180    MOV  R0,R6                 ; Control parameter
            5002 0246  F ANDI R6,0008  ; No CRLF bit
            5004 0008
            5006 0240  @ ANDI R0,0004  ; No SPEC bit
            5008 0004
CHKSPEC     500A 1643  C JNE  NOSPEC   ; Nothing special?
            500C 9824  $ CB   @29B8(R4),@BYDLE      ; Space compression?
            500E 29B8 )
            5010 409F @
            5012 1614    JNE  NOCOMP
SPCOMP      5014 D933  3 MOVB *R3+,@SERBUF(R4)      ; Resolve space compression
            5016 29B8 )
            5018 04C7    CLR  R7                    ; # of compressed spaces
            501A D824  $ MOVB @SERBUF(R4),@PASCALWS+15            ; To low byte R7
            501C 29B8 )
            501E 838F
            5020 0227  ' AI   R7,-32   ; Number of spaces is code-32
            5022 FFE0
            5024 1109    JLT  ZERCOMP               ; None?
            5026 1308    JEQ  ZERCOMP
            5028 D820    MOVB @SPACES,@BLKNUM
            502A 40A2 @
            502C 29AA )
LOOP41      502E 0420    BLWP @DOIO                 ; Handle characters
            5030 51E8 Q
            5032 29A8 )  .WORD IOCTRL
            5034 0607    DEC  R7                    ; Number of spaces to expand
            5036 16FB    JNE  LOOP41
ZERCOMP     5038 0602    DEC  R2                    ; Byte count?
            503A 132E  . JEQ  URET1
NOCOMP      503C D933  3 MOVB *R3+,@SERBUF(R4)      ; Done with space expansion (if any)
            503E 29B8 )
            5040 9824  $ CB   @SERBUF(R4),@BYDLE
            5042 29B8 )
            5044 409F @
            5046 1603    JNE  NOCOMP2
            5048 0602    DEC  R2       ; Byte count
            504A 1326  & JEQ  URET1
            504C 10E3    JMP  SPCOMP

NOCOMP2     504E D824  $ MOVB @SERBUF(R4),@BLKNUM   ; Character to block register
            5050 29B8 )
            5052 29AA )
            5054 0420    BLWP @DOIO
            5056 51E8 Q
            5058 29A8 )  .WORD IOCTRL
            505A C820    MOV  @IOCOD,@SYSCOM        ; Ioresult
            505C 29B4 )
            505E 2B6E +n
            5060 161B    JNE  URET1                             ; Error?
            5062 9820    CB   @BLKNUM,@SPCR+1       ; Compare with 13
            5064 29AA )                                         ; Unit #13 is assigned to tape
            5066 409D @
            5068 160C    JNE  NOT13
            506A C186    MOV  R6,R6
            506C 160A    JNE  NOT13
            506E D820    MOVB @BYLF,@BLKNUM                     ; Is 10
            5070 409E @
            5072 29AA )
            5074 0420    BLWP @DOIO
            5076 51E8 Q
            5078 29A8 )  .WORD IOCTRL
            507A C820    MOV  @IOCOD,@SYSCOM        ; Ioresult
            507C 29B4 )
            507E 2B6E +n
            5080 160B    JNE  URET1                 ; Error?
NOT13       5082 C000    MOV  R0,R0                 ; Control
            5084 1302    JEQ  CTLZERO
            5086 04E4    CLR  @SERBUF(R4)
            5088 29B8 )
CTLZERO     508A 0602    DEC  R2
            508C 1305    JEQ  URET1
            508E C000    MOV  R0,R0                 ; Control
            5090 10BC    JMP  CHKSPEC

NOSPEC      5092 D933  3 MOVB *R3+,@SERBUF(R4)
            5094 29B8 )
            5096 10DB    JMP  NOCOMP2

URET1       5098 0460  ` B    @UNITRET
            509A 3510 5

; Unitread/unitwrite
```

```
UNITIO      509C C03A  : MOV  *SP+,R0  ; Control R0
            509E C800    MOV  R0,@CTRLPAR
            50A0 29B2 )
            50A2 C07A  z MOV  *SP+,R1  ; Block number R1
            50A4 C801    MOV  R1,@BLKNUM
            50A6 29AA )
            50A8 C0BA    MOV  *SP+,R2  ; Number of bytes R2
            50AA C802    MOV  R2,@BYTCNT
            50AC 29AC )
            50AE C0FA    MOV  *SP+,R3  ; Index into buffer R3
            50B0 A0FA    A    *SP+,R3  ; Buffer
            50B2 C803    MOV  R3,@BUFADR
            50B4 29AE )
            50B6 C13A  : MOV  *SP+,R4  ; Unit # R4
            50B8 C804    MOV  R4,@DRIVENO
            50BA 29B0 )
            50BC 1160  ` JLT  BADDEV   ; Should be positive Device #<0?
            50BE 04E0    CLR  @IOCOD
            50C0 29B4 )
            50C2 C082    MOV  R2,R2
            50C4 1602    JNE  CONT06   ; Zero bytes?
            50C6 0460  ` B    @UNITRET ; If zero bytes nothing to do
            50C8 3510 5

CONT06      50CA 0284    CI   R4,127D  ; Highest standard unit number
            50CC 007F
            50CE 1B5B  [ JH   EXECIO   ; Unit # higher than 127?
            50D0 0284    CI   R4,36D
            50D2 0024  $
            50D4 1554  T JGT  BADDEV   ; Unit # greater than 36?
            50D6 D164  d MOVB @DRVNUM(R4),R5         ; Unit # to drive #
            50D8 2BFC +
            50DA 1655  U JNE  EXECIO                 ; Blocked drive exists?
            50DC 8820    C    @IOCTRL,@OPWRIT        ; A write?
            50DE 29A8 )
            50E0 2BEE +
            50E2 138E    JEQ  PAGE2B   ; Device not blocked
            50E4 0240  @ ANDI R0,0004
            50E6 0004
            50E8 1639  9 JNE  SPECIAL  ; SPEC bit?
LOOP51      50EA 0420    BLWP @DOIO
            50EC 51E8 Q
            50EE 29A8 )  .WORD IOCTRL  ; Parameter to call
            50F0 C820    MOV  @IOCOD,@SYSCOM         ; Ioresult
            50F2 29B4 )
            50F4 2B6E +n
            50F6 1630  0 JNE  URET3                  ; Back if error
            50F8 0284    CI   R4,0001
            50FA 0001
            50FC 161B    JNE  NOCONS   ; Not CONSOLE:?
            50FE C820    MOV  @OPWRIT,@IOCTRL
            5100 2BEE +
            5102 29A8 )
            5104 0420    BLWP @DOIO
            5106 51E8 Q
            5108 29A8 )  .WORD IOCTRL
            510A C820    MOV  @IOCOD,@SYSCOM         ; Ioresult
            510C 29B4 )
            510E 2B6E +n
            5110 1623  # JNE  URET3                  ; Back if error
            5112 9820    CB   @BLKNUM,@SPCR+1        ; CR?
            5114 29AA )
            5116 409D @
            5118 160D    JNE  NOCONS
            511A D820    MOVB @BYLF,@BLKNUM                      ; Is 10 LF
            511C 409E @
            511E 29AA )
            5120 0420    BLWP @DOIO
            5122 51E8 Q
            5124 29A8 )  .WORD IOCTRL
            5126 C820    MOV  @IOCOD,@SYSCOM         ; Ioresult
            5128 29B4 )
            512A 2B6E +n
            512C 1615    JNE  URET3                  ; Back if error
            512E D820    MOVB @SPCR+1,@BLKNUM        ; CR
            5130 409D @
            5132 29AA )
NOCONS      5134 C820    MOV  @OPREAD,@IOCTRL        ; Not console: IO
            5136 2BEC +
            5138 29A8 )
            513A 0284    CI   R4,0001  ; CONSOLE:?
            513C 0001
            513E 1604    JNE  NOCONS2
            5140 9820    CB   @BLKNUM,@ETXCOD
```

```
             5142 29AA )
             5144 2BC1 +
             5146 1316    JEQ  GOTETX   ; Is <etx>?
NOCONS2      5148 DCE0    MOVB @BLKNUM,*R3+           ; To buffer
             514A 29AA )
             514C 9820    CB   @BLKNUM,@ETXCOD
             514E 29AA )
             5150 2BC1 +
             5152 1302    JEQ  URET3                  ; Back if error
             5154 0602    DEC  R2                     ; Byte count
             5156 16C9    JNE  LOOP51
URET3        5158 0460  ` B    @UNITRET
             515A 3510 5

SPECIAL      515C 0420    BLWP @DOIO
             515E 51E8 Q
             5160 29A8 )  .WORD IOCTRL
             5162 C820    MOV  @IOCOD,@SYSCOM         ; Ioresult
             5164 29B4 )
             5166 2B6E +n
             5168 1604    JNE  LEAVE                  ; Error?
             516A DCE0    MOVB @BLKNUM,*R3+           ; To buffer
             516C 29AA )
             516E 0602    DEC  R2                     ; Byte count?
             5170 16F5    JNE  SPECIAL
LEAVE        5172 100F    JMP  URET4

GOTETX       5174 DCE0    MOVB @ZERO,*R3+             ; To buffer
             5176 40AC @
             5178 0602    DEC  R2                     ; Byte count
             517A 16FC    JNE  GOTETX
             517C 100A    JMP  URET4

BADDEV       517E C820    MOV  @SBITMSK+2,@SYSCOM     ; Ioresult "Bad device number"
             5180 2D94 -
             5182 2B6E +n
             5184 1006    JMP  URET4

EXECIO       5186 0420    BLWP @DOIO                  ; Perform IO for legal unit number
             5188 51E8 Q
             518A 29A8 )  .WORD IOCTRL
             518C C820    MOV  @IOCOD,@SYSCOM         ; Ioresult
             518E 29B4 )
             5190 2B6E +n
URET4        5192 0460  ` B    @UNITRET
             5194 3510 5

; Unit clear
UNCLR        5196 C820    MOV  @OPCLR,@IOCTRL         ; Opcode
             5198 2BF0 +
             519A 29A8 )
             519C 04E0    CLR  @IOCOD   ; Completion code
             519E 29B4 )
             51A0 C07A  z MOV  *SP+,R1  ; Unit #
             51A2 C801    MOV  R1,@DRIVENO
             51A4 29B0 )
             51A6 11EB    JLT  BADDEV   ; Bad device #?
             51A8 0281    CI   R1,127
             51AA 007F
             51AC 1BEC    JH   EXECIO   ; Bad device #?
             51AE 0281    CI   R1,36
             51B0 0024  $
             51B2 15E5    JGT  BADDEV   ; Unit # should be max 36
             51B4 D860  ` MOVB @IOCOD,@SERBUF(R1)
             51B6 29B4 )
             51B8 29B8 )
             51BA 10E5    JMP  EXECIO

; Unit status
UNSTAT       51BC C820    MOV  @OPSTAT,@IOCTRL        ; Opcode
             51BE 2BF2 +
             51C0 29A8 )
             51C2 04E0    CLR  @IOCOD   ; Completion code
             51C4 29B4 )
             51C6 C07A  z MOV  *SP+,R1  ; Control parameter
             51C8 C801    MOV  R1,@CTRLPAR
             51CA 29B2 )
             51CC C0BA    MOV  *SP+,R2  ;
             51CE C802    MOV  R2,@BUFADR             ; Buffer address
             51D0 29AE )
             51D2 C0FA    MOV  *SP+,R3  ; Unit #
             51D4 C803    MOV  R3,@DRIVENO
             51D6 29B0 )
             51D8 11D2    JLT  BADDEV   ; Check for valid unit #
```

```
             51DA 0283    CI   R3,127
             51DC 007F  
             51DE 1BD3    JH   EXECIO
             51E0 0283    CI   R3,36                 ; Predefined device?
             51E2 0024  $
             51E4 15CC    JGT  BADDEV
             51E6 10CF    JMP  EXECIO

; Unit IO handler
DOIO         51E8 83C0    .WORD INTWS
             51EA 51EC Q  .WORD DOIOX

DOIOX        51EC 04CC    CLR  R12
             51EE 1F02    TB   VDPIRQ   ; Test VDP interrupt bit
             51F0 1302    JEQ  NOVDP
             51F2 0420    BLWP @VDPINT  ; Execute VDP interrupt code
             51F4 2AAA *
NOVDP        51F6 C2BE    MOV  *R14+,R10; Fetch pointer to IOCTRL record
             51F8 04EA    CLR  @12D(R10); IO result code
             51FA 000C
             51FC C06A  j MOV  @8(R10),R1             ; Drive number
             51FE 0008
             5200 0281    CI   R1,128D
             5202 0080
             5204 1B18    JH   USERS                  ; Drive number>128
             5206 1A02    JL   KEEP                   ; Drive number<128, keep it
             5208 0201    LI   R1,31
             520A 001F
KEEP         520C D021  ! MOVB @DRVNUM(R1),R0         ; Get the real drive number
             520E 2BFC +
             5210 0980    SRL  R0,8
             5212 CA80    MOV  R0,@14D(R10)           ; To physical drive number
             5214 000E
             5216 0A11    SLA  R1,1                   ; Word index
             5218 C821  ! MOV  @PCBPNT(R1),@PPOINT    ; Fetch PCB pointer
             521A 29DC )
             521C 2960 )`
             521E C061  a MOV  @UNITPNT(R1),R1        ; Fetch IO code pointer
             5220 2C20 ,
             5222 1322  " JEQ  BADVOL   ; Nonexistent?
FNDBIOS      5224 C0AA    MOV  @0(R10),R2             ; Get opcode
             5226 0000
             5228 0282    CI   R2,6     ; Highest allowed code (read, write, init, status are 0, 2, 4, 6)
             522A 0006
             522C 1B1D    JH   BADVOL
             522E A042  B A    R2,R1
             5230 C091    MOV  *R1,R2   ; Address of supporting BIOS code
             5232 131A    JEQ  BADVOL   ; No support for operation?
             5234 0452  R B    *R2                   ; Execute BIOS function on p-code card page 2

USERS        5236 8801    C    R1,@USERDEF            ; Compare with highest user defined device number
             5238 2BF4 +
             523A 1B16    JH   BADVOL
             523C 0221  ! AI   R1,-128  ; Zero adjust user defined volume number
             523E FF80
             5240 0A11    SLA  R1,1
             5242 C0A0    MOV  @PCBPNT,R2             ; Uses same table as standard units!
             5244 2BF8 +
             5246 1303    JEQ  CHKBUF   ; Not implemented?
             5248 A081    A    R1,R2
             524A C812    MOV  *R2,@PPOINT            ; To PCB pointer
             524C 2960 )`
CHKBUF       524E C0A0    MOV  @BUFAPNT,R2            ; Buffer address table pointer
             5250 2BFA +
             5252 1303    JEQ  CHKUNIT
             5254 A081    A    R1,R2
             5256 CA92    MOV  *R2,@14(R10)           ; Physical drive number
             5258 000E
CHKUNIT      525A C0A0    MOV  @UNPNT,R2; Unit pointer table
             525C 2BF6 +
             525E 1304    JEQ  BADVOL
             5260 A081    A    R1,R2
             5262 C052  R MOV  *R2,R1   ; BIOS code address
             5264 1301    JEQ  BADVOL   ; Not supported?
             5266 10DE    JMP  FNDBIOS  ; Find the BIOS code

BADVOL       5268 CAA0    MOV  @NOVOL,@12(R10)        ; Return "no such volume" if word device
             526A 2CDA ,
             526C 000C
             526E 0380    RTWP

; bios_con_init
CONINIT      5270 06A0    BL   @BEEP
             5272 480E H
```

```
              5274 02E0    LWPI SUB2WS
              5276 2896 (
              5278 0206    LI   R6,FF40  ; Buffer empty flag
              527A FF40  @
              527C 04C7    CLR  R7                    ; Buffer store pointer
              527E 04C8    CLR  R8                    ; Buffer fetch pointer
              5280 020A    LI   R10,VDPWA
              5282 8C02
              5284 02E0    LWPI INTWS
              5286 83C0
              5288 882A  * C    @8(R10),@SBITMSK+2     ; Points at drive # in IOCTRL. Compare to 0002H
              528A 0008
              528C 2D94 -
              528E 1302    JEQ  SYSTERM  ; Unit #2 (SYSTERM:)?
              5290 06A0    BL   @RESCON  ; Reset console
              5292 52B2 R
SYSTERM       5294 04E0    CLR  @CURFLAG
              5296 2886 (
              5298 0200    LI   R0,0BE0H ; Kyeboard buffer in VDP RAM
              529A 0BE0
              529C C800    MOV  R0,@KBUFPNT
              529E 288A (
              52A0 0200    LI   R0,0D80  ; Address of keyboard layout area in VDP RAM
              52A2 0D80
              52A4 C800    MOV  R0,@KBLAYO            ; Keyboard layout
              52A6 288C (
              52A8 04E0    CLR  @FLUSHFL
              52AA 2894 (
              52AC 04E0    CLR  @STOPFLG
              52AE 2890 (
              52B0 0380    RTWP

; Reset CONSOLE:
RESCON        52B2 C14B  K MOV  R11,R5
              52B4 0200    LI   R0,4
              52B6 0004
LOOP64        52B8 C040  @ MOV  R0,R1
              52BA D821  ! MOVB @SOUND20+1(R1),@SNDGEN
              52BC 535B S[
              52BE 8400
              52C0 D861  a MOVB @SOUND20+1(R1),@SOUND2+3(R1)
              52C2 535B S[
              52C4 27DB '
              52C6 04C2    CLR  R2
              52C8 D842  B MOVB R2,@SOUND9+7(R1)
              52CA 27B3 '
              52CC 0A11    SLA  R1,1                  ; Index multiplied by 2
              52CE C860  ` MOV  @SBITMSK,@TOPMEM(R1)  ; 0001H goes to SOUND1 list right after TOPMEM
              52D0 2D92 -
              52D2 2782 '
              52D4 04E1    CLR  @SOUND8+6(R1)
              52D6 2792 '
              52D8 04E1    CLR  @SOUND10+6(R1)
              52DA 279A '
              52DC 04E1    CLR  @SOUND13+2(R1)
              52DE 27DE '
              52E0 0600    DEC  R0
              52E2 16EA    JNE  LOOP64

              52E4 0203    LI   R3,VDPWA
              52E6 8C02
              52E8 0201    LI   R1,40                 ; Screen width in text mode
              52EA 0028  (
              52EC C801    MOV  R1,@WIDTH
              52EE 2CC6 ,
              52F0 0200    LI   R0,VDPCOP2            ; Default VDP register settings
              52F2 2CB0 ,
              52F4 0201    LI   R1,VDPCOPY            ; Current VDP register settings
              52F6 2810 (
              52F8 0204    LI   R4,7
              52FA 0007
LOOP47        52FC C090    MOV  *R0,R2
              52FE 06A0    BL   @BYTEWR  ; R2 to *R3, here VDPWA (writes to register 0, to turn off blanking)
              5300 534E SN
              5302 CC70  p MOV  *R0+,*R1+; Restore VDP registers
              5304 0604    DEC  R4
              5306 16FA    JNE  LOOP47
              5308 06A0    BL   @DISPCUR
              530A 47D2 G
              530C 0202    LI   R2,4BC0H ; VDP address BC0H. Start of the pattern color table
              530E 4BC0 K
              5310 06A0    BL   @BYTEWR
              5312 534E SN
              5314 0202    LI   R2,32D   ; 32 color definitions
```

```
              5316 0020
              5318 0201    LI   R1,1700H ; Default color black on cyan
              531A 1700
LOOP48        531C D8C1    MOVB R1,@-2(R3)             ; To VDP write data
              531E FFFE
              5320 0602    DEC  R2
              5322 16FC    JNE  LOOP48
              5324 0202    LI   R2,4C00  ; VDP address C00H, sprite name table
              5326 4C00 L
              5328 06A0    BL   @BYTEWR
              532A 534E SN
              532C 0202    LI   R2,32                  ; 32 sprite definitions
              532E 0020
LOOP49        5330 0201    LI   R1,SPRDAT
              5332 5358 SX
              5334 0200    LI   R0,4                   ; Each sprite defined by four bytes
              5336 0004
LOOP50        5338 D8F1    MOVB *R1+,@-2(R3)           ; To VDP write data
              533A FFFE
              533C 0600    DEC  R0
              533E 16FC    JNE  LOOP50
              5340 0602    DEC  R2
              5342 16F6    JNE  LOOP49
              5344 04E0    CLR  @SPRBIT1 ; Sprite bitmaps cleared
              5346 280A (
              5348 04E0    CLR  @SPRBIT2
              534A 280C (
              534C 0455  U B    *R5      ; Saved return address

; Writes R2 to *R3, bytewise
BYTEWR        534E 06C2    SWPB R2
              5350 D4C2    MOVB R2,*R3
              5352 06C2    SWPB R2
              5354 D4C2    MOVB R2,*R3
              5356 045B  [ B    *R11

; Data for default sprite settings
SPRDAT        5358 C000    .WORD C000H   ; Vertical 192, horizontal 0
              535A 0000    .WORD 0                     ; Character 0, transparent, late clock

              535C 9FBF    .WORD 9FBFH
              535E DFFF    .WORD DFFFH

; bios_con_stat
CONSTAT       5360 04C4    CLR  R4
              5362 C02A  * MOV  @10D(R10),R0           ; Fetch control parameter
              5364 000A
              5366 2020    COC  @SBITMSK,R0            ; 0001H
              5368 2D92 -
              536A 160D    JNE  NOKEY
              536C C120    MOV  @KBUFSTO,R4            ; Buffer store pointer
              536E 28A4 (
              5370 6120 a  S    @KBUFFTC,R4            ; Buffer fetch pointer
              5372 28A6 (
              5374 1508    JGT  NOKEY
              5376 1105    JLT  ROLLKEY
              5378 C020    MOV  @KBUFMOD,R0
              537A 28A2 (
              537C 2020    COC  @SBITMSK+12,R0
              537E 2D9E -
              5380 1302    JEQ  NOKEYH
ROLLKEY       5382 0224  $ AI   R4,32D
              5384 0020
NOKEY         5386 C02A  * MOV  @6(R10),R0             ; Get buffer address
              5388 0006
              538A C404    MOV  R4,*R0
              538C 0380    RTWP

; bios_con_read
CONREAD       538E 04E0    CLR  @FLUSHFL
              5390 2894 (
              5392 04C2    CLR  R2
              5394 C120    MOV  @CURXY,R4; Current 80 col position
              5396 2CC4 ,
              5398 D094    MOVB *R4,R2   ; Read value there
              539A C120    MOV  @KBUFMOD,R4
              539C 28A2 (
              539E 0AA4    SLA  R4,10
              53A0 172C  , JNC  BUFEMPT  ; Buffer empty
              53A2 0203    LI   R3,SCANKBD
              53A4 53D4 S
WINCHK        53A6 C020    MOV  @WIDTH,R0; Check if in window
              53A8 2CC6 ,
              53AA 0910    SRL  R0,1
```

```
              53AC 3820 8  MPY  @WINDOW,R0
              53AE 2CCA ,
              53B0 8060  ` C    @CURCOL,R1
              53B2 2CC2 ,
              53B4 1107    JLT  WINLEFT  ; Window left?
              53B6 A060  ` A    @WIDTH,R1
              53B8 2CC6 ,
              53BA 8060  ` C    @CURCOL,R1
              53BC 2CC2 ,
              53BE 1405    JHE  WINRGHT  ; Window right?
              53C0 0693    BL   *R3                   ; Display or scan (DISPCUR or SCANKBD)
              53C2 1008    JMP  SCANKBD

WINLEFT       53C4 04E0    CLR  @WINDOW
              53C6 2CCA ,
              53C8 1002    JMP  WINMOVE

WINRGHT       53CA 05A0    INC  @WINDOW
              53CC 2CCA ,
WINMOVE       53CE 0203    LI   R3,DISPCUR
              53D0 47D2 G
              53D2 10E9    JMP  WINCHK

SCANKBD       53D4 06C2    SWPB R2
              53D6 06A0    BL   @SCRECHO
              53D8 5436 T6
              53DA 0203    LI   R3,500D
              53DC 01F4
              53DE 0603    DEC  R3
              53E0 13F9    JEQ  SCANKBD
              53E2 04CC    CLR  R12      ; 9901 PSI
              53E4 1F02    TB   VDPIRQ
              53E6 1303    JEQ  53EE     ; No VDP interrupt?
              53E8 1D02    SBO  2
              53EA 0420    BLWP @VDPINT
              53EC 2AAA *
              53EE 0420    BLWP @KSCANV
              53F0 41B6 A
              53F2 C120    MOV  @KBUFMOD,R4
              53F4 28A2 (
              53F6 0AA4    SLA  R4,10
              53F8 18F2    JOC  53DE
BUFEMPT       53FA 04C1    CLR  R1
              53FC 02E0    LWPI SUB2WS
              53FE 2896 (
              5400 C008    MOV  R8,R0                 ; Buffer fetch pointer
              5402 A020    A    @KBUFPNT,R0           ; Add keyboard buffer base
              5404 288A (
              5406 0420    BLWP @VMBR                 ; Read from keyboard buffer
              5408 4774 Gt
              540A 2896 (  .WORD SUB2WS  ; Address in R0
              540C 83C2    .WORD INTWS+2 ; To INTWS R1
              540E 0001    .WORD 1
              5410 0588    INC  R8                    ; Fetch pointer
              5412 0248  H ANDI R8,001F  ; mod 32
              5414 001F
              5416 81C8    C    R8,R7                 ; Same as store pointer?
              5418 1602    JNE  NOEMPT
              541A 0266  f ORI  R6,0040  ; Set buffer empty
              541C 0040  @
NOEMPT        541E 0246  F ANDI R6,FF7F  ; Not full
              5420 FF7F
              5422 02E0    LWPI INTWS
              5424 83C0
              5426 CA81    MOV  R1,@2(R10)            ; Fetched key value in R1
              5428 0002
              542A D082    MOVB R2,R2
              542C 1603    JNE  5434
              542E 06C2    SWPB R2
              5430 06A0    BL   @SCRECHO
              5432 5436 T6
              5434 0380    RTWP

; Echo to 80 col screen and physical screen, if the position is shown
SCRECHO       5436 C0E0    MOV  @CURXY,R3; Current position on screen
              5438 2CC4 ,
              543A D4C2    MOVB R2,*R3   ; Character to 80 col screen
              543C C0E0    MOV  @WIDTH,R3; Physical screen width
              543E 2CC6 ,
              5440 0913    SRL  R3,1                  ; Half
              5442 38E0 8  MPY  @WINDOW,R3            ; Multiply by current window
              5444 2CCA ,
              5446 8120    C    @CURCOL,R4            ; Compare with current column
              5448 2CC2 ,
```

```
             544A 1118    JLT  ECHORET  ; Return
             544C A120    A    @WIDTH,R4; Add screen width
             544E 2CC6 ,
             5450 8120    C    @CURCOL,R4            ; Compare with current column
             5452 2CC2 ,
             5454 1413    JHE  ECHORET  ; Return
             5456 C120    MOV  @CURLINE,R4           ; Calculate address on screen
             5458 2CC0 ,
             545A 3920 9  MPY  @WIDTH,R4
             545C 2CC6 ,
             545E C0E0    MOV  @WIDTH,R3
             5460 2CC6 ,
             5462 0913    SRL  R3,1
             5464 38E0 8  MPY  @WINDOW,R3
             5466 2CCA ,
             5468 6144 aD S    R4,R5
             546A A160  ` A    @CURCOL,R5
             546C 2CC2 ,
             546E 0225  % AI   R5,VSCREEN            ; Screen start in VDP memory
             5470 0800
             5472 0420    BLWP @VMBW                ; Write character echo on physical screen
             5474 478E G
             5476 83C4    .WORD INTWS+4 ; R2
             5478 83CA    .WORD INTWS+10; R5
             547A 0001    .WORD 1
ECHORET      547C 045B  [ B    *R11

; bios_con_write
CONWRIT      547E C060  ` MOV  @FLUSHFL,R1
             5480 2894 (
             5482 1301    JEQ  NOFLUSH
             5484 0380    RTWP

NOFLUSH      5486 D06A  j MOVB @2(R10),R1           ; R10 is IOCTRL record pointer. Block number. Char to
write?
             5488 0002
             548A 0981    SRL  R1,8
             548C C0E0    MOV  @CURFLAG,R3          ; 2 here if preceeded by chr(30)
             548E 2886 (                            ; chr(30) chr(col+32) chr(row+32)
             5490 1321  ! JEQ  CHKDISP  ; Check if displayable if CURFLAG=0
             5492 0221  ! AI   R1,-32D
             5494 FFE0
             5496 0620    DEC  @CURFLAG
             5498 2886 (
             549A 1308    JEQ  GOY                  ; Go to line number in R1?
             549C 0281    CI   R1,80D
CON080       549E 0050  P
             54A0 1102    JLT  NOTRGHT  ; Not past rightmost column
             54A2 0201    LI   R1,79D   ; Set to rightmost existing column
             54A4 004F  O
NOTRGHT      54A6 C801    MOV  R1,@SAVCOL
             54A8 2888 (
             54AA 0380    RTWP

GOY          54AC C120    MOV  @SAVCOL,R4           ; Previously saved column number
             54AE 2888 (
             54B0 8801    C    R1,@HEIGHT           ; Compare with screen height (24)
             54B2 2CC8 ,
             54B4 1103    JLT  NOBELOW  ; Not below the screen?
             54B6 C060  ` MOV  @HEIGHT,R1
             54B8 2CC8 ,
             54BA 0601    DEC  R1                   ; Last line
NOBELOW      54BC C801    MOV  R1,@CURLINE          ; Calculate position on 80 col screen
             54BE 2CC0 ,
             54C0 C804    MOV  R4,@CURCOL
             54C2 2CC2 ,
             54C4 3860 8` MPY  @CON080,R1
             54C6 549E T
             54C8 A102    A    R2,R4
             54CA A120    A    @SCREN80,R4
             54CC 2CBE ,
             54CE C804    MOV  R4,@CURXY; Set to current position
             54D0 2CC4 ,
             54D2 0380    RTWP

CHKDISP      54D4 0281    CI   R1,32D
             54D6 0020
             54D8 1105    JLT  NODISP   ; Not printable?
             54DA 0281    CI   R1,126D
             54DC 007E  ~
             54DE 1502    JGT  NODISP   ; Not printable?
             54E0 0460  ` B    @TODISP
             54E2 5632 V2
```

```
NODISP      54E4 0281    CI   R1,30D   ; Cursor code
            54E6 001E
            54E8 1604    JNE  NOCURS
            54EA C820    MOV  @SBITMSK+2,@CURFLAG    ; Contains 2
            54EC 2D94 -
            54EE 2886 (
            54F0 0380    RTWP

NOCURS      54F2 0281    CI   R1,7     ; BEL code
            54F4 0007
            54F6 1603    JNE  NOTBEEP
            54F8 06A0    BL   @BEEP
            54FA 480E H
            54FC 0380    RTWP

NOTBEEP     54FE 0281    CI   R1,0                   ; NUL?
            5500 0000
            5502 1308    JEQ  EXCONWR
            5504 0281    CI   R1,13D   ; Carriage return?
            5506 000D
            5508 1606    JNE  NOCR
            550A 6820 h  S    @CURCOL,@CURXY         ; Back to first column
            550C 2CC2 ,
            550E 2CC4 ,
            5510 04E0    CLR  @CURCOL
            5512 2CC2 ,
EXCONWR     5514 0380    RTWP

NOCR        5516 0281    CI   R1,10D   ; Linefeed code
            5518 000A
            551A 1303    JEQ  LINEFED
            551C 0281    CI   R1,138D  ; Cursor down
            551E 008A
            5520 161E    JNE  NOLF

LINEFED     5522 05A0    INC  @CURLINE
            5524 2CC0 ,
            5526 A820    A    @CON080,@CURXY
            5528 549E T
            552A 2CC4 ,
            552C 8820    C    @CURLINE,@HEIGHT
            552E 2CC0 ,
            5530 2CC8 ,
            5532 114B  K JLT  EXCONW2  ; Still on screen?
            5534 6820 h  S    @CON080,@CURXY
            5536 549E T
            5538 2CC4 ,
            553A 0620    DEC  @CURLINE
            553C 2CC0 ,
            553E C120    MOV  @LINE23,R4
            5540 2CCC ,
            5542 C0A0    MOV  @SCREN80,R2
            5544 2CBE ,
            5546 C0C2    MOV  R2,R3
            5548 0222  " AI   R2,80D   ; Screen width
            554A 0050  P
            554C 0914    SRL  R4,1
LOOP80      554E CCF2    MOV  *R2+,*R3+; Scroll screen
            5550 0604    DEC  R4
            5552 16FD    JNE  LOOP80
            5554 C0A0    MOV  @SCREN80,R2
            5556 2CBE ,
            5558 A0A0    A    @LINE23,R2
            555A 2CCC ,
            555C 102D  - JMP  CLRROW   ; Clear bottom of screen

NOLF        555E 0281    CI   R1,136D  ; Cursor left (same as backspace)
            5560 0088
            5562 1303    JEQ  GOLEFT
            5564 0281    CI   R1,8                   ; Backspace
            5566 0008
            5568 1608    JNE  NOBACK
GOLEFT      556A C020    MOV  @CURCOL,R0
            556C 2CC2 ,
            556E 1304    JEQ  ATLEFT   ; Alread at leftmost column?
            5570 0620    DEC  @CURCOL  ; Backspace
            5572 2CC2 ,
            5574 0620    DEC  @CURXY
            5576 2CC4 ,
ATLEFT      5578 0380    RTWP

NOBACK      557A 0281    CI   R1,139D  ; Cursor up
            557C 008B
            557E 1626  & JNE  NOTUP
```

```
             5580 C020    MOV  @CURLINE,R0
             5582 2CC0 ,
             5584 1306    JEQ  ATTOP                  ; At top of screen?
             5586 0620    DEC  @CURLINE ; Go up one line
             5588 2CC0 ,
             558A 6820 h  S    @CON080,@CURXY
             558C 549E T
             558E 2CC4 ,
             5590 0380    RTWP

; Scroll screen downwards
ATTOP        5592 C120    MOV  @LINE23,R4             ; Bytes in 23 lines
             5594 2CCC ,
             5596 C0E0    MOV  @SCREN80,R3            ; 80 column screen start position
             5598 2CBE ,
             559A C083    MOV  R3,R2
             559C 0222  " AI   R2,80D   ; Start of second line
             559E 0050  P
             55A0 A084    A    R4,R2                  ; First word after screen
             55A2 0642  B DECT R2
             55A4 A0C4    A    R4,R3
             55A6 0643  C DECT R3
             55A8 0914    SRL  R4,1                   ; To count words
LOOP78       55AA C493    MOV  *R3,*R2
             55AC 0643  C DECT R3
             55AE 0642  B DECT R2
             55B0 0604    DEC  R4
             55B2 16FB    JNE  LOOP78
             55B4 C0A0    MOV  @SCREN80,R2
             55B6 2CBE ,
CLRROW       55B8 0203    LI   R3,80D   ; Clear a row
             55BA 0050  P
CLREND       55BC C060  ` MOV  @SPACES,R1
             55BE 40A2 @
LOOP79       55C0 DC81    MOVB R1,*R2+
             55C2 0603    DEC  R3
             55C4 16FD    JNE  LOOP79
             55C6 06A0    BL   @DISPCUR
             55C8 47D2 G
EXCONW2      55CA 0380    RTWP

NOTUP        55CC 0281    CI   R1,130D  ; Cursor home (top left)
             55CE 0082
             55D0 160D    JNE  CURHOME
             55D2 06A0    BL   @TOPLEFT
             55D4 55D8 U
             55D6 0380    RTWP

; Go to top left corner
TOPLEFT      55D8 04E0    CLR  @CURLINE
             55DA 2CC0 ,
             55DC 04E0    CLR  @CURCOL
             55DE 2CC2 ,
             55E0 C820    MOV  @SCREN80,@CURXY
             55E2 2CBE ,
             55E4 2CC4 ,
             55E6 04E0    CLR  @WINDOW
             55E8 2CCA ,
             55EA 045B  [ B    *R11

CURHOME      55EC 0281    CI   R1,137D  ; Move cursor right
             55EE 0089
             55F0 1325  % JEQ  CURRGHT
             55F2 0281    CI   R1,142D  ; Clear to end of screen
             55F4 008E
             55F6 160B    JNE  NOCLEND
             55F8 C0A0    MOV  @HEIGHT,R2             ; Clears to end of line from current position
             55FA 2CC8 ,
             55FC 38A0 8  MPY  @CON080,R2
             55FE 549E T
             5600 60E0 `  S    @CURXY,R3
             5602 2CC4 ,
             5604 A0E0    A    @SCREN80,R3
             5606 2CBE ,
CLINEND      5608 C0A0    MOV  @CURXY,R2
             560A 2CC4 ,
             560C 10D7    JMP  CLREND

NOCLEND      560E 0281    CI   R1,141D  ; Clear line to right end of screen
             5610 008D
             5612 1605    JNE  NOCLINE
             5614 0203    LI   R3,80D
             5616 0050  P
             5618 60E0 `  S    @CURCOL,R3
```

```
             561A 2CC2 ,
             561C 10F5    JMP  CLINEND

NOCLINE      561E 0281    CI   R1,12D   ; Form feed (page output)
             5620 000C
             5622 1607    JNE  TODISP
             5624 06A0    BL   @TOPLEFT
             5626 55D8 U
             5628 C0A0    MOV  @HEIGHT,R2
             562A 2CC8 ,
             562C 38A0 8  MPY  @CON080,R2
             562E 549E T
             5630 10EB    JMP  CLINEND

; Display character in R1
TODISP       5632 06C1    SWPB R1
             5634 C081    MOV  R1,R2
             5636 06A0    BL   @SCRECHO
             5638 5436 T6
             563A 06C1    SWPB R1
CURRGHT      563C C0E0    MOV  @CURCOL,R3
             563E 2CC2 ,
             5640 0583    INC  R3
             5642 0283    CI   R3,80D
             5644 0050  P
             5646 1304    JEQ  DISPRDY  ; At rightmost column?
             5648 C803    MOV  R3,@CURCOL             ; Move right
             564A 2CC2 ,
             564C 05A0    INC  @CURXY
             564E 2CC4 ,
DISPRDY      5650 0380    RTWP

; bios_dsk_stat
; R10 is IOCTRL pointer
DSKSTAT      5652 C06A  j MOV  @6(R10),R1             ; Buffer address
             5654 0006
             5656 04F1    CLR  *R1+
             5658 C26A  j MOV  @14D(R10),R9           ; Physical drive number
             565A 000E
             565C 0289    CI   R9,16D   ; Unit loaded from tape
             565E 0010
             5660 1308    JEQ  TAPEUNI
             5662 1B0A    JH   GROMUNI
             5664 C0E0    MOV  @SECSIZE,R3            ; Sector size
             5666 2CE6 ,
             5668 0200    LI   R0,9                   ; Default sectors per track
             566A 0009
             566C 0202    LI   R2,40D   ; Default tracks per side
             566E 0028  (
             5670 1009    JMP  RETSTAT  ; Return status

TAPEUNI      5672 06A0    BL   @TAPUNST ; Fetch data for unit loaded from tape
             5674 58D6 X
             5676 1002    JMP  CONTI01

GROMUNI      5678 06A0    BL   @GROMUST ; Fetch data for GROM unit
             567A 58FC X
CONTI01      567C 0202    LI   R2,1                   ; Single track
             567E 0001
             5680 C0E0    MOV  @GROMSEC,R3
             5682 2CE8 ,
RETSTAT      5684 CC43  C MOV  R3,*R1+  ; Sector size
             5686 CC40  @ MOV  R0,*R1+  ; Size of GROM unit
             5688 CC42  B MOV  R2,*R1+  ; Tracks per side
             568A 0380    RTWP

; bios_dsk_init
; R10 is pointer to IOCTRL
DSKINIT      568C C06A  j MOV  @8(R10),R1             ; Drive number
             568E 0008
             5690 C0AA    MOV  @14D(R10),R2           ; Physical drive number
             5692 000E
             5694 0282    CI   R2,16D   ; Number for unit loaded from tape
             5696 0010
             5698 1318    JEQ  TAPUNI2  ; A tape unit?
             569A 1B20    JH   GRMUNI2  ; A GROM unit?
             569C 06C2    SWPB R2
             569E 0582    INC  R2
             56A0 C802    MOV  R2,@FAC+2
             56A2 834C  L
             56A4 C820    MOV  @DSKBUF,@FAC+4
             56A6 2CEA ,
             56A8 834E  N
             56AA C820    MOV  @SBITMSK+4,@FAC+6      ; Value is 4
```

```
             56AC 2D96 -
             56AE 8350  P
             56B0 0420    BLWP @PERCALL ; Do the IO with a device that has a DSR
             56B2 33F6 3
             56B4 D020    MOVB @FAC+6,R0
             56B6 8350  P
             56B8 1604    JNE  BADCODE
GOODEND      56BA D860  ` MOVB @ZERO,@SERBUF(R1)
             56BC 40AC @
             56BE 29B8 )
             56C0 0380    RTWP
BADCODE      56C2 D860  ` MOVB @ALLFF,@SERBUF(R1)     ; -1
             56C4 40B8 @
             56C6 29B8 )
             56C8 1036  6 JMP  FAILRET

TAPUNI2      56CA 0420    BLWP @VMBR                  ; Tape unit living in VDP RAM
             56CC 4774 Gt
             56CE 2D6E -n .WORD ADDROFS+2
             56D0 83C0    .WORD INTWS
             56D2 0001    .WORD 1
GRMRET       56D4 9800    CB   R0,@VALIDVO            ; Does this GROM have a valider as first byte?
             56D6 2CE4 ,
             56D8 13F0    JEQ  GOODEND
             56DA 10F3    JMP  BADCODE

GRMUNI2      56DC 0222  " AI   R2,-15
             56DE FFF1
             56E0 0A12    SLA  R2,1
             56E2 C162  b MOV  @CODREAD(R2),R5        ; Find read data address
             56E4 2CFA ,
             56E6 0225  % AI   R5,WAOFFS              ; Change to write address location
             56E8 0402
             56EA C1A2    MOV  @ADDROFS(R2),R6        ; Address offset for GROM
             56EC 2D6C -l
             56EE 06A0    BL   @GRMRDR6 ; Fetch word from GROM to R6
             56F0 59B4 Y
             56F2 10F0    JMP  GRMRET

; Get parameters for disk access (WP 83C0)
; R0          Serial buffer (IO-result code)
; R1          Buffer address
; R2          Byte count
; R3          Block/sector number
; R4          Control parameter
; R5
; R6
; R7
; R8          Drive number
; R9          Physical drive number
; R10         IO control block pointer

; Reads data from IO control block pointed to by R10
DISKPAR      56F4 C0EA    MOV  @2(R10),R3                          ; Block number -> R3
             56F6 0002
             56F8 C0AA    MOV  @4(R10),R2                          ; Byte count -> R2
             56FA 0004
             56FC C06A  j MOV  @6(R10),R1                          ; Buffer address -> R1
             56FE 0006
             5700 C26A  j MOV  @14D(R10),R9                        ; Physical drive number -> R9
             5702 000E
             5704 C12A  * MOV  @10D(R10),R4                        ; Control parameter -> R4
             5706 000A
             5708 04EA    CLR  @12D(R10)              ; IO result code cleared in IO control block
             570A 000C
             570C C22A  * MOV  @8(R10),R8                          ; Drive number -> R8
             570E 0008
             5710 D028  ( MOVB @SERBUF(R8),R0         ; Contains IO result code -> R0
             5712 29B8 )
             5714 1610    JNE  FAILRET
             5716 C220    MOV  @GPLWS+30,R8                        ; GPLWS R15 (VDPWA)
             5718 83FE
             571A D820    MOVB @INTWS+19D,@FAC+2      ; INTWS R10 LSBy
             571C 83D3
             571E 834C  L
             5720 C820    MOV  @DSKBUF,@FAC+4         ; Buffer location in VDP RAM
             5722 2CEA ,
             5724 834E  N
             5726 0289    CI   R9,15D   ; Logical drive (has no physical sector access)
             5728 000F
             572A 1B04    JH   GOODRET
             572C 2120 !  COC  @SBITMSK+2,R4                       ; Contains 0002, physical sector access
             572E 2D94 -
             5730 1301    JEQ  GOODRET
```

```
                5732 0A13    SLA  R3,1                  ; Block number to sector number when doing physical
access
GOODRET         5734 045B  [ B    *R11

FAILRET         5736 DAA0    MOVB @SECTERR+1,@13(R10)    ; IO result code
                5738 2CEF ,
                573A 000D
                573C 0380    RTWP

ETX             573E 0003    .WORD 3

; bios_dsk_write
DSKWRIT         5740 04E0    CLR  @FAC+3   ; A byte address! Write flag
                5742 834D  M
                5744 06A0    BL   @DISKPAR
                5746 56F4 V
                5748 0289    CI   R9,16D   ; Tape unit (if there is one)
                574A 0010
                574C 1A04    JL   WRITEOK  ; Not allowed to write to tape unit nor GROM.
                574E CAA0    MOV  @ETX,@12(R10)          ; IO result code "Illegal operation on device"
                5750 573E W>
                5752 000C
                5754 0380    RTWP

WRITEOK         5756 C1E0    MOV  @SECSIZE,R7            ; Byte count
                5758 2CE6 ,
                575A 0200    LI   R0,4000  ; Set VDP for write
                575C 4000 @
                575E 06A0    BL   @VDPBUF  ; Set up VDP to write to location from DSKBUF
                5760 59A6 Y
                5762 04C5    CLR  R5
                5764 0244  D ANDI R4,C000  ; Control parameter. 4000H means VDP address
                5766 C000
                5768 130D    JEQ  LOOP87   ; Standard IO? (All zero)
                576A 1107    JLT  GRMBUFF  ; GROM buffer? MSbit set
                576C C181    MOV  R1,R6                  ; Swap R0 <-> R1
                576E C040  @ MOV  R0,R1                  ;
                5770 C006    MOV  R6,R0                  ;
                5772 06A0    BL   @VDPMOV  ; R0 from, R1 to, R7 count
                5774 592C Y,
                5776 C040  @ MOV  R0,R1                  ; Buffer address back
                5778 1009    JMP  MAKEWRT

GRMBUFF         577A C181    MOV  R1,R6                  ; Save buffer address
                577C 06A0    BL   @GROMBUF ; From GROM to VDP buffer
                577E 5970 Yp
                5780 C046  F MOV  R6,R1                  ; Restore buffer address
                5782 1004    JMP  MAKEWRT

LOOP87          5784 DA31  1 MOVB *R1+,@-2(R8)           ; Buffer to VDP write data address
                5786 FFFE
                5788 0607    DEC  R7
                578A 16FC    JNE  LOOP87
MAKEWRT         578C 06A0    BL   @BLOCKIO
                578E 5892 X
                5790 C082    MOV  R2,R2                  ; Remaining byte count
                5792 15E1    JGT  WRITEOK
                5794 0380    RTWP

; bios_dsk_read
DSKREAD         5796 0720    SETO @FAC+3   ; A byte address! Read flag
                5798 834D  M
                579A 04E0    CLR  @GRMADDR ; GROM address
                579C 2CE2 ,
                579E 06A0    BL   @DISKPAR
                57A0 56F4 V
LOOP99          57A2 0289    CI   R9,16D   ; Physical drive number
                57A4 0010
                57A6 1320    JEQ  VIRTAPE  ; Virtual tape unit
                57A8 1B37  7 JH   INGROM2  ; GROM unit
                57AA 06A0    BL   @BLOCKIO
                57AC 5892 X
NEWSECT         57AE C1E0    MOV  @SECSIZE,R7
                57B0 2CE6 ,
                57B2 C082    MOV  R2,R2                  ; Remaining byte count
                57B4 1502    JGT  NOFIT2
                57B6 1301    JEQ  NOFIT2
                57B8 A1C2    A    R2,R7                  ; Byte count is always negative here, so subtracts
from sector size
NOFIT2          57BA 04C0    CLR  R0
                57BC 06A0    BL   @VDPBUF
                57BE 59A6 Y
NXTSEC          57C0 04C5    CLR  R5
                57C2 0244  D ANDI R4,C000  ; Control word bit 14 and 15
```

```
             57C4 C000
             57C6 1309    JEQ  LOOP63
             57C8 1103    JLT  CTRL15   ; Control bit 15?
             57CA 06A0    BL   @VDPMOV  ; Move to buffer if CTRL 14 set
             57CC 592C Y,
             57CE 1009    JMP  CONT4

CTRL15       57D0 C181    MOV  R1,R6                 ; Control bit 15
             57D2 06A0    BL   @TOGRAM  ; Move to GRAM if bit 15 is set
             57D4 5952 YR
             57D6 C046  F MOV  R6,R1
             57D8 1004    JMP  CONT4

LOOP63       57DA DC68  h MOVB @-WAOFFS(R8),*R1+      ; Read from mapped memory
             57DC FBFE
             57DE 0607    DEC  R7
             57E0 16FC    JNE  LOOP63
CONT4        57E2 C082    MOV  R2,R2                 ; Remaining byte count
             57E4 15DE    JGT  LOOP99
             57E6 0380    RTWP

VIRTAPE      57E8 06A0    BL   @TAPUNST
             57EA 58D6 X
             57EC 80C0    C    R0,R3
             57EE 1A4D  M JL   IOCODRT
             57F0 C003    MOV  R3,R0
             57F2 05C0    INCT R0
             57F4 0A10    SLA  R0,1
             57F6 A020    A    @ADDROFS+2,R0
             57F8 2D6E -n
             57FA 0420    BLWP @VMBR
             57FC 4774 Gt
             57FE 83C0    .WORD INTWS   ; R0
             5800 83C0    .WORD INTWS   ; R0
             5802 0002    .WORD 2
             5804 C1E0    MOV  @GROMSEC,R7
             5806 2CE8 ,
             5808 8087    C    R7,R2                 ; Compare sector size to remaining byte count
             580A 1201    JLE  NOFIT                 ; Doesn't fit in sector?
             580C C1C2    MOV  R2,R7
NOFIT        580E 6087 `  S    R7,R2                 ; Subtract one sector from remaining
             5810 06A0    BL   @VDPADDR
             5812 59AA Y
             5814 0583    INC  R3
             5816 10D4    JMP  NXTSEC

; Data is in user defined GROM device
INGROM2      5818 06A0    BL   @GROMUST ; Validate GROM unit
             581A 58FC X
             581C C1A0    MOV  @GRMADDR,R6           ; GROM address
             581E 2CE2 ,
             5820 1609    JNE  HASADDR  ; Already set up?
             5822 80C0    C    R0,R3                 ; R3 is block number, R0 second word in GROM unit
             5824 1A32  2 JL   IOCODRT  ; Outside volume?
             5826 C183    MOV  R3,R6
             5828 05C6    INCT R6
             582A 0A16    SLA  R6,1
             582C A18C    A    R12,R6   ; R12 is here GROM start read data address for the unit accessed
             582E 06A0    BL   @GRMRDR6 ; Find address
             5830 59B4 Y
             5832 C180    MOV  R0,R6
HASADDR      5834 C1E0    MOV  @SECSIZE,R7
             5836 2CE6 ,
             5838 0200    LI   R0,4000  ; Write enable bit
             583A 4000 @
             583C 06A0    BL   @VDPBUF
             583E 59A6 Y
             5840 21A0 !  COC  @LASTGRM,R6           ; Check if in GROM address gap
             5842 2CDE ,
             5844 1602    JNE  NOGEND1
             5846 A1A0    A    @GROMGAP,R6
             5848 2CE0 ,
NOGEND1      584A D546  F MOVB R6,*R5   ; Update GROM address to pass GROM address gap
             584C 06C6    SWPB R6
             584E D546  F MOVB R6,*R5
             5850 06C6    SWPB R6
LOOP90       5852 DA25  % MOVB @-WAOFFS(R5),@-2(R8)   ; GROM data -> VDP data
             5854 FBFE
             5856 FFFE
             5858 0586    INC  R6
             585A 21A0 !  COC  @LASTGRM,R6           ; Check if in GROM address gap
             585C 2CDE ,
             585E 1606    JNE  NOGEND2
             5860 A1A0    A    @GROMGAP,R6
```

```
                5862 2CE0 ,
                5864 D546  F MOVB R6,*R5   ; Update GROM address to pass GROM address gap
                5866 06C6    SWPB R6
                5868 D546  F MOVB R6,*R5
                586A 06C6    SWPB R6
NOGEND2         586C 0607    DEC  R7                    ; Sector size counter
                586E 16F1    JNE  LOOP90
                5870 60A0 `  S    @SECSIZE,R2
                5872 2CE6 ,
                5874 C1E0    MOV  @GRMADDR,R7
                5876 2CE2 ,
                5878 1304    JEQ  ISZERO   ; Stored address zero?
                587A 04E0    CLR  @GRMADDR
                587C 2CE2 ,
                587E 0583    INC  R3       ; Block number
                5880 1002    JMP  NEXTSEC

ISZERO          5882 C806    MOV  R6,@GRMADDR           ; R6 is GROM address here
                5884 2CE2 ,
NEXTSEC         5886 0460  ` B    @NEWSECT
                5888 57AE W

IOCODRT         588A DAA0    MOVB @SECTERR+3,@13(R10)    ; IO result code and return
                588C 2CF1 ,
                588E 000D
                5890 0380    RTWP

BLOCKIO         5892 04CC    CLR  R12
                5894 1F02    TB   VDPIRQ   ; VDP interrupt
                5896 1302    JEQ  NOVINT
                5898 0420    BLWP @VDPINT  ; Execute VPD interrupt
                589A 2AAA *
NOVINT          589C C00B    MOV  R11,R0
                589E C803    MOV  R3,@FAC+6; Sector number
                58A0 8350  P
                58A2 0420    BLWP @PERCALL ; Do the IO with a device that has a DSR
                58A4 33F6 3
                58A6 D320    MOVB @FAC+6,R12
                58A8 8350  P
                58AA 1604    JNE  DSKFAIL
                58AC 0583    INC  R3                    ; Sector number
                58AE 60A0 `  S    @SECSIZE,R2           ; Decrease byte count
                58B0 2CE6 ,
                58B2 0450  P B    *R0      ; Saved return address

DSKFAIL         58B4 098C    SRL  R12,8
                58B6 0206    LI   R6,4
                58B8 0004
LOOP86          58BA 99A0    CB   @FAC+6,@SECTERR(R6)    ; Look in error code conversion table
                58BC 8350  P
                58BE 2CEE ,
                58C0 1306    JEQ  HASERR
                58C2 0646  F DECT R6
                58C4 18FA    JOC  LOOP86
                58C6 DAA0    MOVB @FAC+6,@13(R10)        ; IO result code not in SECTERR table
                58C8 8350  P
                58CA 000D
                58CC 1003    JMP  DSKRET

HASERR          58CE DAA6    MOVB @SECTERR+1(R6),@13(R10)             ; IO result code in SECTERR table
                58D0 2CEF ,
                58D2 000D
DSKRET          58D4 0380    RTWP

TAPUNST         58D6 0420    BLWP @VMBR
                58D8 4774 Gt
                58DA 2D6E -n .WORD ADDROFS+2             ; Pointer to pointer to address to read from
                58DC 83C0    .WORD INTWS   ; Destination address      R0
                58DE 0001    .WORD 1                    ; Byte count
                58E0 9800    CB   R0,@VALIDVO
                58E2 2CE4 ,
                58E4 1302    JEQ  TAPGOOD
                58E6 0460  ` B    @BADVOL  ; Generate "Bad volume error"
                58E8 5268 Rh

TAPGOOD         58EA C1A0    MOV  @ADDROFS+2,R6
                58EC 2D6E -n
                58EE 05C6    INCT R6
                58F0 0420    BLWP @VMBR                 ; Now read two bytes after first read above
                58F2 4774 Gt
                58F4 83CC    .WORD INTWS+12; R6
                58F6 83C0    .WORD INTWS   ; R0
                58F8 0002    .WORD 2                    ; Byte count
                58FA 045B  [ B    *R11
```

```
GROMUST       58FC C309    MOV  R9,R12   ; Physical drive number, 15 is first in GROM (P-code card)
              58FE 022C  , AI   R12,-15D
              5900 FFF1
              5902 0A1C    SLA  R12,1                 ; WORD pointer
              5904 C16C  l MOV  @CODREAD(R12),R5      ; Find read data address
              5906 2CFA ,
              5908 130B    JEQ  NOTGROM  ; CPU RAM address
              590A 0225  % AI   R5,WAOFFS; Change to write address
              590C 0402
              590E C32C  , MOV  @ADDROFS(R12),R12     ; GROM access offset
              5910 2D6C -l
              5912 C18C    MOV  R12,R6   ; GROM address
              5914 C1CB    MOV  R11,R7   ; Save return link
              5916 06A0    BL   @GRMRDR6 ; Fetch GROM word pointed to by R6
              5918 59B4 Y
              591A 9800    CB   R0,@VALIDVO           ; Check if it's a valid volume
              591C 2CE4 ,
              591E 1302    JEQ  GRMGOOD
NOTGROM       5920 0460  ` B    @BADVOL
              5922 5268 Rh

GRMGOOD       5924 05C6    INCT R6
              5926 06A0    BL   @GRMRDR6 ; Fetch GROM word after valider, which is size of GROM unit in
blocks.
              5928 59B4 Y
              592A 0457  W B    *R7                   ; Return with saved address

; Move data in VDP RAM, byte by byte
; R0 address to read from
; R1 address to write to
; R7 byte count
; R8 VDPWA
; Used if CONTROL has bit 14 set during unit IO
VDPMOV        592C 0261  a ORI  R1,4000  ; Address for write
              592E 4000 @
LOOP62        5930 06C0    SWPB R0                    ; Load read address
              5932 D600    MOVB R0,*R8
              5934 06C0    SWPB R0
              5936 D600    MOVB R0,*R8
              5938 0580    INC  R0                    ; For next byte
              593A D168  h MOVB @-WAOFFS(R8),R5       ; Read one byte
              593C FBFE
              593E 06C1    SWPB R1                    ; Load write address
              5940 D601    MOVB R1,*R8
              5942 06C1    SWPB R1
              5944 D601    MOVB R1,*R8
              5946 0581    INC  R1                    ; For next byte
              5948 DA05    MOVB R5,@-2(R8)            ; Store the read byte
              594A FFFE
              594C 0607    DEC  R7                    ; Count bytes
              594E 15F0    JGT  LOOP62
              5950 045B  [ B    *R11

TOGRAM        5952 0225  % AI   R5,9C26  ; Skip ten GROM device access addresses
              5954 9C26  &
              5956 0606    DEC  R6
LOOP89        5958 D546  F MOVB R6,*R5   ; Write GROM address
              595A 06C6    SWPB R6
              595C D546  F MOVB R6,*R5
              595E 06C6    SWPB R6
              5960 0586    INC  R6
              5962 D968  h MOVB @-WAOFFS(R8),@-2(R5)   ; VDP read data to GROM write data
              5964 FBFE
              5966 FFFE
              5968 0607    DEC  R7                    ; Count bytes
              596A 16F6    JNE  LOOP89
              596C 0586    INC  R6
              596E 045B  [ B    *R11

; Here's some fancy trick for something that looks like using a GROM buffer

GROMBUF       5970 0225  % AI   R5,9C26  ; Ten GROM address displacements
              5972 9C26  &
              5974 21A0 !  COC  @LASTGRM,R6            ; Check for GROM address gap
              5976 2CDE ,
              5978 1602    JNE  NOGEND3
              597A A1A0    A    @GROMGAP,R6           ; Bridge gap if needed
              597C 2CE0 ,
NOGEND3       597E D546  F MOVB R6,*R5   ; Write new GROM address to bridge gap
              5980 06C6    SWPB R6
              5982 D546  F MOVB R6,*R5
              5984 06C6    SWPB R6
LOOP88        5986 DA25  % MOVB @-WAOFFS(R5),@-2(R8)   ; From GROM to VDP
```

```
             5988 FBFE
             598A FFFE
             598C 0586    INC  R6
             598E 21A0 !  COC  @LASTGRM,R6            ; GROM address gap
             5990 2CDE ,
             5992 1606    JNE  NOGEND4
             5994 A1A0    A    @GROMGAP,R6
             5996 2CE0 ,
             5998 D546  F MOVB R6,*R5   ; Write new GROM address to bridge gap
             599A 06C6    SWPB R6
             599C D546  F MOVB R6,*R5
             599E 06C6    SWPB R6
NOGEND4      59A0 0607    DEC  R7
             59A2 16F1    JNE  LOOP88
             59A4 045B  [ B    *R11

; Set up buffer address for VDP
VDPBUF       59A6 E020    SOC  @DSKBUF,R0
             59A8 2CEA ,
VDPADDR      59AA 06C0    SWPB R0
             59AC D600    MOVB R0,*R8
             59AE 06C0    SWPB R0
             59B0 D600    MOVB R0,*R8
             59B2 045B  [ B    *R11

; Reads word at GROM address in R6 to R0
GRMRDR6      59B4 D546  F MOVB R6,*R5   ; Load GROM address
             59B6 06C6    SWPB R6
             59B8 D546  F MOVB R6,*R5
             59BA 06C6    SWPB R6
             59BC D025  % MOVB @-WAOFFS(R5),R0        ; Read GROM data
             59BE FBFE
             59C0 06C0    SWPB R0
             59C2 D025  % MOVB @-WAOFFS(R5),R0
             59C4 FBFE
             59C6 06C0    SWPB R0
             59C8 045B  [ B    *R11

; bios_therm_init
TPINIT       59CA 04E0    CLR  @THRMIO1
             59CC 295C )\
             59CE 04E0    CLR  @THRMIO2 ; Byte count
             59D0 295E )^
             59D2 04E0    CLR  @IOOPCOD
             59D4 2962 )b
             59D6 0420    BLWP @PERCALL ; Do the IO with a device that has a DSR
             59D8 33F6 3
             59DA CAA0    MOV  @IOWS,@12(R10)         ; IO result code
             59DC 2964 )d
             59DE 000C
             59E0 C060  ` MOV  @PPOINT,R1             ; PCB pointer
             59E2 2960 )`
             59E4 0221  ! AI   R1,12D
             59E6 000C
             59E8 0420    BLWP @VMBR
             59EA 4774 Gt
             59EC 83C2    .WORD INTWS+2
             59EE 83C4    .WORD INTWS+4
             59F0 0001    .WORD 1
             59F2 0982    SRL  R2,8
             59F4 C802    MOV  R2,@THRMIO3            ; Head of thermal printer buffer
             59F6 28D6 (
             59F8 0380    RTWP

; bios_therm_write
TPWRIT       59FA C820    MOV  @WRCODE,@IOOPCOD
             59FC 2CF6 ,
             59FE 2962 )b
             5A00 D02A  * MOVB @2(R10),R0             ; Block number ?
             5A02 0002
             5A04 1342  B JEQ  THMEXIT  ; Exit if zero
             5A06 9800    CB   R0,@SPCR+1             ; Carriage return
             5A08 409D @
             5A0A 1607    JNE  NOTHMCR  ; Not CR
             5A0C 0720    SETO @THRMIO1 ; Carriage return flag?
             5A0E 295C )\
             5A10 06A0    BL   @DOTHMIO
             5A12 5A8C Z
             5A14 04E0    CLR  @THRMIO2 ; Byte count
             5A16 295E )^
             5A18 1038  8 JMP  THMEXIT

NOTHMCR      5A1A 9800    CB   R0,@BYLF
             5A1C 409E @
```

```
              5A1E 1612    JNE  NOTHMLF
              5A20 C020    MOV  @THRMIO1,R0            ; Line feed, check CR flag
              5A22 295C )\
              5A24 1303    JEQ  NOCRFLG
              5A26 04E0    CLR  @THRMIO1
              5A28 295C )\
              5A2A 102F  / JMP  THMEXIT

NOCRFLG       5A2C 06A0    BL   @DOTHMIO
              5A2E 5A8C Z
              5A30 0201    LI   R1,THRMBUF
              5A32 28D8 (
              5A34 C0A0    MOV  @THRMIO2,R2            ; Byte count
              5A36 295E )^
              5A38 1328   ( JEQ  THMEXIT
LOOP91        5A3A DC60   ` MOVB @SPACES,*R1+          ; Space fill
              5A3C 40A2 @
              5A3E 0602    DEC  R2
              5A40 16FC    JNE  LOOP91
              5A42 1023   # JMP  THMEXIT

NOTHMLF       5A44 04E0    CLR  @THRMIO1
              5A46 295C )\
              5A48 9800    CB   R0,@FORMFED            ; Form feed command
              5A4A 40A0 @
              5A4C 160B    JNE  NOTHMFF
              5A4E 06A0    BL   @DOTHMIO ; Print current line
              5A50 5A8C Z
              5A52 04E0    CLR  @THRMIO2 ; Reset byte count
              5A54 295E )^
              5A56 0200    LI   R0,0005  ; Number of lines for form feed
              5A58 0005
LOOP92        5A5A 06A0    BL   @DOTHMIO ; Print blank line
              5A5C 5A8C Z
              5A5E 0600    DEC  R0
              5A60 16FC    JNE  LOOP92
              5A62 1013    JMP  THMEXIT

NOTHMFF       5A64 05A0    INC  @THRMIO2 ; Byte count
              5A66 295E )^
              5A68 0201    LI   R1,THRMIO3+1           ; Right before thermal IO buffer
              5A6A 28D7 (
              5A6C A060   ` A    @THRMIO2,R1           ; Add byte count
              5A6E 295E )^
              5A70 D440   @ MOVB R0,*R1   ; Char into buffer
              5A72 8820    C    @THRMIO2,@THRMIO3
              5A74 295E )^
              5A76 28D6 (
              5A78 1608    JNE  THMEXIT  ; Line not completed?
              5A7A 06A0    BL   @DOTHMIO
              5A7C 5A8C Z
              5A7E C820    MOV  @SPACES,@THRMBUF
              5A80 40A2 @
              5A82 28D8 (
              5A84 C820    MOV  @SBITMSK+2,@THRMIO2    ; Contains 2
              5A86 2D94 -
              5A88 295E )^
THMEXIT       5A8A 0380    RTWP

DOTHMIO       5A8C C0E0    MOV  @PPOINT,R3             ; PCB pointer
              5A8E 2960 )`
              5A90 0223   # AI   R3,13D   ; Byte 9 in PAB
              5A92 000D
              5A94 0420    BLWP @VMBW
              5A96 478E G
              5A98 295F )_ .WORD THRMIO2+1
              5A9A 83C6    .WORD INTWS+6 (R3)          ; Address of name length pointer
              5A9C 0001    .WORD 1
              5A9E 0420    BLWP @I2VMBW
              5AA0 47B2 G
              5AA2 2CF4 ,  .WORD THBUFPT ; Print buffer
              5AA4 2CEA ,  .WORD DSKBUF  ; VPD RAM buffer pointer
              5AA6 295E )^ .WORD THRMIO2 ; Byte count
              5AA8 0420    BLWP @PERCALL ; Call peripheral
              5AAA 33F6 3
              5AAC CAA0    MOV  @IOWS,@12D(R10)        ; IO result code
              5AAE 2964 )d
              5AB0 000C
              5AB2 045B   [ B    *R11

; bios_rem_init
REMINIT       5AB4 04E0    CLR  @IOOPCOD
              5AB6 2962 )b
              5AB8 06A0    BL   @DOPCALL
```

```
             5ABA 5B04 [
             5ABC 0380    RTWP

; bios_rem_read
REMREAD      5ABE C820    MOV  @RDCODE,@IOOPCOD
             5AC0 2CF8 ,
             5AC2 2962 )b
             5AC4 06A0    BL   @DOPCALL
             5AC6 5B04 [
             5AC8 0420    BLWP @VMBR
             5ACA 4774 Gt
             5ACC 2CEA ,  .WORD DSKBUF  ; VPD RAM buffer pointer
             5ACE 83C2    .WORD INTWS+2
             5AD0 0001    .WORD 1
             5AD2 CA81    MOV  R1,@2(R10)             ; Block number
             5AD4 0002
             5AD6 0380    RTWP

; bios_rem_write
REMWRIT      5AD8 C06A  j MOV  @2(R10),R1             ; Block number
             5ADA 0002
             5ADC 0420    BLWP @VMBW
             5ADE 478E G
             5AE0 83C2    .WORD INTWS+2
             5AE2 2CEA ,  .WORD DSKBUF  ; VPD RAM buffer pointer
             5AE4 0001    .WORD 1
             5AE6 C020    MOV  @PPOINT,R0             ; PCB pointer
             5AE8 2960 )`
             5AEA 0220    AI   R0,13                  ; Byte 9 in PAB part
             5AEC 000D
             5AEE 0420    BLWP @VMBW
             5AF0 478E G
             5AF2 2C9C ,  .WORD REMIO
             5AF4 83C0    .WORD INTWS
             5AF6 0001    .WORD 1
             5AF8 C820    MOV  @WRCODE,@IOOPCOD
             5AFA 2CF6 ,
             5AFC 2962 )b
             5AFE 06A0    BL   @DOPCALL
             5B00 5B04 [
             5B02 0380    RTWP

; Do peripheral call, return IO result code
DOPCALL      5B04 0420    BLWP @PERCALL ; Do the IO with a device that has a DSR
             5B06 33F6 3
             5B08 CAA0    MOV  @IOWS,@12(R10)         ; IO result code
             5B0A 2964 )d
             5B0C 000C
             5B0E 045B  [ B    *R11

; bios_tape_init
TAPINIT      5B10 0720    SETO @TAPEIO1
             5B12 2820 (
             5B14 04E0    CLR  @TAPEIO2
             5B16 2822 ("
             5B18 04E0    CLR  @TAPEIO3
             5B1A 2844 (D
             5B1C 04CC    CLR  R12
             5B1E 1E16    SBZ  CS1MOT   ; CS1 motor
             5B20 0380    RTWP

; bios_tape_write
TAPWRIT      5B22 0204    LI   R4,DOTAPWR             ; Address to tape write
             5B24 5C38 \8
             5B26 06A0    BL   @PREPTAP ; Prepare for IO (returns if tape)
             5B28 5BAA [
             5B2A D0AA    MOVB @2(R10),R2             ; Block number
             5B2C 0002
             5B2E 9802    CB   R2,@BYLF ; Contains LF (10D)
             5B30 409E @
             5B32 1604    JNE  NBLKTEN
             5B34 9820    CB   @TAPEIO3+1,@SPCR+1     ; Contains CR (13D)
             5B36 2845 (E
             5B38 409D @
             5B3A 1315    JEQ  TAPEXIT
NBLKTEN      5B3C D802    MOVB R2,@TAPEIO3+1          ; Block number
             5B3E 2845 (E
             5B40 C060  ` MOV  @TAPEIO2,R1
             5B42 2822 ("
             5B44 D842  B MOVB R2,@TAPEIO4(R1)
             5B46 2846 (F
             5B48 0581    INC  R1
             5B4A 9802    CB   R2,@ETXCOD             ; Is it End of Text (3)?
             5B4C 2BC1 +
```

```
             5B4E 1603    JNE  NOETX
             5B50 06A0    BL   @DOTAPIO
             5B52 5B98 [
             5B54 10DD    JMP  TAPINIT

NOETX        5B56 8801    C    R1,@2CCE
             5B58 2CCE ,
             5B5A 1A03    JL   TAPEND
             5B5C 06A0    BL   @DOTAPIO
             5B5E 5B98 [
             5B60 04C1    CLR  R1
TAPEND       5B62 C801    MOV  R1,@TAPEIO2
             5B64 2822 ("
TAPEXIT      5B66 0380    RTWP

; bios_tape_read
TAPREAD      5B68 0204    LI   R4,DOTAPRD
             5B6A 5D3C ]<
             5B6C 06A0    BL   @PREPTAP ; Prepare for IO (returns if tape)
             5B6E 5BAA [
             5B70 C060    ` MOV  @TAPEIO2,R1
             5B72 2822 ("
             5B74 1602    JNE  5B7A
             5B76 06A0    BL   @DOTAPIO
             5B78 5B98 [
             5B7A D0A1    MOVB @TAPEIO4(R1),R2
             5B7C 2846 (F
             5B7E DA82    MOVB R2,@2(R10)            ; Block number
             5B80 0002
             5B82 9802    CB   R2,@ETXCOD
             5B84 2BC1 +
             5B86 13C4    JEQ  TAPINIT
             5B88 0581    INC  R1
             5B8A 8801    C    R1,@2CCE
             5B8C 2CCE ,
             5B8E 1A01    JL   5B92
             5B90 04C1    CLR  R1
             5B92 C801    MOV  R1,@TAPEIO2
             5B94 2822 ("
             5B96 0380    RTWP

DOTAPIO      5B98 C0A0    MOV  @2CCE,R2
             5B9A 2CCE ,
             5B9C 0203    LI   R3,TAPEIO4
             5B9E 2846 (F
             5BA0 0414    BLWP *R4                   ; Execute IO action (read or write)
             5BA2 CAA0    MOV  @BIOSWS,@12D(R10)     ; IO status
             5BA4 2824 ($
             5BA6 000C
             5BA8 045B  [ B    *R11

PREPTAP      5BAA C06A  j MOV  @8(R10),R1            ; Drive number
             5BAC 0008
             5BAE 0281    CI   R1,31D
             5BB0 001F
             5BB2 1309    JEQ  ISTAPE   ; Unit is TAPE:
             5BB4 C0AA    MOV  @4(R10),R2            ; Byte count
             5BB6 0004
             5BB8 C0EA    MOV  @6(R10),R3            ; Buffer address
             5BBA 0006
             5BBC 0414    BLWP *R4                   ; Execute relevant IO operation
             5BBE CAA0    MOV  @BIOSWS,@12D(R10)     ; To IO result code
             5BC0 2824 ($
             5BC2 000C
             5BC4 0380    RTWP
ISTAPE       5BC6 045B  [ B    *R11

PRSKEY       5BC8 2896 (  .WORD SUB2WS  ; Vector
             5BCA 5BCC [  .WORD PRSKEYX

PRSKEYX      5BCC 0204    LI   R4,0800  ; VDP address setup, screen start
             5BCE 0800
             5BD0 0420    BLWP @VMBW
             5BD2 478E G
             5BD4 5F40 _@ .WORD TXTAST+2; Asterisk line
             5BD6 289E (  .WORD SUB2WS+8; VDP address from R4 SUB2WS
             5BD8 001E    .WORD 30
             5BDA A120    A    @WIDTH,R4; Next line
             5BDC 2CC6 ,
             5BDE 0420    BLWP @IVMBW
             5BE0 47A8 G
             5BE2 2824 ($ .WORD BIOSWS
             5BE4 289E (  .WORD SUB2WS+8; VDP address from R4 SUB2WS
             5BE6 000E    .WORD 14
```

```
             5BE8 0224  $ AI   R4,14
             5BEA 000E
             5BEC 0420    BLWP @VMBW
             5BEE 478E G
             5BF0 5F30 _0 .WORD TXTANY  ; 'Press any key'
             5BF2 289E (  .WORD SUB2WS+8; VDP address from R4 SUB2WS
             5BF4 0010    .WORD 16
             5BF6 A120    A    @WIDTH,R4; Next line
             5BF8 2CC6 ,
             5BFA 1002    JMP  JPGROM2

; Labels are repeated here, since to the CPU it's the same thing
GROMRD       5BFC 0006    ;GROM read data
GROMRA       5BFE 6906 i  ;GROM read address

JPGROM2      5C00 0224  $ AI   R4,-14
             5C02 FFF2
             5C04 0420    BLWP @VMBW
             5C06 478E G
             5C08 5F40 _@ .WORD TXTAST+2
             5C0A 289E (  .WORD SUB2WS+8; VDP address from R4 SUB2WS
             5C0C 001E    .WORD 14
LOOP44       5C0E 06A0    BL   @KSCAN
             5C10 42C2
             5C12 9820    CB   @8375,@ALLFF
             5C14 8375  u
             5C16 40B8 @
             5C18 16FA    JNE  LOOP44   ; Any key?
LOOP42       5C1A 06A0    BL   @KSCAN
             5C1C 42C2 B
             5C1E 9820    CB   @8375,@ALLFF
             5C20 8375  u
             5C22 40B8 @
             5C24 13FA    JEQ  LOOP42   ; No key?
LOOP43       5C26 06A0    BL   @KSCAN
             5C28 42C2 B
             5C2A 9820    CB   @8375,@ALLFF
             5C2C 8375  u
             5C2E 40B8 @
             5C30 16FA    JNE  LOOP43   ; Any key?
             5C32 06A0    BL   @DISPCUR
             5C34 47D2 G
             5C36 0380    RTWP

DOTAPWR      5C38 2824 ($ .WORD BIOSWS  ; Vector used by tape write
             5C3A 5C3C \< .WORD TAPWRX
TAPWRX       5C3C 0202    LI   R2,TXTREC; Pointer "*SET TO RECORD
             5C3E 5F22 _"
             5C40 0203    LI   R3,23H   ; CRU bits 0010 0011 () Timer setup
             5C42 0023  #
             5C44 06A0    BL   @TAPEMOT
             5C46 5CAE \
             5C48 0208    LI   R8,1E19  ; Instruction SBZ  MAGOUT (25)
             5C4A 1E19
             5C4C 0209    LI   R9,TAPEOUT
             5C4E 5D0A ]
             5C50 0202    LI   R2,768D
             5C52 0300
LOOP82       5C54 04C4    CLR  R4       ; Data to output
             5C56 0699    BL   *R9      ; TAPEOUT
             5C58 0602    DEC  R2
             5C5A 16FC    JNE  LOOP82
             5C5C 0704    SETO R4       ; Data to output
             5C5E 0699    BL   *R9      ; TAPEOUT
             5C60 C105    MOV  R5,R4
             5C62 06C4    SWPB R4       ; Data to output
             5C64 0699    BL   *R9      ; TAPEOUT
             5C66 C105    MOV  R5,R4
             5C68 06C4    SWPB R4       ; Data to output
             5C6A 0699    BL   *R9      ; TAPEOUT
             5C6C 04C3    CLR  R3       ; Repeat twice flag
LOOP104      5C6E 0202    LI   R2,8
             5C70 0008
LOOP83       5C72 04C4    CLR  R4
             5C74 0699    BL   *R9      ; TAPEOUT
             5C76 0602    DEC  R2
             5C78 16FC    JNE  LOOP83
             5C7A 0704    SETO R4
             5C7C 0699    BL   *R9      ; TAPEOUT
             5C7E 0202    LI   R2,64D
             5C80 0040  @
             5C82 04C7    CLR  R7
LOOP84       5C84 C130  0 MOV  *R0+,R4
             5C86 B1C4    AB   R4,R7
```

```
             5C88 0699    BL   *R9      ; TAPEOUT
             5C8A B1C4    AB   R4,R7
             5C8C 0699    BL   *R9      ; TAPEOUT
             5C8E 0642  B DECT R2
             5C90 16F9    JNE  LOOP84
             5C92 C107    MOV  R7,R4
             5C94 0699    BL   *R9      ; TAPEOUT
             5C96 0220    AI   R0,-64D
             5C98 FFC0
             5C9A 0543  C INV  R3
             5C9C 16E8    JNE  LOOP104
             5C9E 0220    AI   R0,64D
             5CA0 0040  @
             5CA2 0605    DEC  R5
             5CA4 16E4    JNE  LOOP104
             5CA6 06A0    BL   @TMRWAIT ; Wait for timer interrupt
             5CA8 5D30 ]0
             5CAA 0460  ` B    @CLRHALF
             5CAC 5E82 ^

TAPEMOT      5CAE C28B    MOV  R11,R10
             5CB0 04CC    CLR  R12                   ; Console CRU
             5CB2 C160  ` MOV  @TAPEIO1,R5
             5CB4 2820 (
             5CB6 1318    JEQ  5CE8
             5CB8 1D16    SBO  CS1MOT   ; CS1 motor enable
             5CBA 0200    LI   R0,TXTREW; Text pointer "Rewind tape"
             5CBC 5F06 _
             5CBE 0420    BLWP @PRSKEY  ; Show text, wait for key
             5CC0 5BC8 [
             5CC2 1E16    SBZ  CS1MOT   ; CS1 motor disable
             5CC4 C002    MOV  R2,R0                 ; Text pointer
             5CC6 0420    BLWP @PRSKEY  ; Show text "Press any key", wait for key
             5CC8 5BC8 [
             5CCA 1D16    SBO  CS1MOT   ; CS1 motor enable
             5CCC C120    MOV  @TIMELOW,R4           ; Remember time
             5CCE 2BA4 +
LOOP45       5CD0 1F02    TB   VDPIRQ   ; Wait for VDP interrupt
             5CD2 13FE    JEQ  LOOP45   ; Off?
             5CD4 1D02    SBO  2                     ; VDP interrupt            clear
             5CD6 0420    BLWP @VDPINT
             5CD8 2AAA *
             5CDA C160  ` MOV  @TIMELOW,R5
             5CDC 2BA4 +
             5CDE 6144 aD S    R4,R5                 ; Elapsed time
             5CE0 0285    CI   R5,600D  ; 10 or 12 seconds
             5CE2 0258  X
             5CE4 16F5    JNE  LOOP45
             5CE6 1E16    SBZ  CS1MOT   ; CS1 motor disable
             5CE8 C16D  m MOV  @4(R13),R5
             5CEA 0004
             5CEC 0225  % AI   R5,63D
             5CEE 003F  ?
             5CF0 0965  e SRL  R5,6
             5CF2 C02D  - MOV  @6(R13),R0
             5CF4 0006
             5CF6 04E0    CLR  @TAPEIO1
             5CF8 2820 (
             5CFA 1E00    SBZ  CONTROL  ; 9901 control set to IO mode
             5CFC 1E01    SBZ  EXTMASK  ; External interrupt
             5CFE 1E02    SBZ  VDPMASK  ; VDP interrupt
             5D00 1E0C    SBZ  12                    ; PSI timer. Restarts the counter
             5D02 1D16    SBO  CS1MOT   ; CS1 motor enable
             5D04 1D03    SBO  TMRMASK  ; 9901 timer interrupt
             5D06 33C3 3  LDCR R3,15                 ; Load timer preset
             5D08 045A  Z B    *R10                  ; Saved return address

; Data output to tape
; Zero is single pulse, one is two pulses
TAPEOUT      5D0A 0206    LI   R6,8                  ; Loop counter, store 8 bits
             5D0C 0008
             5D0E C28B    MOV  R11,R10
LOOP81       5D10 06A0    BL   @TMRWAIT ; Wait for timer interrupt
             5D12 5D30 ]0
             5D14 0488    X    R8                    ; SBZ MAGOUT is 1E19H
             5D16 2A20 *  XOR  @CON300H,R8           ; Flip to SBO MAGOUT (25)  Level change
             5D18 5D2E ].
             5D1A 06A0    BL   @TMRWAIT ; Wait for timer interrupt
             5D1C 5D30 ]0
             5D1E 0A14    SLA  R4,1                  ; Control of what to store
             5D20 1703    JNC  ZEROBIT  ; Shift out a zero bit?
             5D22 0488    X    R8                    ; SBO MAGOUT is 1D19H
             5D24 2A20 *  XOR  @CON300H,R8           ; Flip to SBZ MAGOUT       Level change
             5D26 5D2E ].
```

```
ZEROBIT     5D28 0606    DEC  R6                    ; Loop counter
            5D2A 16F2    JNE  LOOP81
            5D2C 045A  Z B    *R10                  ; Saved return address

CON300H     5D2E 0300    .WORD 300H

TMRWAIT     5D30 1F0F    TB   INTREQ   ; Interrupt signal
            5D32 16FE    JNE  TMRWAIT  ; No interrupt
            5D34 1E00    SBZ  CONTROL  ; 9901 control IO mode
            5D36 1D03    SBO  TMRMASK  ; Timer interrupt
            5D38 1D00    SBO  CONTROL  ; Timer mode
            5D3A 045B  [ B    *R11

; Tape read function
DOTAPRD     5D3C 2824 ($ .WORD BIOSWS  ; Vector used by tape read
            5D3E 5D40 ]@ .WORD TAPRDX

TAPRDX      5D40 04C1    CLR  R1
            5D42 0202    LI   R2,TXTPLAY            ; Press play on tape
            5D44 5F14 _
            5D46 0203    LI   R3,2BH   ; CRU bits 0010 1011 ()
            5D48 002B  +
            5D4A 06A0    BL   @TAPEMOT
            5D4C 5CAE \
            5D4E 04C0    CLR  R0
LOOP101     5D50 0208    LI   R8,30000D
            5D52 7530 u0
NEAR6       5D54 0206    LI   R6,LOOP102
            5D56 5D60 ]`
            5D58 D060    ` MOVB @NEAR6+1,R1         ; Happens to be 06
            5D5A 5D55 ]U
            5D5C 0203    LI   R3,002BH
            5D5E 002B  +
LOOP102     5D60 0608    DEC  R8
            5D62 1602    JNE  5D68
            5D64 0460    ` B    @CLRHALF
            5D66 5E82 ^

            5D68 0202    LI   R2,0030H
            5D6A 0030  0
            5D6C C000    MOV  R0,R0
            5D6E 1601    JNE  5D72
            5D70 A082    A    R2,R2
            5D72 06A0    BL   @TAPEINP
            5D74 5E9A ^
            5D76 1001    JMP  5D7A                  ; Tape and phase bit different
            5D78 10F3    JMP  LOOP102  ; Tape and phase bit equal

            5D7A 0602    DEC  R2
            5D7C 16FA    JNE  5D72
            5D7E 0209    LI   R9,7FFF  ; Max timer value (349 milliseconds)
            5D80 7FFF
            5D82 0202    LI   R2,8                  ; Loop counter (2.79 seconds)
EIGHT       5D84 0008
            5D86 33C9 3  LDCR R9,15                 ; Load timer for 349 milliseconds
            5D88 1E00    SBZ  CONTROL  ; 9901 control to interrupt
            5D8A 1D03    SBO  TMRMASK  ; Allow timer interrupt
LOOP100     5D8C 06A0    BL   @GETTAPE ; Read bit from tape
            5D8E 5EEC ^
            5D90 1001    JMP  TPEQ1                 ; Tape and phase bit equal
            5D92 10FC    JMP  LOOP100  ; Tape and phase bit different

TPEQ1       5D94 0602    DEC  R2
            5D96 16FA    JNE  LOOP100
            5D98 1D00    SBO  CONTROL  ; Timer mode
            5D9A 37C3 7  STCR R3,15                 ; Read timer value
            5D9C 6243 bC S    R3,R9                 ; Subtract from timer start value
            5D9E C0C9    MOV  R9,R3
            5DA0 0A29  ) SLA  R9,2
            5DA2 A0C9    A    R9,R3
            5DA4 0963  c SRL  R3,6
            5DA6 0263  c ORI  R3,0001H
            5DA8 0001
            5DAA 020A    LI   R10,5DB8
            5DAC 5DB8 ]
            5DAE 0283    CI   R3,001F
            5DB0 001F
            5DB2 11D4    JLT  5D5C
            5DB4 0460    ` B    @5EA6
            5DB6 5EA6 ^

            5DB8 06A0    BL   @TAPEINP
            5DBA 5E9A ^
            5DBC 10FD    JMP  5DB8                  ; Tape and phase bit different
```

```
                5DBE 0202    LI   R2,7                  ; Tape and phase bit equal
                5DC0 0007
                5DC2 06A0    BL   @TAPEINP
                5DC4 5E9A ^
                5DC6 10CA    JMP  5D5C                  ; Tape and phase bit different
                5DC8 0602    DEC  R2                    ; Tape and phase bit equal
                5DCA 16FB    JNE  5DC2
                5DCC 0206    LI   R6,5E16
                5DCE 5E16 ^
                5DD0 C000    MOV  R0,R0
                5DD2 160E    JNE  5DF0
                5DD4 0580    INC  R0
                5DD6 D060  ` MOVB @EIGHT+1,R1
                5DD8 5D85 ]
                5DDA 04C7    CLR  R7
                5DDC 06A0    BL   @5ED2
                5DDE 5ED2 ^
                5DE0 8105    C    R5,R4
                5DE2 1A4F  O JL   CLRHALF
                5DE4 C144  D MOV  R4,R5
                5DE6 0507    NEG  R7
                5DE8 06A0    BL   @5ED2
                5DEA 5ED2 ^
                5DEC 164A  J JNE  CLRHALF
                5DEE 10B0    JMP  LOOP101

                5DF0 0200    LI   R0,TAPEIO4
                5DF2 2846 (F
                5DF4 0202    LI   R2,0040
                5DF6 0040  @
                5DF8 04C7    CLR  R7
                5DFA D060  ` MOVB @DEC14,R1
                5DFC 5E98 ^
LOOP103         5DFE 06A0    BL   @5ED2
                5E00 5ED2 ^
                5E02 06C4    SWPB R4
                5E04 DC04    MOVB R4,*R0+
                5E06 0602    DEC  R2
                5E08 16FA    JNE  LOOP103
                5E0A 0247  G ANDI R7,00FF
                5E0C 00FF
                5E0E 0507    NEG  R7
                5E10 06A0    BL   @5ED2
                5E12 5ED2 ^
                5E14 1307    JEQ  5E24
                5E16 C145  E MOV  R5,R5
                5E18 1134  4 JLT  CLRHALF
                5E1A 0505    NEG  R5
                5E1C A002    A    R2,R0
                5E1E 0220    AI   R0,-64D
                5E20 FFC0
                5E22 1096    JMP  LOOP101

                5E24 C145  E MOV  R5,R5
                5E26 1108    JLT  5E38
                5E28 0202    LI   R2,0049
                5E2A 0049  I
                5E2C 0206    LI   R6,5E34
                5E2E 5E34 ^4
                5E30 06A0    BL   @5ED2
                5E32 5ED2 ^
                5E34 0602    DEC  R2
                5E36 16FA    JNE  5E2C
                5E38 0745  E ABS  R5
                5E3A 0200    LI   R0,TAPEIO4
                5E3C 2846 (F
                5E3E C1ED    MOV  @4(R13),R7
                5E40 0004
                5E42 1303    JEQ  5E4A
                5E44 0287    CI   R7,0040
                5E46 0040  @
                5E48 1102    JLT  5E4E
                5E4A 0207    LI   R7,0040
                5E4C 0040  @
                5E4E 6B47 kG S    R7,@4(R13)
                5E50 0004
                5E52 C22D  - MOV  @6(R13),R8
                5E54 0006
                5E56 AB47  G A    R7,@6(R13)
                5E58 0006
                5E5A C1AD    MOV  @20D(R13),R6
                5E5C 0014
                5E5E C1A6    MOV  @10D(R6),R6
                5E60 000A
```

```
              5E62 0A16    SLA  R6,1
              5E64 1104    JLT  5E6E
LOOP94        5E66 DE30  0 MOVB *R0+,*R8+
              5E68 0607    DEC  R7
              5E6A 16FD    JNE  LOOP94
              5E6C 1005    JMP  5E78

              5E6E 0420    BLWP @I2VMBW
              5E70 47B2 G
              5E72 2824 ($ .WORD BIOSWS  ; BIOSWS R0
              5E74 2834 (4 .WORD BIOSWS+16            ; BIOSWS R8
              5E76 2832 (2 .WORD BIOSWS+14            ; BIOSWS R7
              5E78 0605    DEC  R5
              5E7A 1302    JEQ  CLRALL   ; Clear all of the return code
              5E7C 0460  ` B    @LOOP101
              5E7E 5D50 ]P

CLRALL        5E80 7041 pA SB   R1,R1    ; Clear most significant byte of R1
              5E82 06C1    SWPB R1
              5E84 7041 pA SB   R1,R1
              5E86 C001    MOV  R1,R0                 ; Return result code via BIOSWS R0
              5E88 1E03    SBZ  TMRMASK  ; 9901 timer interrupt
              5E8A 1E16    SBZ  CS1MOT   ; CS1 motor
              5E8C 1D0C    SBO  12                    ; 9901 reserved
              5E8E 1D01    SBO  EXTMASK  ; External interrupt
              5E90 1D02    SBO  VDPMASK  ; VDP interrupt
              5E92 1E00    SBZ  CONTROL  ; 9901 control
              5E94 0380    RTWP

TWO           5E96 0002    .WORD 2       ; Constants
DEC14         5E98 0E      .BYTE 14D
              5E99 00           .BYTE 0

TAPEINP       5E9A C28B    MOV  R11,R10  ; Save return address
              5E9C 06A0    BL   @TMRWAIT ; Wait for timer interrupt
              5E9E 5D30 ]0
              5EA0 06A0    BL   @GETTAPE ; Read tape input
              5EA2 5EEC ^
              5EA4 05CA    INCT R10                   ; Tape and phase bit equal
              5EA6 2460 $` CZC  @TWO,R1  ; Tape and phase bit different
              5EA8 5E96 ^
              5EAA 1305    JEQ  LOOP97   ; Phase bit zero?
; Loops here if phase bit was one
LOOP95        5EAC 1F0F    TB   INTREQ   ; Interrupt signal (from timer)
              5EAE 130E    JEQ  TIMEOUT  ; Timeout?
              5EB0 1F1B    TB   MAGINP   ; Tape input
              5EB2 13FC    JEQ  LOOP95   ; Tape bit off?
              5EB4 1004    JMP  ENDGET

; Loops here if phase bit was zero
LOOP97        5EB6 1F0F    TB   INTREQ   ; Interrupt signal
              5EB8 1309    JEQ  TIMEOUT  ; Timeout?
              5EBA 1F1B    TB   MAGINP   ; Tape input
              5EBC 16FC    JNE  LOOP97   ; Tape bit on?
ENDGET        5EBE 33C3 3  LDCR R3,15                 ; 9901 timer setup reload value
              5EC0 1E00    SBZ  CONTROL  ; PSI interrupt mode
              5EC2 1D03    SBO  TMRMASK  ; Timer interrupt
              5EC4 1D00    SBO  CONTROL  ; Timer mode
              5EC6 2860 (` XOR  @TWO,R1  ; Phase bit inverted
              5EC8 5E96 ^
              5ECA 045A  Z B    *R10                  ; Saved return address (incremented if tape and phase
was equal)

; End due to timeout
TIMEOUT       5ECC 1E00    SBZ  CONTROL  ; 9901 control
              5ECE 1D03    SBO  TMRMASK  ; Timer interrupt
              5ED0 0456  V B    *R6                   ; Location depends on where it was when timeout
occured.

              5ED2 0208    LI   R8,8
              5ED4 0008
              5ED6 04C4    CLR  R4
              5ED8 C24B  K MOV  R11,R9
LOOP96        5EDA 0A14    SLA  R4,1
              5EDC 06A0    BL   @TAPEINP
              5EDE 5E9A ^
              5EE0 1001    JMP  5EE4                  ; Tape and phase bit different
              5EE2 0584    INC  R4                    ; Tape and phase bit equal
              5EE4 0608    DEC  R8
              5EE6 16F9    JNE  LOOP96
              5EE8 A1C4    A    R4,R7
              5EEA 0459  Y B    *R9                   ; Saved return address

; Get tape input
```

```
; If input and phase bit in R1 are equal, then invert phase bit and return.
; If input and phase bit in R1 are different, then keep phase bit but advance return one word.
GETTAPE       5EEC 1F1B    TB   MAGINP   ; Tape input
              5EEE 1306    JEQ  NOTMAG   ; Off?
              5EF0 2460 $` CZC  @TWO,R1  ; Phase bit
              5EF2 5E96 ^
              5EF4 1306    JEQ  SKIPRET  ; Phase bit off?
PBITOFF       5EF6 2860 (` XOR  @TWO,R1  ; Swap phase
              5EF8 5E96 ^
              5EFA 045B  [ B    *R11

NOTMAG        5EFC 2460 $` CZC  @TWO,R1  ; Phase bit
              5EFE 5E96 ^
              5F00 13FA    JEQ  PBITOFF  ; Phase bit OFF?
SKIPRET       5F02 05CB    INCT R11
              5F04 045B  [ B    *R11

TXTREW        5F06 2A20 *  .ASCII "* REWIND TAPE "
              5F08 5245 RE
              5F0A 5749 WI
              5F0C 4E44 ND
              5F0E 2054  T
              5F10 4150 AP
              5F12 4520 E
TXTPLAY       5F14 2A20 *  .ASCII "* SET TO PLAY "
              5F16 5345 SE
              5F18 5420 T
              5F1A 544F TO
              5F1C 2050  P
              5F1E 4C41 LA
              5F20 5920 Y
TXTREC        5F22 2A53 *S .ASCII "*SET TO RECORD"
              5F24 4554 ET
              5F26 2054  T
              5F28 4F20 O
              5F2A 5245 RE
              5F2C 434F CO
              5F2E 5244 RD
TXTANY        5F30 2D3E -> .ASCII "-> PRESS A KEY"
              5F32 2050  P
              5F34 5245 RE
              5F36 5353 SS
              5F38 2041  A
              5F3A 204B  K
              5F3C 4559 EY

TXTAST        5F3E 202A  * .ASCII " ********************************"
              5F40 2A2A **
              5F42 2A2A **
              5F44 2A2A **
              5F46 2A2A **
              5F48 2A2A **
              5F4A 2A2A **
              5F4C 2A2A **
              5F4E 2A2A **
              5F50 2A2A **
              5F52 2A2A **
              5F54 2A2A **
              5F56 2A2A **
              5F58 2A2A **
              5F5A 2A2A **
              5F5C 2A2A **

; Called by RELOCSEG. But there should be only one argument according to Internal Architecture Guide.
; R3 is procedure number*2
RLOCEXE       5F5E C17A  z MOV  *SP+,R5  ; A boolean. If false only segment relative relocation is performed.
              5F60 0B15    SRC  R5,1                  ; Put value in sign bit.
              5F62 C07A  z MOV  *SP+,R1  ; E_Rec_P
              5F64 C1A1    MOV  @2(R1),R6; Get Env_Vect
              5F66 0002
              5F68 C1FA    MOV  *SP+,R7  ; Base of segment
              5F6A C07A  z MOV  *SP+,R1  ; Code segment pointer
              5F6C C0E1    MOV  @2(R1),R3; Relocation list pointer (word offset)
              5F6E 0002
              5F70 1311    JEQ  RLOCRET  ; If zero return immediately (no relocation list)
              5F72 0A13    SLA  R3,1                  ; Relocation list pointer (byte offset)
              5F74 A0C1    A    R1,R3                 ; Add code segment base address
LOOP52        5F76 04C4    CLR  R4
              5F78 D813    MOVB *R3,@PASCALWS+9       ; Low byte R4. Reloc_Type
              5F7A 8389
              5F7C 0643  C DECT R3                    ; Point at listsize. Number of pointers in list
              5F7E 0284    CI   R4,3                  ; Reloc_Type must not be 4 (Proc_Rel), since that's
only for non-linked assembly
              5F80 0003
```

```
             5F82 1525  % JGT  UNKNOWN  ; Allowed range is 0..3. Otherwise "unknown instruction" error
             5F84 0A14    SLA  R4,1                  ; Word index
             5F86 C124  $ MOV  @EXECTAB(R4),R4
             5F88 5F8C _
             5F8A 0454  T B    *R4

EXECTAB      5F8C 5F94 _  .WORD RLOCRET ; Reloc_End
             5F8E 5F96 _  .WORD SEGREL  ; Seg_Rel
             5F90 5F9A _  .WORD BASEREL ; Base_Rel
             5F92 5FA8 _  .WORD INTPREL ; Interp_Rel

; End of relocation list
RLOCRET      5F94 045C  \ B    *R12                  ; Returns to RELOCSEG in this case (RLOCRT)

; Segment relative relocation
SEGREL       5F96 C087    MOV  R7,R2                 ; Relative to base of segment
             5F98 1010    JMP  CONT2A

BASEREL      5F9A D123  # MOVB @3(R3),R4; Data_Seg_Num.
             5F9C 0003
             5F9E 0974  t SRL  R4,7                  ; Make word pointer
             5FA0 A106    A    R6,R4                 ; Add environment pointer to get into the E_Vect.Map
             5FA2 C114    MOV  *R4,R4   ; Get E_Rec_P to local segemnt from map
             5FA4 C094    MOV  *R4,R2   ; Get global data pointer for local segment
             5FA6 1002    JMP  CONT3A

INTPREL      5FA8 0202    LI   R2,INTERP; Relative to PME's interp-relative table
             5FAA 4016 @
CONT3A       5FAC C145  E MOV  R5,R5                 ; Check relocation flag
             5FAE 1105    JLT  CONT2A   ; If true

             5FB0 C013    MOV  *R3,R0   ; List size
             5FB2 0643  C DECT R3                    ; Next relocation pointer
             5FB4 0A10    SLA  R0,1
             5FB6 60C0  ` S    R0,R3                 ; Skip all relocation pointers
             5FB8 10DE    JMP  LOOP52

CONT2A       5FBA C013    MOV  *R3,R0   ; Number of pointers in list
             5FBC 1005    JMP  CONT2B

LOOP53       5FBE 0643  C DECT R3                    ; Next pointer in list
             5FC0 C113    MOV  *R3,R4   ; Fetch value pointed to by relocation list item
             5FC2 A101    A    R1,R4                 ; Add code segment base address to find location to
relocate
             5FC4 A502    A    R2,*R4   ; Do the relocation
             5FC6 0600    DEC  R0                    ; Number of pointers in list
CONT2B       5FC8 15FA    JGT  LOOP53
             5FCA 0643  C DECT R3                    ; Point to next relocation list
             5FCC 10D4    JMP  LOOP52

UNKNOWN      5FCE 0460  ` B    @UNKINS  ; Unknown p-code
             5FD0 355E 5^

             5FD2 0000    .WORD 0
             5FD4 0000    .WORD 0
             5FD6 0000    .WORD 0
             5FD8 0000    .WORD 0
             5FDA 0000    .WORD 0
             5FDC 0000    .WORD 0
             5FDE 0000    .WORD 0
             5FE0 0000    .WORD 0
             5FE2 0000    .WORD 0
             5FE4 0000    .WORD 0
             5FE6 0000    .WORD 0
             5FE8 0000    .WORD 0
             5FEA 0000    .WORD 0
             5FEC 0000    .WORD 0
             5FEE 0000    .WORD 0
             5FF0 0000    .WORD 0
             5FF2 0000    .WORD 0
             5FF4 0000    .WORD 0
             5FF6 0000    .WORD 0
             5FF8 0000    .WORD 0
             5FFA 0000    .WORD 0

; Labels are repeated here, since to the CPU it's the same thing, regardless of which memory page used.
GROMWD       5FFC 0606    ;GROM write data (never used)
GROMWA       5FFE 0606    ;GROM write address

; Second p-code ROM page ends here ---------------------------------------

Scratch pad RAM under the p-system

; First the inner PME, which also handles immediate data after instructions
```

```
; Version loaded below for code in CPU RAM
FETCH         8300 D078  x MOVB *R8+,R1  ; Instruction fetch
              8302 0971  q SRL  R1,7                 ; Make word
              8304 C0A1    MOV  @OPCODE(R1),R2 ;2DB2 Fetch address to interpreter
              8306 2DB2 -
              8308 C032  2 MOV  *R2+,R0  ; Fetch execution pointer
              830A 0450  P B    *R0                  ; Branch to instruction interpreter/data fetch
              830C 1000    NOP                       ; Filler for INC R8, which isn't needed for CPU RAM
code
LDUBB         830E 04C4    CLR  R4
              8310 D838  8 MOVB *R8+,@PASCALWS+9     ; Lsby R4 Load unsigned byte when there are two
              8312 8389
              8314 1000    NOP                       ; Filler for INC R8, which isn't needed for CPU RAM
code
LDUB          8316 04C3    CLR  R3
              8318 D838  8 MOVB *R8+,@PASCALWS+7     ; Lsby R3 Load unsigned byte
              831A 8387
              831C 0452  R B    *R2                  ; Interpret instruction
              831E 1000    NOP                       ; Filler for INC R8, which isn't needed for CPU RAM
code
LDUBBG        8320 04C5    CLR  R5
              8322 D838  8 MOVB *R8+,@PASCALWS+11    ; Lsby R5 Load unsigned byte when there are three
              8324 838B
              8326 1000    NOP                       ; Filler for INC R8, which isn't needed for CPU RAM
code
LDUBIG        8328 04C4    CLR  R4
              832A D838  8 MOVB *R8+,@PASCALWS+9     ; Lsby R4 Load unsigned byte
              832C 8389
              832E 1000    NOP                       ; Filler for INC R8, which isn't needed for CPU RAM
code
LDBIG         8330 04C3    CLR  R3
              8332 D0F8    MOVB *R8+,R3
              8334 1102    JLT  ISBIG                ; "Big" value? (15 bits)
              8336 06C3    SWPB R3                   ; Unsigned byte value 0..127
              8338 0452  R B    *R2                  ; Interpret instruction
ISBIG         833A 0243  C ANDI R3,7F00  ; Remove sign bit
              833C 7F00 
              833E D838  8 MOVB *R8+,@PASCALWS+7     ; Lsby R3 low byte Make 15 bit value
              8340 8387
              8342 0452  R B    *R2                  ; Interpret instruction
              8344 1D02    SBO  2                    ; Overflow from code transfer (CPU code is shorter)
              8346 D820    MOVB @8802,@0300          ; Overflow from code transfer (CPU code is shorter)
              8348 8802                              ; Overflow from code transfer (CPU code is shorter)
; Floating point accumulator (FAC)
FAC           834A 0300    .WORD         ; Start of buffer area for some data transfers
              834C 3F00 ?  .WORD
              834E 0101    .WORD
              8350 0000    .WORD

              8352 0019    .WORD
NAMLEN        8354 0007    .WORD         ; Name length base name of device, error code from floating point
operations
PERIOD        8356 3EA9 >  .WORD         ; First period in device name
              8358 0000    .WORD
              835A 0000    .WORD
; Floating point argument (ARG)
FARG          835C 0000    .WORD
              835E 0000    .WORD
              8360 0000    .WORD
              8362 0000    .WORD

              8364 346C 4l .WORD
              8366 41F4 A  .WORD
              8368 453E E> .WORD
              836A 4580 E  .WORD
              836C 4320 C  .WORD
              836E 0000    .WORD         ; End of buffer area

VDPTOP        8370 3EDF >  .WORD         ; VDP RAM top pointer (GPL)
              8372 FF           .BYTE
GPLSTK  8373 7C    | .BYTE ; Sub stack pointer (GPL), initialized to 7C
KBD           8374 00      .BYTE         ; Initialized to 04FF
KEYCODE       8375 FF           .BYTE    ; FAC sign (00 or FF for positive or negative)
              8376 0000    .WORD         ; FAC exponent copy
              8378 0001    .WORD
              837A 00C2    .WORD
GPLST         837C 0000    .WORD         ; GPL Status
              837E 6002 `  .WORD

; Pascal main workspace
PASCALWS 8380 0000   .WORD              ; R0
              8382 BCE2    .WORD         ; R1 Opcode*2 (p-code) when entering the interpreter
              8384 000C    .WORD         ; R2
              8386 0002    .WORD         ; R3
```

```
             8388 D930  0 .WORD         ; R4
             838A AD20    .WORD         ; R5
             838C AD96    .WORD         ; R6
             838E FA4A  J .WORD         ; R7
             8390 AF79  y .WORD         ; R8 IPC
             8392 FB5A  Z .WORD         ; R9 Frame pointer for local activation record on stack.
             8394 FA3E  > .WORD         ; R10 SP
             8396 4E28 N( .WORD         ; R11 Return link.
             8398 8300    .WORD FETCH   ; R12 PME fetch instruction address.
             839A 0000    .WORD         ; R13 Read data address for currently executing code. VDPRD, PGRMRD
or 0 (for CPU RAM).
             839C A91C    .WORD         ; R14 Global data frame pointer for current segment.
             839E 0000    .WORD         ; R15 Code flag. =0: CPU RAM, <0: VDP RAM, >0: GROM.

;Subsidiary workspace, used for simulated interrupt
SUBWIN       83A0 0000    .WORD
             83A2 2970 )p .WORD
             83A4 297C )| .WORD
             83A6 0000    .WORD
             83A8 2780 '  .WORD
             83AA 0000    .WORD
             83AC 0000    .WORD
             83AE 0000    .WORD
             83B0 0000    .WORD
             83B2 3E98 >  .WORD
             83B4 8C02    .WORD         ; R10 VDPWA
             83B6 4784 G  .WORD
             83B8 0000    .WORD
             83BA 2964 )d .WORD
             83BC 348E 4  .WORD
             83BE C400    .WORD

;Workspace, standard interrupt
INTWS        83C0 3E9D >  .WORD
             83C2 0A00    .WORD         ; Interrupt flag byte
             83C4 5AD8 Z  .WORD         ; User defined interrupt (not with p-system)
             83C6 0000    .WORD
             83C8 0000    .WORD
             83CA 0BB9    .WORD
             83CC E1E6    .WORD
             83CE 0000    .WORD
             83D0 8C02    .WORD
             83D2 0001    .WORD
             83D4 29A8 )  .WORD         ; R10
             83D6 5B02 [  .WORD
             83D8 0000    .WORD
             83DA 8380    .WORD PASCALWS
             83DC 507A Pz .WORD
             83DE C000    .WORD

;GPL workspace
GPLWS        83E0 0000    .WORD         ; R0
             83E2 7E9D ~  .WORD
             83E4 4540 E@ .WORD
             83E6 0000    .WORD
             83E8 83E0    .WORD         ; R4 The RS232 DSR does a STWP R4
             83EA 42FA B  .WORD
             83EC 0A4E  N .WORD
             83EE 0000    .WORD
             83F0 4076 @v .WORD
             83F2 83DA    .WORD
             83F4 0002    .WORD
             83F6 346C 4l .WORD
             83F8 1300    .WORD         ; R12 address of RS232
             83FA 9800    .WORD GRMRD   ; R13
             83FC 0108    .WORD
             83FE 8C02    .WORD VDPWA   ; R15
```